BỘ SÁCH CHINH PHỤC C++







Sách Hay Nên Có Trong Tủ Sách Nhà Bạn

Đặt sách tại: www.facebook.com/nguyendang.phu.79

Nguyễn Đặng Phú - GV Trường THPT chuyên Lam Sơn
Phạm Gia Việt Anh - Hoàng Minh Thái (cựu hs chuyên Tin)

LẬP TRÌNH C++ THẬT GIẢN ĐƠN

Nguyễn Đặng Phú - GV Trường THPT chuyên Lam Sơn
Phạm Gia Việt Anh - Hoàng Minh Thái (cựu hs chuyên Tin)

LẬP TRÌNH C++
THẬT GIẢN ĐƠN



Sách Hay Nên Có Trong Tủ Sách Nhà Bạn

Thanh Hóa, 2023

LỜI NÓI ĐẦU

Hiện nay trên mạng có rất nhiều tài liệu viết về ngôn ngữ lập trình C++; nếu như các thầy/cô, các em học sinh, sinh viên tự đọc và nghiên cứu mà đang còn gặp khó khăn thì hãy đọc cuốn sách này. Với mong muốn để các thầy/cô, các em học sinh, sinh viên tiếp cận một cách hiệu quả và dễ dàng hơn, nên chúng tôi viết cuốn sách "***Lập Trình C++ Thật Giản Đơn***".

Tập này gồm 2 phần:

+ ***Phần I:*** với nội dung cơ bản và dễ hiểu nhất, để thầy/cô có thể dạy được ngôn ngữ C++ cho học sinh và các em học sinh, sinh viên có thể tự học. Phần I này được đúc kết qua quá trình đi giảng dạy, nội dung trong tài liệu mang tính thực dụng, những vấn đề thường hay áp dụng trong thi cử và giúp mọi người có kiến thức cơ bản về lập trình C++.

+ ***Phần II:*** với nội dung nâng cao hơn như: *Độ phức tạp của thuật toán, sắp xếp, tìm kiếm nhị phân, kỹ thuật nhánh cận, các dạng bài toán quy hoạch động*… giúp các thầy cô, các em học sinh, sinh viên làm những bài tập vận dụng kiến thức khó hơn, ở mức độ HSG cấp Tỉnh. Tất cả các bài tập ở phần này đều có test để chấm (*Link download ở trang 277*).

Toàn bộ các bài tập trong cuốn sách này được chúng tôi sưu tầm từ nhiều nguồn tài liệu khác nhau.

Tác giả

Mục lục

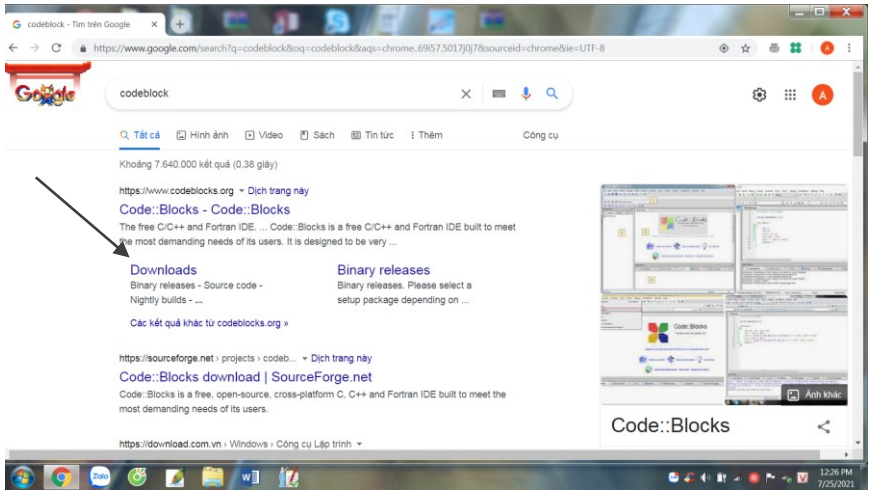
	Trang		*Trang*
PHẦN I		Gợi ý cách giải	116
Cài đặt Codeblock	3	Hàm không trả về giá trị	121
Sử dụng Codeblock	5	Hàm đệ quy	125
Lệnh vào ra trong C++	12	Bài tập	128
Các kiểu dữ liệu cơ bản	15	Gợi ý cách giải	130
Biến và hằng	17	**PHẦN II**	
Phép toán, biểu thức, câu lệnh	22	Độ phức tạp thuật toán	134
Bài tập	26	Hàm sort trong thư viện	137
Gợi ý cách giải	29	Bài tập	140
Câu lệnh và khối lệnh	33	Gợi ý cách giải	143
Đọc ghi dữ liệu từ tệp	33	Tìm kiếm nhị phân	146
Cấu trúc rẽ nhánh	35	lower_bound	148
Bài tập	41	upper_bound	149
Gợi ý cách giải	43	Bài tập	153
Cấu trúc lặp	44	Gợi ý cách giải	157
Bài tập	53	Kỹ thuật nhánh cận	163
Gợi ý cách giải	56	Bài tập	166
Kiểu mảng	68	Gợi ý cách giải	170
Bài tập	74	Quy hoạch động	179
Gợi ý cách giải	76	Dãy các phần tử dài nhất	180
Kiểu xâu	86	Biến đổi xâu	187
Bài tập	90	Dạng bài toán cái túi	194
Gợi ý cách giải	93	Ghép cặp	197
Kiểu bản ghi	103	Di chuyển	201
Bài tập	106	Vị trí cấu hình	205
Gợi ý cách giải	107	Trạng thái	209
Đọc dữ liệu từ tệp khi không biết trước số lượng phần tử	108	Gợi ý cách giải	214
Hàm	112	Lỗi thí sinh hay mắc phải	276
Bài tập	115	Lời nói cuối	277

PHẦN I

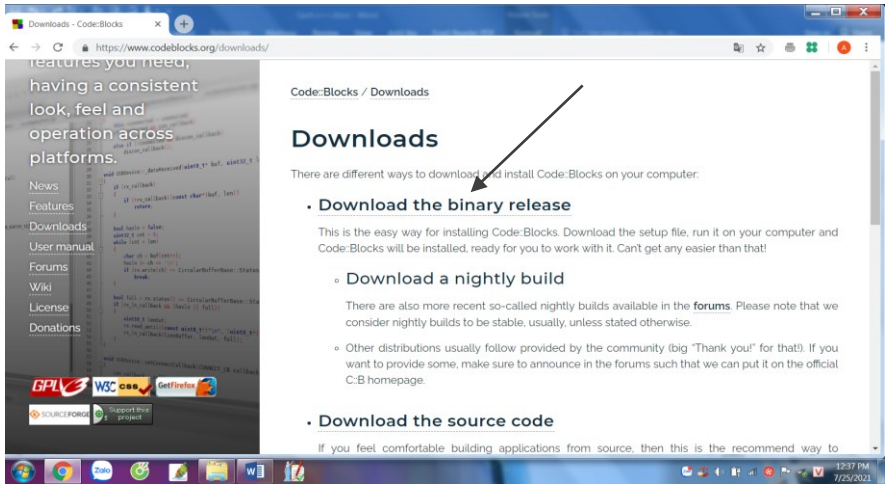
I. Tải và cài đặt Codeblock

Đây là trình soạn thảo và biên dịch chương trình C++ mà học sinh phổ thông dùng phổ biến hiện nay.

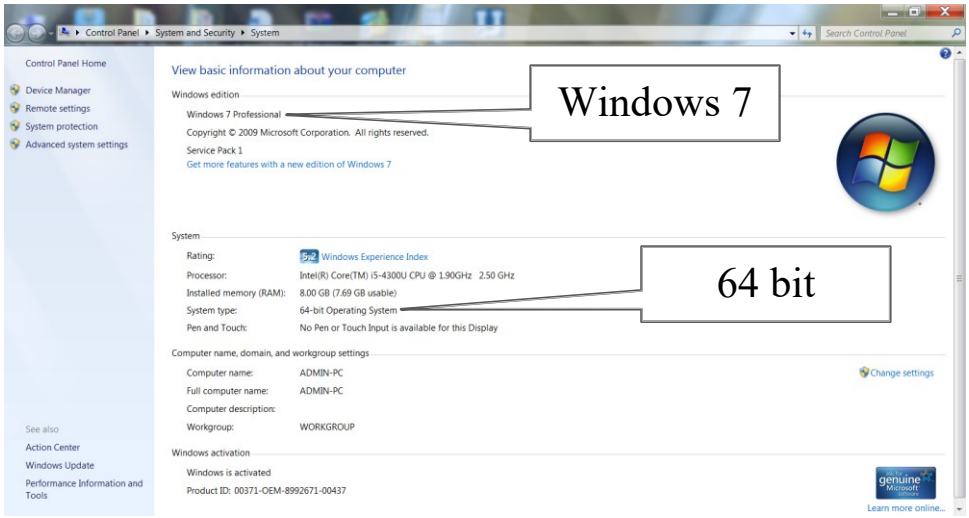
Bước 1: vào google gõ từ khóa "**codeblock**" rồi click vào ***Downloads*** để tải phiên bản mới nhất.



Bước 2: Click vào "**Download the binary release**"

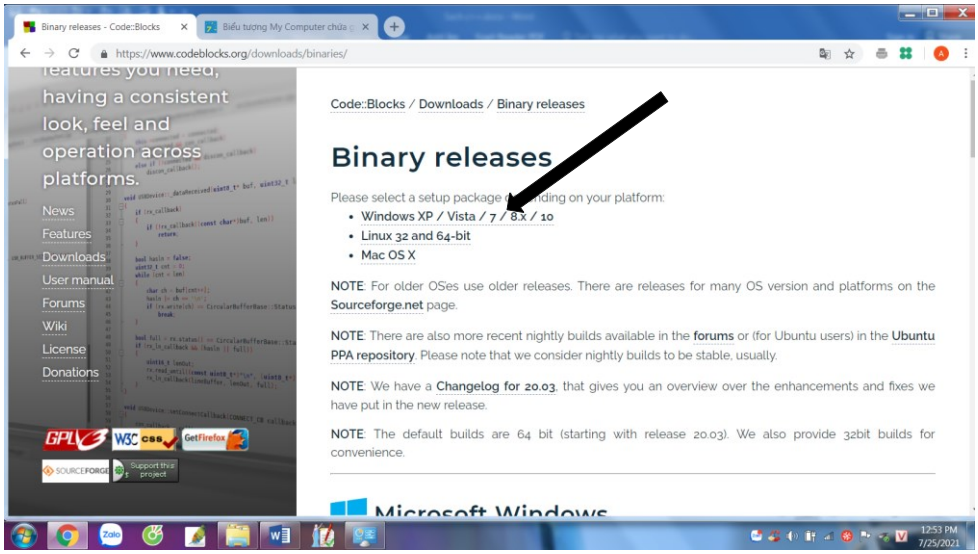


Bước 3: Chọn phiên bản tương ứng với hệ điều hành máy tính của mình (nếu muốn biết máy tính mình hệ điều hành gì thì đa số chúng ta dùng hệ điều hành **Windows**, ta chuột phải vào biểu tượng **My computer** chọn **Properties**)



Ảnh trên xác định máy là hệ điều hành windows 7 bản 64 bit

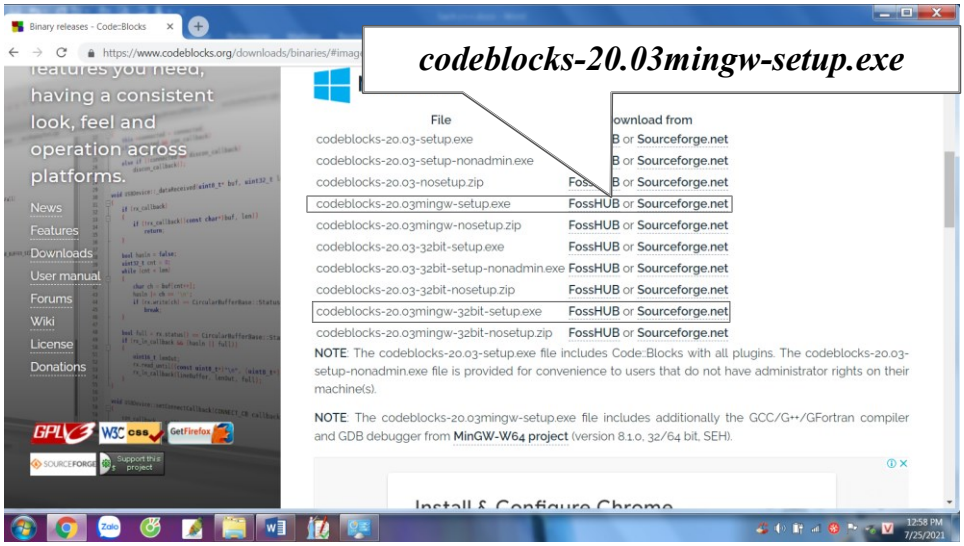
Sau khi xác định được hệ điều hành, ví dụ máy tính như ảnh thì ta click chuột trái vào **Windows XP / Vista / 7 / 8.x / 10**



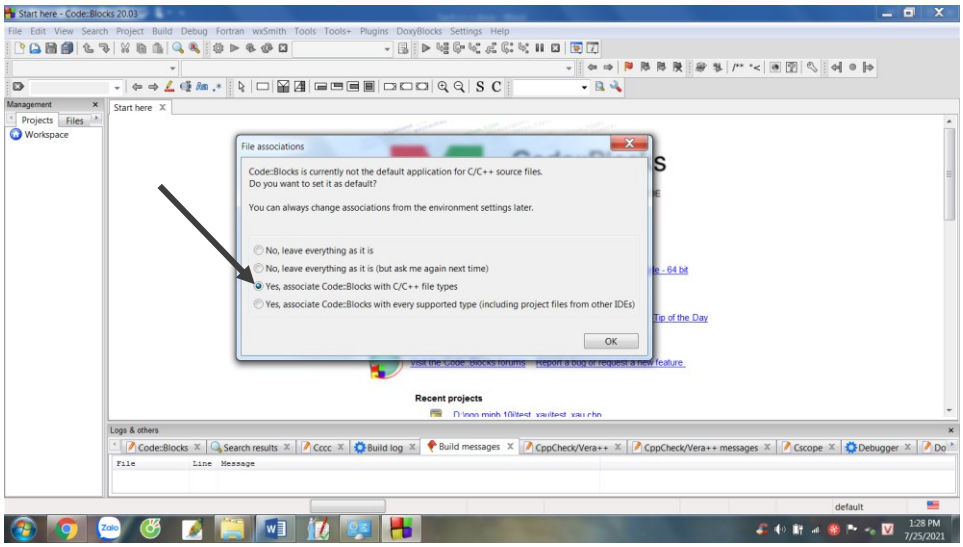
Bước 4: Nếu là windows bản ***64 bit*** thì download ***codeblocks-20.03mingw-setup.exe***, nếu là windows bản ***32 bit*** thì download ***codeblocks-20.03mingw-32bit-setup.exe*** (chú ý ***download*** đúng, có một số ***download*** không đúng phiên bản nên khi soạn thảo xong chương trình mà không chạy được vì không có trình biên dịch nên chỉ có chức năng soạn thảo *code*);

Chúng ta click vào **FossHUB** or **Sourceforge.net** của hàng tương ứng (trong 2 link ***download*** thì chọn cái nào cũng được); Ví dụ tôi click vào mũi tên như

hình phía dưới, rồi chọn đường dẫn để lưu vào máy. Tại thời điểm này thì phiên bản mới nhất của ***Codeblock*** là bản 20.03



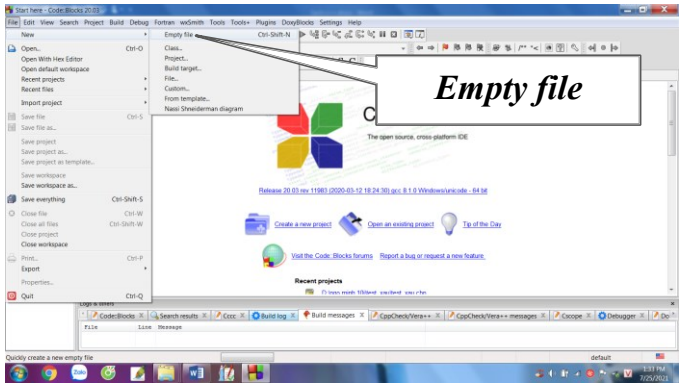
Bước 5: Sau khi ***download*** về máy tính ta click đúp vào ***codeblocks-20.03mingw-setup.exe*** để cài đặt, chúng ta cứ để mặc định để cài bình thường; Lần đầu tiên mở ***Codeblock*** thì chọn lựa như ảnh rồi ***Ok***



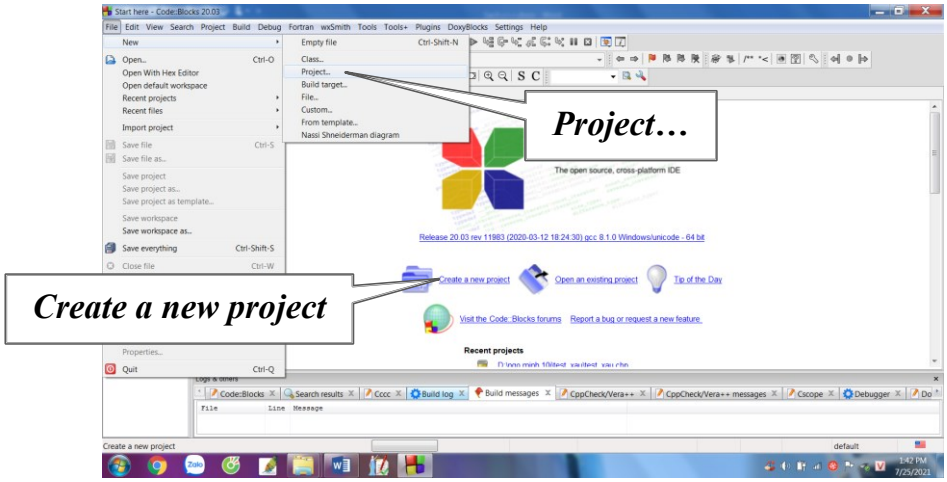
II. Sử dụng Codeblock

Cách 1: Vào ***File → New → Empty file*** (hoặc tổ hợp phím tắt ***Ctrl*** + ***Shift*** + ***N***) rồi soạn thảo chương trình trong khung soạn thảo; đối với cách này thì sau này muốn debug chương trình thì không được, mà ta chỉ có thể xem được giá

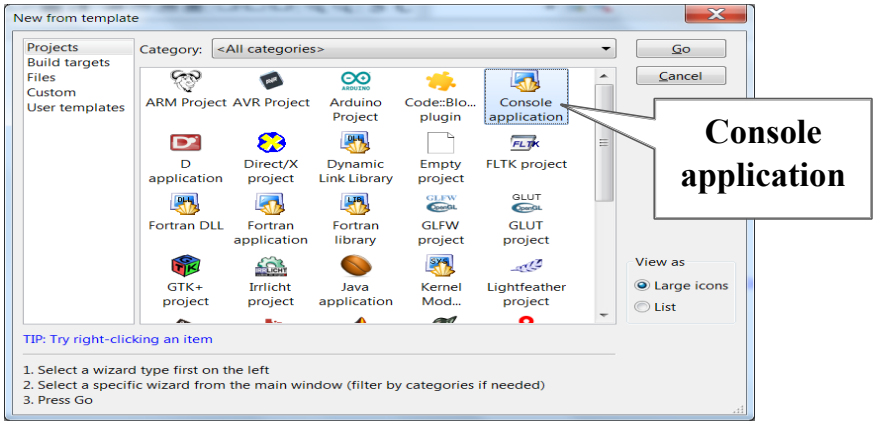
trị của các biến bằng cách in trực tiếp ra màn hình hoặc tệp; thông thường tôi không sử dụng cách 1 này;

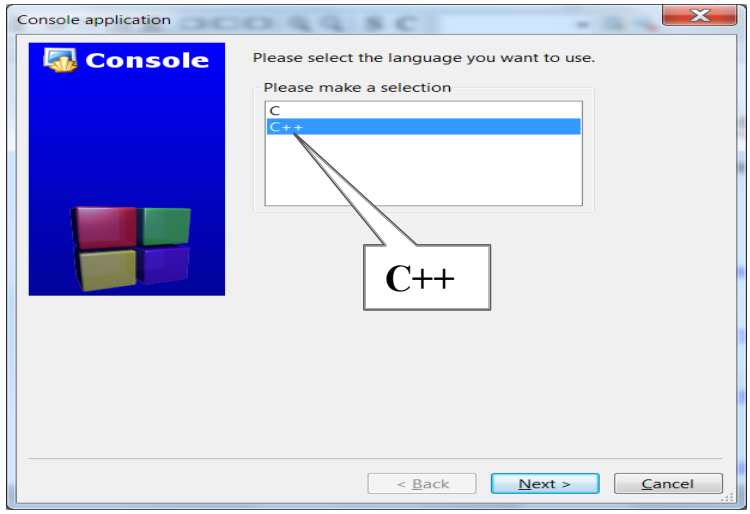


Cách 2: Vào ***File → New → Project…*** Hoặc chọn ***Create a new project*** như hình dưới



Xuất hiện hộp thoại ***New from template***, rồi ta chọn ***Console application*** như hình dưới; Mặc định là ***C++*** rồi click ***Next***

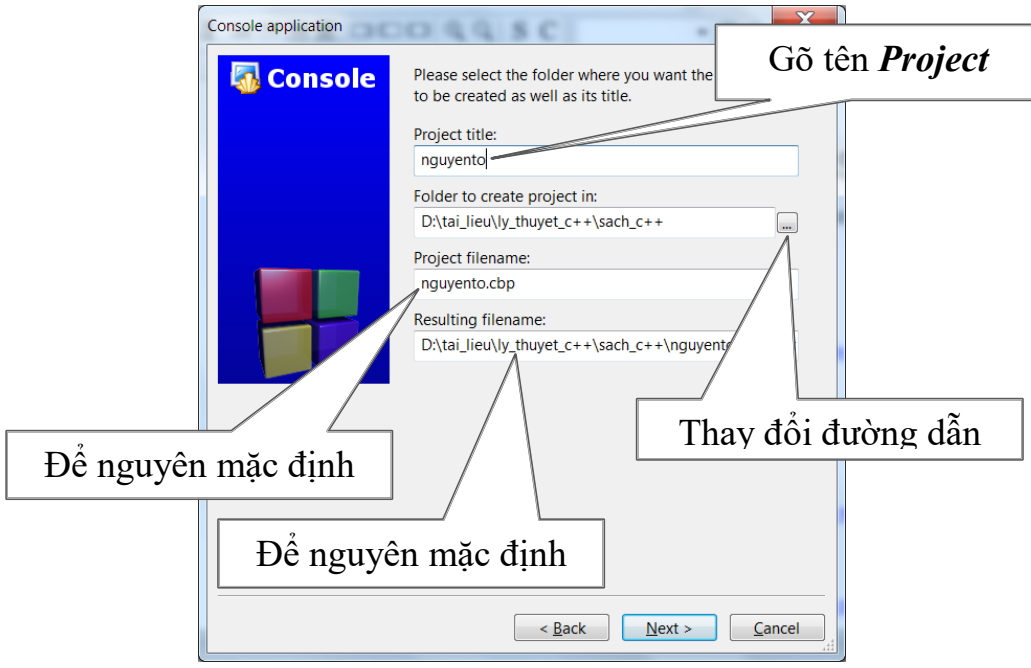




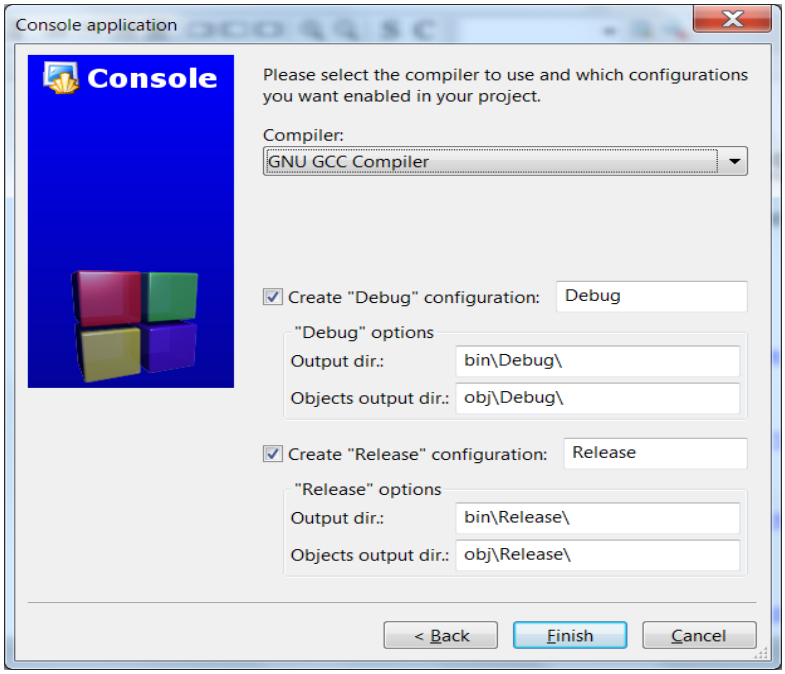
Xuất hiện hộp thoại ***Console application***:

+ Ở phần ***Project title***: chính là gõ tên của bài, ví dụ như: bài đó yêu cầu được đặt tên là ***nguyento.cpp*** thì thông thường ta nên đặt tên cho ***Project*** là ***nguyento*** hoặc có thể đặt tên khác tùy mình.

+ Ở phần ***Folder to create project in***: đặt đường dẫn cho ***Project*** nằm ở vị trí nào trong ổ đĩa. Chú ý: đường dẫn phải là tên không được viết dấu. Nếu đường dẫn để có tiếng việt thì khi dịch chương trình sẽ bị lỗi.

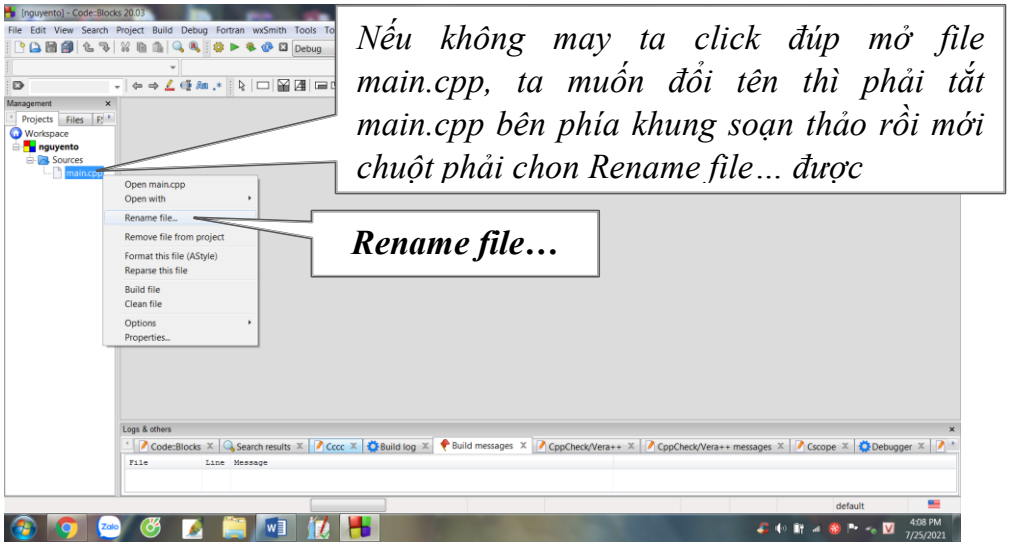


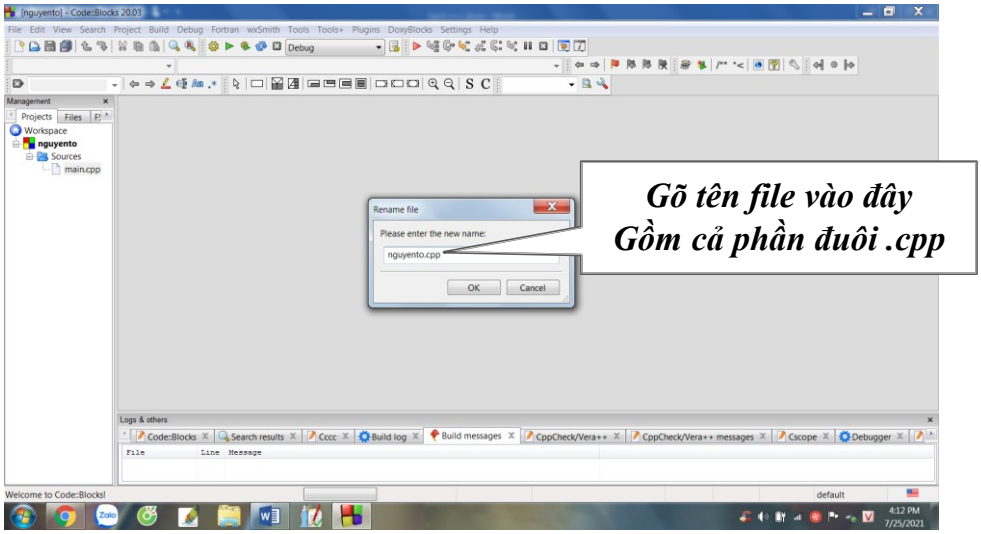
Còn các phần khác ta cứ để mặc định rồi ***Next → Finish***



Sau đó nhìn khung phía trái phần ***Projects*** click vào ***Sources*** → chuột phải vào ***main.cpp*** chọn ***Rename file…*** xuất hiện hộp thoại ***Rename file*** và mình đổi tên thành ***nguyento.cpp*** như hình dưới rồi chọn ***OK*** (trong trường hợp này do bài này yêu cầu đặt tên là ***nguyento.cpp*** nên tôi đặt tên như vậy, nếu bài có tên khác thì các thầy cô đặt tên khác tương tự)

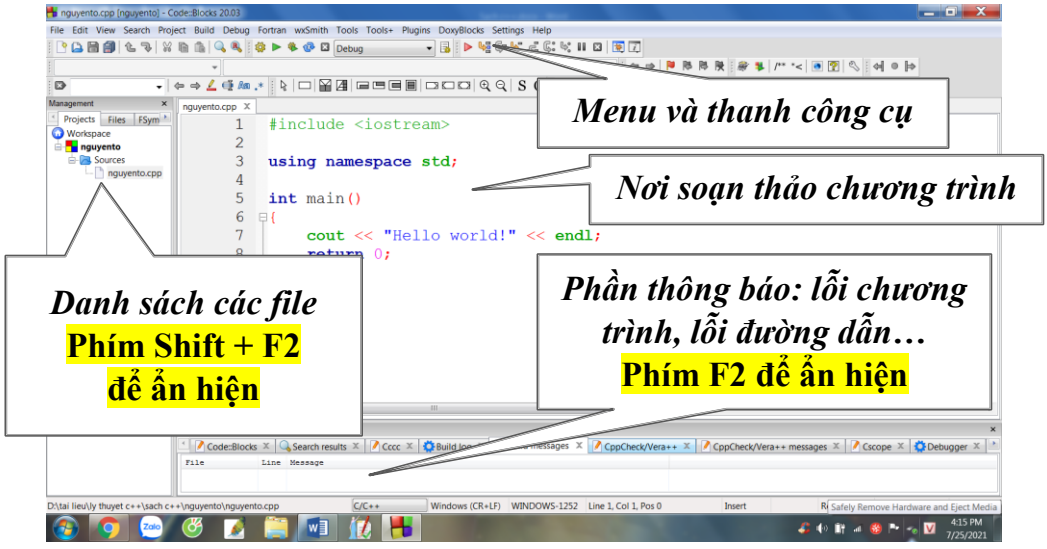
Chú ý: nếu khung phần **Projects** không hiển thị thì chúng ra ấn tổ hợp phím ***Shift + F2*** để hiển thị. Nếu lệnh ***Rename file…*** không thấy thì ta phải tắt ***file*** đó bên màn hình soạn thảo, rồi chuột phải thì mới xuất hiện ***Rename file…***



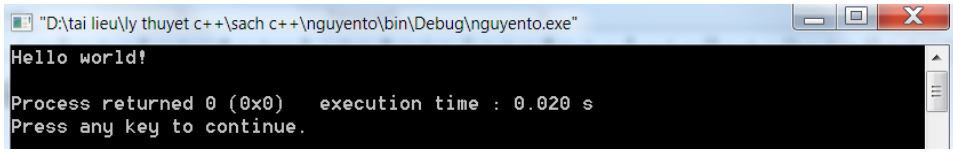


Sau đó click đúp vào ***nguyento.cpp*** ở khung bên trái như hình vẽ sẽ ra phần khung ở phía giữa để soạn thảo chương trình. Nó xuất hiện một chương trình mẫu để in ra dòng chữ: **Hello world!**

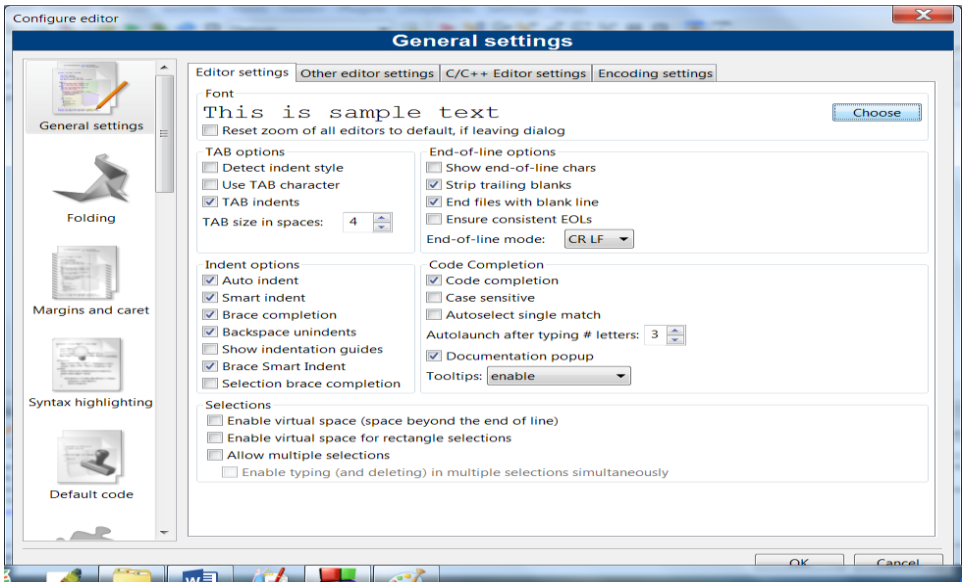
Cách chạy chương trình: Trước khi chạy chương trình thì ta phải dịch chương trình (ấn tổ hợp phím ***Ctrl + F9***) → để chạy chương trình ấn tổ hợp phím ***Ctrl + F10***; Hoặc vừa dịch và chạy là ấn phím ***F9***; Chúng ta luôn nhớ: khi sửa lại code của chương trình thì trước khi chạy thì phải dịch lại chương trình, nếu không dịch lại chương trình mà chạy luôn thì nó sẽ chạy chương trình của lần trước khi sửa. Với cách tạo chương trình bằng ***project*** thì ta có thể dùng chức năng ***debug*** để chạy từng dòng lệnh và xem kết quả của từng biến khi chạy để ta gỡ lỗi thuật toán.



Chương trình trên in ra dòng chữ **Hello world!** Và hiển thị thời gian chạy chương trình, như ví dụ này là thời gian chạy là 0.020 giây. Nếu sau này khi chạy chương trình có phần nhập dữ liệu thì thời gian chỉ chính xác khi nhập dữ liệu từ tệp, còn nhập dữ liệu từ bàn phím thì nó tính luôn cả thời gian ta thao tác để nhập dữ liệu vào.



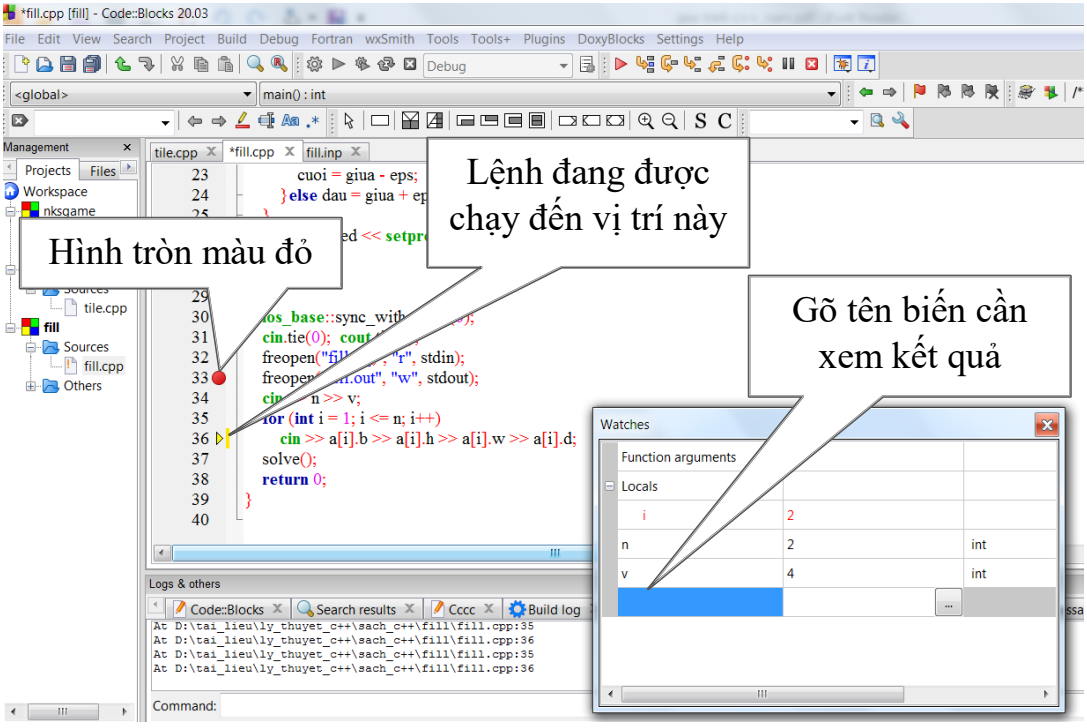
Để thay đổi cỡ chữ trong ***codeblock*** ta vào ***Settings → Editor…***: Ở ***General settings*** chọn Tab: ***Editor settings*** → trong phần khung ***Font*** chọn ***Choose*** để thay đổi cỡ chữ



Chú ý khi muốn mở lại chương trình thì ta mở file ***.cbp*** (như ví dụ này là ***nguyento.cbp***), rồi nhìn bên trái click vào ***Sources*** → click tiếp vào file ***.cpp*** (như ví dụ mình là ***nguyento.cpp***), cách mở này là mở theo kiểu ***project***; Khi nộp bài thi để chấm thì ta nộp mình file ***.cpp*** (ví dụ mình là ***nguyento.cpp***)

Cách xem giá trị của từng biến khi chạy từng câu lệnh của chương trình: muốn chương trình bắt đầu chạy từ đâu, thì ta ***click*** chuột trái vào bên trái của *1* dòng lệnh (xuất hiện hình tròn có màu đỏ ngay tại vị trí vừa ***click***, nếu ta muốn bỏ chọn thì ***click*** thêm 1 lần nữa là hình tròn đỏ mất), sau đó dùng phím ***F8*** thì máy sẽ thực hiện từ đầu chương trình đến dòng lệnh được đánh

dấu và dừng lại. Để kiểm tra kết quả của giá trị từng biến thì ta mở cửa sổ ***Watches*** (***Debug→Debugging windows→watches***). Trong cửa sổ ***Watches*** ta gõ tên các biến cần xem giá trị vào cuối cột đầu tiên (mỗi biến một ô), thì giá trị tương ứng của biến đó ở bên cột thứ 2. Để chạy tiếp những dòng lệnh tiếp theo ta dùng phím ***F7*** (hoặc ***Shift + F7***). Sự khác nhau giữa ***F7*** và ***Shift + F7*** là: nếu chạy từng dòng lệnh ở chương trình, mà trong chương trình chính có chương trình con thì nó sẽ không nhảy vào chương trình con; còn nếu dùng ***Shift + F7*** sẽ nhảy vào chương trình con để chạy từng dòng lệnh trong chương trình con. Khi chạy từng dòng lệnh ta sẽ thấy được sự thay đổi giá trị của từng biến (nếu có) ở cửa sổ ***Watches***. Nếu ta muốn kết thúc việc chạy từng dòng lệnh thì ấn lại phím ***F8***.



Nếu chương trình mà không biên dịch được, mà báo lỗi do đường dẫn thì ta vào ***Settings → Compiler…→*** vào Tab ***Toolchain executables*** sửa lại đường dẫn thành: ***C:\Program Files\CodeBlocks\MinGW***

Nếu chương trình mà không ***Debug*** được bằng ***F8*** (khi cài phiên bản 20.03 mới nhất thì tôi bị lỗi này) và ta sửa như sau: vào ***Settings → Debugger…*** bên phía trái ***click*** vào ***Default*** và nhìn bên phải sửa đường dẫn ở phần ***Executable path*** thành ***C:\Program Files\CodeBlocks\MinGW\bin\gdb.exe***

III. Lập trình C++

Trong ngôn ngữ C++ có phân biệt chữ in hoa và in thường, nó khác với pascal. Ngôn ngữ C++ là ngôn ngữ kế thừa của ngôn ngữ C.

1. Lệnh vào ra trong C++

a. In ra màn hình

Cú pháp: *cout << biểu thức 1 << biểu thức 2 << .... << biểu thức N;*

Hoặc không thích viết gộp thì viết tách như sau:

cout << biểu thức 1;
cout << biểu thức 2;
……
cout << biểu thức N;

Lệnh trên nó sẽ ghi ra màn hình các giá trị của *biểu thức 1, biểu thức 2 ... biểu thức N;* sau mỗi câu lệnh là dấu chấm phẩy (;)

Ví dụ: *cout << "Xin chao cac ban" << "Toi di hoc C++";* Câu lệnh này nó sẽ in ra màn hình câu: Xin chao cac banToi di hoc C++

Chú ý: trong C++ thì xâu được đặt trong cặp dấu nháy kép còn ký tự thì đặt trong cặp dấu nháy đơn.

cin: được đọc là si in
cout : được đọc là si out

b. Nhập dữ liệu từ bàn phím

Cú pháp: *cin >> biến 1 >> biến 2 >> ... >> biến N;*

Hoặc viết tách ra như sau:

cin >> biến 1;
cin >> biến 2;
……..
cin >> biến N;

Chú ý: >> là 2 dấu lớn hơn viết sát nhau; << là 2 dấu bé hơn viết sát sau. Để nhớ 2 toán tử này khi lập trình: >> ta nhớ nó đẩy giá trị vào biến (nhập vào - *cin*), << ta nhớ nó đẩy giá trị ra ngoài (xuất ra – *cout*)

c. Chương trình mẫu

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    cout << "Hello world!" << endl;
    return 0;
}
```

Giải thích ví dụ trên:

#include <bits/stdc++.h> // Khai báo dùng thư viện, tệp này nó chứa tất cả các thư viện chuẩn của C++. Khi *include* nó thì không cần phải dùng các thư viện khác nữa.

using namespace std; // Đây là dùng không gian tên std

int main() // Đây là hàm chính của chương trình, nếu chương trình có nhiều chương trình con thì nó sẽ chạy vào chương trình chính trước

cout << "Hello world!" << endl; // cout là lệnh in ra màn hình; endl là lệnh xuống dòng nó viết tắt của từ end line

return 0; // trả giá trị 0 cho hàm main và kết thúc hàm. Đây là một kết thúc bình thường trong chương trình khi không có một lỗi nào trong quá trình thực hiện. Thực tế ở chương trình chính có thể cuối chương trình không cần return 0 cũng được, nhưng chúng ta nên có.

Cặp ngoặc *{}* nó tương đương như *begin ...end* trong pascal, nó thể hiện thực hiện một khối lệnh

Nếu chương trình trên chúng ta không sử dụng không gian tên *std*: *using namespace std;* thì chúng ta phải viết chương trình như sau:

```
#include <bits/stdc++.h>
int main(){
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Để dễ nhớ và viết gọn hơn thì ta nên dùng: *using namespace std;*

Trong ví dụ trên ta có *endl* nghĩa là xuống dòng hoặc có 1 ký tự ký hiệu *'\n'* cũng là lệnh xuống dòng. *endl* và *'\n'* ý nghĩa đều giống nhau, ta dùng cái

nào cũng được. Dòng lệnh *#include <bits/stdc++.h>* ta luôn nhớ phía cuối không có dấu chấm phẩy và sau này tất cả dòng lệnh có dấu # đứng đầu thì phía sau nó không có dấu chấm phẩy (đừng có hỏi là vì sao, vì nó là quy ước)

Khi dạy và học đến phần in ra màn hình bằng *cout* thì ta có thể làm một số bài tập sau:

Câu 1: Hãy in ra 2 câu thơ sau:

Trau oi ta bao trau nay:
Trau ra ngoai ruong trau cay voi ta.

Rất dễ ta sẽ làm sai như sau:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
      cout << "Trau oi ta bao trau nay:
                  Trau ra ngoai ruong trau cay voi ta." << endl;
   return 0;
}
```

Chương trình trên nó báo lỗi ở phần *cout*, nó không hiểu được cách in theo cú pháp như vậy. Ta có chương trình đúng như sau:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
      cout << “Trau oi ta bao trau nay:” << endl; // in rồi xuống dòng
      cout << “Trau ra ngoai ruong trau cay voi ta.”;
   return 0;
}
```

Hoặc ta có thể viết theo cách gộp như sau:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
   cout << “Trau oi ta bao trau nay:” << endl << “Trau ra ngoai
   ruong trau cay voi ta.”;
   return 0;
}
```

Ta có dấu // để ghi lời chú thích trên 1 dòng, khi dịch chương trình nó sẽ bỏ qua dòng này. Hoặc dùng /* … */ để chú thích đoạn

Từ ví dụ trên ta có thể làm thêm các bài tập như:

+ In ra màn hình một bài thơ (tự lấy một đoạn thơ, mà các câu thơ nằm trên nhiều dòng).

+ In ra màn hình tam giác sao có các dạng sau:

```
*          *          * * * *        * * * *
* *        * *        * * *           * * *
* * *      * * *      * *              * *
* * * *    * * * *    *                 *
```

2. Các kiểu dữ liệu cơ bản

- Kiểu Số nguyên:

int: có kích thước là 4 byte

phạm vi của nó là -2147483648 .. 2147483647

long long: có kích thước là 8 byte,

phạm vi biểu diễn là -2^{63} …$2^{63}-1$

Nếu ta thêm từ *unsigned* vào phía trước 2 kiểu trên thì nó trở thành kiểu dữ liệu số nguyên không âm và đều có kích thước tương đương như vậy:

unsigned int: có kích thước là 4 byte

phạm vi của nó là 0 … $2^{32}-1$

unsigned long long: có kích thước là 8 byte,

phạm vi biểu diễn là 0 …$2^{64}-1$

Cách nhớ phạm vi của 4 kiểu dữ liệu trên ta nên nhớ về kích thước của nó, từ kích thước ta suy ra được phạm vi. Ta luôn có 1byte = 8 bit. Ví dụ như kiểu *int* có kích thước là 4 byte = 4x8 = 32bit; từ 32 bit (bit là 0 hoặc 1) ta suy ra được có 2^{32} số được biểu diễn. Chúng ta tưởng tượng, có 32 ô trống mà mỗi ô trống chỉ bỏ số 0 hoặc số 1 vào thì ta sẽ có 2^{32} cách bỏ. Mà *int* là kiểu số nguyên nên giá trị nó sẽ nằm từ -2^{31} đến $2^{31}-1$ (trong phạm vi này nó có 2^{32} số nguyên). Tương tự kiểu *long long* có kích thước là 8 byte = 64 bit thì sẽ biểu

diễn được 2^{64} số, suy ra được phạm vi của nó là từ -2^{63} đến $2^{63}-1$. Với cách suy luận tương tự thì ta cũng có phạm vi của kiểu dữ liệu *unsigned int* và *unsigned long long.*

- Kiểu ký tự

char: có kích thước là 1 byte nó biểu diễn được $2^8 = 256$ ký tự

- Kiểu logic

bool: có kích thước là 1 byte, biểu diễn giá trị true (1 hay số khác 0), giá trị false (0)

- Kiểu số thực

double: có kích thước là 8 byte nhưng nó có miền giá trị lớn hơn nhiều so với số nguyên, nó lấy chính xác đến khoảng 15, 16 chữ số thập phân. Đối với kiểu số thực thì giá trị của nó được biểu diễn dưới dạng dấu chấm động và dấu chấm tĩnh. Dấu chấm tĩnh ví dụ như: 3.12; 4.34565; 34.25… Còn dấu chấm động được biểu diễn dưới dạng $\pm xE \pm y$ giá trị của nó bằng $\pm x * 10^{\pm y}$, ví dụ như: 1E8 là 10^8; 2E-4 là $2*10^{-4}$….

Chú ý: nếu ta không nhớ được kích thước của mỗi kiểu dữ liệu thì trong C++ có hàm *sizeof(tên kiểu dữ liệu)*, nó sẽ cho ra được kích thước của kiểu dữ liệu đó với đơn vị tính là byte. Ví dụ như *cout << sizeof (char);* sẽ in ra số 1; *cout<< sizeof (long long);* in ra số 8…

Trên đây, tôi chỉ đưa ra các kiểu dữ liệu hay sử dụng trong lập trình, ngoài ra ta có thể tham khảo thêm ở trong mỗi kiểu dữ liệu còn có các kiểu khác nữa.

3. Biến và hằng

- Biến:

Cú pháp:

Kiểu_dữ_liệu Danh_sách_biến;

Nếu danh sách biến có nhiều biến thì các biến được cách nhau bởi dấu phẩy

Ví dụ:

```
int a, b, c;
long long d, e;
char ch;
bool kt;
double f;
```

Tên biến phải được đặt đúng theo quy tắc đặt tên: gồm các chữ cái hoặc chữ số hoặc dấu gạch dưới; bắt đầu bằng chữ cái hoặc dấu gạch dưới; tên không nên đặt quá dài;

Chúng ta có thể vừa khai báo biến vừa khởi tạo cho biến đó;

Ví dụ:
```
int a = 0;
bool kt = true;
char ch1 = 'a', ch2;
```

Trong **C++** lệnh gán giá trị là dấu = (ngôn ngữ **Pascal** là :=)

- Phạm vi của biến

Cũng giống như phạm vi biến trong **pascal**: biến toàn cục và biến cục bộ; trong C++ biến được khai báo linh hoạt ở nhiều vị trí trong chương trình.

Ta xét ví dụ sau:

```
#include <bits/stdc++.h>
using namespace std;
int a = 0, b = 1; // a, b này được gọi là biến toàn cục
int main(){
    cout << a+b << endl; /*Kết quả in ra màn hình số 1; vì a, b này là
    biến toàn cục */
    // a, b, c dưới đây là biến cục bộ
    int a = 5;
```

```
    int b = 2;
    int c = 1;
    cout << a+b+c; // Kết quả in ra màn hình số 8 (vì 5+2+1=8)
    return 0;
}
```

Chúng ta chú ý một điều các biến không cùng phạm vi thì có thể được đặt trùng tên và giá trị của nó chỉ có tác dụng trong phạm vi đó. Sau này khi đến phần chương trình con thì chúng ta sẽ dễ hiểu hơn.

Đến phần này ta có thể cho học sinh làm các bài tập đơn giản để học sinh tiếp cận với C++ một cách hiệu quả. Và cái quan trọng là phải tạo hứng thú học của học sinh ngay từ đầu.

***Ví dụ*:** Nhập vào 2 số nguyên *a* và *b*. Yêu cầu tính tổng của 2 số đó và in ra màn hình

Chương trình:

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int main(){
   cout << "moi ban nhap 2 so a và b";
   cin >> a >> b;
   cout << "Tong cua 2 so la: " << a+b;
   return 0;
}
```

Hoặc:

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int main(){
   cin >> a >> b;
   cout << a+b;
   return 0;
}
```

Chương trình sau khi chạy thì nó sẽ hiện màn hình đen và ta phải nhập 2 số trên bàn phím (2 số được nhập bằng dấu cách hoặc dấu ***Enter***). Còn chương

trình đầu thì nó in ra màn hình có các lời dẫn cho người chạy chương trình làm theo. Cả 2 chương trình này đều đúng.

Cũng chương trình đó và nếu ta nhập *a = 2000000000 b = 2000000000* thì kết quả in ra tổng là bao nhiêu? Đa số sẽ trả lời là *4000000000* nhưng đây không phải là kết quả của bài toán, kết quả chạy in ra số âm. Đây chính là hiện tượng bị tràn số vì kiểu *int* có giới hạn lớn nhất là 2^{31}*-1* (khoảng $2.1*10^9$). Kiểu như có 1 cái cốc đựng được *300 ml* nước: lần *1* ta rót vào *200 ml*, lần *2* ta rót vào *200ml* hỏi trong cốc có bao nhiêu *ml*? Để giải quyết trường hợp này, ta có các cách giải quyết sau:

+ Khai báo kiểu dữ liệu của lớn hơn: chẳng hạn *long long a, b;*

+ Ép kiểu dữ liệu: cú pháp: *(tên kiểu) (biến hay giá trị hay biểu thức cần ép)*. Ví dụ trong trường hợp trên *a, b* ta khai báo là *int* thì *cout << (long long)(a) + b;* lúc này kết quả khi ta nhập *a =2000000000 b=2000000000* thì sẽ cho kết quả là *4000000000* hoặc ta viết *cout << a + (long long)b;* ở đây chỗ *b* ta không cần dấu *(b)* cũng được thì kết quả vẫn cho ra đúng. Nhưng viết *cout << (long long)(a+b);* thì kết quả sẽ ra không đúng. Trường hợp này ta giải thích như sau: nó lấy *a+b* trước rồi được kết quả mới đi ép kiểu, như vậy vẫn xảy ra hiện tượng tràn số khi thực hiện phép toán. Ở đây ta chú ý khi hai toán hạng (số hạng) tác động vào nhau bởi toán tử (các phép toán) thì kết quả sẽ trả về kiểu dữ liệu cao nhất của 2 toán hạng. Ví dụ trên là *a* và *b* đều là kiểu *int*, nếu ta viết *a + (long long)b* nghĩa là giá trị có kiểu *int* cộng với ép giá trị của *b* về kiểu *long long* nên kết quả của phép cộng là kiểu giá trị cao hơn (vì kiểu *long long* > *int*)

Để hiểu thêm ta có đoạn chương trình sau:

```
int  a, b, c;
long long d;
d = a+ b + c; // xảy ra hiện tượng tràn số
d = (long long)a + b + c; //không bị tràn số (kết quả ra đúng)
d = a + b + (long long)c; /*bị tràn số phần a+b vì nó tính giá trị từ trái qua phải */
```

- Hằng: là đại lượng có giá trị không bị thay đổi trong quá trình thực hiện chương trình.

+ Hằng số nguyên: số nguyên thông thường ở hệ thập phân, theo mặc định hằng số nguyên nếu có giá trị trong phạm vi *int* thì giá trị của nó thuộc kiểu int, nếu giá trị của nó ngoài phạm vi int và trong phạm vi *long long* thì nó thuộc kiểu *long long*. Muốn giá trị trong phạm vi *int* có kiểu *long long* thì ta ép kiểu hoặc thêm 2 ký tự *LL* hoặc *ll* vào phía sau số đó.

Ví dụ:

cout << 2000000000 + 2000000000; Xảy ra hiện tượng tràn số vì nó hiểu 2000000000 là kiểu *int* mà kiểu *int* tối đa là 2^{31}-1 ≈ 2,1*10^9

cout << 2000000000 + 3000000000; In ra đúng kết quả vì *2000000000* là kiểu *int*, *3000000000* là kiểu *long long* nên kết quả trả về là kiểu *long long*

cout << 2000000000LL + 2000000000; In ra đúng kết quả số *2000000000LL* thuộc kiểu *long long*

cout << (long long)2000000000 + 2000000000; In ra đúng kết quả

+ Hằng số thực: thông thường chúng ta có 2 cách biểu diễn số thực:

Dấu chấm tĩnh: *2.34; 4.567; 1234.4567 …….*

Dấu chấm động: *1E-9; -23E6; 12E+7* ... Ý nghĩa *1E-9 =1*10^{-9}; -23E6=-23*10^6; 12E+7=12*10^7;* tổng quát lên $\pm XE\pm Y = \pm X * 10^{\pm Y}$ (E có thể là in thường hoặc in hoa)

Từ đây để đỡ bị nhầm số ta có thể khai báo và gán như ví dụ sau:

int a = 2E9; hoặc *int a = 2e9;* thay vì khai báo *int a = 2000000000*; (dễ bị nhầm số lượng số *0*) *2E9* là số thực nhưng được gán lại cho *a*, mà *a* là số nguyên nên kết quả *a* là số nguyên (không phải số thực).

+ Hằng ký tự: trong C++ quy ước hằng ký tự được đặt trong cặp dấu nháy đơn *‘’*; Ta có hằng ký tự đặc biệt *‘\n’* nó tương đương với *endl* (xuống dòng)

Ta có ***ví dụ*** sau:

```
#include <bits/stdc++.h>
using namespace std;
char kta = 'a';
char ktc = 99;
int main(){
   cout << kta << endl; // Ghi ra ký tự a
```

```
    cout << ktc << '\n'; // Ghi ra ký tự c
    cout << kta + ktc <<endl; // Ghi ra số 196
    cout << (int)ktc<< endl; // Ghi ra số 99
    cout << kta +3 << endl; // Ghi ra số 100
    cout << (char)(kta+3)<< endl; // Ghi ra ký tự d
    return 0;
}
```

Qua ví dụ trên, ta muốn chuyển ký tự sang số trong bảng mã ASCII hoặc ngược lại thì ta có thể dùng ép kiểu. Chúng ta nên nhớ một số ký tự trong bảng mã *ASCII*: Ký tự '*A*' là *65*, ký tự '*0*' là *48*, ký tự in thường lớn hơn ký tự in hoa là *32* (nên ký tự *'a'* là *97*); Nếu ta quên thì có thể dùng lệnh in ra để xem kết quả. Ví dụ như không nhớ được ký tự *'h'* thì ta in như sau:

cout << (int)'h'; //nó sẽ in ra được giá trị của *'h'* trong bảng mã *ASCII*.

+ Hằng xâu ký tự: là một dãy ký tự bất kỳ được đặt trong dấu nháy kép. Ví dụ như *"Toi dang hoc C++"*, *"C++ that la thu zi"*, *"a"*,… Chú ý một ký tự được đặt trong cặp dấu nháy kép thì cũng được hiểu đó là hằng xâu, *"a"* nó khác với *'a'*. Chúng ta có xâu rỗng (hai dấu nháy kép viết sát nhau*""*), chứ không có ký tự rỗng.

Bài tập

Câu 1. Cho phương trình *a*x + b = 0*. Hãy nhập các số nguyên *a, b* và in ra phương trình tương ứng với mỗi giá trị đó. Ví dụ *a = 1, b = -2* thì kết quả in ra là *1*x + -2 = 0*

Câu 2. Cho phương trình *a*x^2 + b*x + c = 0*. Hãy nhập các số nguyên *a, b, c* và in ra phương trình tương ứng với mỗi giá trị đó. Ví dụ nhập *a=2, b=3, c=-2*; thì in ra màn hình phương trình là *2*x^2 + 3*x + -2 = 0*

Gợi ý cách giải

Câu 1.

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int main(){
   cin >> a >> b;
   cout << a << "*x + " << b << " = 0";
   return 0;
}
```

Câu 2.

```
#include <bits/stdc++.h>
using namespace std;
int a, b, c;
int main(){
   cin >> a >> b >> c;
   cout << a << "*x^2 + " << b << "*x + " << c << " = 0";
   return 0;
}
```

4. Phép toán, biểu thức, câu lệnh gán

- Phép toán: tương tự như các ngôn ngữ lập trình pascal thì C++ cũng có các phép toán:

+ Phép toán số học: + (cộng); - (trừ); * (nhân); / (chia); % (lấy phần dư). Ta chú ý phép chia (/) nó là lấy phần nguyên khi 2 toán hạng là số nguyên, kết quả nó là phép chia khi ít nhất có 1 toán hạng là kiểu số thực. Ví dụ: 5 / 2 = 2; 5 / 2.0 = 2.5; 5.0 / 2 = 2.5; 5.0 / 2.0 = 2.5. Đối với phép % (lấy dư) nó chỉ áp dụng trên kiểu dữ liệu là số nguyên (không áp dụng cho số thực). Ví dụ 7 % 2 = 1; Nhưng 7.0 % 2 thì chương trình sẽ báo lỗi.

+ Phép toán quan hệ (so sánh): < (nhỏ hơn); <= (nhỏ hơn hoặc bằng); > (lớn hơn); >= (lớn hơn hoặc bằng); == (bằng); != (khác). Kết quả của phép toán quan hệ cho ra giá trị lôgic

+ Phép toán lôgic: ! (phủ định); && (và); || (hoặc)

+ Ngoài ra C++ còn đưa ra 2 phép toán một ngôi là ++ và -- để tăng thêm 1 hoặc giảm đi 1 các biến có kiểu dữ liệu số nguyên hoặc số thực. Nếu phép toán ++ hoặc -- đứng trước toán hạng sẽ có ý nghĩa là tăng hoặc giảm đi 1 trước khi nó được sử dụng; còn ++ hoặc – đứng sau toán hạng thì sẽ có nghĩa là tăng hoặc giảm 1 sau khi giá trị của nó được sử dụng (sử dụng trước rồi sau mới tăng hoặc giảm);

Ví dụ:

n = 10; x = ++n + 1; // giá trị của x = 12 và n = 11;
n = 10; x = --n + 1; // giá trị của x = 10 và n = 9;
n = 10; x = n++ + 1; // giá trị của x = 11 và n = 11;
n = 10; x = n-- + 1; // giá trị của x = 11 và n = 9;

+ Biểu thức số học: tương tự như pascal. Có một số hàm hay sử dụng:

sqrt(x): cho giá trị căn bậc 2 của *x* (kết quả trả về là kiểu số thực)

abs(x): cho giá trị tuyệt đối của *x* (kết quả trả về là cùng kiểu với *x*)

pow (x, y): cho giá trị của x^y (kết quả trả về là kiểu số thực)

exp(x): lũy thừa của số e (tính e mũ *x*) (kết quả trả về là số thực)

sin(x): tính sin của *x* (*x* có đơn vị góc là radian, kết quả trả về là kiểu thực)

cos(x): tính cos của *x* (*x* có đơn vị góc là radian, kết quả trả về là kiểu thực)

Ngoài ra còn nhiều hàm khác….

+ Biểu thức quan hệ: trong biểu thức có dùng đến các phép toán quan hệ kết quả trả về *true* (1 hay một số khác 0) hoặc *false* (hay 0)

<biểu thức 1> <phép toán quan hệ> <biểu thức 2>

Thứ tự ưu tiên trong biểu thức: tính giá trị các biểu thức 1 và 2; rồi mới thực hiện phép toán quan hệ. ví dụ: *2 + 3 < 4 + 1* sẽ cho ra kết quả là *false*. Nếu không nhớ thứ tự ưu tiên thì ta cứ thêm () vào như sau: *(2 + 3) < (4 + 1)*

+ Biểu thức lôgic: là biểu thức dùng các phép toán lôgic. Ví dụ biểu diễn biểu thức toán học: *1 < x < 5* thì ra sẽ có kết quả như sau: *(x > 1) && (x < 5)*. Kết quả biểu thức sẽ trả về *true* hay *false*

+ Lệnh gán: Lệnh gán trong C++ có dạng:

<tên biến> = <biểu thức>;

Nó khác với pascal dùng *:=*; Ví dụ: *x = x + 1; x = 10;*

Trong C++ nó còn cho phép viết rút gọn như sau:

x = x +1; có thể viết như sau: *x++; hoặc ++x; hoặc x+=1;*

x = x + y; có thể viết như sau: *x += y;*

x = x / (y+3); có thể viết như sau: *x /= y+3;*

Tổng quát lên ta có bảng sau:

Viết bình thường	**Viết rút gọn**
x = x + y;	*x += y;*
x = x - y;	*x -= y;*
*x = x * y;*	*x *= y;*
x = x / y;	*x /= y;*
x = x % y;	*x %= y;*

Trong C++ cho phép gán cả biểu thức như ví dụ sau:

x = 2; y = 2; z = 2; thì ta có thể viết *x = y = z = 2;* (ý nghĩa là *2* sẽ được gán cho *z*, rồi *z* lại được gán cho *y*, rồi *y* là được gán cho *x*; ví dụ khác: *x = (y = 1+1) * (z = 2);* thì ta hiểu kết quả của biểu thức này sẽ là *x = y * z* (mà *y=2* và *z = 2* nên kết quả sẽ là *4*)

+ Biểu thức điều kiện: Có cú pháp: *e1 ? e2 : e3* trong đó *e1, e2, e3* là các biểu thức. Giá trị của biểu thức điều kiện này bằng *e2* nếu giá trị của *e1* là *true* (hoặc khác *0*), bằng *e3* nếu như giá trị *e1* là *false* (hoặc bằng *0*).

Ví dụ:

```
x = 5 > 3 ? 4 : 2; // suy ra x = 4
x = (4 – 5  > 5) ? 3 : 2; // suy ra x = 2
x = 2 ? 4 : 2; // suy ra x = 4 vì e1 = 2 (khác 0) nên cho ra giá trị là e2
x = 0 ? 4 : 2; // suy ra x = 2;
x = -2 ? 3 : 4; // suy ra x = 3;
x = a < b ? a : b; // giá trị nhỏ nhất của 2 số a và b được gán cho x;
```

cout << (a > b ? a : b); // ghi ra giá trị lớn nhất của 2 số *a* và *b*; chú ý khi ghi trực tiếp ở *cout* thì phải có cặp ngoặc *()* của cả biểu thức điều kiện,

nếu không thì kết quả sẽ sai vì thứ tự ưu tiên của toán tử << sẽ thực hiện trước (nó in ra giá trị của *a*)

+ Thứ tự ưu tiên các phép toán như bảng sau:

Toán tử	Thứ tự ưu tiên
() [] -> . ++ - -	Trái sang phải
+ - ! ~ ++ - - (type)* & sizeof	Phải sang trái
* / %	Trái sang phải
+ -	Trái sang phải
<< >>	Trái sang phải
< <= > >=	Trái sang phải
== !=	Trái sang phải
&	Trái sang phải
^	Trái sang phải
|	Trái sang phải
&&	Trái sang phải
||	Trái sang phải
?:	Phải sang trái
= += -= *= /= %=>>= <<= &= ^= |=	Phải sang trái
,	Trái sang phải

Trong khi lập trình nếu không nhớ được thứ tự ưu tiên thì ta nên dùng cặp ngoặc nhọn () để ưu tiên thực hiện, vì nó có độ ưu tiên cao nhất

Bài tập

Câu 1. Hãy chuyển biểu thức toán học dưới đây thành biểu thức trong C++

$$x + \frac{nx}{y} + \frac{n(n-1)+ x^2}{z+y}$$

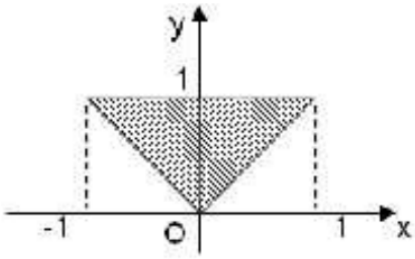
Ngoài ra ta có thể tự lấy nhiều biểu thức khác để chuyển đổi.

Câu 2. Chuyển các biểu thức trong C++ sang biểu thức toán học tương ứng

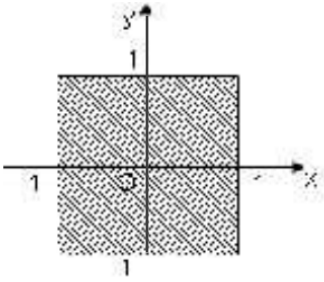
a. a/b*c b. a*b/4 c. 1/a*b/2 d. a/b*c/d*3

e. a/sqrt(x*x+b) f. a/b/(b+4)*d g. a+b/c*d+4/a

Câu 3. Hãy viết biểu thức lôgic cho kết quả *true* (khác 0) khi tọa độ (x, y) là điểm nằm trong vùng gạch chéo kể cả biên của 2 hình sau



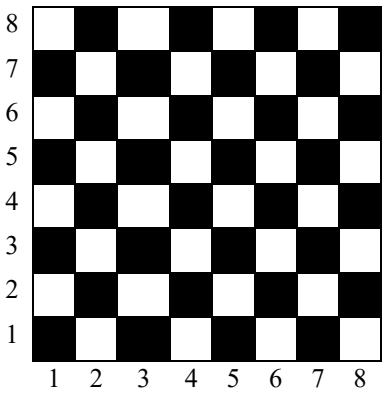
Hình a



Hình b

Câu 4. Hãy viết biểu thức lôgic cho giá trị *true* (khác 0) khi 3 số thực a, b, c lập thành 3 cạnh của 1 tam giác.

Câu 5. Cho bàn cờ vua như hình 2, ô ở hàng i cột j được ký hiệu là ô (i, j). Giả sử có 2 ô trên bàn cờ tọa độ (i1, j1) và (i2, j2). Hãy viết biểu thức lôgic cho kết quả *true* (khác 0) khi:



a. Hai ô (i1, j1) và (i2, j2) cùng màu

b. Quân xe ở ô (i1, j1) khống chế được ô (i2, j2)

c. Quân hậu ở ô (i1, j1) khống chế được ô (i2, j2)

d. Quân mã ở ô (i1, j1) khống chế được ô (i2, j2)

Câu 6. Viết chương trình nhập vào 2 số nguyên a và b; yêu cầu đổi chỗ giá trị 2 số nguyên đó và in ra giá trị sau khi đổi chỗ

Câu 7. Viết chương trình tìm giá trị lớn nhất của 3 số nguyên a, b, c được nhập từ bàn phím.

Câu 8. Nhập vào số N. Yêu cầu tính giá trị của tổng $S = 1+2+3+....+N$

Câu 9. Nhập vào 1 số nguyên a; Nếu a là số chẵn thì in ra màn hình số 0; a là số lẻ thì in ra màn hình số 1; Ví dụ a=4 thì in ra màn hình số 0; a=5 in ra màn hình số 1;

Câu 10. Nhập vào 1 số nguyên a; Nếu a là số chẵn thì in ra màn hình số 1; a là số lẻ thì in ra màn hình số 0; Ví dụ a=4 thì in ra màn hình số 1; a=5 in ra màn hình số 0;

Câu 11. Nhập 3 số nguyên dương a, b, c luôn là 3 cạnh của tam giác (lúc nhập ta luôn nhập nó thỏa mãn 3 cạnh tam giác). Yêu cầu: Tính chu vi và diện tích của tam giác đó.

Câu 12. Hãy viết chương trình nhập số thực r là bán kính của đường tròn. Hãy tính chu vi và diện tích của hình tròn.

Câu 13. Tên bài: **tamgiac01.cpp**

Nhập vào 3 số nguyên a, b, c là 3 cạnh của một tam giác (chú ý lúc nhập phải thỏa mãn là 3 cạnh của tam giác). Yêu cầu in ra chiều dài của đường cao lớn nhất trong 3 đường cao của tam giác

Từ câu 13 trở đi: yêu cầu khi in kết quả nếu là số thập phân thì in ra đúng 3 phần thập phân. Ví dụ kết quả là 2.34 thì in ra 2.340

Câu 14: Tên bài: **tamgiac02.cpp**

Nhập vào 3 số nguyên a, b, c là 3 cạnh của một tam giác (chú ý lúc nhập phải thỏa mãn là 3 cạnh của tam giác). Yêu cầu: tính độ dài 3 đường trung tuyến của tam giác và đưa kết quả ra màn hình

Câu 15. Tên bài: **tamgiac03.cpp**

Nhập vào 3 số nguyên a, b, c là 3 cạnh của một tam giác (chú ý lúc nhập phải thỏa mãn là 3 cạnh của tam giác). Yêu cầu: trong 3 đường trung tuyến của tam giác, đưa ra đường trung tuyến có độ dài lớn nhất là bao nhiêu?

Câu 16. Tên bài: **tamgiac04.cpp**

Nhập vào 3 số nguyên a, b, c là 3 cạnh của một tam giác (chú ý lúc nhập phải thỏa mãn là 3 cạnh của tam giác). Yêu cầu tính diện tích của hình tròn nội tiếp tam giác trên

Câu 17. Tên bài: **tamgiac05.cpp**

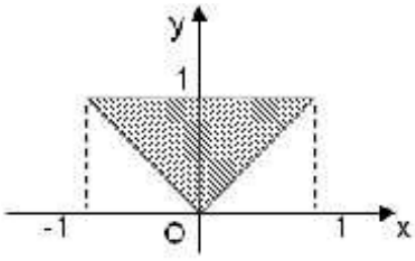
Nhập vào 3 số nguyên a, b, c là 3 cạnh của một tam giác (chú ý lúc nhập phải thỏa mãn là 3 cạnh của tam giác). Yêu cầu tính diện tích của hình tròn ngoại tiếp tam giác trên

Gợi ý cách giải

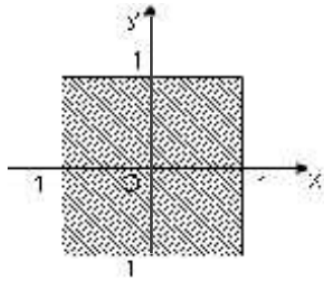
Câu 1. Đáp án: x + n*x/y + (n*(n-1) + x*x) / (z+y)

Câu 2. Tự làm.

Câu 3.



Hình a



Hình b

Ta có nhận xét 1 đường thẳng trong hình học phẳng (hệ trục tọa độ oxy) sẽ chia mặt phẳng đó làm 2 miền. Khi thế mọi tọa độ (x, y) trong một miền vào phương trình đường thẳng thì luôn luôn được kết quả cùng dấu. Ở hình a: ta có phần gạch được tạo bởi 3 đường thẳng: *y = 1; y = x; y = -x;* Ta có phần gạch thỏa mãn điều kiện *y <=1 và y >= x và y>=-x*.

Vậy ta có biểu thức lôgic hình *a* là:

(y <=1) && (y >= x) && (y>=-x).

Tương tự hình *b* ta có kết quả như sau:

(x<=1) && (x>=-1)&&(y<=1)&&(y>=-1)

Câu 4. *(a>0) && (b>0) && (c>0) && (a+b>c)&&(a+b>c)&&(b+c>a)*

Câu 5.

a. Ta nhận thấy tất cả các ô màu đen trên bàn cờ thì tổng của chỉ số hàng và chỉ số cột là số chẵn, còn tất cả các ô màu trắng thì tổng chỉ số hàng và chỉ số cột là số lẻ. Như 2 ô mà cùng màu thì tổng của chỉ số hàng và chỉ số cột phải cùng tính chẵn lẻ. Ta có biểu thức*: (i1+j1) %2 == (i2+j2)%2*

b. Quân xe khống chế được tất cả các ô có cùng hàng hay cùng cột nên ta có biểu thức: *(i1 == i2) || (j1 == j2)*

c. Ta nhận thấy quân hậu khống chế thêm cả phần đường chéo: mà điều kiện 2 ô cùng nằm trên đường chéo khi $|i1 - i2| = |j1 - j2|$ nên ta có biểu thức sau:

(i1 == i2) || (j1 == j2) || ((abs(i1 – i2) == abs(j1 – j2))

d. Ta thấy quân mã sẽ khống chế được ô khác theo hình chữ L:

(i1+1==i2 && j1+2==j2)||(i1+2==i2 && j1+1==j2)||(i1+2==i2 && j1-1==j2) ||(i1+1==i2 && j1-2==j2)||(i1-1==i2 && j1-2==j2)||(i1-2==i2 && j1-1==j2) ||(i1-2==i2 && j1+1==j2)||(i1-1==i2 && j1+2==j2)

Câu 6.

```
#include <bits/stdc++.h>
using namespace std;
int a, b, tg;
int main(){
    cin >> a >> b;
    tg = a;
    a = b;
    b = tg;
    cout << a << ' '<< b;
    return 0;
}
```

Sẽ có bạn làm *a = b; b = a;* như vậy b không được đổi chỗ; Hỏi cách sau có đúng không: a = a + b; b = a – b; a = a – b; // cách làm này chỉ đúng khi $a + b$ không bị tràn số (sẽ sai khi $a + b$ tràn số). Vì vậy khi đổi chỗ 2 số thì ta nên dùng cách dùng biến trung gian *tg*.

Câu 7. Vì đến đây chưa học cấu trúc *if(...)* nên ta dùng biểu thức điều kiện như sau:

Cách 1:

```
res = a > b ? a: b;
res = res > c ? res: c;
cout << res;
```

Cách 2: `cout <<((a > b ? a : b) > c ? (a > b ? a : b):c);`

Cách 2 đúng nhưng ta không nên làm vì giá trị lớn nhất của *a* và *b* được tính đến 2 lần.

Câu 8. Ta có công thức *1+2+3+...+N = N*(N+1)/2*. ở đây ta có *N*(N+1)* luôn là số chẵn nên phép */ 2* đúng trong trường hợp này (phép / là phép lấy phần nguyên đối với 2 toán hạng là số nguyên)

Câu 9. cout << a %2 ;

Câu 10. cout << (a%2==0 ? 1 : 0);

Câu 11. Chu vi là tổng 3 cạnh. Diện tích ta dùng công thức *S = sqrt(p*(p-a)*(p-b)*(p-c))* trong đó *p* là nửa chu vi *(a+b+c)/2.0* (vì *a, b, c* là số nguyên nên ta phải chia cho *2.0* chứ không phải *2* (vì / 2 thì kết quả là lấy phần nguyên))

Câu 12. Chu vi hình tròn là 2*PI*r; diện tích hình tròng là PI*r*r; trong đó PI = 3.1415926… Tùy vào độ chính xác đề bài yêu cầu mà ta lấy đến mấy chữ số phần thập phân của của số PI. Ví dụ trong trường hợp này mình lấy độ chính xác đến 2 chữ số: CV = 2*3.141*r; S = 3.141*r*r;

Khi đến phần in số thập phân thì ta học cách in định dạng số thập phân: để in chính xác có đúng *n* chữ số sau phần thập phân ta sử dụng đồng thời các hàm sau: *fixed*

setprecision (n)

Trong đó *fixed* chỉ cần viết 1 lần, nó sẽ áp dụng cho tất cả các lệnh in làm tròn số thực phía sau. Chỉ cần viết 1 lần *setprecision (n)* thì nó sẽ áp dụng cho tất cả lệnh in số thực phía sau (in ra *n* phần thập phân), khi muốn in ra khác số phần thập phân thì phải viết lại *setprecision (k)* (trong đó *k* là số phần thập phân cần in ra). Ta xét ví dụ đơn giản và dễ hiểu sau:

```
#include <bits/stdc++.h>
using namespace std;
double f = 3.14159;
double e = 4.1111145;
int main (){
  cout << fixed << setprecision(4) << f << '\n'; // ghi ra 3.1416
  cout << setprecision(5) << f << '\n'; // ghi ra 3.14159
  cout << e << '\n'; // ghi ra 4.11111
  cout << setprecision(9) << f << '\n';// ghi ra 3.141590000
  return 0;
}
```

Câu 13. Tên bài: **tamgiac01.cpp**

Tính diện tích tam giác dùng công thức *heron*. Áp dụng công thức S = ½*a*h để tính ra 3 đường cao, rồi ta tìm được đường cao lớn nhất

```
#include <bits/stdc++.h>
using namespace std;
int a, b, c;
double s, p;
double ha, hb, hc;
double res;
int main (){
    cin >> a >> b >> c;
    p = (a+b+c)/2.0;
    s = sqrt(p*(p-a)*(p-b)*(p-c));
    ha = 2*s/a;
    hb = 2*s/b;
    hc = 2*s/c;
    res = ha > hb ? ha : hb;
    res = res > hc ? res : hc;
    // Đưa kết quả lấy chính xác 3 phần thập phân
    cout << fixed << setprecision(3) << res;
    return 0;
}
```

Câu 14: Tên bài: **tamgiac02.cpp**

Ta áp dụng công thức tính đường trung tuyến trong tam giác:

$$m_a = \sqrt{\frac{b^2 + c^2}{2} - \frac{a^2}{4}}$$

$$m_b = \sqrt{\frac{a^2 + c^2}{2} - \frac{b^2}{4}}$$

$$m_c = \sqrt{\frac{b^2 + a^2}{2} - \frac{c^2}{4}}$$

Câu 15. Tên bài: **tamgiac03.cpp**

Tương tự câu 2 rồi ta tìm giá trị lớn nhất của 3 số.

Câu 16. Tên bài: **tamgiac04.cpp**

Ta có công thức tính bán kính đường tròn nội tiếp tam giác r = s / p; trong đó s là diện tích tam giác, p là nửa chu vi. Tính được r thì ta tính diện tích tương tự như câu trên

Câu 17. Tên bài: **tamgiac05.cpp**

Ta dùng công thức *r = (a*b*c)/(4*s);* trong đó s là diện tích tam giác. Từ đây ta tính diện tích hình tròn đơn giản khi biết bán kính r

5. Câu lệnh và khối lệnh

- Trong C++ mỗi câu lệnh được viết trên 1 dòng hoặc nhiều dòng và được kết thúc bằng dấu chấm phẩy (;)

- Khối lệnh hay câu lệnh ghép là dãy các câu lệnh được bao bởi cặp dấu ngoặc nhọn {}. Trong các khối lệnh ta có thể khai báo các biến, kể cả khai báo trùng tên; nhưng biến nào của khối lệnh nào thì chỉ có tác dụng trong khối lệnh đó. Để đơn giản ta có ví dụ sau:

```
int a = 2, b =3;
{
      int a = 5;
       b = a+b;
       cout << a << ' '<< b << endl;// in ra 5 8
}
cout << a << ' '<< b << endl; // in ra 2 8
```

6. Đọc ghi dữ liệu từ tệp (file)

Cú pháp:

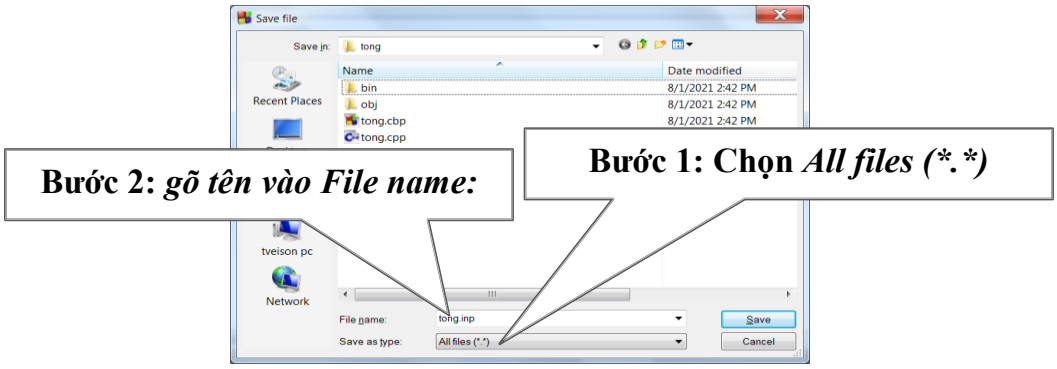
```
// mở file input để đọc dữ liệu (r tức là read)
freopen("Tên file input", "r", stdin);
// mở file output để ghi dữ liệu (w tức là write)
freopen("Tên file ouput", "w", stdout);
```

Thông thường khi lập trình chúng ta cứ cho luôn 2 dòng trên vào đầu chương trình chính thì sẽ đọc dữ liệu từ tệp và ghi dữ liệu ra tệp; nếu ta bỏ 2 dòng đó đi thì sẽ là nhập từ bàn phím và xuất ra màn hình. Ví dụ ta có chương trình tính tổng 2 số đơn giản sau:

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int main(){
   freopen("tong.inp", "r",stdin);
   freopen("tong.out", "w", stdout);
    cin >> a >> b;
    cout << a+b;
    return 0;
}
```

Để chạy được chương trình trên ta cần tạo file ***tong.inp*** bằng cách đơn giản sau: ***File → New → Empty file*** (***Ctrl*** + ***Shift*** + ***N***; ở ô ***Save as type*** ta kéo xuống phía cùng chọn ***All files (*.*)*** *(chú ý chọn cái này trước khi gõ vào File name)*, ô ***File name*** là ***tong.inp*** rồi click vào ***Save*** (Chú ý khi đặt tên đúng với đề bài yêu cầu, tránh trường hợp đặt tên ***tong.inp.c*** *hoặc* ***tong.inp.cpp***). Rồi nhập dữ liệu vào tệp và ***save*** lại *(**Ctrl** + **S**)*. Sau khi tạo file ***tong.inp*** và gõ dữ liệu vào; ta chạy chương trình ***tong.cpp*** (***Ctrl*** + ***F9*** là dịch, nếu không lỗi thì ***Ctrl*** + ***F10*** là chạy chương trình). Vì kết quả ghi ra tệp nên ta phải mở tệp ***.out*** để xem: vào ***file → open*** để mở tệp **tong.out** (***Ctrl*** + ***O***)

Nếu chương trình có lỗi thì khi dịch chương trình nó sẽ hiển thị bảng thông báo lỗi ở phía dưới, nếu không thấy hiển thị thì ta bấm phím ***F2*** để hộp thoại thông báo lỗi xuất hiện.



Chú ý: khi ta không mở được file ***.inp*** hay ***.out*** bằng ***codeblock*** thì ta làm như sau: Vào ***Settings → Environment… →*** xuất hiện hộp thoại ở khung phía trái ta kéo xuống chọn ***Files extension handling →*** Trong khung ***Registered wildcard*** chọn từng cái ****.inp*** và ****.out*** tick vào ***Open it in a Code::Blocks editor → Ok***

7. Cấu trúc rẽ nhánh

- Cấu trúc if :

+ Dạng thiếu

```
if (biểu thức)
        Khối lệnh;
```

Trong đó *biểu thức* có thể có giá trị nguyên hoặc giá trị thực.

Ý nghĩa: nếu biểu thức đúng (hoặc khác 0) thì Khối lệnh sẽ được thực hiện; nếu biểu thức sai (hoặc bằng 0) thì Khối lệnh sẽ bị bỏ qua và chuyển đến các lệnh viết sau nó.

Ví dụ:

```
if (delta < 0)
        cout << "phuong trinh vo nghiem";
```

Hoặc ví dụ trên được viết theo thói quen dùng cặp *{}*, vì sau if nếu có nhiều câu lệnh thì mới cần đặt trong cặp *{}*:

```
if(delta < 0){
        cout << "phuong trinh vo nghiem";
}
```

Các ví dụ khác:

```
if(true)
        cout << "Lệnh if được thực hiện";
if(0)
        cout << "Lệnh if không được thực hiện";
if(1)
        cout << "Lệnh if được thực hiện";
if(2.1)
        cout << "Lệnh if được thực hiện";
if(t = 0)
        cout << "Lệnh if không được thực hiện";
t = 0;
if(t == 1)
        cout << "Lệnh if không được thực hiện";
if(t=1)
        cout << "Lệnh if được thực hiện";
```

Qua ví dụ trên, sau này khi lập trình ta hay viết nhầm lệnh so sánh là dấu = (theo quy ước lệnh so sánh là dấu ==) nên dẫn đến kết quả không theo ý muốn (đối với pascal thì sẽ báo lỗi, còn C++ nó vẫn thực hiện được bình thường): *if (t=1)* lệnh này nghĩa là gán *1* cho *t* và được hiểu là *if(1)* nên lệnh *if* vẫn được thực hiện mặc dù trước câu lệnh *if* giá trị *t* bằng *0*.

Ví dụ 1. Cho tệp **songuyen.inp** gồm 1 số nguyên dương ($n <= 10^9$). Hãy ghi ra tệp **songuyen.out** từ "*chan*" nếu *n* là số chẵn, ghi ra từ "*le*" nếu n là số lẻ.

Chương trình

```
#include <bits/stdc++.h>
using namespace std;
int n;
int main(){
   freopen("songuyen.inp", "r",stdin);
   freopen("songuyen.out", "w", stdout);
   cin >> n;
   if (n%2==0)
      cout << "chan";
   if (n%2==1)
      cout << "le";
   return 0;
}
```

Ví dụ 2. Cho tệp **lonnhat.inp** gồm 3 số nguyên *a, b, c*. Các số này thuộc phạm vi *long long*. Yêu cầu: tìm số lớn nhất của ba số trên và ghi kết quả ra tệp **lonnhat.out**

Chương trình:

```
#include <bits/stdc++.h>
using namespace std;
long long a, b, c, res;
int main(){
   freopen("lonnhat.inp", "r",stdin);
   freopen("lonnhat.out", "w", stdout);
   cin >> a >> b >> c;
   res = a;
   if(res < b)
      res = b;
```

```
    if(res < c)
       res = c;
    cout << res;
    return 0;
}
```

+ *Dạng đủ:*

```
    if(Biểu thức)
            Khối lệnh 1;
    else
            Khối lệnh 2;
```

Ý nghĩa: Nếu *Biểu thức* đúng (hoặc có giá trị khác 0) thì *Khối lệnh 1* sẽ được thực hiện. Ngược lại, nếu *Biểu thức* sai (hoặc có giá trị bằng 0) thì *Khối lệnh 2* sẽ được thực hiện

Ví dụ 1.

```
    if( n%2 == 0)
            cout <<  n << " là số chẵn";
    else
            cout << n << " là số lẻ";
```

Chú ý: trong *Khối lệnh* có thể có các câu lệnh *if* (người ta gọi là *if* lồng nhau)

Ví dụ 2. Cho phương trình *a*x+b=0;* với *a, b* là các số nguyên *(|a| <=10^9, |b|<=10^9)*. Hãy giải phương trình trên. Cho tệp **pt.inp** gồm 2 số *a* và *b* (2 số được cách nhau bởi dấu cách). Ghi ra tệp **pt.out** là *vn* nếu như phương trình vô nghiệm, là *vsn* nếu phương trình vô số nghiệm, nếu có 1 nghiệm thì ghi ra nghiệm đó với chính xác đến 2 phần thập phân.

Chương trình:

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int main(){
   freopen("tong.inp", "r",stdin);
   freopen("tong.out", "w", stdout);
   cin >> a >> b;
   if(a == 0)
      if(b == 0) cout <<"vsn";
```

```
        else
            cout << "vn";
    else
        cout << fixed << setprecision(2) << (double)(-b)/a;
    return 0;
}
```

Chú ý: khi dùng *else* phải biết của *if* nào, nếu cần thiết thì ta thêm cặp dấu ngoặc {} vào để cho dễ phân biệt hơn. Chúng ta nên chú ý đến phong cách lập trình, cái nào là con thì phải viết thụt vào; cái nào ngang hàng nhau thì phải viết thẳng cột.

- Câu lệnh switch: thông thường khi dùng câu lệnh *if* cho dạng lựa chọn ít khả năng. Nếu có nhiều lựa chọn (nhiều nhánh) thì dùng câu lệnh *if* vẫn làm được nhưng nó sẽ viết một cách dài dòng. Để giải quyết vấn đề có nhiều lựa chọn thì ta nên dùng câu lệnh *switch*

Cú pháp:

```
switch (Biểu thức){
        case n1:
                các câu lệnh;
                break;
        case n2:
                các câu lệnh;
                break;
        ...
        case nk:
                các câu lệnh;
                break;
        [default:
                các câu lệnh;]
}
```

+ *Biểu thức* là biểu thức có giá trị nguyên hay kí tự

+ *n1, n2, ...nk*: là các số nguyên, hằng kí tự hay biểu thức hằng

+ *default*: là một thành phần không bắt buộc.

+ Hoạt động của *switch*: Nếu *Biểu thức* có giá trị *ni* nào đó *(1<=i<=k)* thì máy sẽ nhảy đến câu lệnh có nhãn *case ni*, làm xong trường hợp này nó gặp lệnh *break;* sẽ thoát khỏi cấu trúc *switch*. Còn nếu *Biểu thức* có giá trị khác tất cả *ni* thì nó sẽ thực hiện câu lệnh của *default*, vì câu lệnh của *default* ở cuối của *switch* nên ta không cần dùng break trong *default* cũng được.

Ví dụ 1. Cho tệp **tkb.inp** gồm 1 số nguyên *n*. Hãy in ra thời khóa biểu của thứ n đó, nếu không có thứ n đó trong thời khóa biểu thì ghi ra thông báo "Bạn đã nhập sai thông tin". Kết quả ghi ra tệp **tkb.out**. Cho thời khóa biểu như sau:

Thứ 2: Sinh hoạt, Toán, Toán, Tin, Tin
Thứ 3: Tin, Tin, Sinh, Anh, Anh
Thứ 4: Hóa, Lý, Sinh, Thể Dục, Thể Dục
Thứ 5: Văn, Văn, GDCD, Địa lý, Tin
Thứ 6: Toán, Lý, Sử, Công nghệ
Thứ 7: Hóa, Anh, Văn, Tin, Sinh hoạt

Chương trình:

```
#include <bits/stdc++.h>
using namespace std;
int n;
int main(){
   freopen("tkb.inp", "r",stdin);
   freopen("tkb.out", "w", stdout);
   cin >> n;
   switch (n){
      case 2:
         cout << "Sinh hoạt, Toán, Toán, Tin, Tin";
         break;
      case 3:
         cout << "Tin, Tin, Sinh, Anh, Anh";
         break;
      case 4:
         cout << "Hóa, Lý, Sinh, Thể Dục, Thể Dục";
         break;
      case 5:
         cout << "Văn, Văn, GDCD, Địa lý, Tin";
```

```
            break;
        case 6:
            cout << "Toán, Lý, Sử, Công nghệ";
            break;
        case 7:
            cout << "Hóa, Anh, Văn, Tin, Sinh hoạt";
            break;
        default:
            cout << "Bạn đã nhập sai thông tin";
    }
    return 0;
}
```

Ví dụ 2. Cho tệp **ngay.inp** gồm 2 số nguyên dương *m* và *y* (trong đó *m* là tháng, *y* là năm). Yêu cầu ghi ra tệp **ngay.out** một số nguyên duy nhất là số ngày của tháng năm đó. Dữ liệu đầu vào luôn đúng đắn.

Để làm được bài này, học sinh phải tính được số ngày của năm:

+ Tháng có 31 ngày: 1, 3, 5, 7, 8, 10, 12;

+ Tháng có 30 ngày: 4, 6, 9, 11;

+ Tháng 2 của năm nhuận có 29 ngày, năm không nhuận có 28 ngày.

Theo khái niệm chính xác về năm nhuận: năm nhuận là năm chia hết cho 400 hoặc là năm chia hết cho 4 nhưng không chia hết cho 100;

Chương trình:

```
#include <bits/stdc++.h>
using namespace std;
int m, y;
int sn;
int main(){
    freopen("ngay.inp", "r",stdin);
    freopen("ngay.out", "w", stdout);
    cin >> m >> y;
    switch (m){
        case 4:
        case 6:
        case 9:
```

```
        case 11:
            sn = 30;
            break;
        case 2:
            if((y%400==0)||((y%4==0)&&(y%100!=0)))
                sn = 29;
            else    sn = 28;
            break;
        default:
            sn = 31;
    }
    cout << sn;
    return 0;
}
```

Trong ***ví dụ*** trên ta thấy nếu nhiều nhãn case mà có câu lệnh thực hiện giống nhau thì ta có thể viết ngắn gọn như cách viết ở trên.

Bài tập

Câu 01. Nhập vào 3 số nguyên dương a, b, c; Kiểm tra xem 3 số đó có lập thành 1 tam giác hay không? Nếu nó là 3 cạnh của tam giác thì đó là tam giác vuông, hay nhọn, hay tù?

Câu 02. Hãy giải phương trình bậc 2 có dạng $ax^2 + bx + c = 0$; trong đó a, b, c là các số nguyên được nhập từ bàn phím (a != 0). Nếu có nghiệm thì in ra 2 phần thập phân

Câu 03. Hãy giải phương trình có dạng $ax^2 + bx + c = 0$; trong đó a, b, c là các số thực được nhập từ bàn phím. Nếu có nghiệm thì in ra 2 phần thập phân

Câu 04. Hãy giải hệ phương trình 2 ẩn sau

$$a_1x + b_1y = c_1$$
$$a_2x + b_2y = c_2$$

Trong đó a_1 b_1 c_1 a_2 b_2 c_2 là các số nguyên được nhập từ bàn phím

Câu 05. Cho tệp **toantu.inp** gồm 2 số thực x, y và một dấu phép toán (+, -, *, /). Yêu cầu ghi ra tệp **toantu.out** gồm 1 số là kết quả của phép toán (in ra 2 chữ số thập phân):

+ x + y: nếu dấu phép toán là dấu +
+ x – y: nếu dấu phép toán là dấu –
+ x * y: nếu dấu phép toán là dấu *
+ x / y: nếu dấu phép toán là dấu /

toantu.inp	toantu.out
3.1 2.3 +	5.40

Câu 06. Cho lịch luyện tập của đội tuyển như sau:

+ Thứ 2, 3, 4: Học lý thuyết
+ Thứ 5, 7: Nghỉ
+ Thứ 6: Làm bài tập

Cho tệp **lich.inp** gồm 1 số n (là thứ trong tuần). Hãy ghi ra **lich.out** là lịch làm việc của thứ n đó.

lich.inp	lich.out
2	Học lý thuyết

Câu 07. Nhập vào 1 số tự nhiên N. Kiểm tra xem số N có phải là số chính phương hay không?

Gợi ý cách giải

Câu 01. Điều kiện để a, b, là 3 cạnh tam giác: a+b >c và a+c>b và b+c>a. Nếu thỏa mãn là 3 cạnh của tam giác thì:

+ Điều kiện là tam giác vuông $a^2 + b^2 = c^2$ hoặc $a^2+c^2=b^2$ hoặc $b^2+c^2=a^2$

+ Điều kiện là tam giác tù: $a^2 + b^2 < c^2$ hoặc $a^2+c^2<b^2$ hoặc $b^2+c^2<a^2$

+ Ngược lại 2 điều kiện trên là tam giác nhọn.

Câu 02. Dùng cách giải phương trình bậc 2, chú ý a, b, c là số nguyên mà nghiệm có thể là số thực, nên cẩn thận khi dùng phép chia (/).

Câu 03. Ta xét trường hợp a == 0 thì trở về giải phương trình bx+c=0 như bài ở trên. Ngược lại thì ta giải phương trình bậc 2.

Câu 04. Dùng cách giải định thức cấp 2 ta tính các D, Dx, Dy:

$$D = \begin{vmatrix} a1 & b1 \\ a2 & b2 \end{vmatrix} = a1*b2 - a2*b1;\ Dx = \begin{vmatrix} c1 & b1 \\ c2 & b2 \end{vmatrix} = c1*b2 - c2*b1;\ Dy = \begin{vmatrix} a1 & c1 \\ a2 & c2 \end{vmatrix} = a1*c2 - a2*c1$$

+ Điều kiện phương trình có 1 cặp nghiêm: D != 0

+ Điều kiện hệ pt vô số cặp nghiệm: D == 0 và Dx == 0 và Dy == 0

+ Ngược lại của 2 điều kiện trên thi hệ pt vô nghiệm

Câu 05. Dùng cấu trúc switch để làm; ta khai báo 2 biến x, y là kiểu số thực, 1 biến ch có kiểu char để lưu dấu của phép toán và 1 biến res để lưu kết quả

```
switch(ch){
        case '+': res = x + y; break;
        case '-': res = x - y; break;
        case '*': res = x * y; break;
        case '/': res = x / y; break;
}
cout << fixed << setprecision(2) << res;
```

Câu 06. Tương tự ta dùng cấu trúc switch:

```
switch(n){
        case 5:
        case 7: cout << "Nghỉ"; break;
        case 6: cout << "Làm bài tập"; break;
```

```
        default: cout << "Học lý thuyết";
    }
```

Câu 07.

```
int k = sqrt(n);
if( k*k == n) cout << n << " la so chinh phuong";
else cout << n << " khong phai la so chinh phuong";
```

8. Cấu trúc lặp

Cấu trúc lặp trong C++ gồm 3 dạng cơ bản sau:

- Cấu trúc lặp for

Cú pháp: *for (Biểu thức 1; Biểu thức 2; Biểu thức 3)*
Khối lệnh;

Trong đó:

Biểu thức 1: là biểu thức khởi đầu

Biểu thức 2: là biểu thức điều kiện để lặp

Biểu thức 3: là bước nhảy, làm thay đổi bước nhảy

Các biểu thức này có thể vắng mặt trong cấu trúc *for*, nhưng các dấu chấm phẩy *;* vẫn phải có.

Ý nghĩa của vòng *for*

+ **Bước 1**: thực hiện *Biểu thức 1* rồi chuyển sang **Bước 2**

+ **Bước 2**: Kiểm tra điều kiện ở *Biểu thức 2*, nếu điều kiện sai (hoặc bằng 0) thì thoát khỏi vòng lặp *for*, nếu điều kiện đúng (hoặc khác 0) thì sẽ thực hiện *Khối lệnh* rồi chuyển sang **Bước 3;**

+ **Bước 3**: thực hiện *Biểu thức 3*, rồi quay lại về **Bước 2**

Ví dụ. Ghi ra các số từ 1 đến 10:

```
for (int i = 1; i<=10; i++)
        cout << i << " ";
```

Chú ý: theo thói quen ta có thể dùng cặp {} khi sau *for* khi có 1 lệnh; nếu cần thực hiện nhiều câu lệnh sau *for* thì ta phải dùng cặp ngoặc {}

```
for (int i = 1; i<=10; i++){
        cout << i << " ";
}
```

Như vậy câu lệnh trên ta có thể khai báo luôn biến *i* ở trong ngoặc () của cấu trúc *for*. Ở cách làm trên thì thoát khỏi vòng lặp thì *i* có giá trị là giá trị của biến *i* ở ngoài vòng lặp. Với bài trên ta cũng có thể có các cách sau:

```
i = 1;
for (int i = 1; i<=10; i++){
        cout << i << " ";
}
```

Đoạn chương trình trên thì sau khi thoát khỏi vòng lặp for thì biến *i* có giá trị bằng 1 vì (biến *i* khai báo ở for trùng tên với biến *i* ở ngoài, nên giá trị của biến *i* trong for chỉ có tác dụng trong for, khi ra ngoài vòng *for* nó sẽ không còn tác dụng). Ta có cách làm khác như sau

```
int i = 1;
for (; i<=10;){
        cout << i << " ";
        i++;
}
```

Như vậy ta có thể bỏ các biểu thức trong cấu trúc *for* nhưng phải để lại dấu chấm phẩy (;). Ở cách làm trên thì thoát khỏi vòng lặp *i* có giá trị là 11

```
int i = 1;
for ( ; ; ){
        cout << i << " ";
        if (i==10) break;
        i++;
}
```

for(; ;) là vòng lặp chạy không có biểu thức nào, nó sẽ lặp mãi mãi. Vì vậy muốn dừng vòng lặp này thì trong vòng lặp phải có lệnh thoát khỏi vòng lặp hoặc lệnh dừng chương trình. Ở ví dụ trên ta dùng *break;* là lệnh thoát khỏi vòng lặp gần nhất với nó. Ở cách làm trên thì sau khi thoát khỏi vòng lặp thì giá trị của *i* là 10

Bằng cách dùng dấu phẩy, ta có thể dùng nhiều lệnh trong các biểu thức 1, biểu thức 2, biểu thức 3:

```
for(i=1, j =10; i<=j; i++, j--){
        cout << i << ' '<< j << '\n';
}
```

Đoạn chương trình trên sẽ in ra *i , j* như sau:

```
1 10
2 9
3 8
4 7
5 6
```

Thoát khỏi vòng *for* thì giá trị của *i* bằng 6 và giá trị của *j* là 5.

Chú ý: trong C++ thì khai báo biến rất linh hoạt (không giống như pascal). Trong khối lệnh của vòng lặp *for*, chúng ta không nên tác động làm thay đổi các giá trị của các biến dùng làm lệnh nhảy (vì sau này sẽ rất khó kiểm soát giá trị của nó); Ví dụ sau:

```
#include <bits/stdc++.h>
using namespace std;
int i = 0, j = 0;
int main(){
   for( int i=1, j=10; i<=j; i++, j--){
      cout << i << ' ' << j << '\n';
      i++;
      j++;
   }
   cout << i << ' ' << j << '\n';
   return 0;
}
```

Chương trình trên in ra màn hình:

```
1 10
3 10
5 10
7 10
9 10
0 0
```

Ví dụ 1. Tính tổng S = 1 + 2 + 3 + …+N. Với N là số nguyên dương được nhập từ bàn phím (N <=10^6)

Gợi ý: Yêu cầu không được tính tổng S bằng cách dùng công thức S = N*(N+1)/2; Làm sao để lần đầu tiên S =1; sau đó S = 1+2; sau đó S = 1+2+3; cứ lặp như vậy thì S = 1+2+3+…+N;

```
#include <bits/stdc++.h>
using namespace std;
int n;
long long s = 0;
int main(){
   //freopen("tong.inp", "r",stdin);
   //freopen("tong.out", "w", stdout);
   cin >> n;
   for(int i = 1; i <= n; i++)
      s = s + i;
   cout << s;
   return 0;
}
```

Ví dụ 2. Nhập vào số nguyên *n* (0 <= *n* <=20). Tính *n*!

```
#include <bits/stdc++.h>
using namespace std;
int n;
long long gt = 1;
int main(){
   //freopen("giaithua.inp", "r",stdin);
   //freopen("giaithua.out", "w", stdout);
   cin >> n;
   for(int i = 2; i <= n; i++)
      gt = gt*i;
   cout << gt;
   return 0;
}
```

- Cấu trúc lặp while

Cú pháp:

while (biểu thức)
Khối lệnh;

Ý nghĩa của *while:*

+ **Bước 1**: tính giá trị *biểu thức* trong cặp ngoặc () sau *while*. Nếu giá trị của *biểu thức* là sai (hoặc bằng 0) thì kết thúc vòng lặp *while*. Ngược lại nếu giá trị của *biểu thức* đúng (hoặc khác 0) thì chuyển sang **Bước 2**;

+ **Bước 2**: thực hiện *Khối lệnh*, rồi quay lại **Bước 1**.

Khối lệnh trong *while* có thể được thực hiện 1 hoặc nhiều lần hoặc không được thực hiện lần nào nếu như *biểu thức* sau *while* sai (hoặc bằng 0) ngay từ đầu.

Ví du 1. Viết lại 2 chương trình ở ví dụ trên bằng việc cùng cấu trúc *while*

Tính S = 1 + 2 + 3 + …+N

```
#include <bits/stdc++.h>
using namespace std;
long long s;
int n;
int main(){
  cin >> n;
  s = 0;
  int i = 1;
  while (i<=n){
    s = s + i;
    i++;
  }
  cout << s;
  return 0;
}
```

Tính n!

```
#include <bits/stdc++.h>
using namespace std;
```

```
long long gt;
int n;
int main(){
  cin >> n;
  gt = 1;
  int i = 1;
  while (i<=n){
    gt = gt *i;
    i++;
  }
  cout << gt;
  return 0;
}
```

Ví dụ 2. Tính và đưa ra kết quả tổng $S = 1 + \frac{1}{2} + \frac{1}{3} + ... + \frac{1}{N}$ cho đến khi $\frac{1}{N} < 0.0001$ lấy kết quả s chính xác đến 4 phần thập phân

Gợi ý: Như vậy ta phát biểu lại bài toán trong khi $\frac{1}{N} \geq 0.0001$ hoặc $1 \geq 0.0001 * N$ thì ta cứ tính tổng S

```
#include <bits/stdc++.h>
using namespace std;
double  s;
int n;
int main(){
  s = 1;
  int n = 2;
  while (1.0/ n >= 0.0001){
    s = s + 1.0 / n;
    n++;
  }
  cout << fixed << setprecision (4) << s;
  return 0;
}
```

- Cấu trúc do … while

***Cú pháp*:**

do
Khối lệnh;
while(biểu thức);

Ý nghĩa của *do…while*:

+ **Bước 1**: thực hiện *Khối lệnh* rồi chuyển sang **Bước 2**;

+ **Bước 2**: tính giá trị *biểu thức* trong cặp ngoặc () sau *while*. Nếu giá trị của *biểu thức* là sai (hoặc bằng 0) thì kết thúc vòng lặp *while*. Ngược lại nếu giá trị của *biểu thức* đúng (hoặc khác 0) thì quay lại **Bước 1**;

Cấu trúc lặp *do ... while* nó cũng tương tự như cấu trúc lặp *while* nhưng nó thực hiện *Khối lệnh* trước, sau đó mới kiểm tra điều kiện. Như vậy nếu *biểu thức* có giá trị sai (hoặc bằng 0) ngay từ đầu thì *Khối lệnh* vẫn được thực hiện 1 lần rồi mới thoát khỏi vòng lặp *do ... while*. Chúng ta cứ hiểu nôm na là: làm *Khối lệnh* trong khi điều kiện đúng, thoát khi điều kiện sai (làm trước rồi mới kiểm tra điều kiện)

Ví dụ 1. Nhập vào một số nguyên dương n và in ra số vừa nhập, nếu bạn nhập không phải số nguyên dương thì yêu cầu nhập lại cho đúng, cho đến lúc đúng thì mới in ra kết quả, còn cứ nhập sai thì cứ phải nhập lại

```
#include <bits/stdc++.h>
using namespace std;
int n;
int main(){
  cout << "moi ban nhap so n: ";
  do{
    cin >> n;
    if (n < 0)  cout << "moi ban nhap lai n: ";
  } while (n < 0);
  cout << "gia tri cua n vua nhap la " << n;
  return 0;
}
```

Ví dụ 2. Tính và đưa ra kết quả tổng $S = 1 + \frac{1}{2} + \frac{1}{3} + ... + \frac{1}{N}$ cho đến khi $\frac{1}{N} < 0.0001$. Kết quả được làm tròn đến 4 chữ số thập phân.

Làm tương tự như ví dụ trên ta có chương trình sau:

```
#include <bits/stdc++.h>
using namespace std;
int n;
double   s;
int main(){
  s = 1;
  n = 2;
  do {
    s = s + 1.0 / n;
    n++;
  } while (1.0 / n >= 1E-4);
  cout << fixed << setprecision(4) << s;
  return 0;
}
```

- Lệnh break; là lệnh thoát khỏi vòng lặp gần nhất với nó. (giống y như lệnh break; trong pascal)

Ví dụ.

```
for (int i = 2000; ; i++)
    if (i % 100 != 0 && i % 4 == 0){
      cout << i;
      break;
  }
```

Đoạn chương trình trên sẽ in ra số bé nhất, lớn hơn hoặc bằng 2000 mà không chia hết cho 100 và chia hết cho 4

- Lệnh continue; là lệnh bỏ qua phần còn lại của vòng lặp và nhảy sang lần lặp tiếp theo. (giống y như lệnh **continue** trong pascal)

Ví dụ.

```
s = 0;
for (int i  = 1; i <= 3; i++){
```

```
    s = s + i;
    if (i >= 2) continue;
    s = s+ 1;
    cout << s << '\n';
  }
cout << s;
```

Ở ***ví dụ*** trên, khi *n >=2* thì nó sẽ không thực hiện các câu lệnh phía sau *continue* ở trong vòng lặp *for* và quay lên thực hiện lần lặp tiếp theo của *for*

Chú ý: khi làm việc với số thực chúng ta không được thực hiện lệnh so sánh bằng (==) hoặc khác (*!*=). Vì khi làm việc với số thực sẽ xảy ra hiện tượng sai số. Ta xét ví dụ sau:

```
#include <bits/stdc++.h>
using namespace std;
double  i;
int main(){
  i = 0.1;
  do {
    cout << i << endl;
    i = i + 0.1;
  } while (!(i == 1)); // hoặc while (i != 1)
  return 0;
}
```

Theo suy nghĩ thông thường, vòng lặp *do ... while* trên sẽ thoát khi *i* bằng 1. Nhưng do *i* là số thực và được tính toán trong các câu lệnh nên kết quả trả về cho *i* bị sai số nên *i* không bao giờ bằng chính xác 1. Vòng lặp *do ... while* ở trên sẽ xảy ra hiện tượng lặp vô tận (in ra giá trị của *i* mãi mãi không dừng). Như vậy khi làm việc với số thực ta nên dùng các phép so sánh <, <=, > , >=. Trong trường hợp muốn so sánh bằng == thì nên so sánh với sai số delta. Ví dụ: thay vì ta so sánh x == y thì ta xét *abs(x-y) <= delta*. Trong đó *delta* là sai số mà ta suy luận từ đề bài, *delta* càng nhỏ thì độ chính xác càng cao. Ví dụ đề bài cho in số thực lấy chính xác đến 4 chữ số thập phân, thì ta muốn so sánh *x* có bằng *y* hay không thì ta làm như sau: *abs(x-y) <= 0.00001* (vì lấy đến chính xác *4* chữ số thập phân nên ta so sánh hiệu đó với 10^{-5} hoặc với số nhỏ hơn 10^{-5} thì càng chính xác.

Bài tập

Câu 1. Nhập vào N số tự nhiên, Hãy đếm xem có bao nhiêu số chẵn trong N số tự nhiên đó.

Câu 2. Nhập vào N số tự nhiên. Hãy tìm số lớn nhất trong N số đó

Câu 3. Tính a^x. Với a là số thực, x là số tự nhiên

Câu 4. Tính a^x. Với a là số thực, x là số nguyên

Câu 5. Viết chương trình in ra màn hình mã ASCII của các chữ cái từ 'a' đến 'z' và từ 'A' đến 'Z' và chữ số từ '0' đến '9'.

Câu 6. Giải bài toán cổ sau:

Vừa gà, vừa chó
Bó lại cho tròn
Ba mươi sáu con
Một trăm chân chẵn.

Hỏi có bao nhiêu con mỗi loại

Câu 7. Giải bài toán cổ sau:

Có một trăm con trâu
Một trăm bó cỏ
Trâu đứng ăn 5
Trâu nằm ăn 3
3 trâu già già ăn 1

Tính số lượng mỗi loại trâu?

Câu 8. Cho số tự nhiên N (N <=10^9). Yêu cầu tính tổng các chữ số của số N. Ví dụ N = 132 suy ra đáp án tổng các chữ số của N là 1+2+3=6

Câu 9. Cho số tự nhiên N (N <=10^9). Tìm chữ số đầu tiên bên trái của số N.

Câu 10. Cho số tự nhiên N (N <=10^9). Yêu cầu tìm chữ số lớn nhất trong các chữ số của số N. Ví dụ: N = 1345673 thì chữ số lớn nhất là 7

Câu 11. Cho số tự nhiên n. Hãy lập trình tính các bài toán sau: Nếu kết quả là số thực thì lấy kết quả chính xác đến 4 phần thập phân.

a. $S = 1 + 3 + ... + (2 * n + 1)$

b. $s = \frac{1}{2} + \frac{1}{4} + ... + \frac{1}{2 * n}$

c. $s = 1 + \frac{1}{2^2} + \frac{1}{3^2} + ... + \frac{1}{n^2}$

d. $s = (1 + \frac{1}{2^2})(1 + \frac{1}{3^2})...(1 + \frac{1}{n^2})$

e. $s = \frac{1}{2} + \frac{2}{3} + ... + \frac{n}{n+1}$

f. $S = 1! + 2! + 3! + ... + n!$

g. $s = 1 + \frac{1}{2!} + \frac{1}{3!} + ... + \frac{1}{n!}$

h. $S = 1! - 2! + 3! - 4! + ...n!$

i. $s = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + ... + \frac{(-1)^{n+1}}{n}$

k. $e(n) = 2 + \frac{1}{2!} + \frac{1}{3!} + ... + \frac{1}{n!} + ...$ cho đến khi $\frac{1}{n!} < 2.10^{-6}$

l. Tính số π biết $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - ... \frac{1}{k}$ cho đến khi $\frac{1}{k} < 10^{-4}$

Câu 12. Nhập vào số tự nhiên N. Kiểm tra xem N có phải là số nguyên tố hay không?

Câu 13. Nhập vào một số tự nhiên N. Kiểm tra xem N có phải là số hoàn hảo hay không? Số hoàn hảo là số có tổng các ước trừ nó ra thì bằng chính nó. Ví dụ số 6, 28… là các số hoàn hảo.

Câu 14. Nhập vào một số tự nhiên N, yêu cầu in ra số đảo ngược lại của số N. Ví dụ: N = 1324 thì in ra số đảo ngược là 4231. Số N =100 thì in ra số đảo ngược là 1 (bỏ số 0 đứng trước nếu có). Số 0 đảo ngược là 0

Câu 15. Nhập vào số tự nhiên N, yêu cầu in ra số đó ra dưới dạng số nhị phân.

Câu 16. Nhập vào 2 số tự nhiên m và n (không đồng thời bằng 0). Tính ước chung lớn nhất và bội chung nhỏ nhất của 2 số m và n

Câu 17. Hai số a và b được gọi là hai số bạn bè nếu tổng các các ước của số này (trừ 1 và chính nó) bằng số kia hoặc ngược lại. Ví dụ số 6 và 8 là hai số bạn bè vì tổng các ước của 8 trừ 1 và 8 thì bằng 2+4 = 6.

Câu 18. Nhập số nguyên dương N. In ra tất cả các thừa số nguyên tố. Ví dụ: N= 48 thì kết quả in ra 2 2 2 2 3

Câu 19. Nhập vào số nguyên dương N. Tính số fibonacci thứ N. Biết F(1) = 1; F(2) = 1; F(N) = F(N-2) + F(N-1)

Gợi ý cách giải

Câu 1. Nhập được số nào thì kiểm tra luôn số đó.

```
#include <bits/stdc++.h>
using namespace std;
int n, a;
int res;
int main(){
  cout << "moi ban nhap so n: ";
  cin >> n;
  res = 0;
  for (int i = 1; i <= n; i++){
    cout << "moi ban nhap so thu " << i << ": ";
    cin >> a;
    if (a%2 == 0)  res++;
  }
  cout << "so luong so chan la: " << res;
  return 0;
}
```

Nếu cũng câu này nhưng làm việc với tệp: Cho tệp number.inp gồm dòng đầu tiên là số nguyên dương N. Dòng thứ 2 là dãy N số nguyên a1, a2, ..aN. Hãy đếm xem trong dãy số a1, a2, …aN có bao nhiêu số chẵn và ghi kết quả ra tệp number.out

Cách làm: ta tạo file *number.cpp* như sau (đây là file mã nguồn, sau này học sinh đi thi, chỉ cần nộp file *.cpp* này):

```
#include <bits/stdc++.h>
using namespace std;
int n, a;
int res;
int main(){
  ios_base::sync_with_stdio(false);
  cin.tie (0); cout.tie (0);
  freopen("number.inp","r", stdin);
  freopen("number.out", "w", stdout);
  cin >> n;
  res = 0;
```

```
    for (int i = 1; i <= n; i++){
        cin >> a;
        if (a%2 == 0)  res++;
    }
    cout << res;
    return 0;
}
```

Ở ví dụ trên khi đọc và ghi dữ liệu từ tệp thì ta không được dùng: mời bạn nhập N, mời bạn nhập số thứ i…Vì khi dùng những câu đó tức là bạn dùng lệnh *cout* (là lệnh ghi những câu đó ra tệp *.out*). Trong ví dụ trên ta có dùng thêm các câu lệnh đặt trước *freopen()*

```
ios_base::sync_with_stdio(false);
cin.tie(0); cout.tie(0);
```

Ý nghĩa của 2 dòng trên là ta vô hiệu hóa việc đồng bộ giữa các luồng tiêu chuẩn C và C++ và làm cho chương trình có thể chạy nhanh hơn. Ta có thể search thêm trên mạng về vấn đề này.

Như vậy, khi dùng với tệp ta thêm 4 dòng này vào phần đầu của hàm main. Nếu bỏ nó đi thì sẽ là đọc từ bàn phím và xuất ra màn hình:

```
ios_base::sync_with_stdio(false);
cin.tie(0); cout.tie(0);
freopen("Tên file input","r", stdin);
freopen("Tên file ouput", "w", stdout);
```

Trước khi chạy chương trình trên ta phải dịch chương trình xem chương trình có lỗi hay không (***Ctrl + F9***). Chương trình sẽ hiển thị thông báo ở phía dưới. Ta phải tạo tệp **number.inp** như đã hướng dẫn ở trên và nhập thử các bộ dữ liệu vào và lưu lại. Rồi vào lại mã nguồn **number.cpp** để chạy chương trình (**Ctrl + F10**). Để xem kết quả ta mở file **number.out** để xem.

Câu 2. Ta biết rằng kết quả tồi tệ nhất là số lớn nhất bằng 0. Nên ta dùng một biến res để lưu kết quả. Khởi tạo res = 0; nhập đươc số nào thì ta so sánh với res để tìm số lớn nhất hiện tại

```
cin >> n;
res = 0;
for(int i = 1; i <= n; i++){
```

```
        cin >> a;
        if (res < a)     res = a;
    }
    cout << res;
```

Câu 3. a^x bằng x con a nhân lại với nhau, khi lập trình chú ý trường hợp $x = 0$ thì kết quả là *1*; nếu cần in ra mấy phần thập phân thì ta dùng thêm *fixed* và *setprecision* để in.

Câu 4. Bài này tương tự bài trên. Nhưng ta chú ý thêm trường hợp nếu x là số âm thì ta dùng công thức để tính $a^{-x} = 1/(a^x)$

Câu 5. Ta dùng ép kiểu để in bảng mã ASCII của các kí tự

```
for (char ch = 'a'; ch <= 'z'; ch++)
   cout << (int) ch << " " ;
cout << endl;
for (char ch = 'A'; ch <= 'Z'; ch++)
   cout << (int) ch << " ";
cout << endl;
for (char ch = '0'; ch <= '9'; ch++)
   cout << (int) ch << " ";
```

Câu 6. Gọi số gà là x, số chó là y thì ta có hệ sau:

x + y = 36
2* x + 4*y = 100

Ta dễ nhận thấy ngay 0 <= x, y <= 36 (Nếu muốn điều kiện chặt hơn nữa thì ta rút x thế vào dưới và rút y thế vào dưới để tìm điều kiện). Ta thử tất cả trường hợp của x và y trong đoạn [0, 36] xem x và y nào thõa mãn 2 phương trình ở trên.

```
for (int x = 0; x <= 36; x++)
    for (int y =0; y <= 36; y++)
       if (x + y == 36 && 2*x + 4*y == 100)   cout << x <<" "<< y << endl;
```

Câu 7. Cũng giải tương tự như câu 6. Ta gọi số trâu đứng là x, số trâu nằm là y, số trâu già là z thì ta được hệ sau:

x + y + z = 100
x*5 + y*3 + z/3 = 100 ta nên thay bằng đk: x*15 + y*9 + z = 300

Ta dễ nhận thấy 0 <= x, y, z <= 100 hoặc nhìn vào phương trình thứ 2 để lấy điều kiện ngặt hơn.

```
for (int x = 0; x <= 100; x++)
  for (int y = 0; y <= 100; y++)
    for (int z = 0; z <= 100; z++)
      if (x + y + z == 100 && 15*x + 9*y + z == 300)
          cout << x <<" "<< y <<  " " << z << endl;
```

Câu 8. Ta tách từng chữ số, được chữ số nào ta cộng vào kết quả

```
cin >> n;
res = 0;
while (n>0){
  res = res + n % 10;
  n = n / 10;
}
cout << res
```

Câu 9.

```
cin >> n;
res = 0;
while (n>10){
  n = n / 10;
}
cout << n;
```

Câu 10. Khởi tạo kết quả bằng 0, Ta tìm từng chữ số của số đó, rồi so sánh với kết quả để cập nhật lại.

```
cin >> n;
res = 0;
while (n>0){
  a = n % 10;
  if ( res < a) res = a;
  if (res == 9) break;
  n = n / 10;
}
cout << res;
```

Câu 11.

a. $S = 1 + 3 + ... + (2 * n + 1)$

```
cin >> n;
s = 0;
for (int i = 1; i <= 2*n+1; i = i+ 2)
   s = s + i;
cout << s;
```

b. $s = \frac{1}{2} + \frac{1}{4} + ... + \frac{1}{2 * n}$

Khai báo s là số thực;

```
cin >> n;
s = 0;
for (int i = 2; i <= 2*n; i = i+ 2)
   s = s + 1.0 / i;
cout << fixed << setprecision(4) << s;
```

c. $s = 1 + \frac{1}{2^2} + \frac{1}{3^2} + ... + \frac{1}{n^2}$

```
cin >> n;
s = 1;
for (int i = 2; i <= n; i = i+ 1)
   s = s + 1.0 / (i*i);
cout << fixed << setprecision(4) << s;
return 0;
```

d. $s = (1 + \frac{1}{2^2})(1 + \frac{1}{3^2})...(1 + \frac{1}{n^2})$

```
cin >> n;
s = 1;
for (int i = 2; i <= n; i = i+ 1)
   s = s * (1 + 1.0 / (i*i));
cout << fixed << setprecision(4) << s;
```

e. $s = \frac{1}{2} + \frac{2}{3} + ... + \frac{n}{n+1}$

Khai báo s là kiểu double; n kiểu int

```
cin >> n;
s = 0;
for (int i = 1; i <= n; i = i+ 1)
   s = s + 1.0 * i / (i+1);
cout << fixed << setprecision(4) << s;
```

f. $S = 1! + 2! + 3! + ... + n!$

Khai báo n kiểu int; s, gt kiểu long long

```
cin >> n;
s = 1;
gt = 1;
for (int i = 2; i <= n; i = i+ 1){
   gt = gt * i;
   s = s + gt;
}
cout << s;
```

g. $s = 1 + \frac{1}{2!} + \frac{1}{3!} + ... + \frac{1}{n!}$

Khai báo n kiểu int, s kiểu số thực *double*, gt kiểu *long long*

```
cin >> n;
s = 1;
gt = 1;
for (int i = 2; i <= n; i = i+ 1){
   gt = gt * i;
   s = s + 1.0 / gt;
}
cout << fixed << setprecision(4) << s;
```

h. $S = 1! - 2! + 3! - 4! + ... n!$

Ta dùng 1 biến để đảo dấu, khai báo biến s, gt là kiểu long long

```
cin >> n;
s = 1;
gt = 1;
dau = -1;
for (int i = 2; i <= n; i = i+ 1){
   gt = gt * i;
```

```
    s = s + dau * gt;
    dau = - dau;
  }
 cout << s;
```

i. $s = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + ... + \frac{(-1)^{n+1}}{n}$

Khai báo s là kiểu số thực double; n là kiểu int.

```
    cin >> n;
    s = 1;
    dau = -1;
    for (int i = 2; i <= n; i = i+ 1){
      s = s + 1.0 / i * dau;
      dau = - dau;
    }
    cout << s;
```

k. $e(n) = 2 + \frac{1}{2!} + \frac{1}{3!} + ... + \frac{1}{n!} + ...$ cho đến khi $\frac{1}{n!} < 2.10^{-6}$

Nên dùng *while* hoặc *do ... while*

```
 e = 2;
 i = 2;
 gt = 2;
 while (1.0 / gt  >= 2E-6){
   e = e + 1.0 / gt;
   i++;
   gt = gt * i;
 }
 cout << fixed << setprecision(4) << e;
```

l. Tính số π biết $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - ... \frac{1}{k}$ cho đến khi $\frac{1}{k} < 10^{-4}$

```
  pi = 1;
  i = 3;
  dau = -1;
  while (1.0 / i  >= 1E-4){
    pi = pi + 1.0 / i* dau;
    i = i + 2;
    dau = - dau;
```

```
}
cout << fixed << setprecision(4) << pi*4;
```

Câu 12. Duyệt i từ 2 đến $\sqrt{N}$ nếu N chia hết cho i thì kết luận luôn N không phải số nguyên tố, nếu duyệt hết mà không có N chia hết cho i nào thì N là số nguyên tố. Tại sao ta chỉ cần duyệt đến $\sqrt{N}$. Chứng minh: giả sử tồn tại một số a là số lớn hơn $\sqrt{N}$ mà N chia hết cho a. Lúc này N = a*b (vì N chia hết cho a nên ta phân tích được như vậy). Ta đã có a > $\sqrt{N}$ nên suy ra b <$\sqrt{N}$ (Nếu b>$\sqrt{N}$ thì a*b > $\sqrt{N}$ *$\sqrt{N}$ là điều vô lý). Như để kiểm tra N chia hết cho a thì ta chỉ cần kiểm tra N chia hết cho b. Như vậy ta chỉ cần duyệt đến $\sqrt{N}$ Thay vì phép lấy căn, ta cho i chạy từ 2 đến i*i <= N (vì phép nhân thực hiện nhanh hơn phép lấy căn)

```
#include <bits/stdc++.h>
using namespace std;
int n;
bool kt;
int main(){
  cin >> n;
  kt = true;
  if (n<2) kt = false;
  else{
    for (int i = 2; i*i <= n; i++)
    if (n % i == 0){
      kt = false;
      break;
    }
  }
  if (kt)  cout << n << " la so nguyen to";
  else cout << n << " khong phai la so nguyen to";
  return 0;
}
```

Câu 13. Ta muốn tìm tất cả các ước của số N thì chỉ cần duyệt i chạy từ 2 đến N/2. Phía sau số N/2 (trừ số N) thì không có số nào mà N chia hết.

```
#include <bits/stdc++.h>
using namespace std;
```

```
int n, tong;
bool kt;
int main(){
  cin >> n;
  if (n<6){
    cout << n << " khong phai la so hoan hao";
    return 0;
  }
  tong = 1;
  for (int i = 2; i <= n/2; i++)
    if (n % i == 0)   tong = tong + i;
  if (tong == n) cout << n << " la so hoan hao";
  else cout <<n << " khong phai la so hoan hao";
  return 0;
}
```

Câu 14.

```
#include <bits/stdc++.h>
using namespace std;
long long n;
long long res;
int a;
int main(){
  cin >> n;
  res = 0;
  while (n != 0){
    a = n % 10;
    res = res *10 +a;
    n = n / 10;
  }
  cout << res;
  return 0;
}
```

Câu 15. Ta dùng một biến để lưu kết quả, với cách làm này ta chỉ làm được với số nhị phân tầm 19 đến 20 chữ số

```
#include <bits/stdc++.h>
using namespace std;
```

```
long long n;
long long res, t;
int a;
int main(){
  cin >> n;
  res = 0;
  t = 1;
  while (n != 0){
    a = n % 2;
    res = res + t * a;
    n = n / 2;
    t = t* 10;
  }
  cout << res;
  return 0;
}
```

Câu 16. Dùng cách tìm ước chung lớn nhất bằng thuật toán ơclit. Ví dụ m =28; n = 16; thuật toán làm như sau:

+ Lấy 28 chia **16** được 1 dư **12**

+ 16 sẽ thế vào vị trí của 28; 12 thế vào vị trí của 16: rồi ta lại lấy 16 chia **12** được 1 dư **4**

+ 12 sẽ thế vào vị trí của 16; 4 sẽ thế vào vị trí của 12. Rồi ta lấy 12 chia 4 được 3 dư 0. Khi có phép chia hết thì ta dừng và kết luận 4 là ước chung lớn nhất.

Ở thuật toán này ta không cần quan tâm m lớn hơn hay nhỏ hơn n, vì n có lớn hơn thì sau lần chia thứ nhất thì giá trị m và n sẽ đổi chỗ cho nhau: Ví dụ m = 16; n = 28: Ta lấy 16 chia **28** được 0 dư **16**, tiếp theo là 28 sẽ thế vào vị trí m, 16 sẽ thế vào vị trí n….

Để tính BCNN(m, n) thì ta có công thức. BCNN (m, n) = (m*n)/UCLN(m,n) ở đây vì UCLN (m, n) đều là ước của m và n nên m*n luôn chia hết cho UCLN(m, n). Khi lập trình ta nên lưu giá trị của 2 biến trước khi tính UCLN(m, n).

```
#include <bits/stdc++.h>
```

```
using namespace std;
int  m, n, a, b;
int r;
int main(){
   cin >> m >> n;
   a = m;
   b = n;
   while (n != 0){
      r = m % n;
      m = n;
      n = r;
   }
   cout << "uoc chung lon nhat la: " << m << endl;
   cout << "boi chung nho nhat la: " << (long long)a * b / m;
   return 0;
}
```

Câu 17.

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int ma, mb;
int main(){
   cin >> a >> b;
   ma = mb = 0;
   for (int i = 2; i <= a/2; i++ )
      if (a % i == 0)   ma  += i;
   for (int i  = 2; i <= b/2; i++)
      if (b % i == 0)   mb +=i;
   if (a == mb || b == ma) cout << a << " va " << b << " la hai so ban
be";
   else  cout << a << " va " << b << " khong phai la hai so ban be";
   return 0;
}
```

Câu 18.

```
#include <bits/stdc++.h>
using namespace std;
```

```
int i, n;
int main(){
  cin >> n;
  if (n == 1){
    cout << n;
    return 0;
  }
  i = 2;
  while (n != 1){
      while (n % i == 0){
        cout << i << " ";
        n = n / i;
      }
      i++;
  }
  return 0;
}
```

Câu 19.

```
#include <bits/stdc++.h>
using namespace std;
int n;
long long f, f1, f2 ;
int main(){
  cin >> n;
  if (n <= 2) f = 1;
  else{
    f1 = 1;
    f2 = 1;
    for (int i = 3; i <= n; i++){
      f = f1 + f2;
      f2 = f1;
      f1 = f;
    }
  }
  cout << f;
  return 0;
}
```

9. Kiểu mảng

- Mảng 1 chiều

Mảng là một dãy hữu hạn các phần tử có cùng kiểu được đặt liên tiếp trong bộ nhớ và có thể truy xuất đến từng phần tử bằng cách thêm một chỉ số vào sau tên của mảng.

+ Khai báo theo cú pháp sau:

Kiểu dữ liệu Tên biến mảng[số phần tử];

Ví dụ.

```
int a[10], b[20];
long long c[30];
char   ch[100];
bool kt[1000];
double r[20];
```

Chú ý: số phần tử bên trong cặp ngoặc [] phải là giá trị hằng khi khai báo mảng. Theo quy ước của C++ mảng được bắt đầu từ chỉ số 0. Ví dụ int a[10]; thì mảng a sẽ gồm 10 phần tử từ a[0] đến a[9] và khi khai báo như vậy nghĩa là phần tử a[10] sẽ không có trong mảng (đây là lỗi học sinh hay mắc phải khi lập trình)

+ Khởi tạo: ta có thể vừa khai báo và vừa khởi tạo mảng như ví dụ sau:

```
Cách 1:      int a[5] ={0, 1, 3, 0, 4};
Cách 2:      int b[] = {5, 1, 3, 4};
```

Đối với cách 1 thì mảng a có phần tử lần lượt là các giá trị trong {}, còn cách 2 thì mảng b có 4 phần từ lần lượt trong {}.

+ Truy xuất đến các phần tử của mảng

Cũng giống như pascal C++ dùng cặp [] để truy xuất các phần tử của mảng:

Tên_mảng[chỉ số]

Với khởi tạo ở cách 1: *a[0] = 0; a[1] = 1; a[2] = 3; a[3] = 0; a[4] = 4;*
ở cách 2 thì ta có *b[0] = 5; b[1] = 1; b[2] = 3; b[3] = 4;*

Nói chung giống như pascal, chú ý mỗi chỉ số mảng trong C++ được đánh từ vị trí 0; Cách tính độ lớn của dữ liệu chính là bằng số phần tử của mảng

*nhân với độ lớn kiểu dữ liệu. Ví dụ int a[20]; thì mảng a sẽ có độ lớn là 20*4byte= 80byte;*

Ví dụ 1. Nhập vào *N* số nguyên *a1, a2, ... aN* ($N<=10^5$; $|a_i| <= 10^9$). Hãy tính tổng của *N* số trên

```
#include <bits/stdc++.h>
#define  N 100002
using namespace std;
int a[N];
int n;
long long s = 0;
int main(){
  cin >> n;
  for (int i = 1; i <= n; i++){
    cin >> a[i];
    s = s + a[i];
  }
  cout << s;
  return 0;
}
```

Trong ví dụ trên ta có lệnh *#define N 100002 nghĩa là gán hằng số 100002 cho N*. Chú ý tất cả các lệnh có # ở đầu thì phía cuối lệnh không có dấu chấm phẩy (;). Nếu trong chương trình ta phải dùng đến *100002* nhiều lần, thay vì viết *100002* thì ta viết N, nếu cần thay đổi giá trị *100002* thì ta chỉ cần sửa 1 lần ở *#define N 100002*. Ngoài ra chúng ta có thể dùng khai báo hằng số *const int N = 100002;* vì nó là hằng số nên ta nên viết hoa, như ở ví dụ dưới.

Ví dụ 2. Nhập vào *N* số nguyên *a1, a2, ...aN*. Tìm số lớn nhất của *N* số trên.

```
#include <bits/stdc++.h>
const    int   N = 100002;
using namespace std;
int a[N];
int n;
long long res = 0;
int main(){
  cin >> n;
  for (int i = 1; i <= n; i++){
```

```
    cin >> a[i];
  }
  res = a[1];
  for (int i = 2; i <= n; i++)
    if (res < a[i])   res = a[i];
  cout << res;
  return 0;
}
```

Ví dụ 3. Nhập vào *N* số nguyên *a1, a2, ... aN (1 <= N <= 10^5; $|a_i|$ <= 10^9)* Hãy đếm số lượng số không âm của dãy số trên.

```
#include <bits/stdc++.h>
#define  N  100002
using namespace std;
int a[N];
int n;
int res;
int main(){
  cin >> n;
  for (int i = 1; i <= n; i++){
    cin >> a[i];
  }
  res = 0;
  for (int i = 1; i <= n; i++)
    if (a[i] >= 0)   res++;
  cout << res;
  return 0;
}
```

- Mảng 2 chiều: Tương tự cách khai báo của mảng 1 chiều, ta chỉ thêm 1 chiều nữa. Vì mảng 2 chiều là mảng 1 chiều của mảng 1 chiều. Các thao tác và truy xuất các phần tử của mảng tương tự như mảng 1 chiều.

Ví dụ. *int a[3][5];*
long long b[3][3];

Mảng 2 chiều được biểu diễn giống như ma trận. Kiểu như ta quan niệm chỉ số đầu là hàng, chỉ số sau là cột. Nhưng khi lưu trữ trên máy tính là các ô nhớ có địa chỉ liên tiếp nhau.

a[0][0]	*a[0][1]*	*a[0][2]*	*a[0][3]*	*a[0][4]*
a[1][0]	*a[1][1]*	*a[1][2]*	*a[1][3]*	*a[1][4]*
a[2][0]	*a[2][1]*	*a[2][2]*	*a[2][3]*	*a[2][4]*

Chúng ta có thể vừa khai báo vừa khởi tạo mảng 2 chiều theo cách như sau (ngoài ra còn một số cách khác):

```
int a[3][5] = {{1, 2, 3, 4, 5},
               {1, 3, 4, 2, 3},
               {2, 3, 2, 3, 2}};
```

Ví dụ 1. Nhập vào một ma trận gồm các số nguyên có *m* hàng và *n* cột; Hãy in ra ma trân vừa nhập (Các số trên một hàng thì được cách nhau bởi 1 dấu cách). Ma trận là một cái bảng có *m* hàng *n* cột.

```
#include <bits/stdc++.h>
#define  M 1002
#define  N  1002
using namespace std;
int a[M][N];
int m, n;
int main(){
  cout << "moi ban nhap 2 so m, n: ";
  cin >> m >> n;
  // Nhập ma trân A
  cout << "nhap ma tran A:" << endl;
  for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++){
      cout << "nhap a[" << i << "][" <<j << "]: ";
      cin >> a[i][j];
    }
  // In ra ma trân A
  cout << "In ra ma tran A" << endl;
  for (int i = 1; i <= m; i++){
    for (int j = 1; j <= n; j++)
      cout << a[i][j] << " ";
    cout << endl;
  }
  return 0;
}
```

Ví dụ 2. Cho 2 ma trận cấp *m*n*. Hãy tính tổng của 2 ma trận trên và in ra ma trận tổng đó. Ta có ma trân *A[m, n] + B[m, n] = C[m, n]* trong đó *C[i, j] = A[i, j] + B[i, j]*

```
#include <bits/stdc++.h>
#define  M 1002
using namespace std;
int a[M][M], b[M][M], c[M][M];
int m, n;
int main(){
  cout << "moi ban nhap 2 so m, n: ";
  cin >> m >> n;
  // Nhập ma trân A
  cout << "nhap ma tran A:" << endl;
  for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++){
      cout << "nhap a[" << i << "][" << j << "]: ";
      cin >> a[i][j];
    }
  // Nhập ma trận B
  cout << "nhap ma tran B:" << endl;
  for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++){
      cout << "nhap b[" << i << "][" << j << "]: ";
      cin >> b[i][j];
    }
  // Tính tổng 2 ma trận
  for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++)
      c[i][j] = a[i][j] + b[i][j];
  // In ra ma trân C
  cout << "In ra ma tran C" << endl;
  for (int i = 1; i <= m; i++){
    for (int j = 1; j <= n; j++)
      cout << c[i][j] << " ";
    cout << endl;
  }
  return 0;
}
```

Ví dụ 3. Cho ma trân *A* vuông cấp *n*. Tính tổng các phần từ nằm trên đường chéo chính của ma trận

Đường chéo chính là đường có các phần từ *A[1][1], A[2][2]... A[n][n]*

```
#include <bits/stdc++.h>
#define  N 1002
using namespace std;
int a[N][N];
int n;
long long res = 0;
int main(){
  cin >> n;
  // Nhập ma trân A
  cout << "nhap ma tran A:" << endl;
  for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++){
      cout << "nhap a[" << i << "][" << j << "]: ";
      cin >> a[i][j];
    }
  //Tinh tong cac phan tu duong cheo chinh
  for (int i = 1; i <= n; i++)
    res = res + a[i][i];
  cout << res;
  return 0;
}
```

- Tương tự mảng 2 chiều, chúng ta sẽ có mảng n chiều (n>=3). Ta chỉ cần thêm số chiều vào khai báo. Ví dụ:

```
int a[2][3][3]; //đây là mảng 3 chiều.
int b[4][2][3][3]; //đây là mảng 4 chiều….
```

Chú ý: Khi khai bảo mảng có số phần tử lớn hơn 10^5 phần tử thì ta phải khai báo ở phần khai báo biến toàn cục (ngoài hàm *main* và ngoài chương trình con nếu có). Vì khi khai báo mảng trong hàm *main* thì mảng đó được lưu trữ vào bộ nhớ *stack* (bộ nhớ *stack* có dung lượng thấp), nếu khai bảo mảng lớn trong hàm main thì *stack* sẽ bị tràn bộ nhớ dẫn tới chương trình không chạy được; còn khai báo ở biến toàn cục thì biến đó sẽ được lưu trữ ở vùng data,

vùng này được load lên khi chương trình khởi động, nên có thể khai báo mảng có số phần tử lớn hơn.

Bài tập

Câu 1. Hãy chuyển số tự nhiên N sang số nhị phân.

Câu 2. Hãy nhập vào 1 mảng gồm N số nguyên không âm. Hãy đưa ra số lượng số chẵn, số lượng số lẻ, số lẻ lớn nhất, số chẵn lớn nhất.

Câu 3. Cho dãy Số nguyên a1, a2, … aN. Hãy tìm một số x có trong dãy hay không. Nếu có hãy in ra vị trí cuối của X trong dãy.

Câu 4. Cho dãy số nguyên a1, a2, …aN. Hãy in ra dãy đã được sắp xếp không giảm.

Câu 5. Cho dãy số nguyên a1, a2, … aN. Cho một số nguyên K. Kiểm tra xem có tồn tại trong dãy 2 số ai và aj (i != j) sao cho trung bình cộng của 2 số này có bằng K hay không

Câu 6. Tính số fibonaci thứ N. biết f(1) = 1; f(2) = 1; f(N) = f(N-2) + F(N-1)

Câu 7: Cho dãy a gồm m số nguyên ($|a_i| <= 10^9$), dãy b gồm n số nguyên ($|b_i| <= 10^9$). 2 dãy này đã được sắp xếp không giảm. Hãy in ra một dãy c có các phần tử gồm 2 dãy số trên cũng được sắp xếp không giảm.

Câu 8. Cho dãy số gồm có N phần tử. Hãy đổi vị trí của phần tử lớn nhất và nhỏ nhất cho nhau. Nếu có nhiều phần tử lớn nhất và nhỏ nhất thì đổi chỗ phần tử lớn nhất có vị trí lớn nhất cho phần tử nhỏ nhất có vị trí nhỏ nhất cho nhau.

Câu 9. Cho dãy số a1, a2, … aN ($2<=N<=10^6$, $|a_i| <= 10^9$. Tìm số lượng cực tiểu địa phương. ai được gọi là cực tiểu địa phương nếu:

+ Nếu i = 1 thì a[1] < a[2];

+ Nếu i = N thì a[N] < a[N-1]

+ Ngược lại của 2 điều kiên trên thì a[i-1] > a[i] < a[i+1]

Câu 10. Tương tự câu 9, in ra các vị trị của cực đại địa phương theo thứ tự tăng dần nếu có.

Câu 11. Nhập vào số nguyên dương N (N<= 10^7), in ra tất cả các số nguyên tố từ 1 đến N.

Câu 12. Cho dãy số nguyên a1, a2, …aN (1<= N<= 10^6; 0<= a_i<=10^6) . Hãy tìm số có tần số xuất hiện lớn nhất trong dãy số trên, nếu có nhiều số thì hãy in ra số có giá trị lớn nhất.

Câu 13. Cho N điểm có tọa độ nguyên trên mặt phẳng tọa độ. Tìm khoảng cách nhỏ nhất của 2 điểm trong N điểm trên. In ra 3 phần thập phân

Câu 14. Cho bảng gồm m hàng và n cột. Trên mỗi ô của bảng là một số nguyên. Với mỗi hàng thì ta tính tổng các phần tử của mỗi hàng. Hãy in ra tổng lớn nhất của m hàng trên.

Câu 15. Nhập vào một bảng m hàng và n cột. Tính tổng các ô nằm ở viền ngoài của bảng

Câu 16. Cho 2 ma trân A[m, p], B[p, n] . Tính tích của hai ma trận trên

Hai ma trận nhân được với nhau khi số cột của ma trận này phải bằng số hàng của ma trận kia. Ta có tích của 2 ma trân trên là ma trận C[m][n]. với mọi C[i][j] được tính theo công thức sau: $C[i][j] = \sum_{k=1}^{p} A[i][k] * B[k][j]$

Gợi ý cách giải

Câu 1. Ở phần trước ta đã làm bài này rồi, nhưng chỉ lưu được số nhị phân tầm 19 đến 20 chữ số. Khi học đến mảng, ta dùng mảng để lưu thì ta có thể làm được với số nhị phân có độ dài lớn hơn nhiều.

```
#include <bits/stdc++.h>
const int  N = 1E6;
using namespace std;
int a[N];
int n, i;
int main(){
  cin >> n;
  if (n == 0){
    cout << 0;
    return 0;
  }
  i = 0;
  while (n != 0){
    i++;
    a[i] = n % 2;
    n = n / 2;
  }
  for (int j = i; j >=1; j--)
    cout << a[j];
  return 0;
}
```

Ở bài trên ta dùng *const int N = 1E6;* Nếu ta dùng *#define N 1E6* thì chương trình sẽ báo lỗi. Vì *1E6* là số thực nên không dùng được trong khai báo số phần tử của mảng. Ở cách dùng *const, 1E6* là số thực được gán cho *N*, như *N* là kiểu int tên kết quả *N = 1000000;* (số nguyên).

Câu 2. Ta dùng một biến đếm số lượng số chẵn, khởi tạo 2 biến để lưu số lẻ lớn nhất và số chẵn lớn nhất (giá trị khởi tạo là -1). Duyệt tất cả các phần tử của mảng. Nếu nó là số lẻ thì tăng đếm lẻ lên 1 và so sánh để tìm số lẻ lớn nhất. Ngược lại thì ta so sánh để tìm số chẵn lớn nhất. Kết quả in ra được số lượng số lẻ; số lượng số chẵn = N – số lượng số lẻ; nếu số chẵn lớn nhất hoặc

số lẻ lớn nhất bằng -1 nghĩa là dãy đó không có số chẵn lớn nhất hoặc số lẻ lớn nhất.

Câu 3. Duyệt từ cuối mảng về đầu, nếu gặp số nào bằng X thì in ra vị trí và thoát khỏi vòng lặp.

Câu 4. Có thể dùng cách sắp xếp đơn giản sau:

```
for (int i =1 ; i < n; i++)
    for (int j = i+1; j <= n; j++)
        if (a[i] > a[j]) swap(a[i], a[j]);
```

Lấy phần tử thứ nhất so sánh với tất cả phần tử phía sau, lấy phần tử thứ 2 so sánh với tất cả phần tử phía sau, … lấy phần tử cuối -1 so sánh với phần tử cuối. Ngoài thuật toán này ta có thể học thêm thuật toán sắp xếp nổi bọt... Các thuật toán sắp xếp ở trên đều có độ phức tạp là $O(n^2)$. Hoặc chúng ta có thể dùng *sort* trong thư viện có sẵn được trình bày như ở gần cuối tài liệu có độ phức tạp là *O(nlogn)*

Câu 5. Ta duyệt tất cả trường hợp (i , j). Với mỗi trường hợp ta kiểm tra (a[i] + a[j]) *2 == K hay không (mình nhân 2 lên thì đỡ phải biện luận cách dùng /2). Nếu có thì thông báo và thoát luôn khỏi vòng lặp. Nếu duyệt hết vòng lặp mà không có thì thông báo không có.

Câu 6. Khai báo mảng *f* là mảng chứa các số fibonacci

```
f[1] = f[2] = 1;
for (int i = 3; i <= n; i++) f[i] = f[i-2] + f[i-1];
```

Câu 7. Ta thêm vào phía cuối của 2 mảng a và b một số lớn hơn tất cả các số của 2 mảng. Khi lấy đủ m+n phần tử vào mảng C thì chắc chắn số thêm vào cuối của 2 mảng sẽ không có trong mảng C.

```
#include <bits/stdc++.h>
const int   N = 1E6+2;
using namespace std;
int a[N+1], b[N+1], c[2*N];
int m, n, i, j;
int main(){
  cin  >> m ;
  for (int i = 1; i <= m; i++)
```

```
    cin >> a[i];
  cin >> n;
  for (int i = 1; i <= n; i++)
    cin >> b[i];
  a[m+1] = b[n+1] = 1E9+1;
  i = j = 1;
  int k = 0; int t = m+n;
  while (k <= t){
    k++;
    if (a[i] < b[j]){
      c[k] = a[i];
      i++;
    }
    else {
      c[k] = b[j];
      j++;
    }
  }
  // In ra mảng C
  for (int i = 1; i <= t; i++)  cout << c[i]<< " ";
  return 0;
}
```

Câu 8.

```
#include <bits/stdc++.h>
const int   N = 1E6+2;
using namespace std;
int a[N];
int n;
int maxr, minl, vtmin, vtmax;
int main(){
  cin  >> n ;
  for (int i = 1; i <= n; i++)
    cin >> a[i];
  maxr = minl = a[1];
  vtmax = vtmin = 1;
  for (int i = 2; i <= n; i++){
    if (maxr <= a[i]){
```

```
            maxr = a[i];
            vtmax = i;
        }
        if (minl > a[i]){
            minl = a[i];
            vtmin = i;
        }
    }
    swap(a[vtmax], a[vtmin]);
    // in ra mảng a sau khi đổi chỗ
    for (int i = 1; i <= n; i++)
        cout << a[i]<< " ";
    return 0;
}
```

Câu 9.

```
#include <bits/stdc++.h>
const int   N = 1E6+2;
using namespace std;
int a[N];
int n;
int res = 0;
int main(){
    cin  >> n ;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    if (a[1]<a[2]) res++;
    if (a[n-1]>a[n]) res++;
    for (int i = 2; i <= n-1; i++)
        if (a[i]<a[i-1] && a[i]<a[i+1])
            res++;
    cout << res;
    return 0;
}
```

Câu 10.

```
#include <bits/stdc++.h>
const int   N = 1E6+2;
```

```
using namespace std;
int a[N];
int n;
int main(){
  cin  >> n ;
  for (int i = 1; i <= n; i++)
    cin >> a[i];
  if (a[1] > a[2]) cout << 1 <<" ";
  for (int i = 2; i <= n-1; i++)
    if (a[i]>a[i-1] && a[i]>a[i+1])
      cout << i << " ";
  if (a[n-1] < a[n]) cout << n;
  return 0;
}
```

Câu 11. Bài này nếu ta làm theo cách kiểm tra một số X có phải nguyên tố hay không như cách làm ở phía trên, ta duyệt i từ 2 đến N nếu kiểm tra i là số nguyên tố thì ta in ra i. Độ phức tạp của thuật toán là $O(N*\sqrt{N})$ Như vậy với N <= 10^7 thì nó sẽ bị quá thời gian ở những trường hợp N lớn. Ở bài này ta dùng thuật toán sàng nguyên tố: đại loại như hiểu như thế này: coi giả sử tất cả các số từ 1 đến N đều là số nguyên tố, giờ chúng ta dùng cái sàng để sàng dần những số không phải là nguyên tố. Những số không phải là số nguyên tố là nó bằng tích của 2 số i*j (2 <= i, j) Sau khi sàng xong, những số còn lại trên cái sàng là số nguyên tố. Để làm được điều này chúng ta dùng một mảng để đánh dấu. Độ phức tạp của thuật toán sàng nguyên tố là O(NlogN)

```
#include <bits/stdc++.h>
const int   N = 1E7+2;
using namespace std;
int kt[N];
int n;
int main(){
  cin  >> n ;
  // Khởi tạo tất cả từ 2 đến n đều là số nguyên tố
  for (int i = 2; i <= n; i++)
       kt[i] = true;
  // sàng nguyên tố
  for (int i = 2; i <= sqrt(n); i++)
```

```
    if (kt[i])
      for (int j = 2; j <= n/i; j++)
        kt[i*j] = false;
  // in ra các số nguyên tố
  for (int i = 2; i <= n; i++)
    if (kt[i])  cout << i << " ";
  return 0;
}
```

Ta chỉ cần cho *j* chạy đến *n/i* để tích *i*j* đạt lớn nhất bằng *n* (vì đề bài chỉ cần in số nguyên tố <= *n*). Để tối ưu hơn, thay vì ta kiểm tra *i <= sqrt (n)* thì ta kiểm tra *i*i <= n*; (vì phép nhân thực hiện nhanh hơn phép lấy căn. Nhận xét tiếp theo ta thấy ở chỗ *kt[i*j] = false;* thì khi *j =2* thì đó là kt[2*i], khi j = 3 thì đó là kt[3*i], khi j = 4 thì đó là kt[4*j]… Thay vì phép nhân ta chuyển sang phép cộng như code dưới đây (vì phép cộng thực hiện nhanh hơn phép nhân):

```
#include <bits/stdc++.h>
const int   N = 1E7+2;
using namespace std;
int kt[N];
int n;
int main(){
  cin  >> n ;
  // Khởi tạo tất cả từ 2 đến n đều là số nguyên tố
  for (int i = 2; i <= n; i++)
       kt[i] = true;
  // sàng nguyên tố
  for (int i = 2; i * i <= n; i++)
    if (kt[i])
      for (int j = 2 * i; j <= n; j = j +i)
        kt[j] = false;
  // in ra các số nguyên tố
  for (int i = 2; i <= n; i++)
    if (kt[i])  cout << i << " ";
  return 0;
}
```

Câu 12. Ta dùng tương tự như thuật toán đếm phân phối vì các giá trị của a_i nằm trong phạm vi [0, 10^6];

```
#include <bits/stdc++.h>
const int   N = 1E6+5;
using namespace std;
int a[N];
int   dem[N];
int n, k, res;
int main(){
  cin  >> n;
  for (int i = 1; i <= n; i++)
    cin >> a[i];
  memset(dem, 0, sizeof(dem)); // khởi tạo tất cả phần tử mảng dem bằng 0
  for (int i = 1; i <= n; i++ )
    dem[a[i]]++;
  res = 0;
  for (int i = 0; i <= 1E6; i++)
    if (res <= dem[i]) {
        res = dem[i];
        k = i;
    }
  cout << k;
  return 0;
}
```

Cũng với bài này nhưng điều kiện của a_i thay đổi thành $|a_i|<=10^6$. Như vậy nếu để nguyên cách làm trên thì sẽ không làm được. vì lý do trong C++ mảng không có chỉ số âm (trong pascal ta khai báo được chỉ số âm). Để làm được điều này ta sẽ biến tất cả các phần tử của mảng *a* thành số không âm (Cộng tất cả phần tử của một mảng với một số để cho mảng đó không còn số âm, như bài này ta cộng với 10^6 hoặc cộng với trị tuyệt đối của số nhỏ nhất của mảng *a*). Rồi làm tương tự cách trên và khai báo tăng thêm kích thước của mảng *dem*, đến lúc in ra kết quả thì ta trừ đi số lúc cộng vào.

```
#include <bits/stdc++.h>
const int   N = 2E6+5;
using namespace std;
int a[N];
```

```
int   dem[N];
int n, k, res;
int main(){
  cin  >> n;
  for (int i = 1; i <= n; i++){
    cin >> a[i];
    a[i] += 1E6; //cộng thêm vào a[i] với 1000000
  }
  memset(dem, 0, sizeof(dem)); // khởi tạo tất cả phần tử của mảng bằng 0
  for (int i = 1; i <= n; i++ )
    dem[a[i]]++;
  res = 0;
  for (int i = 0; i <= 2E6; i++)
    if (res <= dem[i]) {
        res = dem[i];
        k = i;
    }
  k = k - 1E6;
  cout <<  k;
  return 0;
}
```

Câu 13. Chúng ta phải dùng 2 mảng để lưu tọa độ của N điểm. Chẳng hạn mảng x để lưu hoành độ, mảng y để lưu tung độ. Với ý nghĩa x[1] và y[1] là tọa độ điểm 1; x[2], y[2] là tọa độ điểm thứ 2….x[n], y[n] là tọa độ điểm thứ n. Duyệt tất cả các cặp điểm để tìm khoảng cách nhỏ nhất. Công thức tính khoảng cách giữa 2 điểm là *sqrt ((x2-x1)*(x2-x1) + (y2-y1)*(y2-y1))*

```
#include <bits/stdc++.h>
#define  N 1002
#define oo 1000000 // oo là một số đủ lớn để nó luôn >=kết quả.
using namespace std;
int x[N], y[N];
int m, n, t;
int kc, res = oo;
int main(){
  // Nhap n
  cin  >> n;
  // Nhập N điểm
```

```
    for (int i = 1; i <= n; i++){
       cin >> x[i];
       cin >> y[i];
    }
    for (int i = 1; i < n; i++)
       for (int j = i+1; j <= n; j++){
            kc = (x[j]-x[i])*(x[j]-x[i]) + (y[j]-y[i])*(y[j]-y[i]);
            if (res < kc)  res = kc;
       }
    // in ra khoảng cách
    cout << fixed << setprecision(3) << sqrt (res);
    return 0;
}
```

Ở cách làm này thì thay vì chúng ta so sánh dùng giá trị lấy căn bậc 2 thì ta không cần, đến lúc in ra kết qua ta mới lấy căn. Ta dùng thêm biến *kc* để lưu khoảng cách để đỡ phải tính lại khoảng cách đó. Ta dùng res là kiểu nguyên, vì trong lập trình kiểu nguyên nó thực hiện phép tính nhanh hơn kiểu số thực, tùy vào điều kiện đề bài mà ta khai báo biến, cẩn thận trường hợp bị tràn số.

Câu 14. Duyệt tất cả các hàng, với mỗi hàng ta tính tổng để tìm ra kết quả;

```
    res = 0;
    // Tính tổng hàng 1;
    for (int j = 1; j<= n; j++)
       res = res + a[1][j];
    // Tính tổng các hàng còn lại so sánh với res
    for (int i = 2; i <= m; i++){
       t = 0;
       for (int j = 1; j<= n; j++)
          t = t + a[i][j];
       if (res < t)   res = t;
    }
    cout << res;
```

Tương tự các thầy cô có thể ra thêm bài tập tìm cột có tổng lớn nhất.

Câu 15.

```
 res = 0;
  for( int i =1; i <= m; i++ )
```

```
    res = res + a[i][1] + a[i][n];
  for (int j = 1; j <= n; j++)
    res = res + a[1][j] + a[m][j];
  cout << res;
```

Câu 16. Sử dụng công thức: $C[i][j] = \sum_{k=1}^{p} A[i][k] * B[k][j]$

```
#include <bits/stdc++.h>
#define  M 1002
using namespace std;
int a[M][M], b[M][M], c[M][M];
int m, n, p;
int k, t;
int main(){
  cout << "moi ban nhap 2 so m, n, k: ";
  cin >> m >> n >> p;
   // Nhập ma trân A[m,p]
  cout << "nhap ma tran A:" << endl;
  for (int i = 1; i <= m; i++)
    for (int j = 1; j <= p; j++){
      cout << "nhap a[" << i << "][" << j << "]: ";
      cin >> a[i][j];
    }
  // Nhập ma trận B[p, n]
  cout << "nhap ma tran B:" << endl;
  for (int i = 1; i <= p; i++)
    for (int j = 1; j <= n; j++){
      cout << "nhap b[" << i << "][" << j << "]: ";
      cin >> b[i][j];
    }
  // Tính tích 2 ma trận
  for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++){
      t = 0;
      for (int k = 1; k <= p; k++)
        t = t + a[i][k] * b[k][j];
      c[i][j] = t;
    }
```

```
    // In ra ma trân C[m, n]
    cout << "In ra ma tran C" << endl;
    for (int i = 1; i <= m; i++){
      for (int j = 1; j <= n; j++)
        cout << c[i][j] << " ";
      cout << endl;
    }
    return 0;
}
```

Ở phần tính tích ở trên ta dùng 1 biến t để lưu kết quả, tính xong t rồi thì ra mới trả kết quả cho *C[i][j]* (làm như vậy để không phải truy xuất *C[i][j]* nhiều lần => thời gian sẽ chạy nhanh hơn).

10. Kiểu xâu

Ta có 2 cách biểu diễn:

- Biểu diễn bằng mảng của ký tự.

Ví dụ:

```
char st[100];
```

Được hiểu là mảng st có tối đa là 100 phần tử mà mỗi phần tử là kiểu kí tự. Và chúng ta làm việc và thao tác với nó như mảng một chiều đã học phía trên.

- Biểu diễn bằng kiểu xâu (string)

+ Khai báo *string tên_biến_xâu;*

Ví dụ:

```
string st1, st2;
string st[100]; // Mảng kiểu xâu
```

Ta có thể vừa khai báo vừa khởi tạo như những kiểu khác.

```
string st1 = "", st2 = "Thay phu day c++";
```

Ta luôn chú ý hằng xâu luôn được đặt trong cặp dấu nháy kép *""* còn ký tự đặt trong cặp dấu nháy đơn *''*; Ta cũng có thể gán xâu như sau: *st1 = st2;* Nghĩa là gán xâu st2 cho xâu st1

+ Nhập, xuất xâu:

Xuất xâu: thi ta dùng *cout* như lâu nay đối với các kiểu dữ liệu khác

Ví dụ. *cout << "C++ that gian don" << '\n';*
cout << st; // Trong đó st là biến xâu chẳng hạn

Nhập xâu thì ta có 2 cách:

+ Dùng cú pháp *cin: cin >> biến_xâu;*// Đối với cách nhập này nó chỉ nhập được xâu từ đầu đến trước vị trí dấu cách trống đầu tiên. Nên khi dùng *cin* để nhập xâu, thì xâu đó không được chứa dấu cách hoặc chứa dấu cách ở cuối xâu thì ta nên dùng cách này. Vì nếu xâu chứa dấu cách thì nó sẽ không nhập được tất cả vị trí sau dấu cách.

Ví dụ.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int main(){
   cin >> st;
   cout << st;
   return 0;
}
```

Ở ví dụ trên: nếu chạy chương trình ta nhập xâu: *C++ that don gian* thì nó chỉ in ra màn hình xâu: *C++* . Nếu ta nhập xâu: *C++thatdongian* thì nó in được cả xâu đó. Vì trong xâu đó không có dấu cách.

+ Cách dùng *getline*: *getline(cin, biến_xâu);*

Ví dụ.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int main(){
   getline(cin, st);
   cout << st;
   return 0;
}
```

Ở ***ví dụ*** trên thì khi nhập st có dấu cách hay không có thì nó đều nhập được. Nhưng nếu nhập xâu không có dấu cách thì ta ưu tiên nên dùng cách 1 (vì thời giản xử lý nó nhanh hơn dùng *getline*)

+ **Các phép toán so sánh xâu**: *==, !=, <, <=, >, >=,* được thực hiện so sánh theo quy tắc sau:

+ Hai xâu bằng nhau nếu như chúng giống nhau hoàn toàn.

+ Xâu *st1* lớn hơn xâu *st2* nếu như kí tự đầu tiên khác nhau giữa chúng kể từ trái qua phải trong xâu *st1* có mã ASCII lớn hơn.

+ Nếu xâu *st1* và xâu *st2* là các xâu có độ dài khác nhau và *st1* là đoạn đầu của *st2* thì *st1 < st2*.

Ví dụ. ta có các phép sau: *"abcd" < "abcd1"; "0234" < "a12"; "ab" == "ab"; "bcd" > "abcddd"....*

- Các phép thao tác, phương thức xâu:

+ Lấy chỉ số: dùng cặp ngoạc vuông *[chỉ số]* như kiểu mảng. Ví dụ *st[0], st[1]...*(ta chú ý, chỉ số trong xâu được đánh từ 0).

+ Lấy độ dài xâu: *st.length(); hoặc st.size();* (dùng cái nào cũng được)

Ví dụ. *st = "123456"; cout << st.length(); // in ra số 6;*

Chú ý trong lập trình viết như sau sẽ báo lỗi:

st ="123456"; st.length(); // vì st.length() là có giá trị nên phải gán cho biến nào đó hoặc dùng *cout* in ra.

+ Phép ghép xâu: ta dùng dấu + để ghép xâu. Chúng ta có thể ghép 2 xâu hoặc ghép ký tự vào xâu.

Ví dụ. *string st = 'a';*// ký tự a được gán cho xâu, lúc này *st* là xâu *"a"*

st1 = "hoc c++"; st2 = " that gian don"; st1 = st1 + st2;

Lúc này *st1* sẽ là xâu: *hoc c++ that gian don*

Chú ý ta không được viết như sau:

st1 = "hoc c++" + " that gian don"; // báo lỗi

Muốn cộng xâu như vậy ta làm như sau:

st1 = “”;
st1 = st1 + “hoc c++” + “ that gian don”;

+ Phép chèn xâu: *st1.insert(vt, st2);* // Ý nghĩa chèn xâu *st2* vào xâu *st1* bắt đầu từ vị trí *vt* (vị trí trong xâu được tính bắt đầu từ 0), như vậy *st1* sẽ bị thay đổi còn *st2* vẫn giữ nguyên giá trị.

Ví dụ. st1 = “abcacd”; st2 = “12”; st1.insert(2, st2); // giá trị st1= “ab12cacd”

st1 = “abcacd”; st1.insert(0, “12”); // giá trị st1 = “12abcacd”

Cách viết sai: *st1.insert(0,’1’);* lỗi vì không đúng kiểu, *‘1’* là kí tự.

Viết như sau thì đúng: *st1.insert(0,”1”);// vì “1” là xâu*

+ Xóa xâu: *st.erase(vt, n);* // xóa đi *n* kí tự của xâu *st* bắt đầu từ vị trí *vt*, kết quả xâu *st* sẽ bị thay đổi. (chú ý trong xâu vị trí được bắt đầu từ 0).

Ví dụ. *st = “toi_di_hoc_c++”; st.erase (3,4);* // Xóa 4 kí tự của xâu st bắt đầu từ vị trí 3, kết quả xâu *st = “toihoc_c++”*

+ Đổi chỗ 2 xâu: *st1.swap (st2);* // kết quả sẽ đổi chỗ giá trị của 2 xâu này; thực chất phương thức này thì ta có thể dùng thêm 1 biến *tg* kiểu xâu để đổi chỗ như sau:

tg = st1;
st1 = st2;
st2 = tg;

+ Lấy xâu con: *st.substr(vt, n)* là trích ra xâu con *n* kí tự bắt đầu từ vị trí *vt* của xâu *st*. Như vậy xâu *st* vẫn giữ nguyên giá trị.

Ví dụ. *st1 = “toidihoc”; st2 = st1.substr (3,2); // st2 sẽ là xâu “di”*

+ Tìm kiếm: *st1.find(st2)* tìm vị trí đầu tiên của xâu hoặc kí tự *st2* trong xâu *st1*. Nếu không có thì kết quả trả về giá trị lớn nhất của kiểu *unsigned long long* nếu máy tính *64 bit* thì kết quả trả về là 2^{64}-1. (chú ý trong C++ vị trí của xâu bắt đầu từ 0).

Ví dụ.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n;
```

```
int main(){
        st = "112341236";
        n = st.find("123");
        cout << n;// in ra 1
        n = st.find('7');
        cout << n; // in ra -1
        cout << (int)(st.find("13"); // in ra -1
        cout << (int)(st.find('1'); // in ra 0
        return 0;
}
```

Ở ***ví dụ*** trên thì nếu ta in *cout << (st.find("13");* thì kết quả sẽ ra số 2^{64}*-1*. Còn *cout << (int)(st.find("13");* thì kết quả in ra *-1* vì chúng ta đang ép kiểu *unsign long long* thành kiểu *int*, trong kiểu int không có số 2^{64}*-1* nên khi ép kiểu thì số này là số *-1*. Nên việc so sánh số *2* số nguyên: 2^{64}*-1* và *-1* thì *2* số này bằng nhau.

+ Thay thế: *st1.replace(vt, n, st2);* // thay thế *n* kí tự của xâu *st1* bắt đầu từ vị trí *vt* bằng xâu *st2*. Kết quả *st2* không bị thay đổi, *st1* bị thay đổi.

Ví dụ. *st1= "abcde"; st1.replace(2, 1, "111"); // ta được st1="ab111de"*

Ngoài ra chúng ta còn một số phương thức khác, tất cả các phương thức ở trên chúng ta có thể tự viết để sử dụng được, nếu như không nhớ.

Bài tập

Câu 1. Nhập vào một xâu st. Hãy xóa tất cả ký tự 'a' hoặc 'A' trong xâu, và in kết quả sau khi xóa.

Câu 2. Nhập vào một xâu gồm các chữ cái latin. Hãy đếm xem trong xâu có bao nhiêu kí tự 'a' (không phân biệt chữ cái in hoa và in thường)

Câu 3. Nhập vào một xâu st gồm các ký tự chữ cái và chữ số. Hãy đếm trong xâu có bao nhiêu chữ số.

Câu 4. Nhập vào một xâu st. Kiểm tra xem xâu đó có đối xứng hay không (không phân biệt kí tự in hoa hay in thường).

Câu 5. Nhập vào xâu chỉ gồm các chữ cái *'a'* đến *'z'* hoặc *'A'* đến *'Z'*. Hãy biến đổi xâu đó thành xâu có tất cả kí tự in thường

Câu 6. Nhập vào một xâu chỉ gồm các kí tự ngoặc (và). Xâu s xác định một cách đặt ngoặc đúng, nếu thỏa mãn các điều kiện:

+ Số ngoặc mở bằng số ngoặc đóng

+ Nếu duyệt từ trái sang phải, số lượng ngoặc mở luôn luôn lớn hơn hoặc bằng số ngoặc đóng

Ví dụ: xâu "(())(()()(()))" xác định một cách đặt ngoặc đúng. Xâu "()()()))" là một cách đặt ngoặc sai.

Câu 7. Nhập vào 1 xâu st chỉ chứa các kí tự latin in thường và các kí tự số từ '0' đến '9'. Đếm xem trong xâu đó có bao nhiêu loại kí tự khác nhau.

Ví dụ: st="abcaccac112"; số kí tự khác nhau là 5

Câu 8. Nhập vào một xâu st bất kỳ, hãy chuẩn hóa xâu:

+ Xóa tất cả dấu cách ở đầu xâu nếu có;

+ Xóa tất cả dấu cách ở cuối xâu nếu có;

+ Giữa các từ nếu có nhiều dấu cách thì xóa đi sao cho chỉ còn 1 dấu cách

Câu 9. Nhập vào một xâu st gồm các từ, giữa các từ được cách nhau bởi 1 dấu cách hay nhiều dấu cách. Hãy đếm xem xâu đó có bao nhiêu từ

Câu 10. Nhập vào một xâu st gồm các chữ cái latin. Hãy biến tất cả kí tự in hoa thành in thường và tất cả kí tự in thường thành kí tự in hoa

Câu 11. Nhập vào một xâu gồm một câu, các từ được cách nhau bởi 1 dấu cách, đầu câu và cuối câu không có dấu cách. Hãy viết chương trình in ra câu đảo ngược của câu đó như ví dụ sau: "toi dang hoc c++" được đảo ngược thành "c++ hoc dang toi"

Câu 12. Nhập vào một xâu st dạng số nguyên không âm. Hãy chuyển xâu này thành số.

Câu 13. Nhập vào một số tự nhiên N. Hãy chuyển số này thành xâu

Câu 14. Nhập vào một số nguyên dương n không quá 100 chữ số. Hãy xác định xem số đó có chia hết cho 3 hay không?

Câu 15. Nhập vào một số nguyên dương n. Hãy chuyển số này sang số hexa

Câu 16. Người ta xâu N viên đá quý kích thước giống nhau thành một vòng đeo cổ (5<= N <=120), mỗi viên có một màu trong số các màu được đánh số từ 1 đến 9. Khi đó vòng được biểu diễn bởi một xâu S gồm N kí tự trong tập ['1'..'9']. Để tăng tính độc đáo cho vòng trang sức quý này, người ta định lắp khóa đep vào vị trí sao cho khi mở vòng ra được một dây đá quý có tính chất: không phụ vào việc cầm đầu dây nào bên tay phải và đầu kia bên tay trái, ta đều được một chuỗi hạt giống nhau, tức là viên đá thứ i từ trái sang luôn có màu j không phụ thuộc vào cách cầm.

Ví dụ xâu S: 222222335533

Thì ta có 2 cách cách đặt khóa như sau:

+ 222335533222

+ 533222222335

Gợi ý cách giải

Câu 1. Thay vì đi xóa tất cả thì ta lưu ra xâu khác những kí tự khác kí tự ‘a’ và ‘A’. Độ phức tạp thuật toán là O(độ dài của xâu)

```
#include <bits/stdc++.h>
using namespace std;
string st;
string st1 = "";
int n;
int main(){
  getline(cin, st);
  n = st.length();
  for (int i = 0; i < n; i++)
    if (st[i] != 'a' && st[i] != 'A')
      st1 = st1 + st[i];
  cout << st1;
  return 0;
}
```

Câu 2. Khởi tạo biến dem = 0. Duyệt từ đầu xâu đến cuối xâu, nếu gặp kí tự nào là ‘a’ hoặc ‘A’ thì tăng dem lên 1; Vì bài này cho xâu không có dấu cách nên ta dùng *cin* nhập được.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n, dem = 0;
int main(){
  cin >> st;
  n = st.length();
  for (int i = 0; i < n; i++)
    if (st[i] == 'a' || st[i] == 'A')
      dem++;
  cout << dem;
  return 0;
}
```

Câu 3. Duyệt từ đầu xâu đến cuối xâu, nếu kí tự đó >= ‘0’ và <= ‘9’ thì tăng kết quả lên 1;

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n, dem = 0;
int main(){
  cin >> st;
  n = st.length();
  for (int i = 0; i < n; i++)
    if (st[i] >= '0' && st[i] <= '9')
      dem++;
  cout << dem;
  return 0;
}
```

Câu 4. Biến xâu st thành xâu có tất cả các kí tự in hoa (trong bảng mã ASCII kí tự in thường lớn hơn kí tự in hoa là 32, giả sử xâu có độ dài là n. Vì xâu được đánh chỉ số từ 0. Nên cho i chạy từ 0 đến (n-2)/2 ta đi so sánh st[i] với st[n-1-i] nếu khác nhau thì kết luận luôn xâu không đối xứng. Hoặc dùng 2 biến i, j và vòng lặp while… cho chạy từ 2 đầu vào.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n;
int main(){
  getline (cin, st);
  n = st.length();
  // Chuyển kí tự in thường thành in hoa
  for (int i = 0; i < n; i++)
    if (st[i] >= 'a' && st[i] <= 'z')
      st[i] = st[i] - 32;
  // Kiểm tra tính đối xứng
  for (int i = 0; i<= (n-2)/2; i++ )
  if (st[i] != st[n-1-i]){
    cout << "st khong phai xau doi xung";
    return 0;
  }
  cout << "st la xau doi xung";
  return 0;
}
```

Cách dùng while..

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n, i, j;
int main(){
  getline (cin, st);
  n = st.length();
  // Chuyển kí tự in thường thành in hoa
  for (int i = 0; i < n; i++)
    if (st[i] >= 'a' && st[i] <= 'z')
      st[i] = st[i] - 32;
  // Kiểm tra tính đối xứng
  i = 0; j = n-1;
  while (i < j){
    if (st[i] != st[j]){
      cout << "st khong phai la xau doi xung";
      return 0;
    }
    i++; j--;
  }
  cout << "st la xau doi xung";
  return 0;
}
```

Câu 5. Duyệt từ đầu xâu đến cuối xâu, nếu gặp ký tự nào in hoa thì đổi nó thành in thường (ký tự in hoa cộng thêm 32 ra ký tự in thường).

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n, i, j;
int main(){
  cin >> st;
  n = st.length();
  for (int i = 0; i < n; i++)
    if (st[i] >= 'A' && st[i]<= 'Z')
      st[i] = st[i] +32;
  cout << st;
```

```
    return 0;
}
```

Câu 6. Ta có nhận xét, nếu duyệt từ trái qua phải: số cặp ngoặc đóng mà lớn hơn số cặp ngoặc mở thì xâu ngoặc này chắc chắn sai. Dùng 1 biến *t* để kiểm tra, ta khởi tạo *t = 0*; Duyệt từ đầu xâu đến cuối xâu: nếu gặp dấu mở ngoặc thì ta tăng *t* lên *1*, nếu gặp dấu đóng ngoặc thì ta giảm *t* đi *1*. Trong khi đang duyệt nếu *t < 0* thì kết luận luôn dãy ngoặc đó là không đúng. Nếu duyệt hết xâu mà *t > 0* thì thì dãy ngoặc đó cũng không đúng. Ngược lại nghĩa là *t = 0* thì là dãy ngoặc đúng.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n, t;
bool kt;
int main(){
  cin >> st;
  n = st.length();
  t = 0;
  kt = true;
  for (int i = 0; i < n; i++){
    if (st[i] == '(') t++;
    else t--;
    if (t < 0){
      kt = false;
      break;
    }
  }
  if (t >0) kt = false;
  if (t == 0) cout << "day ngoac la dung";
  else cout << "day ngoac la sai";
  return 0;
}
```

Câu 7. Đối với mảng của C++ thì kiểu chỉ số chỉ có thể là số nguyên từ 0, không có kiểu ký tự. Ta đã biết ý tự ‘0’ trong bảng mã ASCII là 48 và kí tự ‘z’ là 122. Như vậy từ 48 đến 122 sẽ bao gồm toàn bộ các ký tự trong xâu. Còn nếu ta quên bảng mã ASCII thì ta dùng câu lệnh sau để in ra bảng mã

của kí tự đó: *cout << (int)'0'; hoặc* cout << (int) *'a'*; hoặc *cout << (int) 'z';* Ta dùng thuật toán đếm phân phối như sau:

```
#include <bits/stdc++.h>
using namespace std;
string st;
int dem [124];
int n, res = 0;
int main(){
  cin >> st;
  n = st.length();
  // Gán tất cả các phần tử của mang dem bằng 0
  memset(dem, 0, sizeof(dem));
  // Đếm phân phối
  for (int i = 0; i <= n-1; i++ )  dem[(int) st[i]] ++;
  // Tính số kí tự khác nhau
  for (int i = 48; i <= 122; i++)
    if (dem[i] != 0)  res++;
  cout << res;
  return 0;
}
```

Câu 8.

```
#include <bits/stdc++.h>
using namespace std;
string st, st1;
int n;
int main(){
  getline(cin, st);
  n = st.length();
  // Thêm dấu cách vào đầu xâu và cuối xâu, chú ý độ dài xâu tăng thêm 2
  st = ' ' + st + ' ';
  // St1 là xâu chuẩn hóa của xâu st mà cuối xâu chứa 1 dấu cách
  st1 = "";
  for (int i = 1; i <= n; i++)
    if  ((st[i] != ' ') || (st[i] == ' ' && st[i-1] != ' '))
        st1 = st1 + st[i];
  n = st1.length();
```

```
    // Xóa 1 dấu cách ở cuối xâu nếu có
    if (st1[n-1] == ' ') st1.erase (n-1, 1);
    // in ra xâu sau khi chuẩn hóa
    cout << st1;
    return 0;
}
```

Câu 9.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n, res = 0;
int main(){
    getline(cin, st);
    n = st.length();
    // Thêm 1 dấu cách vào cuối xâu
    st = st + ' ';
    for (int i = 0; i < n; i++)
        if (st[i] != ' ' && st[i+1] == ' ')
            res ++;
    cout << res;
    return 0;
}
```

Câu 10. Duyệt từ đầu xâu dến cuối xâu, nếu gặp kí tự in hoa nghĩa là >= 'A' và <= 'Z' thì ta cộng nó với 32 và gán lại cho nó, nếu là in thường thì ta lấy nó - 32 rồi gán lại cho kí tự đó. Tương tự như câu phía trên.

Câu 11. Khai báo thêm 1 mảng kiểu xâu để lưu các từ. Thêm dấu cách ở cuối xâu. Duyệt từ đầu xâu đến cuối xâu -1 để ta tách các từ ra mảng.

```
#include <bits/stdc++.h>
#define N 100002
using namespace std;
string st;
string a[N];
int n, j;
```

```
int main(){
  getline(cin, st);
  n = st.length();
  //Gán xâu rỗng cho tất cả phần tử của mảng a
  for (int i =1; i <= 1E5; i++)
    a[i] = "";
  // Tách các từ ra mảng
  j = 1;
  for (int i = 0; i < n; i++)
    if  (st[i] != ' ') a[j] = a[j] + st[i];
    else j++;
  // In ra xâu đảo ngược các từ
  for (int i = j; i >=1; i--)
  cout << a[i] << ' ';
  return 0;
}
```

Chương trình trên ta in thừa ra 1 dấu cách ở cuối xâu, đối với việc in thừa dấu cách ở cuối xâu thì kết quả khi chấm vẫn đúng (coi như không có dấu cách ở cuối xâu).

Câu 12. Tách từng kí tự của xâu rồi chuyển nó thành số và ghép vào kết quả. Ta có ký tự '0' là số 48 trong bảng mã ASCII, tương tự kí tự '1' là số 49… vậy muốn chuyển kí tự '0' thành số 0 thì ta kí tự '0' sang bảng mã ASCII rồi trừ đi 48 sẽ ra số 0; Vậy kí tự số bất kì ta đổi sang mã ASCII rồi trừ đi 48 sẽ cho ra số của kí tự đó.

```
#include <bits/stdc++.h>
#define  ll long long
using namespace std;
string st;
ll res;
int n;
```

```
int main(){
  cin >> st;
  n = st.length();
  res = 0;
  for (int i = 0; i < n; i++)
    res = res * 10 + st[i] - 48;
  cout << res;
  return 0;
}
```

Qua *code* ở trên ta lại được học cách viết ngắn gọn *long long* thành *ll.* Nếu như trong bài ta phải viết nhiều từ *long long*, thay vì viết như vậy ta dùng *#define* để gán *long long* cho *ll* để viết cho ngắn gọn. Thực chất chương trình trên không cần viết như vậy cũng đc, vì mình dùng có 1 lần *ll*

Câu 13. Ta cũng tách từng chữ số của số N rồi chuyển nó thành kí tự và ghép vào xâu kết quả. Chú ý: ta có kí tự '0' trong bảng mã ASCII là 48. Nên muốn đổi số 0 sang kí tự '0' thì ta phải lấy số 0 cộng với 48, rồi đổi 48 sang đc kí tự '0'. Tương tự số từ 0 đến 9 ta phải cộng với 48 rồi đổi nó sang được kí tự tương ứng.

```
#include <bits/stdc++.h>
#define  ll long long
using namespace std;
string st;
ll n;
int main(){
  cin >> n;
  st = "";
  while (n != 0){
    st = char( n%10 + 48) + st;
    n = n /10;
  }
  if (st == "") cout << "0";
  else cout << st;
  return 0;
}
```

Câu 14. Sử dụng tính chất một số chia hết cho 3 khi tổng các chữ số của nó chia hết cho 3. Vì ta không có kiểu dữ liệu số mà có tối đa 100 chữ số. Nên chúng ta dùng xâu để lưu số dạng này. Thay vì xử lý trên số ta lại xử lý trên xâu:

```
#include <bits/stdc++.h>
using namespace std;
string n;
int res, k;
int main(){
  cin >> n;
  k = n.length();
  res = 0;
  for (int i = 0; i < k; i++ )
    res = res + n[i] - 48;
  if (res % 3 == 0) cout << "chia het cho 3";
  else  cout << "khong chia het cho 3";
  return 0;
}
```

Câu 15. Ta phải hiểu cách chuyển số thập phân sang số hexa. Đại loại là cứ lấy số n chia cho 16 được số dư ta lại chuyển nó thành kí tự rồi dùng xâu để lưu các số dư được viết ngược lại.

```
#include <bits/stdc++.h>
using namespace std;
string st;
long long n;
int t;
int main(){
  cin >> n;
  st = ""; // gán xâu rỗng cho st, viết 2 nháy kép liên tiếp
  while (n != 0){
    t = n%16;
    switch (t){
      case  10:   st = 'A' + st; break;
      case  11:   st = 'B' + st; break;
      case  12:   st = 'C' + st; break;
      case  13:   st = 'D' + st; break;
```

```
        case  14:   st = 'E' + st; break;
        case  15:   st = 'F' + st; break;
        default:    st = char(t + 48) + st;
      }
      n = n / 16;
    }
    if (st == "") cout << 0;
    else  cout << st;
    return 0;
}
```

Câu 16. Đề bài đại loại là cho một vòng hạt gồm n hạt, mỗi hạt được đánh số. Hãy đếm xem có bao nhiêu cách lắp khóa vào vòng hạt trên sao cho sau khi mở khóa thì ta được chuỗi phải có tính chất đối xứng. Ta nhận thấy có tối đa *N* vị trí đặt khóa cần kiểm tra. Để kiểm tra tính đối xứng đơn giản ta nhân đôi xâu đó lên để kiểm tra được *N* cách đặt khóa.

```
#include <bits/stdc++.h>
using namespace std;
string st;
int n, res, dau, cuoi;
bool kt;
int main(){
  cin >> st;
  n= st.length();
  res = 0;
  st = st + st;
  for (int i = 0; i <= n-1; i++  ){
    kt = true;
    dau = i; cuoi = n-1+i;
    while (dau < cuoi){
      if (st[dau] != st[cuoi]) {
        kt = false;
        break;
      }
      dau++; cuoi--;
    }
    if (kt) res++;
  }
```

```
    cout << res;
    return 0;
}
```

Với chương trình như trên, sau này ta viết bằng hàm thì chương trình sẽ nhìn tường minh hơn!

11. Kiểu bản ghi

- Kiểu bản ghi là một kiểu dữ liệu có cấu trúc. Một bản ghi gồm các thành phần (gọi là trường), khác với các kiểu dữ liệu có cấu trúc khác (mảng và xâu), các trường có thể thuộc các kiểu dữ liệu khác nhau.

- Kiểu bản ghi cho phép mô tả nhiều đối tượng có cùng một số thuộc tính, có hữu ích cho nhiều bài toán quản lí.

- Ngôn ngữ lập trình đưa ra quy tắc, cách thức xác định:

+ Tên kiểu bản ghi.

+ Tên các thuộc tính (trường).

+ Kiểu dữ liệu của mỗi trường.

+ Cách khai báo biến.

+ Cách tham chiếu đến trường.

Ví dụ: Ta muốn lưu thông tin về học sinh. Dữ liệu cần lưu trữ bao gồm:

+ Họ tên (Kiểu string).

+ Giới tính (Kiểu string).

+ Điểm thi (Kiểu double).

Để lưu trữ nhiều sinh viên ta có thể sử dụng các cách:

+ Khai báo một mảng bản ghi. Mỗi bản ghi sẽ chứa thông tin về một học sinh (họ tên,giới tính,điểm số…).

+ Khai báo 2 mảng kiểu string để lưu tên và giới tính 1 mảng kiểu double để lưu điểm số.

Rõ ràng cách đầu tiên thuật tiện để lưu trữ hơn cách thứ hai.

+ Cách khai báo

```
struct   Tên_kiểu_bản_ghi {
        Kiểu_dữ_liệu Tên_trường 1;
        Kiểu_dữ_liệu Tên_trường 2;
        ……
        Kiểu_dữ_liệu Tên_trường_n;
};
Tên_kiểu_bản_ghi   Danh_sách_biến;
```

Ví dụ.

```
struct   hocsinh {
        string  hoten;
        string  gioitinh;
        string  lop;
        double dtoan;
        double dvan;
        double tongdiem;
};
hocsinh        hs[100], a, b;
hocsinh        tg;
```

+ Cách truy xuất đến kiểu bản ghi:

Tên_biến.Tên_trường

Ở khai báo trên, nếu ta gán a = b; điều này có nghĩa là:

```
a.hoten = b.hoten;
a.gioitinh = b.gioitinh;
a.lop = b.lop;
a.dtoan = b.dtoan;
a.dvan = b.dvan;
a.tongdiem = b.tongdiem;
```

Ví dụ. Nhập vào n học sinh. Mỗi học sinh gồm Tên, Giới tính, Lớp, điểm toán, điểm văn. Yêu cầu in ra danh sách học sinh vừa nhập gồm các thông tin: Họ và tên, giới tính, lớp, điểm toán, điểm văn và Tổng điểm của 2 môn. Mỗi học sinh trên 1 hàng

```
#include <bits/stdc++.h>
#define  N 1000001
```

```
using namespace std;
struct   hocsinh{
   string ten;
   string gioitinh;
   string lop;
   double   dtoan, dvan;
   double   tdiem;
};
hocsinh    a[N];
int n;
int main(){
   cin >> n;
   // Nhập n học sinh
   for (int i = 1; i <= n; i++ ){
      cout << "nhap thong tin hoc sinh thu"<< i <<":\n ";
      cout << " Ten hoc sinh: ";
      cin >> a[i].ten;
      cout << "gioi tinh: ";
      cin >> a[i].gioitinh;
      cout << "lop: ";
      cin >> a[i].lop;
      cout << "diem toan: ";
      cin >> a[i].dtoan ;
      cout  << "diem van: ";
      cin >> a[i].dvan;
      a[i].tdiem = a[i].dtoan + a[i].dvan;
   }
   // in ra danh sach vua nhap
   for (int i =1; i <= n; i++)
      cout << a[i].ten << " " << a[i].gioitinh << " " << a[i].lop << " "
      << a[i].dtoan << " " << a[i].dvan << " " << a[i].tdiem << endl;
   return 0;
}
```

Bài tập

Câu 1. Tính diện tích đa giác lồi khi biết tọa độ N điểm (giả sử tọa độ điểm là tọa độ nguyên).

Câu 2. Cho *n* điểm. Tìm khoảng cách nhỏ nhất của 2 điểm

Câu 3. Cho n điểm khác nhau có tọa độ nguyên. Yêu cầu tìm điểm có hoành độ nhỏ nhất. Nếu có nhiều đáp án thì in ra ở gần tâm O nhất.

Câu 4. Cho 3 điểm có tọa độ nguyên. Hãy kiểm tra xem 3 điểm trên có thẳng hàng hay không?

Câu 5. Cho 2 điểm A, B có toạ độ nguyên trên hệ trục toạ độ Oxy. Hãy đếm xem có bao nhiêu điểm có toạ độ nguyên thuộc đoạn thẳng AB (nằm trên đoạn thẳng).

Gợi ý cách giải

Câu 1. Ta khai báo một mảng kiểu bản ghi. Mỗi bản ghi gồm 2 thuộc tính là hoành độ và tung độ. Rồi áp dụng công thức tính diện tích đa giác lồi khi biết tọa độ N điểm: $S = \frac{1}{2} |\sum_{i=1}^{N} (x_i - x_{i+1})(y_i + y_{i+1})|$

```
#include <bits/stdc++.h>
#define  N 1000001
using namespace std;
struct   toado{
  int x;
  int y;
};
toado    a[N];
int n;
long long s;
int main(){
  cin >> n;
  // Nhập n điểm
  for (int i = 1; i <= n; i++ )
    cin >> a[i].x >> a[i].y;
  a[n+1] = a[1];
  s = 0;
  for (int i = 1; i <= n; i++)
    s = s + (a[i].x - a[i+1].x)*(a[i].y + a[i+1].y);
  // Cách lấy trị tuyệt đối không dùng hàm abs
  if (s<0) s = -s;
  // Các số nguyên chia cho 2 ra được kết quả số thực
  // Vì làm việc với số nguyên nhanh hơn số thực nên ta có mẹo để in kết quả
  cout << s/2;
  if (s % 2 == 1) cout << ".5";
  return 0;
}
```

Câu 2. Câu này đã từng làm ở trên bằng việc dùng 2 mảng để lưu. Ta làm tương tự, thay vì dùng 2 mảng để lưu, giờ ta dùng 1 mảng để lưu *n* điểm, mỗi phần tử của mảng gồm 2 thuộc tính là tung độ và hoành độ.

Câu 3. Duyệt *n* điểm để tìm ra hoành độ nhỏ nhất. Duyệt lại *n* điểm, điểm nào có hoành độ nhỏ nhất thì tính khoảng cách đến điểm O, rồi cập nhật lại

vào kết quả. Giả sử ban đầu kết quả là số thật lớn. Ta dùng công thức tính khoảng các giữa 2 điểm A và B bằng $\sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$ vì ta chỉ cần so sánh nên không cần lấy căn cũng được.

Câu 4. Theo tính chất của vector trong toán học lớp 10. Giả sử 3 điểm $A(x_A, y_A)$, $B(x_B, y_B)$, $C(x_C, y_C)$. Để *3* điểm *A, B, C* thẳng hàng thì ta có $\overrightarrow{AB} = k\overrightarrow{BC}$ suy ra được điều kiện là $\frac{x_B - x_A}{x_C - x_B} = \frac{y_B - y_A}{y_C - y_B}$ thay vì kiểm tra phép / thì nhân chéo lên để làm việc với số nguyên: $(x_B - x_A)(y_C - y_B) = (y_B - y_A)(x_C - x_B)$

Câu 5. Giả sử toạ độ điểm A (x_1, y_1); điểm B (x_2, y_2). Ta xét 2 trường hợp:

+ Nếu 2 điểm trùng nhau thì in ra kết quả là 1

+ Nếu 2 điểm A, B là phân biệt thì ta có công thức tính số điểm có toạ độ nguyên nằm trên đoạn thẳng AB (chú ý là nằm trên đoạn thẳng chứ không phải nằm trên đường thẳng) là UCLN ($|x_2-x_1|$, $|y_2-y_1|$) +1

Qua bài này ta có thể làm bài tương tự cho đa giác có N đỉnh (đa giác không tự cắt), các đỉnh được sắp xếp theo cùng hoặc ngược chiều kim đồng hồ. Tính số điểm có toạ độ nguyên nằm trên các cạnh của đa giác.

12. Đọc dữ liệu tệp khi không biết trước số lượng phần tử

Trong nhiều bài, thi thoảng gặp các bài toán dữ liệu vào ở dạng tệp (file), nhưng lại không cho trước số lượng phần tử trong tệp. Ta có các ví dụ sau:

Ví dụ 1. Cho tệp *songuyen.inp* gồm các số nguyên trong phạm vi *int* (các số nguyên được cách nhau bởi 1 dấu cách trống). Yêu cầu tính tổng tất cả các số trong tệp và ghi kết quả ra tệp *songuyen.out*. (kết quả tổng luôn đảm bảo thuộc phạm vi *long long*)

```
#include <bits/stdc++.h>
using namespace std;
int x;
long long s = 0;
int main(){
  ios_base::sync_with_stdio(false);
  cin.tie(0); cout.tie(0);
  freopen("songuyen.inp","r", stdin);
```

```
  freopen("songuyen.out","w",stdout);
  while (cin >> x){
    s = s + x;
  }
  cout << s;
  return 0;
}
```

Trong ***ví dụ*** trên: *while (cin >> x)* được hiểu là trong khi còn đọc được vào biến *x* thì cứ đọc, nếu không còn đọc được thì vòng lặp *while* sẽ dừng (đọc hết tệp). Như vậy lệnh *cin >> x* trong điều kiện *while* nó đã thực hiện đọc các số của tệp vào biến *x*. Nếu cần lưu giá trị của các số trong tệp để sau này cần tính toán thì ta làm như sau:

```
#include <bits/stdc++.h>
const  int N = 1E5;
using namespace std;
int i, x, n;
int a[N];
long long s = 0;
int main(){
  ios_base::sync_with_stdio(false);
  cin.tie(0); cout.tie(0);
  freopen("songuyen.inp","r", stdin);
  freopen("songuyen.out","w",stdout);
  i = 0;
  while (cin >> x){
    i++;
    a[i] = x;
  }
  n = i;
  for (int i = 1; i <= n; i++)  s = s + a[i];
  cout << s;
  return 0;
}
```

Với cách làm trên thì các số trong tệp được lưu và mảng *a* và sau nay ta cần tính toán các số trong tệp thì ta chỉ cần sử dụng mảng *a*

Ví dụ 2. Cho tệp *songuyen.inp* gồm các số nguyên khác 0 trong phạm vi *int* (các số nguyên được cách nhau bởi 1 dấu cách trống) và cuối tệp là 1 số nguyên bằng 0 để đánh dấu hết tệp. Yêu cầu: hãy lưu tất cả các số của tệp ra một mảng (trừ số 0 ở cuối tệp).

Với ***ví dụ*** này ta có thể đọc dữ liệu theo cách trên cũng được, ngoài ra ta còn có cách đọc khác: nhận thấy số 0 là dấu hiệu của việc kết thúc tệp, số 0 chỉ có 1 số nằm ở cuối tệp.

```
freopen("songuyen.inp","r", stdin);
freopen("songuyen.out","w",stdout);
i = 0;
while (true){
    cin << x;
    if (x == 0) break;
    i++;
   a[i] = x;
}
n = i;
```

Như vậy ta sẽ lưu được mảng *a* chứa *n* số từ *a[1], a[2],...a[n]* là các số trong tệp *songuyen.inp*

Ví dụ 3. Cho tệp **baitho.inp** gồm *1* bài thơ chứa nhiều câu thơ (từ *2* câu trở lên), mỗi câu thơ nằm trên *1* dòng. Yêu cầu ghi ra tệp **baitho.out** gồm *1* câu thơ có độ dài dài nhất (nếu có nhiều đáp án thì in ra câu thơ ở phía đầu của bài thơ)

Vì các câu thơ có thể chứa dấu cách nên ta dùng *getline* để đọc như sau:

```
#include <bits/stdc++.h>
const  int N = 1E5;
using namespace std;
int i, n, k, res;
string a[N];
string st;
int main(){
  ios_base::sync_with_stdio(false);
  cin.tie(0); cout.tie(0);
  freopen("baitho.inp","r", stdin);
```

```
  freopen("baitho.out","w",stdout);
  i = 0;
  while (getline (cin , st)){
    i++;
    a[i] = st;
  }
  n = i;
  res = 0;
  for (int i = 1; i <= n; i++)
    if (res < a[i].length()){
      res = a[i].length();
      k = i;
    }
  cout << a[k];
  return 0;
}
```

while (getline (cin , st)) có ý nghĩa là trong khi còn đọc được các hàng của tệp vào biến *st*, thì ta cứ đọc, đọc khi nào hết tệp thì dừng. Ngoài cách làm trên ta có thể làm cách đọc được xâu nào thì xử lý luôn xâu đó:

```
#include <bits/stdc++.h>
using namespace std;
int res = 0;
string st1, st2;
int main(){
  ios_base::sync_with_stdio(false);
  cin.tie(0); cout.tie(0);
  freopen("tong.inp","r", stdin);
  freopen("tong.out","w",stdout);
  while (getline (cin , st1)){
      if (res < st1.length()){
        res = st1.length();
        st2 = st1;
      }
  }
  cout << st2;
  return 0;
}
```

Nếu bài toán cho các câu thơ không chứa dấu cách trống thì ta dùng *cin* đọc như sau:

```
res = 0;
while (cin >> st1){
    if (res < st1.length()){
      res = st1.length();
      st2 = st1;
    }
}
cout << st2;
```

13. Hàm

Trong C++ chỉ có khái niệm hàm. Hàm có thể xem là một đơn vị độc lập của chương trình. Các hàm có vai trò ngang nhau, vì vậy không được viết một hàm bên trong các hàm khác. Thực thế ta đã biết được một số hàm khi học đến đây: như hàm *main (); sqrt (x); abs (x); ...*Cấu trúc viết hàm:

```
Kiểu_dữ_liệu_hàm  Tên_hàm ([Khai báo các đối số]){
        [Khai báo các biến cục bộ;]
        Các câu lệnh;
        [return [biểu thức];]
}
```

Trong đó:

Kiểu dữ liệu hàm là kiểu giá trị trả về cho hàm

Tên hàm là tên gọi của hàm được đặt tên đúng theo cách đặt tên.

Khai báo các đối số giống như cách khai báo biến, nhưng không được khai báo gộp.

Khai báo các biến cục bộ nếu cần

return (biểu thức); là trả giá trị của biểu thức cho hàm và kết thúc hàm. Có thể không cần *return* khi kiểu của hàm là *void*.

Cách hay viết hàm ở chương trình cấp phổ thông thì vị trí của các hàm được đặt trước hàm *main()*.

- Lời gọi hàm: *Tên_hàm ([Danh sách các tham số thự sự]);*

Trong đó:

+ Số tham số thực sự phải bằng đối số của hàm.

+ Kiểu của tham số phải phù hợp với kiểu của đối số.

Hàm chỉ được thực hiện khi có lời gọi hàm.

Đối với hàm thì chúng ta nên dạy học sinh thông qua các ví dụ:

Ví dụ 1. Viết chương trình tìm giá trị lớn nhất của 3 số.

```
#include <bits/stdc++.h>
using namespace std;
double  a, b, c;
double  max3(double x, double y, double z){
   double  res;
   res = x;
   if (res < y)  res = y;
   if (res < z)  res = z;
   return  res;
}
int main(){
   cin >> a >> b >> c;
   cout << max3(a, b, c);
   return 0;
}
```

Mặc dù *x, y, z* là các đối số có cùng kiểu dữ liệu double nhưng chúng ta không được viết: *double max3(double x, y, z)*

Nguyên tắc hoạt động của hàm trên: khi gọi hàm *max3(a, b, c)* ở chương trình chính thì lúc này *a, b, c* sẽ truyền giá trị tương ứng cho các đối số *x, y, z* và các lệnh trong hàm *max3* đươc thực hiện, lệnh *return res;* là trả giá trị *res* cho hàm và kết thúc hàm.

Ở trong hàm ta có các biến *x, y, z, res* đều được gọi là biến cục bộ trong hàm, chúng được cấp phát bộ nhớ khi hàm được xét đến và ngay lập tức bị xóa khi ra khỏi hàm. Các đối số và biến cục bộ trong cùng một hàm không được đặt trùng tên. Chúng có thể đặt trùng tên với biến toàn cục hoặc biến của các hàm khác. Ở ***ví dụ*** trên ta có thể đặt trùng tên như sau:

```
#include <bits/stdc++.h>
using namespace std;
double  a, b, c;
double  max3(double a, double b, double c){
    double  res;
    res = a;
    if (res < b)  res = b;
    if (res < c)  res = c;
    return  res;
}
int main(){
    cin >> a >> b >> c;
    cout << max3(a, b, c);
    return 0;
}
```

Khi một hàm được gọi tới, thì các tham số thực sự sẽ được truyền cho các đối số tương ứng, các đối số sẽ là bản sao của các tham số thực sự. Hàm chỉ làm việc trên các đối số, các đối số có thể bị thay đổi trong thân hàm nhưng các tham số thực sự không hệ bị thay đổi. Vị trí của hàm thông thường được viết sát phía trên của hàm main. Trong C++ nó cho phép tên các hàm có thể giống nhau, nhưng đối số phải khác nhau. Mặc dù cho phép trùng tên, nhưng chúng ta không nên đặt trùng tên, dễ gây nhầm lẫn.

Bài tập

Câu 1. Trong C++ có hàm min và max sẵn để tìm giá trị nhỏ nhất và lớn nhất của hai số bất kỳ. Hãy xây dựng 2 hàm min3(a, b, c) và max3(a, b, c) có sử dụng hàm min và max để tìm giá trị nhỏ nhất và lớn nhất của 3 số a, b, c.

Câu 2. Tìm ước chung lớn nhất của 2 số nguyên m, n không âm, không đồng thời bằng 0.

Câu 3. Xây dựng hàm *daoso(long long a)* nhận số tự nhiên *a* và trả về số đảo ngược của số *a*; ví dụ *a = 234* thì *daoso(a)* là *432; a = 200* thì số đảo là *2* (không cần in ra số 0 đứng trước nếu có).

Câu 4. Số hoàn thiện là số có số đảo ngược và chính nó là nguyên tố cùng nhau (hay nói cách khác là có ước chung lớn nhất là 1). Ví dụ số 23 có số đảo ngược là 32 và 2 số này nguyên tố cùng nhau. Nên số 23 là số thân thiện và số 32 cũng là số thân thiện. Nhập vào 2 số nguyên a, b ($10 <= a <= b <= 3*10^4$). Hãy đếm xem trong đoạn [a, b] có bao nhiêu số thân thiện.

Câu 5: Xây dựng hàm *giaithua(int n)* để tính giá trị của *n! (0<= n <= 20)*.

Câu 6: Xây dựng hàm tongchuso(long long n) trả về giá trị là tổng các chữ số của số n (n là số nguyên dương);

Câu 7: Xây dựng hàm kiểm tra một số n có phải là số nguyên tố hay không, sau đó đưa ra màn hình số nguyên tố lớn nhất mà nhỏ hơn 10^4

Câu 8: Xây dựng hàm *hoanhao (int n)* để kiểm tra xem số n có phải số hoàn hảo hay không

Gợi ý cách giải

Câu 1.

```
#include <bits/stdc++.h>
using namespace std;
double  a, b, c;
double  min3(double a, double  b, double c){
   double  res;
   res = min(min(a, b), c);
   return  res;
}
double  max3(double a, double  b, double c){
   double  res;
   res = max(max(a, b), c);
   return  res;
}
int main(){
   cin >> a >> b >> c;
   cout << min3(a, b, c) << '\n';
   cout << max3(a, b, c);
   return 0;
}
```

Hoặc

```
#include <bits/stdc++.h>
using namespace std;
double  a, b, c;
double  min3(double a, double  b, double c){
   return  min(min(a, b), c);
}
double  max3(double a, double  b, double c){
   return  max(max(a, b), c);
}
int main(){
   cin >> a >> b >> c;
   cout << min3(a, b, c) << '\n';
   cout << max3(a, b, c);
   return 0;
}
```

Câu 2.

```
#include <bits/stdc++.h>
using namespace std;
int  m, n;
int   ucln(int m, int n){
  int tg;
  while (n!=0){
    tg = m%n;
    m = n;
    n = tg;
  }
  return m;
}
int main(){
  cin >> m >> n;
  cout << ucln(m, n);
  return 0;
}
```

Trong C++ có 1 hàm được viết sẵn trong thư viện để tính ước chung lớn nhất của 2 số có cú pháp: *__gcd(m, n)* chú ý trước gcd là 2 dấu gạch nối _

Câu 3.

```
long long   daoso(long long a){
  long long   res = 0;
  while (a!=0){
    res = res*10 + a%10;
    a = a / 10;
  }
  return res;
}
```

Câu 4. Như vậy bài toán chúng ta cần sử dụng hàm *daoso(a)* và *__gcd(a, daoso(a))*.

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int res = 0;
```

```
int   daoso(int a){
  int  res = 0;
  while (a!=0){
    res = res*10 + a%10;
    a = a / 10;
  }
  return res;
}
int main(){
  cin >> a >> b;
  for (int i = a; i <= b; i++ )
    if (__gcd(i, daoso(i))==1) res++;
  cout << res;
  return 0;
}
```

Chương trình trên nếu một số x và số đảo của x đều trong phạm vị từ a đến b thì mặc dù ta đã biết được số đảo của x cũng là đáp số nhưng lúc sau ta lại đi tính lại. ta có thể dùng mảng đánh dấu kt[] để viết tối ưu hơn như sau:

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int x;
bool  kt[30002];
int res = 0;
int   daoso(int a){
  int  res = 0;
  while (a!=0){
    res = res*10 + a%10;
    a = a / 10;
  }
  return res;
}
int main(){
  cin >> a >> b;
  memset(kt, false, sizeof(kt));
  for (int i = a; i <= b; i++ )
    if (kt[i]==false){
```

```
        x = daoso(i);
        if (__gcd(i, daoso(i))==1){
          kt [i] = true;
          if  (x>=a && x<= b)     kt[x] = true;
        }
      }
    for (int i = a; i <= b; i++)
      if (kt[i])  res++;
    cout << res;
    return 0;
}
```

Câu 5.

```
#include <bits/stdc++.h>
using namespace std;
long long n;
long long giaithua (int n){
  long long res = 1;
  for (int i = 1; i <= n; i++)
    res = res * i;
  return   res;
}
int main(){
  cin >> n;
  cout << giaithua(n);
  return 0;
}
```

Câu 6.

```
#include <bits/stdc++.h>
using namespace std;
long long n;
int   tongchuso (long long  n){
  int res = 0;
  while (n != 0){
    res = res + n%10;
    n = n / 10;
  }
```

```
    return   res;
}
int main(){
    cin >> n;
    cout << tongchuso(n);
    return 0;
}
```

Câu 7.

```
#include <bits/stdc++.h>
const int N=10000;
using namespace std;
bool  nguyento (int  n){
    if (n < 2) return false;
    for (int i = 2; i*i <= n; i++)
        if (n % i == 0)   return false;
    return true;
}
int main(){
    for (int i = N - 1; ; i--)
        if (nguyento(i)) {
            cout << i;
            break;
        }
    return 0;
}
```

Câu 8.

```
#include <bits/stdc++.h>
using namespace std;
int n;
bool  hoanhao (int  n){
    long long tong = 1;
    if (n<6) return false;
    for (int i = 2; i<= n/2; i++)
        if (n % i == 0)   tong = tong + i;
    if (tong == n) return true;
    return false;
```

```
}
int main(){
    cin >> n;
    if (hoanhao(n))  cout << n << " la so hoan hao";
    else cout << n << " khong phai so hoan hao";
   return 0;
}
```

- Hàm không giá trị: Qua các bài tập, ngoài các kiểu dữ liệu ở trên trả về cho hàm thì ta còn có kiểu dữ liệu *void* (không cần trả dữ liệu về cho hàm, nó giống tương tự như thủ tục trong pascal)

Ví dụ 1. Chương trình nhập 2 số *a* và *b*. tính tổng 2 số

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
void  nhap (){
   cin >> a >> b;
   return;
}
void  xuat(){
   cout << a +b;
}
int main(){
   nhap();
   xuat();
   return 0;
}
```

Ví dụ trên ta dùng hàm không có tham số. Trong thân hàm dùng *void:* ta không được dùng *return biểu thức;* để trả giá trị cho hàm, mà chỉ được dùng *return;* (lệnh này là thoát khỏi hàm) hoặc ta không dùng cũng được (vì hàm dùng *void* là hàm không có giá trị trả về). Đối với mọi hàm có giá trị trả về thì ta cũng có thể viết được bằng hàm *void* (ta dùng biến toàn cục để lưu kết quả). Ví dụ hàm kiểm tra số hoàn hảo ta viết hàm dùng *void* như sau:

```
#include <bits/stdc++.h>
using namespace std;
int n;
```

```
bool  kt = true;
void  hoanhao (int  n){
  long long tong = 1;
  if (n<6) {
    kt = false;
    return;
  }
  for (int i = 2; i<= n/2; i++)
    if (n % i == 0)   tong = tong + i;
  if (tong == n) kt = true;
  else kt = false;
}
int main(){
   cin >> n;
   hoanhao(n);
   if (kt)  cout << n << " la so hoan hao";
   else cout << n << " khong phai so hoan hao";
  return 0;
}
```

- Truyền tham biến và tham trị: khái niệm này nó cũng tương tự như trong pascal, ở các ví dụ trình bày ở trên là truyền tham trị

+ Truyền tham trị là ta chỉ truyền giá trị cho các đối số của hàm, mọi thay đổi của đối số thì không ảnh hưởng gì đến tham số thực sự.

+ Truyền tham biến là ta truyền biến đó luôn cho đối số của hàm, mọi thay đổi của đối số như thế nào thì tham số thực sự đó cũng bị thay đổi theo. Cách truyền tham biến ta thêm & vào trước tên của các đối số ở phần khai báo các đối số của hàm (trừ trường hợp là mảng). Trong một hàm thì có thể có cả tham biến và tham trị.

Ta xét ví dụ sau:

```
using namespace std;
int a, b;
void  tinh (int c, int &d){
  c = c + 2;
  d = d + 2;
  cout << c <<" "<< d << endl;//in ra 5 3
```

```
}
int main(){
  a = 3; b = 1;
  tinh (a, b);
  cout << a << " " << b;// in ra 3 3
  return 0;
}
```

Ở ***ví dụ*** trên ta hiểu giá trị của *a* sẽ truyền cho *c* và biến *b* sẽ truyền cho *d*. Nêu mọi thay đổi gì của *d* sau khi thoát khỏi hàm thì *b* cũng thay đổi như *d* (*b = d*;). Còn *a* là tham trị nên mọi thay đổi của *c* thì *a* vẫn không thay đổi.

Ví dụ khác, khi khai báo trùng tên:

```
#include <bits/stdc++.h>
using namespace std;
int a, b, c;
void  tinh (int &a, int &b, int c){
  a = a + 2;
  b = b + 2;
  c = c + 2;
  cout << a <<" "<< b << " " << c << endl;//In ra 3 4 5
}
int main(){
  a = 1; b = 2; c = 3;
  tinh (a, b, c);
  cout << a << " " << b << " " << c;//in ra 3 4 3
  return 0;
}
```

Ở ví dụ trên thì *a, b* là tham biến; còn *c* là tham trị.

Ví dụ: Xây dựng hàm đổi chỗ hai số nguyên *a* và *b*

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
void  doicho(int &a, int &b){
  int tg;
  tg = a;
  a = b;
```

```
  b = tg;
}
int main(){
  cin >> a >> b;
  doicho(a, b);
  cout << a << " " << b;
  return 0;
}
```

Trong thư viện C++ có hàm *swap(a, b)* thực hiện đổi chỗ 2 số *a* và *b* (kể cả kiểu số thực, ký tự, xâu vẫn dùng *swap* bình thường):

```
#include <bits/stdc++.h>
using namespace std;
int a, b;
int main(){
  cin >> a >> b;
  swap(a, b);
  cout << a << " " << b;
  return 0;
}
```

Tất cả đối số của hàm, nếu là mảng thì đều được truyền theo tham biến. Khi đối số là mảng thì được khai báo như ví dụ sau: Hãy dùng hàm để nhập dữ liệu cho mảng gồm n phần tử.

```
#include <bits/stdc++.h>
#define  N 1000001
using namespace std;
int n, a[N];
void  nhapdl(int n, int a[]){
  for (int i =1; i <= n; i++)
     cin >> a[i];
}
void  indl(int n, int a[]){
  for (int i =1; i <= n; i++)
     cout << a[i] << " ";
}
int main(){
  cin >> n;
```

```
    nhapdl(n, a); // Lời gọi có tham số là mảng
    cout << "mang vua nhap la: ";
    indl (n, a); // Lời gọi có tham số là mảng
    return 0;
}
```

Chú ý: nếu mảng 1 chiều thì tên mảng ở phần đối số phần [] (để trống như ví dụ), Còn lời gọi hàm thì viết mình tên mảng như ví dụ. Còn đối với mảng nhiều chiều, thì ở phần khai báo đối số chiều đầu tiên bỏ trống, các chiều còn lại viết số lượng y hệt như đã khai báo nó ở phía trước lời gọi hàm. Ví dụ: *nhapdl (int n, int a[][30])* thì lúc này mảng a ở chiều thứ 2 là 30 thì khai báo ở biến toàn cục cũng phải là đúng 30, nếu khác nó sẽ báo lỗi. Lời gọi hàm thì chỉ cần tên hàm.

```
#include <bits/stdc++.h>
#define  N 10
using namespace std;
int n, a[N][N];
void  nhapdl(int n, int a[][N]){
  for (int i =1; i <= n; i++)
     for (int j = 1; j <= n; j++)
        cin >> a[i][j];
}void  indl(int n, int a[][N]){
  for (int i =1; i <= n; i++){
     for (int j = 1; j <= n; j++)
     cout << a[i][j] << " ";
     cout << endl;
  }
}
int main(){
  cin >> n;
  nhapdl(n, a);
  cout << "mang vua nhap la: \n";
  indl (n, a);
  return 0;
}
```

- Hàm đệ quy: là trong thân hàm đó có lời gọi hàm của chính nó. (Kiểu như đêm nằm ngủ mơ tự mình gọi tên mình). Thông thường trong thân hàm đệ

quy phải có điểm dừng hay gọi là điểm neo (trường hợp suy biến để thoát khỏi đệ quy). Cú pháp thông thường của thân hàm đệ quy:

```
if ( Trường hợp suy biến) {
    Trình bày cách giải bài toán;
}
else {
    Gọi đệ quy tới hàm với giá trị khác của tham số;
}
```

Hàm đệ quy thì nó cũng tương tự như phương pháp quy nạp toán học. Cách làm bằng đệ quy thường đơn giản và dễ hiểu, nhưng thông thường sẽ tốn bộ nhớ và thời gian hơn. Để hiểu rõ về đệ quy hơn, ta học thông qua các ví dụ:

Ví dụ 1. Chương trình đệ quy như dưới, chương trình in ra màn hình cái gì?

```
#include <bits/stdc++.h>
using namespace std;
void  dequy (int n){
  if (n == 0) cout << n << endl;
  else dequy (n-1);
  cout << n << endl;
}
int main(){
  dequy(3);
  return 0;
}
```

Giải thích ***ví dụ*** trên: khi gọi dequy(3) thì 3 sẽ được truyền cho đối số *n*:

+ vì 3 != 0 nên nó sẽ gọi dequy (3-1) và nhớ 1 lệnh *cout << n << endl*; phía sau lời gọi đệ quy (lúc đó *n =3*)

+ 2 được truyền lên cho *n*, vì 2 != 0 nên nó sẽ gọi *dequy(2-1)* và nhớ 1 lệnh phía sau lời gọi đệ quy (lúc đó *n = 2*)

+ 1 được truyền lên cho *n*, vì 1 != 0 nên nó sẽ gọi *dequy(1-1)* và nhớ 1 lệnh phía sau lời gọi đệ quy (lúc đó *n = 1*)

+ 0 được truyền lên cho *n*, vì 0 == 0 nên in ra số 0 (vì *n=0*), sau đó nó thực hiện tiếp lệnh *cout << n << endl*; Tức là in tiếp ra số 0; rồi thoát khỏi hàm. Giờ nó mới quay lại để xử lý những lần nhớ lúc trước (cái nào nhớ cuối cùng

sẽ được thực hiện đầu tiên, kiểu như đi lần lại dấu vết) Nên kết quả in ra màn hình như sau:

0
0
1
2
3

Ví dụ 2. Chương trình sau in ra màn hình cái gì?

```
#include <bits/stdc++.h>
using namespace std;
void  dequy (int n){
   if (n == 0) return;
   else dequy (n-1);
   cout << n << endl;
}
int main(){
   dequy(3);
   return 0;
}
```

Tương tự ***ví dụ 1***, khi *n == 0* thì sẽ return; (thoát khỏi hàm nên nó không làm lệnh in số *n* ở phía cuối hàm, nhưng nó phải xử lý các lệnh được nhớ lúc trước lại). Nên kết quả in ra là :

1
2
3

Ví dụ 3. Chương trình sau in ra màn hình cái gì

```
#include <bits/stdc++.h>
using namespace std;
int  dequy (int n){
   if (n == 0) return 1;
   else return dequy (n-1);
   cout << n << endl;
}
int main(){
   cout <<  dequy(3);
```

```
    return 0;
}
```

Tương tự vì hàm này có *return dequy (n-1);* nên sau lời gọi đệ quy mặc dù có lệnh in n ra màn hình, nhưng lệnh return là lệnh trả kết quả và thoát khỏi hàm nên nó không làm các lệnh phía sau *return*. Nên kết quả chỉ in ra 1 là giá trị của hàm (in ở chương trình chính).

Bài tập

Câu 1. Lập hàm tìm ước chung lớn nhất của 2 số nguyên không âm, không đồng thời bằng không

Câu 2. Tính *n! (0 <= n <= 20)*

Câu 3. Tính số fibonacci thứ n. Biết:

f(1) = 1;

f(2) = 1;

f(n) = f(n-2) + f(n-1);

Câu 4. Viết hàm đệ quy tính $S(n) = 1^3 + 2^3 + 3^3 + \ldots + n^3$

Câu 5. Viết hàm tính $P(n) = a^n$ (*n* là số nguyên không âm, *a* là số nguyên dương)

Câu 6. Viết chương trình đổi số nguyên dương N sang hệ nhị phân

Câu 7. Tên bài: **THAP.CPP**

Người chơi được cho ba cái cọc và N đĩa có kích thước khác nhau có thể cho vào các cọc này. Ban đầu sắp xếp các đĩa theo trật tự kích thước vào một cọc sao cho đĩa nhỏ nhất nằm trên cùng. Người chơi phải di chuyển toàn bộ số đĩa sang một cọc khác, tuân theo các quy tắc sau:

- Một lần chỉ được di chuyển một đĩa
- Một đĩa chỉ có thể được đặt lên một đĩa lớn hơn

Hãy in ra màn hình cách di chuyển. Kí hiệu 3 cọc là *A, B, C*. Hãy nêu các di chuyển theo mẫu *A -> B* , có nghĩa là di chuyển 1 đĩa trên cùng từ *A* đến *B*.

Lưu ý: ban đầu tất cả các đĩa nằm ở cột *A* và chuyển sang cột *C*. ($1 < N \leq 10$)

Dữ liệu: vào từ file **THAP.INP** gồm 1 số nguyên dương N.

Kết quả: ghi ra file **THAP.OUT** là cách di chuyển.

THAP.INP	THAP.OUT
3	A->C A->B C->B A->C B->A B->C A->C

Câu 8. Tên bài: **TOHOP.CPP**

Tính tổ hợp chập K của N. (K <= N <= 20)

Dữ liệu: vào từ file **TOHOP.INP** gồm 2 số nguyên N và K

Kết quả: ghi ra file **TOHOP.OUT** là kết quả của bài toán

TOHOP.INP	TOHOP.OUT
4 6	15

Gợi ý cách giải

Câu 1.

```
int ucln(int x, int y){
  if (y == 0) return x;
  else return ucln(y, x%y)
}
```

Câu 2.

```
long long  giaithua (int n){
  if (n==0 || n==1) return 1;
  return   giaithua(n-1)*n;
}
```

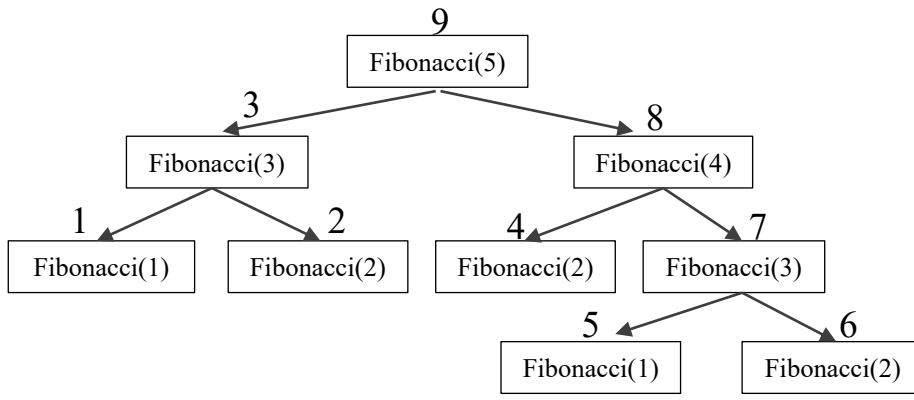
Câu 3.

```
long long  fibonacci (int n){
  if (n==1 || n==2) return 1;
  return   fibonacci(n-2) + fibonacci(n-1);
}
```

Nếu ta gọi *fibonacci(5)* thì hàm sẽ hoạt động như sau:

+ 5 được truyền cho đối số *n* (*n = 5),* vì 5 khác 1 và khác 2 nên nó sẽ làm lệnh *return fibonacci (3) + fibonacci (4);* Như vậy *fibonacci (3)* được tính trước (còn **fibonacci (4)** sẽ được nhớ lại để sau tính). Chuyền 3 cho *n*. Vì 3 khác 1 và 2 nên nó sẽ làm lệnh *return fibonacci (1) + fibonacci (2)*. Như vậy *fibonacci (1)* sẽ được tính tính trước (còn **fibonacci (2)** sẽ được nhớ lại để sau tính). Chuyền 1 cho n. Vì *1 == 1* nên trả kết quả 1 về *cho fibonacci (1)* bằng 1; giờ quay lại để xử lý các lần nhớ để tính (lần ngược, theo kiểu nhớ sau sẽ được tính lại trước). Lúc này **fibonacci (2)** được tính, 2 chuyền lên cho n. Vì *n == 2* nên trả 1 về cho *fibonacci (2).* Lúc này *fibonacci (3)* được tính bằng 1+1 = 2; Tiếp tục nó sẽ tính **fibonacci (4)** mà lúc trước nhớ. 4 truyền lên cho n. Vì 4 khác 1 và khác 2 nên nó sẽ làm lệnh *return fibonacci (2) + fibonacci (3);* Tương tự thì nó sẽ tính lại *fibonacci (2)* và nhớ lại *fibonacci (3)* để lúc sau tính ….. Qua ví dụ này ta thấy mặc dù *fibonacci (1), fibonacci (2), fibonacci (3)* đã được tính lúc trước nhưng sau vẫn được tính lại. Để hình dung hơn thì các thầy cô nên vẽ hoạt động kiểu theo sơ đồ cây như sau (thứ

tự tính ra các *fibonacci* được đánh số như hình vẽ, số 1 được tính đầu tiên ….số 9 được tính cuối cùng)



Câu 4. Ta nhận thấy công thức trên được viết lại như sau:

$$S(n) = S(n-1) + n^3$$

```
long long  tong (int n){
  if (n==1) return 1;
  return   tong(n-1) +1LL* n*n*n;
}
```

Ở bài này ta có 1LL là số 1 kiểu long long hoặc ta dùng ép kiểu (*long long*) *n*n*n* cũng được

Câu 5. Ta viết lại công thức $P(n) = a^{n-1}*a$

```
#include <bits/stdc++.h>
using namespace std;
int a, n;
long long  tinhmu (int n){
  if (n==0) return 1;
  return   tinhmu(n-1)*a;
}
int main(){
  cin >> a >> n;
  cout <<  tinhmu (n);
  return 0;
}
```

Ngoài ra ta có thể dùng công thức sau để tính nhanh hơn. Độ phức tạp là *O(logn)*

P(n) = $a^{n/2}$ *$a^{n/2}$ Nếu n chẵn

= $a^{n/2}$ * $a^{n/2}$ *a Nếu n lẻ

```
#include <bits/stdc++.h>
using namespace std;
int a, n;
long long  tinhmu (int n){
  if (n==0) return 1;
  long long t = tinhmu (n/2);
  if (n%2==0) return t*t;
  else  return t*t*a;
}
int main(){
  cin >> a >> n;
  cout <<  tinhmu (n);
  return 0;
}
```

Ta không nên *code* như phía dưới. Mặc dù code có vẻ ngắn hơn nhưng lại chạy lâu hơn. Vì *tinhmu (n/2)* đã được tính rồi, nhưng vẫn được tính lại.

```
long long  tinhmu (int n){
  if (n==0) return 1;
  if (n%2==0) return (tinhmu (n/2)* tinhmu (n/2));
  else  return tinhmu ((n/2)* tinhmu (n/2)*a);
}
```

Câu 6.

```
#include <bits/stdc++.h>
using namespace std;
long long n;
void  nhiphan( long long n){
  if (n==0) return;
  nhiphan(n/2);
  cout << n%2;
}
int main(){
```

```
    cin >> n;
    nhiphan(n);
    return 0;
}
```

Câu 7. Đầu tiên ta lấy cột C làm cọc trung gian. Chuyển n-1 chiếc đĩa sang cột B.

+ Ta chuyển chiếc đĩa lớn nhất sang cột C

+ Lấy cột A làm cột trung gian chuyển n-1 chiếc đĩa từ cột B sang cột C.

+ Cứ làm tương tự như thế cho đến khi hết số lượng đĩa.

```
#include<bits/stdc++.h>
using namespace std;
int n;
char a,b,c;
void    tower(int n, char a, char b, char c){
    if(n==1){
        cout<<a<<"->"<<c << '\n';
        return;
    }
    tower(n-1,a,c,b);
    tower(1,a,b,c);
    tower(n-1,b,a,c);
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("THAP.INP","r", stdin);
    freopen("THAP.OUT", "w", stdout);
    a = 'A'; b = 'B'; c = 'C';
    cin >> n;
    tower(n, a, b, c);
    return 0;
}
```

Câu 8. Gọi c[x][y] là tổ hợp chập x của y.

+ c[0][n] = c[n][n] = 1;

+ Với 0 < k < n thì c[k][n] = c[k – 1][n – 1] + c[k][n – 1].

```
#include<bits/stdc++.h>
using namespace std;
int c(int k, int n){
    if (k == 0 || k == n) return 1;
    return c(k - 1, n - 1) + C(k, n - 1);
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("TOHOP.inp","r", stdin);
    freopen("TOHOP.out", "w", stdout);
    int k, n;
    cin >> k >> n;
    cout << c(k, n);
    return 0;
}
```

PHẦN II

14. Độ phức tạp của thuật toán

Nói về độ phức tạp của thuật toán thì ta quan tâm tới 2 yếu tố: độ lớn của kiểu dữ liệu và thời gian chạy chương trình. Với cấu hình máy tính hiện nay, thì độ phức tạp của thuật toán ta thường quan tâm đến mặt thời gian hơn, bởi vì nếu một thuật toán cho ra một kết quả đúng nhưng thời gian thực hiện thuật toán cần một khoảng thời gian lớn đến nỗi không thể chấp nhận được (Ví dụ bài toán tháp Hà Nội nếu dùng đệ quy để tính toán thì với tốc độ máy tính hiện nay phải giải nó khoảng vài ngàn năm). Do đó yêu cầu thời gian được đưa ra đầu tiên. Thuật toán được gọi là hay thì phải có thời gian thực hiện ngắn và tiết kiệm tài nguyên máy tính. Sự hao phí tài nguyên máy tính liên quan đến phần cứng máy tính. Vì vậy mà những thuật toán đưa ra thường lấy thời gian để tính độ phức tạp hơn là tài nguyên (vì mỗi máy khác tài nguyên). Vì vậy, độ phức tạp của thuật toán là được hiểu nôm na là thời gian thực hiện của thuật toán. Ký hiệu độ phức tạp của thuật toán là *O* lớn.

Ta có tính chất $O(f(x)+g(x))=max(O(f(x)), O(g(x)))$

Ở đây chúng ta hiểu nôm na là đánh giá về mặt thời gian. Và cách hiểu thông thường nhất chính là số lần lặp tối đa của một đoạn nào đó trong chương trình.

Ví dụ 1. Xét câu lệnh: *x = x+1;*

Độ phức tạp của câu lệnh trên là *O(1)*. Vì đây là câu lệnh đơn nó được thực hiện *1* lần.

Ví dụ 2. Xét câu lệnh sau:

```
for (int i=1; i<=10; i++) x = x+ 1;
```

Câu lệnh trên được lặp *10* lần, mỗi lần lặp thì nó thực hiện một lệnh đơn *x = x+1* suy ra câu lệnh trên có độ phức tạp là *O(10)*.

Ví dụ 3. Xét câu lệnh sau:

```
for(int i=1; i<=n; i++) x = x+ 1;
```

Câu lệnh trên được lặp *n* lần, nên độ phức tạp của thuật toán là *O(n)*.

Ví dụ 4. Xét câu lệnh sau:

```
for(int i=1; i<= m; i++)
        for(int j=1; j<= n; j++)
                x = x+ a;
```

Đây là 2 câu lệnh *for* lồng nhau, nên nó lặp *m*n* lần, suy ra độ phức tạp của thuật toán là *O(m*n)*.

Ví dụ 5. Xét đoạn chương trình sau:

```
x = 1; //O(1)
for (int i =1; i <= n; i++) x = x+i;          //O(n)
for (int i =1; i <= n; i++)
        for (int j=1; j <=n; j++) x = x+1; //O(n^2)
```

Độ phức tạp của đoạn chương trình trên là $max(O(1), O(n), O(n^2)) = O(n^2)$.

Ví dụ 6. Xét đoạn chương trình tìm kiếm nhị phân sau:

```
dau =1; cuoi = n;
while (dau <= cuoi) {
        giua = (dau+cuoi)/2;
        if (a[giua] = x) {
                cout << "tim thay";
                break;
        }
        else    if (a[giua] > x)  cuoi = giua -1;
```

```
            else dau = giua +1;
    }
```

Câu lệnh *dau =1; cuoi = n;* là câu lệnh đơn có độ phức tạp là *O(1)*. Còn cả khối lệnh phía sau là đoạn tìm kiếm nhị phân trên mảng có *n* phần tử *a[1], a[2], a[3],... a[n]*. Nó cứ chia đôi ra để tìm. Suy ra số lần tối đa của việc chia đôi ra chính là độ phức tạp của thuật toán. Giả sử số lần chia đôi là *k*, thì ta dễ dàng thấy $2^k = n$. Từ đây là suy ra $k = log_2 n$. Độ phức tạp của đoạn chương trình trên là $O(log_2 n)$ hay nói một cách ngắn gọn là *O(logn)*.

Ví dụ 7. Xét đoạn chương trình sắp xếp sau:

```
for (int i= 1; i <= n-1; i++)
        for (int j = i+1; j <= n; j++)
                if (a[i]>a[j]){
                        tg = a[i];
                        a[i] = a[j];
                        a[j] = tg;
                }
```

Đoạn chương trình trên có cấu trúc *for* lồng nhau, còn sau lệnh *for* là các câu lệnh đơn

- Khi *i =1* thì nó lặp *n-1* lần;
- Khi *i =2* thì nó lặp *n-2* lần;
- Khi *i =3* thì nó lặp *n-3* lần;
- .....
- Khi *i = n-1* thì nó lặp *1* lần;

Vậy số lần lặp tối đa của đoạn chương trình trên là: *(n-1) + (n-2) + (n-3) + ... + 2 + 1*. Đây là tổng các số tự nhiên từ *1* đến *n-1* có kết quả là *(n*(n-1))/2*. Vậy độ phức tạp của thuật toán là *O(n*(n-1)/2)*. Nhiều lúc chúng ta không cần đánh giá chính xác mà chỉ cần lấy xấp xỉ. Nên ta cũng có thể nói độ phức tạp trên bằng $O(n^2)$.

Từ đây ta có thể dễ dàng đánh giá độ phức tạp của một số thuật toán thông dụng như: tìm ước chung lớn nhất của *2* số *a* và *b*: *max(O(loga), O(logb))*; tìm kiếm nhị phân: *O(logn)*; kiểm tra số nguyên tố thông thường: $O(\sqrt{n})$; sàng nguyên tố: *O(nlogn)*; hàm *sort* trong thư viện: *O(nlogn)*…; Nhiều lúc đánh giá trực tiếp về độ phức tạp là khó khăn: ví dụ như đệ quy sinh nhị phân

độ dài *n*, rõ ràng ta thấy nếu độ dài *n* thì ta đệ quy sinh ra được 2^n dãy nhị phân. Vậy độ phức tạp là $O(2^n)$, vì nó lặp 2^n để sinh ra số nhị phân.

Trong khi thi, thông thường mỗi test có thời gian chấm là *1 giây* (*1s*). Trong *1s* thông thường máy tính làm được khoảng 4.10^8 phép tính. Chính vì vậy khi đọc giới hạn của đề bài thì ta sẽ biết làm bài đó với độ phức tạp bao nhiêu để được điểm tối đa. Đánh giá độ phức tạp thuật toán là cực kỳ quan trọng trong thi cử. Nếu đánh giá sai độ phức tạp thuật toán dẫn đến bài của mình làm thì mình không tự chấm được chính xác bao nhiêu điểm (nếu như code đúng). Ví dụ: nếu cách làm là $O(n)$ thì bài đó mình có thể làm được với $n \leq 4.10^8$; Nếu là $O(nlogn)$ thì ta tính $nlogn = 4.10^8$, suy ra được $n \leq 10^7$; Nếu là $O(n^2)$ thì tương tự ta tính được bài đó làm được với $n \leq 2.10^4$…

15. Sử dụng hàm sort trong thư viện

Giả sử ta có mảng a gồm có *n* phần tử nguyên hoặc thực hoặc kí tự hoặc kiểu xâu, kiểu bản ghi. Trong thư viện của C++ có hàm *sort* dùng để sắp xếp mảng, độ phức tạp của nó là $O(nlogn)$ tương đương với thuật toán sắp xếp *quicksort*;

Cách 1:

sort (a+1, a+n +1); //nghĩa là sắp xếp mảng a thành mảng không giảm (hay gọi là tăng không ngặt) từ vị trí 1 đến vị trí *n* (*a[1]* đến *a[n]*)

sort (a, a+n); //nghĩa là sắp xếp mảng *a* thành mảng không giảm từ phần tử thứ 0 đến phần tử thứ *n-1* (*a[0]* đến *a[n-1]*)

Tổng quát có thể sắp xếp không giảm từ vị trí *i* đến vị trí *j*: *sort (a+i, a+j+1);*

Nếu là mảng các phần tử là kiểu số thì ta đổi dấu tất cả các phần tử, rồi sắp xếp tăng như hàm ở trên, rồi lại đổi dấu lại thì ta được mảng được sắp xếp giảm. Hoặc sau khi sắp xếp tăng, ta duyệt mảng từ phần tử cuối về phần tử đầu → duyệt dãy giảm.

Cách 2: Cách tự viết điều kiện

// sắp xếp theo điều kiện hàm cmp từ phần tử a[1] đến a[n]

```
sort (a+1, a+n+1, cmp);
```

// sắp xếp theo điều kiện hàm cmp từ phần tử a[0] đến a[n-1]

```
sort (a, a+n, cmp);
```

Cách viết hàm *cmp* như sau: hàm *cmp* này phải viết ở trước khi gọi hàm *sort*. *cmp* chỉ là tên do người lập trình đặt, ta có thể đặt tên khác cũng không sao. Ví dụ sosanh thì phải viết hàm có tên là sosanh...

Cách viết hàm *cmp*

```
// Sắp xếp tăng
bool   cmp (int a, int b){
    return a<b;
}
```

```
// Sắp xếp giảm
bool   cmp (int a, int b){
    return a>b;
}
```

Đối với đối số *a, b* trong hàm *cmp* ta cứ hiểu nôm na là *a* đứng trước *b* thì khi mảng sắp xếp giảm thì trả kết quả là true nếu $a > b$ (rõ ràng $a > b$ thì mới có dãy giảm vì *a* đứng trước *b* trong mảng). Còn sắp xếp tăng thì trả kết quả *true* nếu $a < b$ (rõ ràng $a < b$ thì mới có dãy giảm vì *a* đứng trước *b* trong mảng). Ta chú ý thêm 1 điều: mảng có kiểu dữ liệu gì thì đối số trong hàm *cmp* phải có kiểu dữ liệu y như vậy. Ví dụ mảng có các phần tử là *long long* thì các đối số *a và b* của hàm *cmp* cũng có phần tử kiểu *long long*. Nếu mảng *a* là kiểu dữ liệu là bản ghi có tên là *hocsinh* thì đối số của hàm *cmp* cũng phải kiểu dữ liệu *hocsinh*. Ví dụ mảng *a* là kiểu bản ghi có tên là *hocsinh* gồm có trường *ten, ngaysinh, diem*. Yêu cầu hãy sắp xếp theo *diem* giảm dần. Thì ta viết *cmp* như sau:

```
bool   cmp (hocsinh  a, hocsinh  b){
    return a.diem > b.diem;
}
```

Hoặc để dễ hiểu hơn nữa ta viết tường tận như sau:

```
bool   cmp (hocsinh  a, hocsinh  b){
    if( a.diem > b.diem) return true;
    return  false;
}
```

Ví dụ. Hãy nhập N học sinh có các thuộc tính như trên. Sắp xếp theo điểm tăng dần, nếu 2 điểm bằng nhau thì sắp xếp theo tên giảm dần.

```
#include <bits/stdc++.h>
#define  N 1001
using namespace std;
struct   hocsinh{
  string   ten;
  string   ngaysinh;
  double  diem;
};
hocsinh  a[N];
int i, n;
bool  cmp (hocsinh   a, hocsinh  b){
  if (a.diem < b.diem) return true;
  if (a.diem = b.diem)
    if (a.ten > b.ten)   return true;
  return false;
}
int main(){
  cin >> n;
  for (int i =1 ; i<= n; i++) {
    cin >> a[i].ten;
    cin >> a[i].ngaysinh;
    cin >> a[i].diem;
  }
  sort(a+1, a+n+1, cmp);
  // in ra danh sach mỗi học sinh trên 1 hàng
  for (int i = 1; i <= n; i++ )
    cout << a[i].ten << " "  << a[i].ngaysinh << " "<< a[i].diem << endl;
  return 0;
}
```

Bài tập

Câu 1. Tên bài: **vatsua.cpp**

Vào một buổi sáng anh Bo sắp một đàn bò gồm n con bò để vắt sữa. Anh dự kiến là vào sáng hôm đó, con bò thứ *i* có khả năng sẽ vắt được a_i lít sữa. Tuy nhiên đàn bò của anh có đặc tính là cứ mỗi lần vắt sữa một con, những con còn lại trông thấy sợ quá nên sẽ bị giảm sản lượng mỗi con *01* lít sữa. Nếu vắt sữa con bò thứ nhất, *n-1* con còn lại bị giảm sản lượng. Sau đó vắt sữa con bò thứ hai thì *n-2* con còn lại bị giảm sản lượng.... Bạn hãy giúp anh Bo tính xem thứ tự vắt sữa bò như thế nào để số lượng sữa vắt được là nhiều nhất nhé.

Dữ liệu: vào từ file **vatsua.inp** gồm 2 dòng

- Dòng thứ nhất là số nguyên n ($1 \le n \le 10^5$) là số lượng con bò.

- Dòng thứ hai gồm n số nguyên $a_1, a_2,..., a_n$ ($1 \le a_i \le 10^9$) là sản lượng sữa của các con bò.

Kết quả: ghi ra file **vatsua.out:** một số nguyên xác định số lít sữa nhiều nhất mà anh Bo có thể vắt được.

Ví dụ:

vatsua.inp	vatsua.out	vatsua.inp	vatsua.out
4 4 4 4 4	10	4 2 1 4 3	6

- Trong test 1: vắt sữa con bò 1 (được 4), lượng sữa còn lại là 3 3 3; vắt sữa con bò 2 (được 3), lượng sữa còn lại là 2 2, vắt sữa con bò 3 (được 2) và con bò 4 (được 1), tổng cộng 10.

- Trong test 2: vắt sữa con bò 1 (được 2), lượng sữa còn lại là 0 3 2; vắt sữa con bò 3 (được 3) và vắt sữa con bò 4 (được 1) tổng cộng 6.

Câu 2. Tên bài: **tile.cpp**

Nam có *n* viên gạch được đánh số từ *1* đến *n*. Các viên gạch có độ cứng lần lượt là $a_1, a_2,..., a_n$. Một viên gạch có độ cứng *x* nghĩa là Nam có thể chồng lên trên viên gạch đó tối đa *x* viên gạch khác, nếu chồng nhiều hơn thì viên gạch đó bị vỡ. Hỏi Nam có thể sắp được chồng gạch cao nhất là bao nhiêu?

Dữ liệu: vào từ file **tile.inp** gồm:

- Dòng đầu tiên là số nguyên n ($1 \le n \le 10^5$) - là số viên gạch.

- Dòng tiếp theo gồm n số nguyên $a_1, a_2,..., a_n$ ($0 \le a_i \le 10^9$)

Kết quả: ghi ra file **tile.out**: một số nguyên là kết quả của bài toán

Ví dụ:

tile.inp	tile.out	tile.inp	tile.out
3 1 2 1	3	6 0 0 0 0 0 0	1

Trong test 1 viên trên cùng có độ cứng 1, viên giữa có độ cứng 1, viên dưới cùng có độ cứng 2 => chiều cao là 3

Câu 3. Tên bài: **capso.cpp**

Cho một mảng gồm n số nguyên dương $a_1, a_2, a_3, ... a_n$. Hỏi có bao nhiêu cặp số bằng nhau? (Bao nhiêu cặp $a_i = a_j$ với $i \ne j$, (a_i, a_j) và (a_j, a_i) chỉ được tính là 1 cặp)

Dữ liệu: Vào từ file **capso.inp** gồm

- Dòng thứ nhất là chiều dài n của mảng ($1 <= n <= 10^5$)

- Dòng thứ hai gồm n số nguyên $a_1, a_2, a_3, ... a_n$ ($1 <= a_i <= 10^9$)

Kết quả: ghi ra file **capso.out** là số nguyên xác định số lượng các cặp bằng nhau

Ví dụ:

capso.inp	capso.out	capso.inp	capso.out
5 8 2 9 8 1	1	7 6 2 4 2 4 3 4	4

Câu 4. Tên bài: **CAMDIEN.CPP**

Trong nhà Nam hiện đang có n ổ cắm điện rời. Số lượng chỗ cắm trên mỗi ổ cắm điện này lần lượt là $a_1, a_2, a_3, ..., a_n$ chỗ cắm. Trên tường nhà Nam có một chỗ cắm cố định đang có điện. Vậy để cho một ổ cắm điện rời có điện thì phải cắm ổ cắm đó vào chỗ cắm cố định trên tường. Chúng ta cũng có thể cắm ổ cắm điện rời này vào một ổ cắm điện rời khác đang có điện.

Nam có m thiết bị sử dụng điện, để sử dụng thì các thiết bị này cần được cắm vào ổ cắm trên tường hoặc ổ cắm rời đang có điện. Bạn hãy giúp Nam tìm ra số ổ cắm rời ***ít nhất*** cần dùng để có thể sử dụng tất cả *m* thiết bị điện này.

Dữ liệu: vào từ file **CAMDIEN.INP** gồm

- Dòng thứ nhất gồm 2 số nguyên *n, m*. Dữ liệu vào đảm bảo $1 \le n, m \le 10^5$, *n* là số lượng ổ cắm và *m* là số lượng thiết bị.

- Dòng thứ hai gồm *n* số nguyên $a_1, a_2, ..., a_n$ là số chỗ cắm trên các ổ cắm rời tương ứng, mỗi số cách nhau một khoảng trắng, dữ liệu vào đảm bảo $1 \le a_i \le 10^5$.

Kết quả: ghi ra file **CAMDIEN.OUT** là số nguyên cho biết số ổ cắm rời ít nhất cần sử dụng là bao nhiêu. Nếu đã sử dụng hết tất cả ổ cắm rời mà vẫn không đủ, in ra *-1*.

***Ví dụ*:**

CAMDIEN.INP	CAMDIEN.OUT
3 4 3 2 2	2
5 5 1 3 1 2 1	-1

Gợi ý cách giải

Câu 1. Tên bài: **vatsua.cpp**

Ta sắp xếp mảng *a* giảm dần, rồi cứ tham lam từ trái qua phải như code dưới:

```
#include <bits/stdc++.h>
const   int N = 1E5+2;
using namespace std;
int a[N];
int i, k, n;
long long res;
bool    cmp (int a, int b){
   return a > b;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("vatsua.inp", "r", stdin);
   freopen("vatsua.out", "w", stdout);
   cin >> n;
   for (int i = 1; i <= n; i++)    cin >> a[i];
   sort (a+1, a+n+1, cmp);
   res = 0;   k = 0;
   for (int i = 1; i <= n; i++)
      if (a[i] - k >0){
            res = res + a[i] - k;
            k++;
      } else    break;
   cout << res;
   return 0;
}
```

Câu 2. Tên bài: **tile.cpp**

Sắp xếp mảng *a* giảm dần, ta nhận thấy cứ đặt các viên gạch có độ cứng lớn ở phía dưới là phương án tối ưu, ta tham lam như *code* sau:

```
#include <bits/stdc++.h>
const   int N = 1E5+2;
using namespace std;
```

```
int a[N];
int i, k, n;
int res;
bool    cmp (int a, int b){
   return a > b;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("tile.inp", "r", stdin);
   freopen("tile.out", "w", stdout);
   cin >> n;
   for (int i = 1; i <= n; i++)    cin >> a[i];
   sort (a+1, a+n+1, cmp);
   k = a[1]; res = 1;
   for (int i = 2; i <= n; i++)
      if (k  >= 1){
            k--;
            k = min (a[i], k);
            res++;
      }
      else break;
   cout << res;
   return 0;
}
```

Câu 3. Tên bài: **capso.cpp**

Sắp xếp mảng *a* thành mảng tăng dần. Lúc này nếu có các phần tử bằng nhau thì nó phải nằm sát nhau. Ta dùng một biến *dem* để đế số lượng phần tử bằng nhau, được *dem* ta cập nhật luôn vào kết quả. Giả sử nếu có *k* phần tử bằng nhau thì số cách ghép cặp là C_k^2 = k*(k-1)/2 (số cách lấy 2 phần tử trong *k* phần tử). Vì *k*(k-1)* là tích của 1 số chẵn với 1 số lẻ nên tích này ra số chẵn nên phép chia / này là phép chia hết. Để cài đặt được đơn giản ta thêm vào *1* phần tử *a[n+1]* có giá trị khác với tất cả các số trong mảng. Chú ý khi cài đặt, tránh bị hiện tượng tràn số.

```
#include <bits/stdc++.h>
const   int N = 1E5+2;
```

```
using namespace std;
int a[N];
int i, n, dem;
long long res;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("capso.inp", "r", stdin);
    freopen("capso.out", "w", stdout);
    cin >> n;
    for (int i = 1; i <= n; i++)    cin >> a[i];
    sort (a+1, a+n+1);
    dem = 1; res = 0;
    a[n+1] = 1E9+1;
    for (int i = 2; i <= n+1; i++)
        if (a[i] == a[i-1]) dem++;
        else if (dem > 1){
            res = res + (long long)dem*(dem-1)/2;
            dem = 1;
        }
    cout << res;
    return 0;
}
```

Câu 4: Tên bài: **CAMDIEN.CPP**

Trường hợp *m == 1* thì ta cắm trực tiếp trên tường. Trường hợp *m > 1*: ta thấy cứ chọn ổ cắm nào có số lượng chỗ cắm nhiều thì sẽ được lợi hơn. Sắp xếp mảng *a* tăng dần, rồi duyệt từ cuối về đầu để lấy các ổ cắm. Khi dùng ổ cắm khác cắm vào ổ đó thì ổ đó bị giảm đi 1 chỗ cắm.

```
#include <bits/stdc++.h>
#define N 100005
using namespace std;
int n, m, a[N], res = 0;
void solve(){
    if(m == 1){
        cout << 0;
        return;
```

```
    }
    sort(a + 1, a + n + 1);
    for(int i = n; i >= 1; i --){
        res ++;
        if(m <= a[i]){
            cout << res;
            return;
        }
        m = m - (a[i] - 1);
    }
    cout << -1;
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("CAMDIEN.INP", "r", stdin);
    freopen("CAMDIEN.OUT", "w", stdout);
    cin >> n >> m;
    for(int i = 1; i <= n; i ++)
        cin >> a[i];
    solve();
    return 0;
}
```

16. Tìm Kiếm Nhị Phân

Ví dụ. Cho dãy số *a1, a2, ...aN* đã được sắp xếp tăng hoặc giảm. Không mất tính tổng quát ta giả sử nó đã sắp xếp tăng dần (tăng không ngặt). Tìm *X* có trong dãy số trên hay không?

Cách 1*:* Tìm kiếm tuần tự đã trình ở bài tập phía trên. Độ phức tạp là *O(n)*

Cách 2: Tìm kiếm nhị phân: vì dãy này đã được sắp xếp theo tiêu chí tăng dần, nên ta cứ chia đôi ra để tìm: Tìm vị trí ở giữa. So sánh *X* với giá trị ở vị trí giữa. nếu *X* mà bằng thì coi như đã tìm thấy, Nếu *X* lớn hơn giá trị ở giữa thì ta phải tìm phía bên phải của vị trí giữa, ngược lại *X* bé hơn giá trị ở giữa thì ta đi tìm phía bên trái của vị trí giữa… Độ phức tạp là *O(logn)*

```
#include <bits/stdc++.h>
#define  N 10000001
```

```
using namespace std;
int  a[N];
int n, x;
bool  timkiem (int x, int a[], int l, int r){
  while (l <= r){
    int giua = l + (r-l)/2; // lấy vị trí giữa đoạn [l, r] hoặc (l+r)/2
    if (a[giua] == x)  return  true;
    if (a[giua] > x) r = giua -1;
    else l = giua + 1;
  }
  return false;
}
int main(){
  // Nhập n
  cin >> n;
  // Nhập dãy a
  for (int i =1 ; i<= n; i++)  cin >> a[i];
  // Nhập x
  cin >> x;
  if (timkiem(x, a, 1, n) == true)   cout << x << " co trong day";
  else  cout << x << " khong co trong day";
  return 0;
}
```

Hàm *timkiem (int x, int a[], int l, int r)* có chứ năng tìm kiếm *x* có trong đoạn từ *l* đến *r* của mảng *a* hay không. Ở chương trình ta có *if (timkiem(x, a, 1, n) == true)* ta truyền tương ứng tham số tìm *x* có trong mảng *a* từ vị trí *1* đến vị trí *n* hay không. Thực tế ta có viết tắt như sau: *if (timkiem(x, a, 1, n))* (bỏ *== true* đi)

Ngoài ra trong thư viện của C++ có hàm ***binary_search*** là hàm tìm kiếm nhị phân cũng có độ phức tạp là *O(logn)*

Ví dụ. *binary_search(a+1, a+n+1, x)* với ý nghĩa là tìm *x* có trong dãy *a* (đã được sắp xếp tăng dần) từ vị trí *1* đến vị trí *n* hay không? Nếu có kết quả trả về true, ngược lại trả về false;

Tương tự *binary_search(a, a+n, x)* là tìm *x* có trong dãy *a* đã được sắp xếp tăng dần từ vị trí *0* đến vị trí *n-1* hay không?

Tương tự *sort* ta cũng có cách dùng ***binary_search*** bằng tự viết hàm *cmp*

```
binary_search(a+1, a+n+1, x, cmp)
binary_search(a, a+n, x, cmp)
```

Tìm kiếm dãy đã được sắp xếp tăng

```
bool    cmp (int a, int b){
      return a<b;
}
```

Tìm kiếm dãy đã được sắp xếp giảm

```
bool   cmp (int a, int b){
      return a>b;
}
```

Nếu dãy nhập vào là giảm thì ta có chương trình tìm kiếm sau:

```
#include <bits/stdc++.h>
#define  N 10000001
using namespace std;
int  a[N];
int n, x;
bool  cmp (int a, int b){
  return a>b;
}
int main(){
  // Nhập n
  cin >> n;
  // Nhập dãy a
  for (int i =1 ; i<= n; i++)  cin >> a[i];
  // Nhập x
  cin >> x;
  if (binary_search(a+1, a+n+1, x, cmp) == true)
            cout << x << " co trong day";
  else  cout << x << " khong co trong day";
  return 0;
}
```

Chú ý: hàm *cmp* cũng phải được viết phía trên hàm có lời gọi *binary_search* và ta có thể tổng quát lên tìm từ vị trí *L* đến vị trí *R* của mảng như sau: *binary_search (a+L, a+R+1, x)* hoặc : *binary_search(a+L, a+R+1, x, cmp)*

Trong tìm kiếm nhị phân ta có thêm 2 hàm có chức năng tương tự (giả sử mảng a đã được sắp xếp tăng):

- ***lower_bound(a+1, a+n+1, x)*** là cho địa chỉ đầu tiên của ô nhớ có giá trị >= *x* của mảng *a* (*a[1], a[2],..a[n]*) (nếu không có *x* trong mảng *a* thì nó sẽ đưa

ra địa chỉ của ô nhớ sau vị trí *a[n]*. Như vậy ta muốn đưa ra vị trí của phần tử >= *x* đầu tiên thì ta lấy địa chỉ ô nhớ đó trừ đi địa chỉ ô nhớ đầu tiên của mảng *a*: *lower_bound(a+1, a+n+1, x) - a (hoặc - &a[0]).* (vị trí được đánh từ *1, 2, ...n*). Nếu *x* không có thì kết quả ra *n+1.*

Tương tự ***lower_bound(a ,a+n, x) - a*** là cho ra vị trí đầu tiên trong mảng *a (a[0], a[1],...[a_{n-1}])* có giá trị >= *x* (vị trí được đánh từ *0, 1, ..n-1*). Nếu không có *x* trong mảng thì kết quả là *n*. Độ phức tạp của hàm này là *O(logn)*

- upper_bound(a+1, a+n+1, x) là cho địa chỉ đầu tiên của ô nhớ có giá trị > *x* của mảng *a* (*a[1], a[2],..a[n]*). Nếu không có *x* trong mảng *x* thì nó sẽ đưa ra địa chỉ của ô nhớ sau vị trí *a[n]*. Như vậy ta muốn đưa ra vị trí của phần tử > *x* đầu tiên thì ta lấy địa chỉ ô nhớ đó trừ đi địa chỉ ô nhớ đầu tiên của mảng *a*: *upper_bound(a+1, a+n+1, x) - a (hoặc - &a[0]).* (vị trí được đánh từ *1, 2, ...n*). Nếu *x* không có thì kết quả là *n+1*

Tương tự *upper_bound(a ,a+n, x) - a* là cho ra vị trí đầu tiên trong mảng *a (a[0], a[1],...[a_{n-1}])* có giá trị > *x* (vị trí được đánh từ *0, 1, ..n-1*). Nếu không có *x* trong mảng thì kết quả là *n*. Độ phức tạp của hàm này cũng là *O(logn)*

Nếu dãy *a* được sắp xếp giảm thì ta phải viết hàm ***cmp*** y hệt như phần *binary_search* phía trên.

Ví dụ: Tên bài: **khieuvu.cpp**

Câu chuyện tình yêu Elo Cruz và Mara ở Philippines là một minh chứng cho tình yêu đích thực, không màng đến ngoại hình. Họ khiến cư dân mạng thế giới phải khâm phục vì một tình yêu bất chấp những khác biệt về ngoại hình. Tuy nhiên admin rất lo lắng cho Elo, không biết anh chàng này sẽ phải chọn cây ghế cao thế nào để hôn vợ. Cho nên trong buổi tiệc khiêu vũ "Cơn gió đêm hè!" sắp đến đây admin muốn các cặp đôi có chiều cao chênh lệch phải đúng bằng *K* mới được khiêu vũ cùng nhau.

Yêu cầu: Bạn hãy tính giúp cho admin xem có thể có bao nhiêu cách sắp xếp từng cặp đôi với nhau thỏa mãn.

Dữ liệu: Vào từ file **khieuvu.inp** gồm

- Dòng đầu tiên là N - số lượng người tham gia bữa tiệc và số K ($2 \le N \le 10^5$; $0 \le K \le 10^9$)

- Dòng thứ hai là chiều cao của N người tham gia bữa tiệc ($0 < Hi \leq 10^9$)

Kết quả: Ghi ra file **khieuvu.out** một số duy nhất là kết quả của bài toán.

Ví dụ:

khieuvu.inp	khieuvu.out
6 2 1 3 3 3 9 5	6

Gợi ý: không mất tính tổng quát bài toán ta sắp xếp mảng chiều cao *h* thành dãy không giảm (tăng dần), vì nếu nếu *(a, b)* là một cặp thì cặp *(b, a)* không được tính nữa. Ta duyệt *i* từ *1* đến *n-1*, với mỗi *i* ta tìm: vị trí *left* là vị trí đầu tiên trong đoạn từ *[i+1, n]* mà *h[left] == h[i] + k*; vị trí *right* là vị trí cuối cùng trong đoạn từ *[i+1, n]* mà *h[right] == h[i] + k.* Như vậy số cặp ghép với *h[i]* sẽ là *right – left +1* (chú ý biện luận trường hợp không thỏa mãn và kết quả bài toán vượt kiểu int). Độ phức tạp thuật toán là *O(nlogn)*

Cách 1: tự viết hàm tìm kiếm

```
#include <bits/stdc++.h>
const   int     N = 1E5+2;
using namespace std;
int i, n, k;
int h[N];
long long res;
// Tìm vị trí đầu tiên bằng x
int     left(int l, int r, int x){
   int kq = 0;
   while (l <= r){
      int giua = (l+r)/2;
      if (h[giua] == x) {
         kq = giua;
         r = giua -1;
      } else   if (h[giua] > x)    r = giua -1;
               else    l = giua + 1;
   }
   return kq;
}
int     right(int l, int r, int x){
```

```
    int kq = 0;
    while (l <= r){
        int giua = (l+r)/2;
        if (h[giua] == x) {
            kq = giua;
            l = giua +1;
        } else   if (h[giua] > x)    r = giua -1;
                 else    l = giua + 1;
    }
    return kq;
}
void    solve (){
    sort (h+1, h+n+1);
    res = 0;
    int l, r;
    for (int i = 1; i <= n-1; i++){
        l = left (i+1, n, h[i]+k);
        if (l == 0) continue;
        r = right (i+1, n, h[i]+k);
        res = res + r - l +1;
    }
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie (0); cout.tie (0);
    freopen ("khieuvu.inp", "r", stdin);
    freopen ("khieuvu.out", "w", stdout);
    cin >> n >> k;
    for (int i = 1; i <= n; i++)    cin >> h[i];
    solve ();
    cout << res;
    return 0;
}
```

Cách 2: dùng hàm ***lower_bound*** và ***upper_bound*** có sẵn trong thư viện

```
#include <bits/stdc++.h>
const   int     N = 1E5+2;
using namespace std;
int i, n, k;
int h[N];
long long res;
void    solve (){
    int left, right;
    sort (h+1, h+n+1);
    res = 0;
    for (int i = 1; i <= n-1; i++){
        // Tìm vị trí đầu tiên >= h[i] +k trong đoạn [i+1, n]
        left = lower_bound (h+i+1, h+n+1, h[i]+k) - h;
       // Nêu không có giá trị h[i]+k thì ta quay lên vòng lặp thực hiện tiếp
        if (h[left] != h[i]+k) continue;
      // Tìm vị trí cuối cùng bằng h[i]+k
        right = upper_bound (h+i+1, h+n+1, h[i]+k) - h - 1;
      // Cập nhật vào kết quả
        res = res + (right - left +1);
    }
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie (0); cout.tie (0);
    freopen ("khieuvu.inp", "r", stdin);
    freopen ("khieuvu.out", "w", stdout);
    cin >> n >> k;
    for (int i = 1; i <= n; i++)    cin >> h[i];
    solve ();
    cout << res;
    return 0;
}
```

Ở code trên để tìm vị trí cuối có giá trị bằng *h[i]+k* thì ta dùng hàm *upper_bound* để tìm vị trí đầu tiên có giá trị > *h[i]+k* . Rồi lấy kết quả đó trừ đi 1 là ra vị trí cuối có giá trị bằng *h[i]+k* (có được điều này vì trong dãy đã tìm được vị trí đầu tiên có giá trị bằng *h[i]+k*)

Bài Tập

Câu 1. Tên bài: **BETUE.CPP**

Shop Bé Tuệ là một thương hiệu nổi tiếng trong lĩnh vực cung cấp các sản phẩm uy tín cho Mẹ & Bé ở khu vực thành phố Thanh Hóa. Ở đây có rất nhiều sản phẩm phù hợp với người tiêu dùng, giá cả rất hợp lý, chất lượng thì khỏi phải bàn. Nhân dịp trung thu sắp đến, ***Shop Bé Tuệ*** có chương trình tặng quà cho các em thiếu nhi, nhưng do dịch covid nên không thể tổ chức trao quà tại shop được. Vì lý do này, shop đã đăng thông tin lên ***website: shopbetue.com*** để cho các em đăng ký nhận quà miễn phí tại nhà. Sau khi thống kê số lượng đăng ký, thì có *N* em bé tham gia chương trình nhận quà. Shop có rất nhiều quà được đóng vào hộp với *M* màu khác nhau. Shop sẽ chia quà cho các em theo nguyên tắc sau:

- Mỗi em chỉ nhận được những hộp quà có màu giống nhau.
- Tất cả các hộp quà đều phải được chia hết trong dịp này.
- Số lượng hộp quà của em được tặng nhiều nhất phải là nhỏ nhất có thể.

Ví dụ trong trường hợp có 5 em, shop có 4 hộp quà màu đỏ và 7 hộp quà màu xanh, một cách chia hợp lý sẽ như sau: 2 đỏ, 2 đỏ, 2 xanh, 2 xanh, 3 xanh.

Yêu cầu: Bạn không cần đưa ra cách chia cụ thể mà chỉ cần đưa ra số hộp quà của em có nhiều hộp quà nhất.

Dữ liệu: vào từ file văn bản **BETUE.INP** gồm:

- Dòng 1: Gồm hai số nguyên dương *N, M* ($1 <= N <= 10^9$, $1 <= M <= 3.10^5$) lần lượt là số em nhận quà và số màu của các hộp quà.
- *M* dòng tiếp theo: Dòng i ghi một số nguyên dương a_i ($1 <= a_i <= 10^9$) là số hộp quà có màu *i*.

***Kết quả*:** Ghi ra file **BETUE.OUT** là số hộp quà của em có nhiều hộp quà nhất trong cách chia tối ưu. *Chú ý: dữ liệu của bài toán luôn có đáp án*.

BETUE.INP	BETUE.OUT
5 2 4 7	3

Câu 2. Tên bài: **CAKES.CPP**

Nghệ nhân nấu ăn Tư Mập có thể sử dụng hệ thống gồm *n* bếp điện để thực hiện nấu món ăn khiến ông được vinh danh, đó là món "gatô hải sản". Thời gian để thực hiện nấu một suất ăn như vậy trên các bếp điện tương ứng là *t1, t2, ..., tn* giây.

Yêu cầu: Cho biết *s* là số lượng thực khách cần phục vụ, hãy xác định thời gian tối thiểu cần thiết để Nghệ nhân Tư Mập có thể nấu xong s suất ăn trên hệ thống bếp điện của khách sạn. Để nấu mỗi suất ăn chỉ được sử dụng một bếp.

Dữ liệu: vào từ file văn bản **CAKES.INP**:

+ Dòng đầu tiên chứa số lượng suất ăn $s\ (0 < s < 10^{15})$ và số lượng bếp điện $n\ (0 < n < 20)$.

+ Dòng thứ hai chứa n số nguyên dương *t1, t2, ..., tn*, mỗi số nhỏ hơn 500.

Kết quả: Ghi ra file văn bản **CAKES.OUT** duy nhất một số nguyên là thời gian tối thiểu tìm được tính bằng giây.

Ví dụ:

CAKES.INP	CAKES.OUT
3 2 50 70	100

Câu 3. Tên bài: **NKSGAME.CPP**

Hai bạn học sinh trong lúc nhàn rỗi nghĩ ra trò chơi sau đây. Mỗi bạn chọn trước một dãy số gồm n số nguyên. Giả sử dãy số mà bạn thứ nhất chọn là: $b_1, b_2, ..., b_n$; còn dãy số mà bạn thứ hai chọn là: $c_1, c_2, ..., c_n$

Mỗi lượt chơi mỗi bạn đưa ra một số hạng trong dãy số của mình. Nếu bạn thứ nhất đưa ra số hạng $b_i\ (1 \le i \le n)$, còn bạn thứ hai đưa ra số hạng $c_j\ (1 \le j \le n)$ thì giá của lượt chơi đó sẽ là $|b_i+c_j|$.

Ví dụ: Giả sử dãy số bạn thứ nhất chọn là *1, -2;* còn dãy số mà bạn thứ hai chọn là *2, 3*. Khi đó các khả năng có thể của một lượt chơi là *(1, 2), (1, 3), (-2, 2), (-2, 3)*. Như vậy, giá nhỏ nhất của một lượt chơi trong số các lượt chơi có thể là 0 tương ứng với giá của lượt chơi *(-2, 2)*.

Yêu cầu: Hãy xác định giá nhỏ nhất của một lượt chơi trong số các lượt chơi có thể.

Dữ liệu: vào từ file văn bản **NKSGAME.INP** gồm

+ Dòng đầu tiên chứa số nguyên dương n ($n \le 10^5$)

+ Dòng thứ hai chứa dãy số nguyên $b_1, b_2, ..., b_n$ ($|b_i| \le 10^9$, i=1, 2, ..., n)

+ Dòng thứ hai chứa dãy số nguyên $c_1, c_2, ..., c_n$ ($|c_i| \le 10^9$, i=1, 2, ..., n)

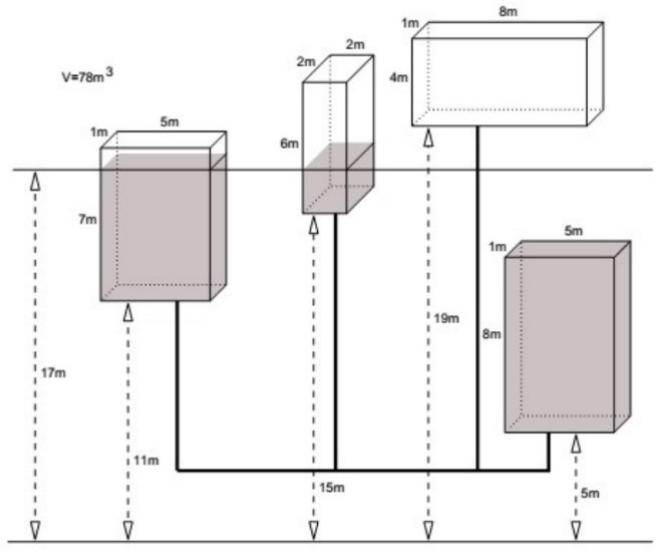
Kết quả: ghi ra file văn bản **NKSGAME.OUT** giá nhỏ nhất tìm được.

Ví dụ:

NKSGAME.INP	NKSGAME.OUT
2 1 -2 2 3	0

Câu 4. Tên bài: **FILL.CPP**

Theo dự báo, ở thế kỉ tiếp theo con người sẽ bị thiếu nguớc ngọt để cung cấp cho sinh hoạt hằng ngày. Để đối phó với tình trạng đó, thành phố *X* đã cho xây *N* bể chứa, tất cả các bể được xây dưới dạng hình hộp chữ nhật. Đáy bể thứ *i* được đặt ở độ cao so với mặt đất là b_i và kích thước các cạnh của bể là h_i, w_i, d_i: tương ứng là chiều cao, chiều rộng và chiều dài của bể. Giữa các bể này có đường ống thông nhau ở đáy bể, để điều tiết nước (như hình vẽ ví dụ). Trong trường hợp thiếu nước, nước ở các bể trên cao hơn sẽ chảy xuống các bể có độ cao thấp hơn



Yêu cầu: Cho thông tin về N bể chứa và lượng nước V. Các bạn hãy cho biết độ cao của mực nước so với mặt đất khi bơm V đơn vị thể tích nước vào các bể chứa. Coi như nước ở ống nối các bể là không đáng kể và độ độ dày của các thành bể cũng không đáng kể.

Dữ liệu: Vào từ file **FILL.INP** gồm:

+ Dòng đầu tiên chứa 2 số nguyên N, V; tương ứng là số bể chứa và lượng nước sẽ bơm vào các bể. ($1 \leq N \leq 10^5; 0 \leq V \leq 2.10^9$)

+ N dòng tiếp theo, mỗi dòng gồm 4 số nguyên là thông số của bể thứ i: b_i, h_i, w_i, d_i. ($0 \leq b_i \leq 10^6; 0 < h_i, w_i, d_i; h_i * w_i * d_i \leq 4.10^4$)

+ Dòng cuối chứa số nguyên T là các trường hợp của test chấm. ($1 \leq T \leq 4$)

Kết quả: Ghi ra file **FILL.OUT** gồm 1 số nguyên duy nhất (làm tròn đến hai chữ số thập phân) là độ cao của mực nước so với mặt đất. *Dữ liệu đảm bảo lượng nước bơm vào không vượt quá tổng dung tích các bể.*

Ví dụ:

FILL.INP	FILL.OUT	FILL.INP	FILL.OUT
2 4 0 1 1 2 2 1 3 1 4	2.67	4 78 11 7 1 5 15 6 2 2 19 4 1 8 5 8 1 5 2	17.00

Ràng buộc:

+Có *15%* số test ứng với $T = 1$: b_i giống nhau ở tất cả các bể và d_i giống nhau ở tất cả các bể

+ Có *20%* số test ứng với $T = 2$: kết quả đảm bảo là số nguyên và nhỏ hơn *50*

+ Có *25%* số test ứng với $T = 3$: kết quả đảm bảo là số nguyên

+ Có *40%* số test còn lại ứng với $T = 4$: không có điều kiện gì thêm.

Gợi ý cách giải

Câu 1. Tên bài: **BETUE.CPP**

Ta nhận thấy kết quả của bài toán luôn nằm trong đoạn [1, 10^9], vì số hộp quà tối đa có cùng màu là 10^9. Vậy ta chặt nhị phân trong đoạn từ 1 đến 10^9. Với mỗi giá trị *x* kiểm tra xem có thỏa mãn bài toán không; nếu thỏa mãn thì ta lại tìm tiếp bên phía trái, nếu không thỏa mãn thì ta tìm bên phía phải.

```
#include <bits/stdc++.h>
const   int M = 3E5+2;
using namespace std;
int i, n, m;
int a[M];
bool    kt (int x){
   int  s = 0;
   for (int i = 1; i <= m; i++){
      s = s+ a[i] / x;
      if (a[i] % x != 0) s = s +1;
   }
   return (s <= n);
}
void    solve (){
   int res = 0;
   int dau = 1, cuoi =  1E9;
   while (dau <= cuoi){
      int giua = (dau+cuoi) / 2;
      if (kt(giua)){
         res = giua;
         cuoi = giua - 1;
      } else  dau = giua + 1;
   }
   cout << res;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie (0); cout.tie (0);
   freopen("betue.inp", "r", stdin);
   freopen("betue.out", "w", stdout);
```

```
    cin >> n >> m;
    for (int i = 1; i <= m; i++)    cin >> a[i];
    solve();
    return 0;
}
```

Câu 2. Tên bài: **CAKES.CPP**

Ta thấy kết quả bài toán nằm trong đoạn *[1, s*500]*. Ta chặt nhị phân trong đoạn trên, với mỗi kết quả ta kiểm tra xem có thỏa mãn hay không, nếu thỏa mãn thì ta lại chặt nhị phân trên đoạn đầu, nếu kết quả không thỏa mãn thì ta lại chặt nhị phân trên đoạn sau. Ta có nhận xét: nếu có thời gian là *x* thì số xuất ăn nấu trên bếp *t[i]* là *x / t[i]* (chia lấy phần nguyên).

```
#include <bits/stdc++.h>
using namespace std;
long long  s;
int i, n, t[21];
bool check (long long x){
    long long  res = 0;
    for (int i = 1; i <= n; i++)  res = res + x / t[i];
    return res >= s;
}
void    solve (){
    long long kq;
    long long  dau =1, cuoi = s*500;
    while (dau <= cuoi){
        long long giua = (dau + cuoi) / 2;
        if (check (giua)){
            kq = giua;
            cuoi = giua - 1;
        } else  dau = giua + 1;
    }
    cout << kq;
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie (0); cout.tie (0);
    freopen("cakes.inp", "r", stdin);
```

```
    freopen("cakes.out", "w", stdout);
    cin >> s >> n;
    for (int i = 1; i <= n; i++)    cin >> t[i];
    solve();
    return 0;
}
```

Câu 3. Tên bài: **NKSGAME.CPP**

Cách 1:

Đầu tiên, ta sẽ sắp xếp dãy *b* tăng dần. Sau đó, duyệt lần lượt các phần tử *c* từ 1 → *n*; với mỗi phần tử *c[i]*, ta sử dụng chặt nhị phân trên dãy *b* để tìm phần tử bé nhất mà lớn hơn hoặc bằng *-c[i]* và phần tử lớn nhất bé hơn hoặc bằng *-c[i]*. Sau đó chỉ cần lấy *min* giá trị tuyệt đối của các tổng số tìm được với *c[i]*. Độ phức tạp của thuật toán là *O(nlogn)*.

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int N = 1e5 + 2;
int n, b[N], c[N], ans;
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);  cout.tie(0);
    freopen("nksgame.inp", "r", stdin);
    freopen("nksgame.out", "w", stdout);
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> b[i];
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    sort(b + 1 , b + n + 1);
    ans = 2e9;
    int id, tmp;
    for (int i = 1; i <= n; i ++) {
        id = lower_bound(b + 1, b + n + 1, -c[i]) - b;
        if (id == n + 1) tmp = b[1];
        else tmp = b[id];
        ans = min(ans , abs(tmp + c[i]));
```

```
        if (id != 1) {
            tmp = b[id - 1];
            ans = min(ans , abs(tmp + c[i]));
        }
    }
    cout << ans;
    return 0;
}
```

Cách 2: sắp xếp mảng *b* và mảng *c* tăng dần. Cho *i* chạy từ *1* và cho *j* chạy từ *n* về: Nếu *b[i] + c[j] > 0* thì ta giảm *j* xuống (để tổng tiến gần về *0*, vì nếu ta tăng *i* thì tổng đó càng lớn hơn tổng cũ), tương tự nếu *b[i] + c[j] < 0* thì ta tăng *i* lên, còn nếu *b[i] + c[j] = 0* thì đó chính là kết quả và thoát luôn. Kết quả của bài toán là *min* trong tất cả các trường hợp

```
#include <bits/stdc++.h>
const   int N = 1E5+2;
using namespace std;
int b[N], c[N];
int i, j, n;
void    solve(){
    sort(b + 1, b+n+1);
    sort(c + 1, c+n+1);
    int res = 2e9;
    i = 1; j = n;
    while (i <= n && j >= 1){
        if (res > abs(b[i]+ c[j]))  res = abs(b[i] +c[j]);
        if (res == 0)  break;
        if (b[i] +c[j] > 0) j--;
        else i++;
    }
    cout << res;
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie (0); cout.tie (0);
    freopen ("nksgame.inp", "r", stdin);
    freopen ("nksgame.out", "w", stdout);
    cin >> n;
```

```
    for (int i = 1; i <= n; i++)    cin >> b[i];
    for (int j = 1; j <= n; j++)    cin >> c[j];
    solve ();
    return 0;
}
```

Câu 4. Tên bài: **FILL.CPP**

Cách làm full điểm. Nhận thấy độ cao của mực nước so với mặt đất nằm trong đoạn [0, 10^6+4.10^4] (10^6 là độ cao nhất của đáy bể + 4.10^4 là độ cao cao nhất của bể có thể). Vì đề bài cho in kết quả làm tròn đến 2 phần thập phân nên ta có thể chặt nhị phân kết quả với sai số là epsilon = 0.001 (có thể lấy sai số nhỏ hơn thì càng chính xác) với mỗi độ cao *x* đó, ta tính được tổng dung tích của nước ở các bể; so sánh tổng này với *V:* nếu mà lớn hơn hoặc bằng V thì ta lại hạ độ cao xuống để tìm tiếp; ngược lại ta phải nâng độ cao lên để tìm tiếp.

```
#include <bits/stdc++.h>
const   int N = 1e5+2;
using namespace std;
struct  bechua{
   int b, h, w, d;
};
bechua  a[N];
int n, v;
double  eps = 0.001;
bool    check (double x){
  double sum = 0;
 // Tính tổng các lượng nước ở các bể khi đạt độ cao x
  for(int i = 1 ; i <= n ; ++i)
  if (a[i].b < x) sum = sum + min((x-a[i].b), (double)a[i].h)*a[i].w*a[i].d;
  return sum >= v;
}
void    solve (){
   double res;
   double dau = 0, cuoi = 1e6 +4e4;
   while (dau <= cuoi){
      double giua = (dau + cuoi) / 2;
      if (check (giua)) {
```

```
            res = giua;
            cuoi = giua - eps;
        }else dau = giua + eps;
    }
    cout << fixed << setprecision (2) << res;
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0);  cout.tie(0);
    freopen("fill.inp", "r", stdin);
    freopen("fill.out", "w", stdout);
    cin >> n >> v;
    for (int i = 1; i <= n; i++)
        cin >> a[i].b >> a[i].h >> a[i].w >> a[i].d;
    solve();
    return 0;
}
```

17. Kỹ Thuật Nhánh Cận

Định nghĩa: Trong một số bài toán, để tìm ra được đáp án, chúng ta phải sử dụng phương pháp liệt kê hết cấu hình để đánh giá và tìm ra cấu hình tốt nhất. Việc liệt kê cấu hình có thể sử dụng bằng việc sinh tuần tự hoặc quay lui. Tuy vậy, trong quá trình liệt kê, chúng ta có thể sẽ liệt kê rất nhiều cấu hình vô nghĩa, gây ra rất nhiều trường hợp khiến chương trình chạy chậm, thậm chí không thể ra được kết quả. Khi đó, một vấn đề đặt ra là trong quá trình liệt kê lời giải ta cần tận dụng những thông tin đã tìm được để loại bỏ sớm những phương án chắc chắn không phải tối ưu. Kỹ thuật đó gọi là kỹ thuật đánh giá nhánh cận trong tiến trình quay lui.

Mô hình nhánh cận:

```
<khởi tạo một cấu hình bất kỳ cho BESTCONFIG>
void Try(i) {
   for <mọi giá trị V có thể gán cho x[i]> {
      <thử x[i] := V>;
      if <việc thử trên vẫn còn hi vọng tìm ra cấu hình tốt hơn
BESTCONFIG> {
            if <x[i] là phần tử cuối cùng trong cấu hình> {
                  <cập nhật BESTCONFIG>
            } else {
                        <ghi nhận việc thử x[i] = V (nếu cần)>;
                        Try(i + 1);
                        <bỏ ghi nhận việc thử x[i] := V (nếu cần)>;
                  }
      }
   }
}
```

Lưu ý : Kỹ thuật nhánh cận thêm vào cho thuật toán quay lui khả năng đánh giá theo từng bước, nếu tại bước thứ *i*, giá trị thử gán cho *x[i]* không có hi vọng tìm thấy cấu hình tốt hơn cấu hình *BESTCONFIG* thì thử giá trị khác ngay mà không cần phải gọi đệ quy tìm tiếp hay ghi nhận kết quả làm gì (đặt cận để bỏ nhánh, cận càng tốt thì sẽ bỏ được nhiều nhánh hơn). Nghiệm của bài toán sẽ được làm tốt dần, bởi khi tìm ra một cấu hình mới (tốt hơn

BESTCONFIG), ta không in kết quả ngay mà sẽ cập nhật *BESTCONFIG* bằng cấu hình mới vừa tìm được.

Ví dụ. Tên bài: **NHIPHANK.cpp**

Cho tệp **NHIPHANK.inp** gồm 2 số nguyên *n, k (1<=n<=20, 1<=k<=n)*.
Hãy ghi ra tệp **NHIPHANK.out** gồm các số nhị phân có đúng *k* số 1. (mỗi số nằm trên mỗi hàng và theo thứ tự từ điển từ nhỏ đến lớn)

Ở ***ví dụ*** này ta chỉ đơn giản đệ quy sinh nhị phân, trong lúc sinh: nếu mà tổng các số 1 lớn hơn *k* thì ta bẻ nhánh đó (vì nếu có đi theo nhánh đó thì không bao giờ cho ra kết quả số nhị phân có đúng *k* số 1 được. Dùng thêm 1 biến *tong* để tính số lượng số 1 của dãy nhị phân. Nếu như ta đang xét đến phần tử *x[i]* mà *tong* > *k* thì ta bỏ nhánh đó đi (vì có đi tiếp thì *tong* này vẫn > *k*)

```
#include <bits/stdc++.h>
using namespace std;
int i, n, k;
int tong = 0;
int x[21];
void    vietcauhinh (){
   // Vì x[n] chưa cập nhật vào tong nên ta phải cộng với tong để so sánh
   if (tong + x[n]==k){
      for (int i = 1; i <= n; i++)    cout << x[i];
      cout << endl;
   }
}
void    nhiphank(int i){
   if ( tong >k) return; // Đặt cận để bẻ nhánh
   for (int j = 0; j <= 1; j++){
      x[i] = j;
      if (i == n ) vietcauhinh();
      else {
         tong = tong + x[i]; // Tính số lượng số 1
         nhiphank (i+1);
         tong = tong - x[i]; // Trả lại giá trị cho tong
      }
   }
}
```

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie (0);
    freopen ("nhiphank.inp", "r", stdin);
    freopen ("nhiphank.out", "w", stdout);
    cin >> n >> k;
    nhiphank (1);
    return 0;
}
```

Ở bài toán này, ta có thể nghĩ thêm 1 cận nữa để cho chương trình chạy nhanh hơn: Khi xét tiếp đến phần tử *x[i]*, Nếu *tong + (n – i +1) < k* thì ta cũng bỏ nhánh đó đi, có đi tiếp nữa thì số lượng con 1 vẫn < *k* (vì trường hợp tốt nhất là tất cả phía sau vị trí *i* đều bằng 1 mà tổng số con 1 vẫn < *k*)

```
void    nhiphank(int i){
    if ( tong >k) return; // Đặt cận để bẻ nhánh
    if (tong + n - i +1) // Đặt thêm cận để bẻ nhánh
    for (int j = 0; j <= 1; j++){
        x[i] = j;
        if (i == n ) vietcauhinh();
        else {
            tong = tong + x[i];
            nhiphank (i+1);
            tong = tong - x[i];
        }
    }
}
```

Bài Tập

Câu 1. Tên bài: **TOURISM.CPP**

Cho *n* thành phố đánh số từ 1 đến *n* và m tuyến đường giao thông hai chiều giữa chúng, mạng lưới giao thông này được cho bởi bảng *C* cấp *n*n*, ở đây $C_{ij} = C_{ji}$ = Chi phí đi đoạn đường trực tiếp từ thành phố *i* đến thành phố *j*. Giả thiết rằng $C_{ii} = 0$ với mọi i, $C_{ij} = +\infty$ nếu không có đường trực tiếp từ thành phố *i* đến thành phố *j*.

Một người du lịch xuất phát từ thành phố 1, muốn đi thăm tất cả các thành phố còn lại, mỗi thành phố đúng 1 lần và cuối cùng quay lại thành phố 1. Hãy chỉ ra cho người đó hành trình với chi phí ít nhất. Bài toán đó gọi là bài toán người du lịch hay bài toán hành trình của một thương gia (Traveling Salesman).

Dữ liệu: vào từ file văn bản **TOURISM.INP**

+ Dòng 1: Chứa số thành phố *n (1 <= n <= 15)* và số tuyến đường *m* trong mạng lưới giao thông

+ *m* dòng tiếp theo, mỗi dòng ghi số hiệu hai thành phố có đường đi trực tiếp và chi phí đi trên quãng đường đó (chi phí này là số nguyên dương <= 100)

Kết quả: ghi ra file văn bản **TOURISM.OUT**, ghi chi phí ít nhất tìm được.

Nếu không tồn tại đường đi, in ra -1.

Ví dụ:

TOURISM.INP	TOURISM.OUT
4 6 1 2 3 1 3 2 1 4 1 2 3 1 2 4 2 3 4 4	6

Câu 2. Tên bài: **CAITUI.CPP**

Trong siêu thị có *n* gói hàng, gói hàng thứ *i* có trọng lượng là $W_i <= 100$ và giá trị $V_i <= 100$. Một Tên trộm đột nhập vào siêu thị, sức của tên trộm

không thể mang được trọng lượng vượt quá *M (M <= 100)*. Hỏi tên trộm sẽ lấy đi những gói hàng nào để được tổng giá trị lớn nhất.

Dữ liệu: vào từ file **CAITUI.INP** gồm

+ Dòng đầu tiên gồm 2 số nguyên dương *n* và *m* (*n <= 40, m <= 100*)

+ *n* dòng tiếp theo, mỗi dòng chứa 2 số nguyên W_i và V_i (W_i, V_i <= *100*) lần lượt là trọng lượng và giá trị của gói hàng thứ *i*.

***Kết quả*:** ghi ra file **CAITUI.OUT** là giá trị lớn nhất tên trộm lấy được.

CAITUI.INP	CAITUI.OUT
3 4 1 4 2 5 3 6	10

Câu 3. Tên bài: **NGOAC.CPP**

Có thể định nghĩa khái niệm dãy ngoặc đúng dưới dạng đệ quy như sau:

1. **()** là dãy ngoặc đúng

2. **C** là dãy ngoặc đúng nếu **C = (A)** hay **C = AB** với **A, B** là các dãy ngoặc đúng.

Ví dụ dãy ngoặc đúng: **(), (()), ()(), (())()**

Ví dụ dãy ngoặc sai: **)(, ((((, ()((,))))),)()(**

Yêu cầu: Bạn hãy viết chương trình liệt kê tất cả các dãy ngoặc đúng có chiều dài *n* (n chẵn)

***Dữ liệu*:** vào từ file NGOAC.INP gồm 1 số nguyên dương *n* (*n* chẵn, $n \leq 20$)

Kết quả: ghi ra file NGOAC.OUT

+ Trong *m* dòng đầu tiên, mỗi dòng liệt kê một dãy ngoặc đúng chiều dài *n*. Các dãy được liệt kê theo thứ tự từ điển**: '(' < ')'**.

+ Dòng cuối cùng là số *m*. (*m* là số lượng các dãy ngoặc đúng có chiều dài *n*)

NGOAC.INP	NGOAC.OUT
4	(()) ()() 2

Câu 4. Tên bài: **TRES.CPP**

Hôm nay, vì sinh nhật Tí, nên Tí mời mọi người đi ăn ở nhà hàng. Mọi việc chuẩn bị đã gần xong nhưng còn một vấn đề làm Tí khá nhức đầu, đó là làm sao chọn được những món ăn mà mọi người cùng thích. Nhà hàng có *M* món ăn khác nhau và thú vị ở chỗ là mỗi món ăn rất nhiều nên có thể đủ cho bao nhiêu người cũng được, vì thế vấn đề là gọi món nào chứ không phải mỗi món gọi bao nhiêu.

Có tất cả *N* người đến dự tiệc sinh nhật (bao gồm cả Tí trong đó). Tí đã tìm hiểu được danh sách những món ăn yêu thích của từng người và muốn rằng đối với mỗi người phải có ít nhất 2 món mà họ thích. Tuy nhiên sau khi ăn xong còn nhiều tiết mục hấp dẫn khác nên bạn cũng muốn rằng bất kỳ ai cũng không có quá 2 món ăn yêu thích trong danh sách được đặt trước. Và vấn đề cuối cùng, đây là tiền của bố mẹ nên cũng không nên tiêu xài quá đáng.

Yêu cầu: Hãy cho biết số tiền ít nhất phải trả để gọi một thực đơn thỏa mãn các yêu cầu trên.

Dữ liệu: vào từ file **TRES.INP**

+ Dòng đầu tiên chứa hai số *M, N (M ≤ 30; N ≤ 10)*

+ Dòng thứ hai chứa *M* số *Pi* là giá của món thứ *i*.

+ Trong *N* dòng cuối cùng, dòng thứ *k* ghi danh sách các món yêu thích của người thứ *k*.

Kết quả: ghi ra file **TRES.OUT** gồm một số duy nhất là kết quả của bài toán, hoặc in ra -1 nếu không có cách gọi món nào thỏa mãn.

TRES.INP	TRES.OUT
5 3 100 150 300 425 200 1 2 4 1 3 4 5 1 4 5	450

Câu 5. Tên bài: **MINE.CPP**

Cho một bãi mìn kích thước *m*n* ô vuông, trên một ô có thể có chứa một quả mìn hoặc không, để biểu diễn bản đồ mìn đó, người ta có hai cách:

+ Cách 1: dùng bản đồ đánh dấu: sử dụng một lưới ô vuông kích thước *m*n*, trên đó tại ô *(i, j)* ghi số 1 nếu ô đó có mìn, ghi số 0 nếu ô đó không có mìn

+ Cách 2: dùng bản đồ mật độ: sử dụng một lưới ô vuông kích thước *m*n*, trên đó tại ô *(i, j)* ghi một số trong khoảng từ 0 đến 8 cho biết tổng số mìn trong các ô lân cận với ô *(i, j)* (ô lân cận với ô *(i, j)* là ô có chung với ô *(i, j)* ít nhất 1 đỉnh).

Giả thiết rằng hai bản đồ được ghi chính xác theo tình trạng mìn trên hiện trường. Về nguyên tắc, lúc cài bãi mìn phải vẽ cả bản đồ đánh dấu và bản đồ mật độ, tuy nhiên sau một thời gian dài, khi người ta muốn gỡ mìn ra khỏi bãi thì vấn đề hết sức khó khăn bởi bản đồ đánh dấu đã bị thất lạc !!. ***Công việc của các lập trình viên là: Từ bản đồ mật độ, hãy tái tạo lại bản đồ đánh dấu của bãi mìn.***

Dữ liệu: vào từ file văn bản **MINE.INP** gồm

+ Dòng 1: Ghi 2 số nguyên dương *m, n* (2 <= *m, n* <= 20)

+ *m* dòng tiếp theo, dòng thứ *i* ghi n số trên hàng *i* của bản đồ mật độ theo đúng thứ tự từ trái qua phải.

Kết quả: ghi ra file văn bản **MINE.OUT**:

+ Dòng 1: ghi tổng số lượng mìn trong bãi

+ *m* dòng tiếp theo, dòng thứ *i* ghi *n* số trên hàng *i* của bản đồ đánh dấu theo đúng thứ tự từ trái qua phải.

MINE.INP	MINE.OUT
2 5 0 2 1 3 2 1 2 2 4 2	5 1 0 1 1 1 0 0 0 0 1

Gợi ý cách giải

Câu 1. Tên bài: **TOURISM.CPP**

Hành trình cần tìm có dạng $(x_1 = 1, x_2, \ldots, x_n, x_{n+1} = 1)$ ở đây giữa x_i và x_{i+1}: hai thành phố liên tiếp trong hành trình phải có đường đi trực tiếp ($C_{ij} \neq +\infty$) và ngoại trừ thành phố 1, không thành phố nào được lặp lại hai lần. Có nghĩa là dãy $(x_1, x_2, \ldots, x_n)$ lập thành 1 hoán vị của $(1, 2, \ldots, n)$.

Duyệt quay lui: x_2 có thể chọn một trong các thành phố mà x_1 có đường đi tới (trực tiếp), với mỗi cách thử chọn x_2 như vậy thì x_3 có thể chọn một trong các thành phố mà x_2 có đường đi tới (ngoài x_1). Tổng quát: x_i có thể chọn 1 trong các thành phố chưa đi qua mà từ x_{i-1} có đường đi trực tiếp tới ($1 \leq i \leq n$).

Nhánh cận: Khởi tạo cấu hình *BestConfig* có chi phí = +∞. Với mỗi bước thử chọn x_i xem chi phí đường đi cho tới lúc đó có < Chi phí của cấu hình *BestConfig* ?, nếu không nhỏ hơn thì thử giá trị khác ngay bởi có đi tiếp cũng chỉ tốn thêm. Khi thử được một giá trị x_n ta kiểm tra xem x_n có đường đi trực tiếp về 1 không ?. Nếu có hãy đánh giá chi phí đi từ thành phố 1 đến x_n cộng với chi phí từ x_n đi đi trực tiếp về 1, nếu nhỏ hơn chi phí của đường đi *BestConfig* thì cập nhật lại *BestConfig* bằng cách đi mới.

Sau thủ tục tìm kiếm quay lui mà chi phí của *BestConfig* vẫn bằng +∞ thì có nghĩa là nó không tìm thấy một hành trình nào thoả mãn điều kiện đề bài để cập nhật *BestConfig*, bài toán không có lời giải, còn nếu chi phí của *BestConfig* < +∞ thì in ra cấu hình *BestConfig* - đó là hành trình ít tốn kém nhất tìm được

```
#include<bits/stdc++.h>
using namespace std;
int n, m;
int c[20][20], x[20], t[20], res;
bool ch[20];
const int inf = 1000000007;
void Try(int i){
   for (int j = 2; j <= n; j ++) // thử các tp từ 2 -> n
      if (!ch[j]){ // tp j chưa được thăm
         x[i] = j;
         t[i] = t[i - 1] + c[x[i - 1]][j]; // chi phí.
```

```
            if (t[i] < res) {
                if (i < n) // nếu chưa đi hết
                {
                    ch[j] = 1;
                    Try(i + 1);
                    ch[j] = 0;
                }
                else {
                    if (t[n] + c[x[n]][1] < res)// quay lại tp 1
                        res = t[n] + c[x[n]][1];
                }
            }

        }
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("tourism.inp","r", stdin);
    freopen("tourism.out","w", stdout);
    cin >> n >> m;
    for (int i = 1; i <= n; i ++)
        for (int j = 1; j <= n; j ++)
            if (i == j) c[i][j] = 0;
            else c[i][j] = inf;
    int u, v;
    for (int i = 1; i <= m; i ++){
        cin >> u >> v;
        cin >> c[u][v];
        c[v][u] = c[u][v];
    }
    memset(ch, 0, sizeof(ch)); //đánh dấu các thành phố chưa thăm
    ch[1] = true; // trừ tp 1.
    x[1] = 1; // xuất phát từ thành phố 1
    t[1] = 0;// chi phí xuất phát là 0.
    res = inf;
    Try(2);
```

```
    if (res == inf) // không có đường đi
        cout << -1;
    else cout << res;
    return 0;
}
```

Câu 2. Tên bài: **CAITUI.CPP**

Ta sẽ sinh hết tất cả các cấu hình, với gói hang thứ *i* thì tên trộm có thể lấy hoặc không lấy. Tuy vậy, ta có thể thêm nhánh cận vào để giảm thời gian chương trình như sau :

+ Nếu trong cấu hình đang xây dựng, tổng $W > m$ thì không cần xây dựng tiếp vì nó không thỏa mãn

+ Sau khi xây dựng được một vài cấu hình, ta được 1 kết quả tạm thời là *res*. Nếu trong cấu hình tiếp theo, khi xây dựng đến gói hàng thứ *i* và có tổng giá trị lấy được từ *1* → *i – 1* là *val*, thì *val + sum_val(i , i + 1, ... , n) <= res* thì không cần xây dựng nữa. Trong code dưới *sum[i]* là tổng các giá trị của các gói hàng *i, i+1, ...n*

```
#include<bits/stdc++.h>
using namespace std;
struct  goihang{
    int w;
    int v;
};
int n, m;
goihang  a[43];
int sum[43];
int res = 0;
int Try(int i, int w, int val){
    if (w > m) return 0; // Đặt cận để bẻ nhánh
    if (val + sum[i] < res) return 0; // Thêm 1 cận
    if (i == n + 1 && w <= m) return val;
    int r1 = Try(i + 1, w + a[i].w, val + a[i].v); // lấy hàng thứ i
    int r2 = Try(i + 1, w, val); // không lấy hàng thứ i
    return res = max(r1, r2);
}
int main(){
    ios_base::sync_with_stdio(false);
```

```
    cin.tie(0); cout.tie(0);
    freopen("CAITUI.inp","r", stdin);
    freopen("CAITUI.out","w", stdout);
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        cin >> a[i].w >> a[i].v;
    sum[n] = a[n].v;
    for (int i = n - 1; i >= 1; i --)
        sum[i] = sum[i + 1] + a[i].v;
    cout << Try(1, 0, 0);
    return 0;
}
```

Câu 3. Tên bài: **NGOAC.CPP**

Dùng thuật toán quay lui với mảng *x* đếm số lượng ngoặc mở và ngoặc đóng. Ta xem ngoặc mở là 0, ngoặc đóng là 1. Tại mỗi bước chọn phải đảm bảo rằng số lượng của dấu ngoặc đó phải nhỏ hơn hoặc bằng *n div 2*. Đó là cận thứ nhất. Tiếp theo phải đảm bảo rằng sau mỗi bước chọn số lượng ngoặc mở phải lớn hoặc bằng số lượng ngoặc đóng. Đó là cận thứ hai.

```
#include<bits/stdc++.h>
using namespace std;
int n, res, x[22];
void output(){
    res++;
    for (int i = 1; i <= n; i ++)
        if (x[i] == 0) cout << '(';
        else cout << ')';
    cout << '\n';
}
void Try(int i, int cnt1, int cnt2,  int dif ){
    if (i == n + 1 && dif == 0) { output();  return; }
    if (cnt1 > n / 2 || cnt2 > n / 2) return;
    if (dif < 0) return;
    for (int j = 0; j <= 1; j ++){
        x[i] = j;
        if (j == 0) Try(i + 1, cnt1 + 1, cnt2, dif + 1);
        else Try(i + 1, cnt1, cnt2 + 1, dif - 1);
```

```
    }
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("NGOAC.inp", "r" , stdin);
    freopen("NGOAC.out", "w" , stdout);
    cin >> n;
    res = 0;
    Try(1, 0, 0, 0);
    cout << res;
    return 0;
}
```

Câu 4. Tên bài: **TRES.CPP**

Mình sẽ sinh tất cả cấu hình các món ăn , sau đó thử từng người khách xem có thỏa mãn không. Đặt cận ở chỗ ta xét nếu người khách nào mà có số món ăn yêu thích > 2 thì không cần xét tiếp cấu hình đó nữa. Thứ hai, nếu tổng số tiền đang xây dựng ở cấu hình này lớn hơn số tiền của cấu hình thỏa mãn trước đó thì không cần xây dựng nữa.

Ta thấy vì mỗi người có số lượng món ăn yêu thích là bất kì (không biết trước số lượng phần tử của tệp) nên khi đọc dữ liệu, ta sẽ dùng hàm *getline(...)* một xâu với mỗi người, sau đó chuyển từ xâu đó sang các số để dễ xử lí.

```
#include<bits/stdc++.h>
using namespace std;
const int base = 1e9 + 7;
int m, n;
int a[40], cnt[40], f[40][40], res = base, sum =0 ;
string s;
void process(){
    for (int i = 1; i <= n;i ++)
        if (cnt[i] < 2 || cnt[i] > 2) return;
    res = min(res, sum);
}
```

```
void Try(int id){
    if (id == m + 1) {process(); return;}
    if (sum > res) return;
    for (int x = 0; x <= 1; x++){
        bool kt = 1;
        if (x){
            sum += a[id];
            for (int i = 1; i <= n; i++){
                cnt[i] += f[i][id];
                if (cnt[i] > 2) kt = 0;
            }
            if (kt) Try(id + 1);
            sum -= a[id];
            for (int i = 1; i <= n; i++)
                cnt[i] -= f[i][id];
            return;
        }
        Try(id + 1);
    }
}
void sol(){
    sum = 0;
    for (int i = 1;i <= n;i ++)
        cnt[i] = 0;
    Try(1);
    if (res == base) cout << -1;
    else cout << res;
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie (0);
    cout.tie(0);
    freopen("tres.inp","r", stdin);
    freopen("tres.out", "w", stdout);
    cin >> m >> n;
    for (int i = 1; i <= m; i++) cin >> a[i];
    getline(cin, s);
    for (int i = 1; i <= n; i++){
```

```
        getline(cin, s);
        int d = 0;
        for (int j = 0; j < s.size(); j ++){
            if (s[j] != ' ') d = d * 10 + (s[j] - '0');
            else {
                f[i][d] = 1;
                d = 0;
            }
        }
        if (d != 0) f[i][d] = 1;
    }
    sol();
    return 0;
}
```

Câu 5. Tên bài: **MINE.CPP**

- Xét một ô, nếu 7 trong 8 ô xung quanh nó đều đã biết có mìn hay không thì có thể suy ra được ô còn lại có mìn hay không. Giả sử đã biết được toàn bộ các ô hàng trên cùng và hàng ngoài cùng bên trái có mìn hay không (dùng đệ quy quay lui để tạo ra giả thiết này)
- Xét ô đặc biệt là ô ở góc trái trên (1, 1), xung quanh nó chỉ có 3 ô, như vậy nếu biết được 2 ô xung quanh: 1 ô nằm ở hàng trên cùng (1, 2), 1 ô nằm ở hàng ngoài cùng bên trái (2, 1) thì ta có thể suy ra được ô thứ ba (1, 1). Lúc này ô (2, 2) lại đóng vai trò như ô (1, 1) ban nãy do đó ta tìm được (3, 2).
- Làm lần lượt ta tìm được toàn bộ bảng. Nếu có một bước nào đó không thực hiện được thì chứng tỏ giả thiết là sai, tiếp tục thực hiện đệ quy quay lui.
- ***Nhánh cận*** : Để giảm tính toán ta không nhất thiết phải tính toàn bộ hàng trên cùng và hàng ngoài cùng rồi mới suy ra toàn bảng để kiểm tra mà có thể vừa tính vừa suy, cụ thể suy ra toàn bộ hàng ngoài cùng bên trái, sau đó mỗi lần chạy đệ quy với một ô của hàng trên cùng ta xác định tất cả các ô cùng cột với nó có mìn hay không. Nếu ở bước nào đó không xác định được thì quay lui.

```
#include<bits/stdc++.h>
using namespace std;
int n, m, a[22][22], mine[22][22];
int h[8] = {-1, -1, 0, 1, 1, 1, 0, -1};
```

```
int c[8] = {0, -1, -1, -1, 0, 1, 1, 1};
void input(){
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> a[i][j];
}
void init(){
    for (int i = 0; i <= n + 1; i++)
        for (int j = 0; j <= m + 1 ; j ++)
            mine[i][j] = 0;
}
bool correct(){
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= m; j ++){
            int cnt = 0;
            for (int k = 0; k < 8; k++ )
                cnt += mine[i + c[k]][j + h[k]];
            if (cnt != a[i][j]) return 0;
        }
    }
    return 1;
}
bool res = false;
void output(){
    if (! correct()) return;
    res = true;
    int cnt  = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cnt += mine[i][j];
    cout << cnt << '\n';
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= m; j++)
            cout << mine[i][j] << " ";
        cout << '\n';
    }
}
```

```
bool check(int i, int j, int k){
   return (i+h[k] - 1>=1 && i+h[k] - 1<=n && j+c[k] - 1>=1 &&
j+c[k] - 1<=m);
}
bool possible(int j){
   // j là cột
   int cnt;
   for (int i = 2; i <= n; i++){
      // i là hàng
      cnt = 0;
      mine[i][j] = 0;
      for (int k = 0; k < 8; k++){
         if(check(i, j, k)) // kiểm tra xem có đi được theo hướng k không
            cnt += mine[i + h[k] - 1][j + c[k] - 1];
      }
      if (a[i - 1][j - 1] - cnt == 0) mine[i][j] = 0;
      else if (a[i - 1][j - 1] - cnt == 1) mine[i][j] = 1;
      else return 0;
   }
   return 1;
}
void try_row(int i){ // thử sai theo hàng
   if (res) return;
   if (i == m + 1) output();
   else for ( int j = 0; j <= 1; j++ ){
      mine[1][i]= j;
      if (possible(i)) // kiểm tra bằng nhánh cận
         try_row(i+1);
   }
}
void try_col(int i){ // thử sai theo cột
   if (res) return; // có kết quả rồi thì không thử nữa.
   if (i == n + 1) try_row(2);
   else for (int j = 1; j >= 0; j-- ){
      mine[i][1]=j;
      try_col(i+1);
   }
}
```

```
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("MINE.inp","r", stdin);
    freopen("MINE.out","w", stdout);
    input();
    init();
    try_col(1);
    return 0;
}
```

18. Quy Hoạch Động

1.Lí thuyết và nhận dạng các bài tập quy hoạch động.

1.1 Lí thuyết.

Nguyên lí Quy hoạch động có thể được hiểu như sau*: Quy hoạch động là lớp các bài toán mà quyết định ở bước thứ i phụ thuộc vào quyết định ở các bước đã xử lý trước đó.*

1.2 Nhận dạng các bài tập về quy hoạch động.

Một bài toán quy hoạch động phải thỏa mãn có thể sử dụng lời giải ở các trạng thái thấp hơn để tìm lời giải cho các trạng thái cao hơn và cuối cùng là lời giải tối ưu cho cả bài toán.

2. Các bước giải quyết một bài toán quy hoạch động.

Bước 1: **Xác định hàm mục tiêu.**

+ Ở đây, theo nguyên lí của quy hoạch động, ta xây dựng hàm mục tiêu F(i) là kết quả của bước thứ i.

Bước 2: **Xây dựng bài toán cơ sở và công thức truy hồi.**

+ Bài toán cơ sở là bài toán con nhỏ nhất mà ta có thể tính được kết quả dễ dàng.

+ Sau khi xây dựng bài toán cơ sở, ta sẽ tiến hành tìm công thức truy hồi của bài toán thứ i dựa trên kết quả của các bài toán trước đó.

Bước 3: **Lập bảng phương án.**

+ Sử dụng công thức truy hồi và nghiệm các bài toán cơ sở tính nghiệm tất cả các bài toán con và lưu trữ chúng vào bảng phương án.

Bước 4: **Kết luận.**

+ Tìm ra được nghiệm của bài toán

Ví dụ: Bài toán tìm số Fibonaci thứ N.

Hàm mục tiêu : Gọi F(i) là số fibonaci thứ i.

Bài toán cơ sở : F(1) = 1, F(2) = 1.

Công thức truy hồi

$$F(i) = F(i - 1) + F(i - 2).$$

Lập bảng phương án :

i	1	2	3	4	5	6	7	...	...
F(i)	1	1	2	3	5	8	13		

Kết luận: kết quả của bài toán là F(N)

3. Các dạng toán quy hoạch động điển hình.

3.1 Dãy các phần tử dài nhất thỏa mãn bài toán.

Câu 1. Tên bài **LIQ.CPP**

Cho dãy A_1, A_2, … A_n. Hãy tìm một dãy con tăng không ngặt dài nhất của dãy (dãy con có thể không liên tiếp). ($N <= 10^3$).

Dữ liệu: vào từ file văn bản **LIQ.INP**

+ Dòng đầu tiên chứa số nguyên N ($1 <= N <= 10^3$)

+ Tiếp theo là 1 dòng gồm N phần tử của dãy. Các số thuộc phạm vi *int*

Kết quả: ghi ra file văn bản **LIQ.OUT**: một số nguyên là kết quả bài toán

LIQ.INP	LIQ.OUT
5 1 3 2 2 5	4

Câu 2. Tên bài: **LIS.CPP**

Cho dãy A_1, A_2, … A_n. Hãy tìm một dãy con tăng ngặt dài nhất của dãy. Dãy con có các phần tử không nhất thiết phải liên tiếp.

Dữ liệu: vào từ file văn bản **LIS.INP**

+ Dòng đầu tiên chứa số nguyên N ($1 <= N <= 10^5$)

+ Dòng tiếp theo gồm N phần tử của dãy. Các số trong phạm vi *int*

Kết quả: ghi ra file văn bản **LIS.OUT**: một số nguyên là kết quả bài toán

Ví dụ:

LIS.INP	LIS.OUT
5 1 3 2 2 5	3

Câu 3. Tên bài: **TOWER.CPP**

Trong một cuộc thi tìm người thông thái nhất đất nước, Tí được cho *N* hình trụ đứng với nhiều kích thước khác nhau và yêu cầu phải xếp được tòa tháp cao nhất từ các hình trụ theo đúng thứ tự từ 1 đến *N* sao cho khối ở trên phải được xếp khít với khối ở dưới, hay đường kính đáy của hình trên không vượt quá đường kính đáy hình dưới. Một khối trụ có thể dùng hoặc không dùng nhưng phải đúng theo thứ tự đã cho. Hãy giúp Tí giải bài toán này nhé các bạn.

Dữ liệu: vào từ file **TOWER.INP**

+ Dòng đầu ghi số nguyên dương *N* ($N \leq 10^5$).

+ *N* dòng tiếp theo mỗi dòng ghi 2 số *Ri* và *Hi* là bán kính đáy và chiều cao hình trụ thứ *i*. ($1 \leq Ri \leq 500$, $1 \leq Hi \leq 500$).

Kết quả: Ghi ra file **TOWER.OUT** một số nguyên duy nhất là chiều cao lớn nhất của tòa tháp xếp được.

TOWER.INP	TOWER.OUT
4 4 2 2 5 1 3 3 1	10

Câu 4. Tên bài: **STADIUM.CPP**

Sân bóng mini *ABC* nhận được rất nhiều đơn đặt sân từ *n* đội bóng. Đội bóng *i* muốn sử dụng sân bóng trong khoảng thời gian từ a_i đến b_i và trả tiền thuê là c_i. Hãy giúp ông chủ sân tính toán sắp xếp lịch để nhận được nhiều tiền nhất và thỏa mãn 2 đội bóng bất kì đều có khoảng thời gian sử dụng sân bóng là không giao nhau.

Dữ liệu: vào từ file **STADIUM.INP** gồm

+ Dòng đầu là số nguyên *n*, là số đội bóng đặt sân bóng ($1 \le n \le 5000$)

+ *n* dòng tiếp theo, mỗi dòng gồm 3 chỉ số *ai, bi, ci* ($1 \le ai, bi, ci \le 10^4$) thể hiện thời gian mở đầu, kết thúc và tiền thuê sân của đội thứ *i*.

Kết quả: ghi ra file **STADIUM.OUT** số tiền lớn nhất mà chủ sân nhận được.

Ví dụ:

STADIUM.INP	STADIUM.OUT
5 1 3 4 1 5 5 3 6 2 4 7 9 5 7 6	13

Câu 5. Tên bài: **EXP.CPP**

Cho N số tự nhiên A1, A2, …, An. Ban đầu các số được đặt liên tiếp theo đúng thứ tự cách nhau bởi dấu "?": A1 ? A2 ? ... ? An. Cho trước số nguyên S, có cách nào thay các dấu ? bằng dấu + hay dấu − để được một biểu thức số học cho giá trị là S không?

Dữ liệu: vào từ file **EXP.INP** gồm:

+ Dòng đầu tiên là số N ($1 \le N \le 500$)

+ Dòng 2 gồm N số là các phần tử Ai ($1 \le Ai \le 50$)

+ Dòng 3 là một số S. ($-25000 \le S \le -25000$)

Kết quả: ghi ra file **EXP.OUT** chữ "YES" nếu tồn tại cách thỏa mãn, ra "NO" nếu không tồn tại.

Ví dụ:

EXP.INP	EXP.OUT
4 1 2 3 4 4	YES
5 2 3 1 4 5 14	NO

Câu 6. Tên bài: **SPSEQ.CPP**

Một dãy được xem là đẹp nếu nó là 1 dãy các số nguyên dương và có các đặc điểm sau:

+ Độ dài của dãy là 1 số lẻ: **L = 2*N + 1**

+ **N + 1** số nguyên đầu tiên của dãy tạo thành 1 dãy tăng

+ **N + 1** số nguyên cuối của dãy tạo thành 1 dãy giảm

+ Không có 2 số nguyên nào cạnh nhau trong dãy có giá trị bằng nhau

Ví dụ: **1, 2, 3, 4, 5, 4, 3, 2, 1** là 1 dãy đẹp độ dài **9**. Tuy nhiên, dãy **1, 2, 3, 4, 5, 4, 3, 2, 2** không là 1 dãy đẹp.

Yêu cầu: Trong các dãy con của dãy số cho trước, tìm dãy đẹp có độ dài dài nhất.

Dữ liệu: vào từ file **SPSEQ.INP** gồm:

+ Dòng 1: số nguyên dương N ($N \leq 10^5$), độ dài dãy số.

+ Dòng 2: N số nguyên dương a_i ($a_i \leq 10^9$).

Kết quả: ghi ra file SPSEQ.OUT là một số nguyên dương duy nhất là độ dài dãy đẹp dài nhất.

Ví dụ:

SPSEQ.INP	SPSEQ.OUT
19 1 2 3 2 1 2 3 4 3 2 1 5 4 1 2 3 2 2 1	9

Câu 7. Tên bài: **BEADS.CPP**

Trong một đợt đi du lịch ở Sầm Sơn, sáng sớm Trạng Tí thường đi dạo dọc bờ biển và nhặt những vỏ ốc rồi xâu chúng lại thành một chuỗi. Nguyên tắc tạo chuỗi ốc của Trạng Tí như sau: Ban đầu từ chuỗi rỗng, không có vỏ ốc; khi gặp một vỏ ốc mới, có thể lấy để xâu vào một trong hai đầu của chuỗi hoặc hoặc bỏ đi không lấy; cuối cùng nhận được một chuỗi vỏ ốc mà tính từ đầu chuỗi đến cuối chuỗi, các vỏ ốc có kích thước tăng dần (tăng ngặt) và gồm càng nhiều vỏ ốc càng tốt.

Yêu cầu: Cho trước dãy A[1], A[2], …, A[n] là kích thước của các vỏ ốc mà Trạng Tí lần lượt gặp khi đi dọc bờ biển, hãy tìm cách nhặt và xâu chuỗi để được chuỗi gồm nhiều vỏ ốc nhất

Dữ liệu: vào từ file **BEADS.INP** gồm:

+ Dòng 1 chứa số nguyên dương n ($n \leq 10^5$)

+ Dòng 2 chứa n số nguyên dương A[1], A[2], …, A[n] ($Ai \leq 10^9$)

Kết quả: ghi ra file **BEADS.OUT** một số nguyên duy nhất là số lượng vỏ ốc trong chuỗi tạo được.

Ví dụ:

BEADS.INP	BEADS.OUT
5 4 4 5 3 1	4

Câu 8. Tên bài: **STICK.CPP**

Bình có N đoạn gỗ cần thi công ngày hôm nay. Để xử lý chúng bác cần thời gian để chuẩn bị :

+ Thời gian chuẩn bị cho đoạn gỗ đầu tiên là 1 phút.

+ Sau khi xử lý xong đoạn gỗ có chiều dài L và trọng lượng W, không mất thời gian xử lý nếu đoạn gỗ tiếp theo có độ dài L’ và trọng lượng W’ thỏa $L \leq L'$ và $W \leq W'$. Ngược lại mất 1 phút để chuẩn bị.

Các bạn hãy giúp Bình tìm thời gian chuẩn bị ít nhất cho N đoạn gỗ.

Ví dụ: có 5 đoạn gỗ với các chỉ số : (9,4), (2,5), (1,2), (5,3) và (4,1) thì thời gian ít nhất là 2 phút vì có thể xử lý theo thứ tự như sau (4,1), (5,3), (9,4), (1,2), (2,5).

Dữ liệu: vào từ file **STICK.INP** gồm

+ Dòng đầu là số lượng test T ($1 \le T \le 100$)

+ Các dòng tiếp theo ứng với mỗi test có định dạng như sau:

+Dòng đầu là số nguyên dương $N \le 5000$.

+Sau đó là N dòng gồm N cặp số nguyên dương L[i] và W[i] tương ứng là độ dài và trọng lượng của các khối gỗ. (L[i], W[i] ≤ 10000)

Kết quả: ghi ra file **STICK.OUT** T dòng, mỗi dòng là thời gian ít nhất để chuẩn bị các đoạn gỗ của test đó.

Ví dụ:

STICK.INP	STICK.OUT
3 5 4 9 5 2 2 1 3 5 1 4 3 2 2 1 1 2 2 3 1 3 2 2 3 1	2 1 3

Câu 9. Tên bài: **NEMCHUA.CPP**

Nem chua Anh Dân là một cở sở sản xuất lớn ở Thanh Hoá. Với thương hiệu đã xây dựng được hơn 30 năm và được nhiều người biết đến. Vào một ngày đẹp trời, VanhG được mời đến để thăm xưởng sản xuất nem chua, anh vô

cùng ngạc nhiên bởi khả năng gói nem siêu đẳng của các công nhân. Các công nhân được ngồi trong phòng khép kín ở khu vực sản xuất nhìn rất chuyên nghiệp, mỗi công nhân có một cái rổ để đựng nem khi gói xong. VanhG với biệt danh là code khủng nhưng hay bị lũng nghĩ ra trong đầu một số nguyên dương M, và có một bài toán để về đố các bạn trên lớp của mình như sau: Cho N công nhân gói nem, mỗi công nhân gói được tương ứng số nem là a_1, a_2, … a_N. Hãy chia N công nhân này thành M nhóm sao cho tổng chi phí của M nhóm là nhỏ nhất có thể. Chi phí của một nhóm được tính bằng chênh lệnh giữa công nhân có số lượng nem nhiều nhất với công nhân có số lượng nem ít nhất trong nhóm đó. Nếu một nhóm có 0 hoặc 1 công nhân, chi phí của nhóm đó bằng 0

Yêu cầu: Hãy giúp VanhG tìm cách phân chia N công nhân vào M nhóm sao cho tổng chi phí là nhỏ nhất có thể.

Dữ liệu: vào từ file **NEMCHUA.INP** gồm:

+ Dòng đầu tiên là hai số nguyên N, M (0<= N, M <= 200)

+ Dòng thứ 2 là N số nguyên a_1, a_2, …a_N tương ứng là số nem mà công nhân thứ i gói được. (0 <= a_i <= 10^9).

Kết quả: ghi ra file **NEMCHUA.OUT** một dòng duy nhất là tổng chi phí nhỏ nhất có thể.

Ví dụ:

NEMCHUA.INP	NEMCHUA.OUT
8 3 5 2 3 10 7 2 6 8	4

Giải thích: Chia thành 3 nhóm (5 7 6 8), (10), và (2 3 2) chi phí từng nhóm tương ứng là 8-5 = 3, 10 – 10 = 0, 3 – 2 = 1 => tổng chi phí là 4

Câu 10. Tên bài: **BUNDAU.CPP**

Bún Đậu Hoàng Long là một nhà hàng mới nổi trong nghành nghề bún đậu mắm tôm. Ở đây có không gian đẹp và thoáng. Có vị trí đắc địa, nằm ngay đối diện cổng trường lái của Học viện cảnh sát tại khu đô thị Hoàng Long (phường Long Anh, TP Thanh Hoá). Ở đây có rất nhiều món ăn đi kèm với bún đậu như: chả cốm, dồi sụn, chân giò…và còn được free nước uống. Tình

cờ VanhG đang lướt facebook thì biết được địa điểm này. Vào một ngày VanhG code toàn bị lũng nên anh quyết định đi đến Bún Đậu Hoàng Long để thưởng thức. Vừa đến nơi thì VanhG thấy trong bếp có N mẹt bún, mỗi mẹt bún có rất nhiều lá bún bên trong, các mẹt bún này được xếp thành hàng ngang trông rất là đẹp mắt, anh liền nghĩ ra một bài toán để đố các bạn trên lớp: cho N mẹt bún, trong các mẹt có các lá bún tương ứng là a_1, a_2, … a_N. Hãy tìm các mẹt bún liên tiếp dài nhất mà có tổng các lá bún là một số nguyên tố.

Yêu cầu: Hãy giúp VanhG giải bài toán trên

Dữ liệu: vào từ file **BUNDAU.INP** gồm:

+ Dòng đầu tiên là số nguyên N ($0 < N <= 5.10^3$)

+ Dòng thứ 2 là N số nguyên a_1, a_2, …a_N là số lá bún của các mẹt. $0 \leq a_i \leq 10^3$

Kết quả: ghi ra file **BUNDAU.OUT** một dòng duy nhất là số lượng mẹt bún thoả mãn đề bài (nếu không có đáp án thoả mãn thì in ra -1)

Ví dụ:

BUNDAU.INP	BUNDAU.OUT
5 1 1 1 1 1	5

3.2 Biến đổi xâu

Câu 1. Tên bài: **STRING.CPP**

Cho hai xâu *S* và *T* chỉ gồm các ký tự in thường. Bạn được phép thực hiện một trong ba thao tác sau trên xâu *S*:

- Chèn một ký tự bất kỳ vào xâu.
- Xoá một ký tự bất kỳ trong xâu.
- Thay đổi một ký tự bất kỳ trong xâu thành ký tự in thường khác.

Yêu cầu: Tìm số thao tác ít nhất để đưa xâu *S* về xâu *T*.

Dữ liệu: vào từ file **STRING.INP** gồm 2 dòng, mỗi dòng chứa một xâu gồm các kí tự trong *'a'* đến *'z'* viết liền nhau, giới hạn độ dài không quá 10^3.

Kết quả: ghi ra file **STRING.OUT** là kết quả tương ứng số thao tác ít nhất cần thực hiện.

STRING.INP	STRING.OUT
abc ae	2

Câu 2. Tên bài: **QBSTR.CPP**

Cho 2 xâu *X, Y*. Hãy tìm xâu con của *X* và của *Y* có độ dài lớn nhất. Biết xâu con của một xâu thu được khi xóa một số kí tự thuộc xâu đó (hoặc không xóa kí tự nào).

Dữ liệu: vào từ file **QBSTR.INP** gồm

+ Dòng đầu gồm xâu *X*

+ Dòng hai gồm xâu *Y*

(*chú ý 2 xâu viết liền không cách, độ dài 2 xâu không quá* 10^3)

Kết quả: ghi ra file **QBSTR.OUT** là độ dài xâu con chung lớn nhất.

Ví dụ:

QBSTR.INP	QBSTR.OUT
abc1def2ghi3 abcdefghi123	10

Câu 3. Tên bài: **NKPALIN.CPP**

Một chuỗi được gọi là đối xứng (palindrome) nếu như khi đọc chuỗi này từ phải sang trái cũng thu được chuỗi ban đầu.

Yêu cầu: tìm chuỗi con đối xứng dài nhất của một chuỗi s cho trước. Chuỗi con là chuỗi thu được khi xóa đi một số ký tự từ chuỗi ban đầu.

Dữ liệu: vào từ file **NKPALIN.INP** gồm một dòng duy nhất chứa chuỗi s, chỉ gồm những chữ cái in thường. (Độ dài xâu s <= 2000)

Kết quả: ghi ra file **NKPALIN.OUT** là một xâu con đối xứng dài nhất của xâu s. Nếu có nhiều kết quả, chỉ cần in ra một kết quả bất kỳ.

NKPALIN.INP	NKPALIN.OUT
lmevxeyzl	level

Câu 4. Tên bài: **COUNTPL.CPP**

Palindrome là xâu ký tự mà nếu đọc nó từ trái sang phải cũng như từ phải sang trái ta được cùng một xâu. Một xâu ký tự bất kỳ luôn có thể biểu diễn như là một dãy các *Palindrome* nếu như ta coi xâu chỉ gồm một ký tự luôn là một palindrome. Ví dụ: Xâu '*bobseesanna*' có thể biểu diễn dưới dạng dãy các palindrome theo nhiều cách, chẳng hạn:

'*bobseesanna*' = '*bob*' + '*sees*' + '*anna*'

'*bobseesanna*' = '*bob*' + 's' + '*ee' + 's' + 'anna'*

'bobseesanna' = 'b' + 'o' + 'b' + 'sees' + 'a' + 'n' + 'n' + 'a'

Yêu cầu: Cho xâu ký tự *s* (chỉ chứa các kí tự chữ cái) cần tìm cách biểu diễn xâu *s* dưới dạng một dãy gồm số ít nhất các *palindrome*. Ví dụ: Cho s = '*bobseesanna*', do ta có '*bobseesanna*' = *'bob' + 'sees' + 'anna'* và không thể biểu diễn *'bobseesanna'* bởi ít hơn là 3 *Palindrome* nên biểu diễn này chính là biểu diễn cần tìm.

Dữ liệu: vào từ file **COUNTPL.INP** xâu ký tự s, không quá 1000 ký tự.

Kết quả: ghi ra file **COUNTPL.OUT** số nguyên k là số lượng ít nhất các *Palindrome* trong biểu diễn tìm được.

Ví dụ:

COUNTPL.INP	COUNTPL.OUT
bobseesanna	3

Câu 5. Tên bài: **PAL.CPP**

Xâu con B là xâu con đối xứng của xâu A khi và chỉ khi xâu B là xâu con của xâu A và xâu B là một xâu đối xứng. Ví dụ: TIT là xâu con đối xứng của xâu TIKTOK, nhưng xâu TIK hay TBT thì không phải. Trạng Tí nhận ra điều này rất hay ho nên đố thử người bạn Ếch Ộp của mình rằng, trong một xâu bất kì, hãy đếm xem có bao nhiêu xâu con là 1 xâu đối xứng. Hãy giúp Ếch Ộp giải quyết bài toán này nhé.

Dữ liệu: vào từ file **PAL.INP** gồm 1 xâu *S* (gồm chữ cái in hoa, độ dài không quá 200 chữ)

Kết quả: ghi ra file **PAL.OUT** là số xâu con đối xứng (lấy dư cho $10^9 + 7$).

Ví dụ:

PAL.INP	PAL.OUT
AAB	4
IOIKATU	9

Câu 6. Tên bài: **GONME.CPP**

Sau khi quá chán với việc tìm xâu con chung của 2 xâu, Tí đã nghĩ ra một bài toán mới lại hơn kết hợp với số học. Cho *m* dãy là hoán vị của *n* số tự nhiên đầu tiên. Hãy tìm độ dài dãy con chung dài nhất của *m* dãy đó. Dù nghĩ ra đề nhưng Tí vẫn chưa thể giải quyết được bài toán này, hãy giúp Tí nhé mọi người.

Dữ liệu: vào từ file **GONME.INP** gồm

+ Dòng đầu chứa số hai số nguyên *n* và *m* (*1 ≤ n ≤ 1000, 1 ≤ m ≤ 10*)

+ *m* dòng sau mỗi dòng chứa *n* số nguyên là dãy hoán vị của *n* số tự nhiên đầu tiên

Kết quả: ghi ra file **GONME.OUT** là độ dài lớn nhất của dãy con chung.

Ví dụ:

GONME.INP	GONME.OUT
5 3 1 5 3 4 2 1 3 4 2 5 3 1 5 4 2	3

Câu 7: Tên bài: **SUBSTRK.CPP**

Một xâu kí tự gọi là xâu nhị phân nếu nó chỉ chứa hai kí tự ‘0’ hoặc ‘1’. Xâu ***v*** gọi là xâu con của xâu ***S*** nếu xâu ***v*** khác rỗng và được tạo bởi các kí tự liên tiếp trong xâu ***S*** (thứ tự giữ nguyên). Hai xâu con ***u*** và ***v*** của xâu ***S*** là khác nhau nếu nó có độ dài khác nhau hoặc được tạo từ các kí tự ở vị trí khác nhau trong xâu ***S***. Ví dụ: xâu “010” có các xâu con là “0”, “1”, “0”, “01”, “10”, “010”.

Yêu cầu: Cho trước một xâu nhị phân ***S*** và số nguyên dương ***k***, hãy đếm xem có bao nhiêu xâu con của xâu ***S*** chứa đúng ***k*** kí tự ‘1’.

Dữ liệu vào: Vào từ file văn bản **SUBSTRK**.INP gồm:

- Dòng 1 chứa một số nguyên k ($0 \le k \le 10^6$).
- Dòng 2 chứa xâu nhị phân S có độ dài không quá 10^6.

Kết quả: Ghi ra file văn bản **SUBSTRK**.OUT một số nguyên duy nhất là kết quả tìm được.

Ví dụ:

SUBSTRK.INP	SUBSTRK.OUT
2 01010	4

Câu 8. Tên bài: **BLAST.CPP**

Cho hai xâu A, B. Mở rộng của 1 xâu X là xâu thu được bằng cách chèn (0, 1, 2 ...) kí tự trống vào xâu. Ví dụ: X là '*abcbcd*', thì *'abcb-cd'*, '*-a-bcbcd-*' và *'abcb-cd-*' là các mở rộng của X. (Dấu cách kí hiệu bằng '-'). A1, B1 là mở rộng của A và B; giả sử chúng cùng độ dài. Khoảng cách giữa A1 và B1 là tổng khoảng cách giữa các kí tự cùng vị trí. Nếu hai kí tự không là dấu cách thì khoảng cách giữa 2 kí tự này là trị tuyệt đối mã ASCII của chúng. Còn ngược lại, khoảng cách là 1 số K cố định.

Yêu cầu: Cho hai xâu A, B. Tìm khoảng cách nhỏ nhất giữa hai xâu mở rộng của nó.

Dữ liệu: vào từ file **BLAST.INP** gồm:

+ Dòng 1 chứa xâu A, dòng 2 chứa xâu B, chỉ gồm các chữ cái in thường từ 'a' đến 'z' và số kí tự ≤ 2000.

+ Dòng thứ 3 là số K, khoảng cách của 1 kí tự bất kỳ với kí tự trống, 1≤ K≤ 100.

Kết quả: ghi ra file **BLAST.OUT** là khoảng cách nhỏ nhất.

Ví dụ:

BLAST.INP	BLAST.OUT
cmc snmn 2	10
mj jao 4	12

Câu 9. Tên bài: **STMERGE.CPP**

Cho 2 xâu ký tự $X = x_1, x_2, .., x_m$ và $Y = y_1, y_2, ..., y_n$. Cần xây dựng xâu $T = t_1, t_2, t_3, .., t_{n+m.}$ gồm tất cả các ký tự trong xâu X và tất cả các ký tự trong xâu *Y*, sao cho các ký tự trong *X* xuất hiện trong *T* theo thứ tự xuất hiện trong *X* và các ký tự trong *Y* xuất hiện trong T theo đúng thứ tự xuất hiện trong *Y*, đồng thời với tổng chi phí trộn là nhỏ nhất. Tổng chi phí trộn hai xâu *X* và *Y* để thu được xâu *T* được tính bởi công thức c(*T*) = sum(c(t_k, t_{k+1})) với k = 1, 2, .., n+m-1; trong đó, các chi phí c(t_k, t_{k+1}) được tính như sau:

+ Nếu hai ký tự liên tiếp t_k, t_{k+1} được lấy từ cùng một xâu X hoặc Y thì c(t_k, t_{k+1}) = 0

+ Nếu hai ký tự liên tiếp t_k, t_{k+1} là x_i y_j thì chi phí phải trả là c(x_i, y_j). Nếu hai ký tự liên tiếp t_k, t_{k+1} là y_j, x_i thì chi phí phải trả là c(y_j, x_i) = c(x_i, y_j)

Dữ liệu: Vào từ file **STMERGE.INP** gồm

+ Dòng đầu tiên chứa Q là số lượng bộ dữ liệu. tiếp đến là Q nhóm dòng, mỗi nhóm cho thông tin về 1 bộ dữ liệu theo khuôn dạng sau:

+ Dòng thứ nhất chứa 2 số nguyên duong m, n (m, n <= 1000);

+ Dòng thứ *i* trong m dòng tiếp theo chứa n số nguyên dương, mỗi số không vượt quá 10^9: c(x_i, y_1), c(x_i, y_2), …, c(x_i, y_n), *i* = 1, 2,…, m.

Kết quả: Ghi ra file STMERGE .OUT gồm Q dòng, mỗi dòng chứa một số nguyên là tổng chi phí theo cách xây dựng xâu *T* tìm được tương ứng với bộ dữ liệu vào.

Ví dụ:

STMERGE.INP	STMERGE.OUT
1 2 3 3 2 30 15 5 4	6

Câu 10. Tên bài: **SUBSTR**

Hôm nay các học viên nhí của Trung tâm Anh ngữ ABC Smart được học về các nguyên âm và phụ âm. Trong tiếng anh có 5 nguyên âm chính là *a, e, i, o, u*; các chữ cái còn lại có thể coi là phụ âm. *VanhG* là một học sinh rất thông

minh, anh đã phát hiện ra một xâu $S_1S_2S_3...S_k$ độ dài k có tính chất kỳ lạ như sau:

+ k là số chẵn

+ Nếu S_i và S_{k-i+1} là nguyên âm nếu i là số chẵn, là phụ âm nếu i là số lẻ. Hoặc ngược lại, nếu S_i và S_{k-i+1} là phụ âm nếu i là số chẵn, là nguyên âm nếu i là số lẻ. (Với mọi $i <= k/2$)

+ Nếu xâu rỗng thì ta coi như xâu có tính chất chất kỳ lạ độ dài bằng *0*

Ví dụ: các xâu *"abba", "baab", "abeaca"* … là các xâu có tính chất kỳ lạ. Các xâu: *"abbbba", "baaaab", "abcd"*…không có tính chất kỳ lạ.

VanhG nghĩ ra một bài toán để đố các bạn của mình trong lớp: cho một xâu *S* có độ dài *n* chỉ gồm các kí tự chữ cái latin từ *"a"* đến "*z*". Hãy xóa đi ít nhất các kí tự để xâu *S* trở thành xâu có tính chất kỳ lạ như trên.

Yêu cầu: Các bạn hãy giải bài toán trên.

Dữ liệu: Vào từ file **SUBSTR.INP** gồm:

+ Dòng đầu tiên là số nguyên dương n ($n <= 5.10^3$)

+ Dòng thứ 2 là xâu *S* có độ dài *n*

Kết quả: Ghi ra file **SUBSTR.OUT:** là một số nguyên là kết quả của bài toán.

Ví dụ:

SUBSTR.INP	SUBSTR.OUT	SUBSTR.INP	SUBSTR. OUT
2 aa	0	2 ab	2

Ràng buộc:

+ Có 50% số test $n \leq 20$

+ Có 50% số test $n \leq 5.10^3$

3.3. Dạng bài toán cái túi và ứng dụng.

Câu 1. Tên bài: **VALIA.CPP**

Có *n* đồ vật, vật thứ *i* có trọng lượng *A[i]* và giá trị *B[i]*. Hãy chọn ra một số các đồ vật, mỗi vật một cái để xếp vào 1 vali có trọng lượng tối đa là *w* sao cho tổng giá trị của vali là lớn nhất.

Dữ liệu: vào từ file **VALIA.INP** gồm:

+ Dòng đầu tiên gồm 2 số nguyên dương *n* và *w* (*n <= 100, w <= 1000*)

+ *n* dòng tiếp theo, mỗi dòng chứa 2 số nguyên dương A_i và B_i (A_i, B_i <= *100*) lần lượt là trọng lượng và giá trị của đồ vật thứ *i*.

Kết quả: ghi ra file **VALIA.OUT** là giá trị lớn nhất của vali.

VALIA.INP	VALIA.OUT
3 4 1 4 2 5 3 6	10

Câu 2: Tên bài: **VALIB.CPP**

Có *n* đồ vật, vật thứ *i* có trọng lượng *A[i]* và giá trị *B[i]*. Hãy chọn ra một số các đồ vật, mỗi vật một cái để xếp vào 1 vali có trọng lượng tối đa là *w* sao cho tổng giá trị của vali là lớn nhất. Lưu ý: mỗi đồ vật có thể chọn được nhiều lần.

Dữ liệu: vào từ file **VALIB.INP** gồm:

+ Dòng đầu tiên gồm 2 số nguyên dương *n* và *w* (*n <= 100, w <= 1000*)

+ *n* dòng tiếp theo, mỗi dòng chứa 2 số nguyên A_i và B_i (A_i, B_i <= *100*) lần lượt là trọng lượng và giá trị của đồ vật thứ *i*.

Kết quả: ghi ra file **VALIB.OUT** một dòng là giá trị lớn nhất của vali

VALIB.INP	**VALIB.OUT**
3 4 1 4 2 5 3 6	16

Câu 3: Tên bài: **SEQ.CPP**

Cho *n* số nguyên dương tạo thành dãy A=*{A_1, A_2, ..., A_n}*. Tìm ra một dãy con của dãy *A* (không nhất thiết là các phần tử liên tiếp trong dãy) có tổng bằng *S* cho trước.

Dữ liệu: vào từ file **SEQ.INP** gồm:

+ Dòng đầu là nguyên dương *N* và *S (0 < n ≤ 200; 0 <S ≤40000)*

+ Các dòng tiếp theo lần lượt ghi *n* số hạng của dãy *A* là các số *A_1, A_2, ..., A_n (0 < A_i ≤ 200)*

Kết quả: ghi ra file **SEQ.OUT** nếu bài toán vô nghiệm thì in ra "NO" ngược lại in ra "YES"

Ví dụ:

SEQ.INP	SEQ.OUT
5 6 1 2 4 3 5	YES

Câu 4. Tên bài: **FARMER.CPP**

Phú ông có *N* mảnh đất và *M* dải đất trong làng. Các mảnh đất có thể được xem là một tứ giác và các dải đất thì như một đường thẳng. Dọc theo các dải đất, phú ông trồng các cây bách, dải đất thứ *i* có A_i cây bách. Phú ông cũng trồng các cây bách trên viền của các mảnh đất, mảnh đất thứ *j* có B_j cây bách. Cả ở trên các mảnh đất và dải đất, xen giữa 2 cây bách, phú ông trồng một cây ôliu. Phú ông vì rất thích quạt mo của Bờm nên muốn đổi cho Bờm các mảnh đất và dải đất tuỳ ý. Tuy vậy, vốn là người khôn ngoan, phú ông đưa thêm điều kiện tổng số cây bách mà Bờm chọn không vượt quá *Q* cây. Bờm phải chọn thế nào để có nhiều cây ôliu (loài cây mà Bờm ta thích nhất). Hãy giúp Bờm nhé các bạn.

Dữ liệu: vào từ file **FARMER.INP** gồm

+ Dòng đầu tiên gồm 3 số *Q, N, M*. (*0 < Q ≤ 30000, 0 ≤ N, M ≤ 1000*)

+ Dòng thứ hai gồm *N* số A_i là số cây bách ở mảnh đất thứ *i* ($3 \le A_i \le 150$)

+ Dòng thứ ba gồm *M* số B_i thể hiện số cây bách ở dải đất thứ *i* ($3 \le B_i \le 150$)

Kết quả: ghi ra file **FARMER.OUT** là số cây oliu mà Bờm có thể lấy được.

Ví dụ:

FARMER.INP	FARMER.OUT
17 3 3 13 4 8 4 8 6	17

Giải thích: Bờm sẽ chọn 2 mảnh đất là mảnh đất thứ nhất và thứ hai.

Câu 5. Tên bài: **MONSTER.CPP**

Bờm phải đi qua một thung lũng đầy cướp. Mỗi tên cướp có một chỉ số thể hiện sự hung dữ. Cụ thể tên cướp thứ *i* có mức độ hung dữ là một số nguyên *H[i]*. Bờm đứng trước nguy cơ bị bọn cướp tấn công, tuy nhiên Bờm ta có thể sống sót bằng những đồng vàng của mình để mua chuộc một số tên cướp. Tên cướp thứ *i* sẽ đòi *T[i]* đồng vàng để bảo vệ cho Bờm. Khi gặp một tên cướp, nếu nó có độ hung dữ lớn hơn tổng độ hung dữ của số tên cướp mà Bờm đã mua chuộc thì tên cướp đó sẽ tấn công Bờm. Nói các khác để tồn tại Bờm bắt buộc phải mua chuộc tên cướp này. Trong trường hợp ngược lại khi gặp một tên cướp có độ hung dữ nhỏ hơn hoặc bằng tổng độ hung dữ của số tên cướp mà Bờm đã mua chuộc thì nó không thể tấn công Bờm và Bờm có thể mua chuộc nó hoặc không. Bạn hãy tính số vàng ít nhất mà Bờm phải dùng để vượt qua được thung lũng.

Dữ liệu: vào từ file văn bản **MONSTER.INP** gồm:

+ Dòng đầu ghi số nguyên N $(1 \le N \le 10^3)$ là số tên cướp và V $(0 \le V \le 10^4)$ là số vàng Bờm có.

+ Dòng thứ 2 ghi N số nguyên $H1, H2, ..., H_N$. $(0 \le Hi \le 10^6)$

+ Dòng thứ 3 ghi N số nguyên dương $T1, T2, ..., T_N$. $(Ti \le 10)$

Kết quả: Ghi ra file **MONSTER.OUT** là số vàng ít nhất Bờm sử dụng nếu vượt qua được thung lũng, ngược lại ghi -1.

MONSTER.INP	MONSTER.OUT
3 5 8 5 10 1 1 2	2

3.4. Ghép cặp

Câu 1. Tên bài: **MPILOT.CPP**

Phú ông vì rất giàu có nên đã mua một vài máy bay và đầu tư để kiếm lời. Có N phi công (N chẵn) và cần có N/2 phi hành đoàn.Mỗi phi hành đoàn gồm 2 người: 1 lái chính, 1 trợ lí. Lái chính phải cao tuổi hơn trợ lý. Hợp đồng cho mỗi phi công có ghi mức lương nếu anh ta là lái chính hoặc là trợ lí. Với mỗi 1 hợp đồng thì lương lái chính > lương trợ lí.

Phú ông muốn tìm cách ghép cặp sao cho tổng lương phải trả cho N người là ít nhất.

Dữ liệu: vào từ file **MPILOT.INP** gồm:

+ Dòng đầu là N (N chẵn), số phi công, $2 \le N \le 10^4$

+ N dòng tiếp theo, mỗi dòng là 2 số X,Y là lương phi công thứ i nếu làm lái chính hoặc trợ lí, $1 \le Y < X \le 100{,}000$.

Các phi công sắp tăng dần theo tuổi.

Kết quả: ghi ra file **MPILOT.OUT** là lương nhỏ nhất cần trả.

MPILOT.INP	MPILOT.OUT
4 5000 3000 6000 2000 8000 1000 9000 6000	19000
6 10000 7000 9000 3000 6000 4000 5000 1000 9000 3000 8000 6000	32000

Câu 2. Tên bài: **FLOWER.CPP**

Trong cuộc thi cắm hoa liên vũ trụ, ban tổ chức cho *N* lọ hoa sắp thẳng hàng và *K* bó hoa được đánh số thứ tự từ nhỏ đến lớn. Yêu cầu là cần cắm *K* bó hoa trên vào *N* lọ sao cho hoa có số thứ tự nhỏ phải đứng trước hoa có số thứ tự lớn. Giá trị thẩm mỹ tương ứng khi cắm hoa *i* vào lọ thứ *j* là *v(i,j)*. Để trái

đất không bị thua cuộc trước người ở hành tinh khác, hãy tìm 1 cách cắm sao cho tổng giá trị thẩm mỹ là lớn nhất. Chú ý rằng mỗi bó hoa chỉ được cắm vào 1 lọ và mỗi lọ cũng chỉ cắm được 1 bó hoa.

Dữ liệu: vào từ file **FLOWER.INP** gồm

+ Dòng đầu tiên gồn 2 số nguyên K và N ($1 \le K \le 100$, $K \le N \le 100$)

+ Tiếp theo gồm K dòng, mỗi dòng gồm N số nguyên thể hiện giá trị thẩm mĩ (các giá trị nằm trong khoảng từ -50 đến 50)

Kết quả: ghi ra file **FLOWER.OUT** một dòng duy nhất là tổng số giá trị thẩm mĩ cao nhất.

Ví dụ:

FLOWER.INP	FLOWER.OUT
3 5 7 23 -5 -24 16 5 21 -4 10 23 -21 5 -4 -20 20	53

Câu 3. Tên bài: **CHAIR.CPP**

Có N phòng học chuyên đề và K nhóm học được đánh số thứ tự từ nhỏ đến lớn. Cần xếp K nhóm trên vào N phòng học sao cho nhóm có số hiệu nhỏ được xếp vào phòng có số hiệu nhỏ, nhóm có số hiệu lớn phải được xếp vào phòng có số hiệu lớn. Với mỗi phòng có học sinh, các ghế thừa phải được chuyển ra hết, nếu thiếu ghế thì lấy vào cho đủ ghế. Biết phòng i ban đầu có A_i ghế, nhóm j có B_j học sinh. Hãy chọn 1 phương án bố trí sao cho tổng số ghế chuyển ra và vào là ít nhất.

Dữ liệu: vào từ file **CHAIR.INP** gồm

+ Dòng đầu tiên gồm 2 số nguyên N và K ($1 \le K \le N \le 1000$)

+ Dòng 2 gồm N số nguyên dương chỉ số ghế của N phòng ($A_i \le 100$)

+ Dòng 3 gồm K số nguyên dương chỉ số học sinh của K nhóm ($B_i \le 100$)

Kết quả: ghi ra file **CHAIR.OUT** một dòng duy nhất là tổng số ghế chuyển đi ít nhất

Ví dụ:

CHAIR.INP	CHAIR.OUT
4 2 3 1 4 2 2 4	1

Giải thích: nhóm 1 vào phòng 1, nhóm 2 vào phòng 3.

Câu 4. Tên bài: **SHOES.CPP**

Trước trận chung kết thế giới, ban lãnh đạo đã dẫn các cầu thủ đi mua giày mới để tăng tính phong thủy. Trong cửa hang có N đôi giày, đôi giày thứ i có size là H_i. Đội bóng gồm K cầu thủ, cầu thủ thứ i thì thường đi giày có size là S_i. Khi cầu thủ i chọn mua đôi giày j thì độ lệch sẽ là $|H_i - S_j|$. Hãy tìm cách chọn mua giày cho K cầu thủ trên sao cho tổng độ lệch là ít nhất, để giúp các cầu thủ đi giày thoải mái nhất trước trận đấu . Biết rằng mỗi người chỉ mua 1 đôi giày và 1 đôi giày cũng chỉ có một người mua.

Dữ liệu: vào từ file **SHOES.INP**

+ Dòng đầu tiên gồm 2 số nguyên N và K ($1 \le K \le N \le 10^3$)

+ Dòng thứ 2 là N số nguyên H_i là chỉ số size của N đôi giày ($1 \le H_i \le 10^2$)

+ Dòng thứ 3 gồm K số nguyên S_i là chỉ số size của K cầu thủ ($1 \le S_i \le 10^2$)

Kết quả: ghi ra file **SHOES.OUT** một dòng duy nhất là độ chênh lệch ít nhất

Ví dụ:

SHOES.INP	SHOES.OUT
5 3 1 3 3 7 6 2 5 1	2

Giải thích: cầu thủ số 1 đi đôi giày số 2, cầu thủ số 2 đô đôi giày số 5, cầu thủ số 3 đi đôi giày số 1.

Câu 5. Tên bài: **BUILT.CPP**

Trong việc xây dựng lại thành phố sau chiến tranh vũ trụ, có N công trình cần thi công. Công trình thứ i cần được cung cấp D[i] đơn vị vật liệu. Vật liệu được cung cấp từ hai kho A và B của nhà nước. Cước vận chuyển một đơn vị

vật liệu từ kho A đến công trường i là A[i]. Cước vận chuyển một đơn vị vật liệu từ kho B đến công trường i là B[i]. Biết kho A có R đơn vị vật liệu và tổng số vật liệu của hai kho vừa đủ cung cấp cho N công trường. Hãy phân phối vật liệu từ hai kho đến các công trường sao cho tổng cước phí vận chuyển là ít nhất.

Dữ liệu: vào từ file **BUILT.INP** gồm

+ Dòng đầu tiên gồm 2 số nguyên N, R ($0 < R , N \leq 500$).

+ Dòng thứ 2 gồm N số nguyên D[1], D[2], …, D[n].

+ Dòng thứ 3 gồm N số nguyên A[1], A[2], … , A[n].

+ Dòng thứ 4 gồm N số nguyên B[1], B[2], … ,B[n].

Các giá trị $0 \leq D_i , A_i, B_i \leq 100$

Kết quả: ghi ra file **BUILT.OUT** là tổng chi phí vận chuyển ít nhất.

Ví dụ:

BUILT.INP	BUILT.OUT
5 100 30 80 80 50 40 3 5 10 4 23 4 6 2 7 4	1070

Câu 6. Tên bài: **CABLE.CPP**

Vì tình trạng chơi game trong giờ Tin học diễn ra thường xuyên, nên người trực phòng máy đã ngắt tất cả các máy tính ra khỏi mạng và xếp chúng thành một dãy trên một cái bàn dài. Ngoài ra, máy tính được gắn chặt xuống mặt bàn rồi đánh số thứ tự từ 1 đến N theo chiều từ trái sang phải. Dù vậy, các học sinh không chịu thua, họ đã quyết định tìm cách nối các máy trên bàn bởi các đoạn dây nối sao cho mỗi máy được nối với ít nhất một máy khác. Để tiến hành công việc này, họ đã đo khoảng cách giữa hai máy liên tiếp. Tuy vậy, để tiết kiệm, các học sinh muốn tổng độ dài dây cáp nối là ít nhất. Bạn hãy giúp các học sinh này tìm cách nối mạng thoả mãn yêu cầu đặt ra.

Dữ liệu: vào từ file **CABLE.INP** gồm:

+ Dòng đầu tiên chứa số lượng máy N ($1 \leq N \leq 25000$).

+ Dòng thứ i trong số N-1 dòng tiếp theo chứa các khoảng cách từ máy i đến máy i+1 (i=1,2,...,N-1). Giả thiết rằng khoảng cách từ máy 1 đến máy N không vượt quá 10^6.

Kết quả: ghi ra file **CABLE**.OUT là độ dài thỏa mãn yêu cầu.

Ví dụ:

CABLE.INP	CABLE.OUT
6 2 2 3 2 2	7

Giải thích: Sử dụng đoạn nối giữa các máy (1) ~ (2), (3) ~ (4), (5) ~ (6). Tổng 2 + 3 + 2 = 7.

3.5. Di chuyển.

Phương pháp chung: Đối với các bài toán có dạng di chuyển thì việc xác định hàm mục tiêu cần căn cứ vào quy luật di chuyển cho phép và cần phải chú ý vào giá trị biên. Đây là dạng toán thường gặp tuy nhiên việc xây dựng hàm mục tiêu và công thức truy hồi là khá dễ dàng

Câu 1. Tên bài: **MAX.CPP**

Cho một bảng A kích thước m x n (1 <= m, n <= 100), trên đó ghi các số nguyên A_{ij} ($|A_{ij}| <= 100$). Một người xuất phát tại ô nào đó của hàng 1, cần sang hàng m (tại ô nào cũng được). Quy tắc đi: Từ ô (i, j) chỉ được quyền sang một trong 3 ô (i + 1, j); (i + 1, j + 1); (i + 1, j - 1)

Dữ liệu: vào từ file **MAX.INP** gồm

+ Dòng đầu tiên hai số m, n là số hàng và số cột của bảng.

+ m dòng tiếp theo, dòng thứ i ghi đủ n số trên hàng i của bảng theo đúng thứ tự từ trái qua phải

Kết quả: ghi ra file **MAX.OUT** là tổng lớn nhất tìm được.

Ví dụ:

MAX.INP	MAX.OUT
5 7 9 -2 6 2 1 3 4 0 -1 6 7 1 3 3 8 -2 8 2 5 3 2 1 -1 6 2 1 6 1 7 -2 6 2 1 3 7	33

Câu 2. Tên bài: **VSTEPS.CPP**

Trạng Tí rất thích chơi trò chơi điện tử Rambo Đại Chiến. Sau nhiều ngày cày cuốc, cuối cùng Trạng cũng đến được stage cuối cùng. Stage này yêu cầu Trạng phải điều khiển Rambo leo lên một cầu thang gồm n bậc. Các bậc thang được đánh số từ 1 đến n từ dưới lên trên. Rambo có thể đi lên một bậc thang, hoặc nhảy một bước lên hai bậc thang. Tuy nhiên có một số bậc thang đã bị thủng do cũ kỹ và Rambo không thể bước chân lên được. Biết ban đầu, Rambo đứng ở bậc thang số 1 (bậc thang số 1 không bao giờ bị thủng). Chơi đến đây, Trạng Tí chợt nảy ra câu hỏi: có bao nhiêu cách để Rambo leo hết được cầu thang? (nghĩa là leo đến bậc thang thứ n). Vì bận chơi game nên Trạng Tí muốn nhờ bạn trả lời câu hỏi này.

Dữ liệu: vào từ file **VSTEPS.INP** gồm:

+ Dòng đầu tiên, 2 số nguyên n và k, là số bậc của cầu thang và số bậc thang bị hỏng ($0 \le k < n \le 100000$).

+ Dòng thứ hai, chứa k số nguyên cho biết chỉ số của các bậc thang bị hỏng theo thứ tự tăng dần.

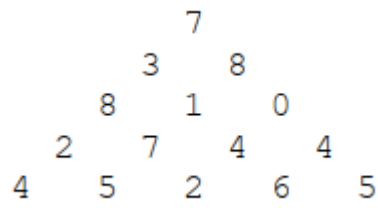
Kết quả: ghi ra file **VSTEPS.OUT** là phần dư của số cách Lucky leo hết cầu thang khi chia cho 14062008.

Ví dụ:

VSTEPS.INP	VSTEPS.OUT
4 2 2 3	0
90000 1 49000	4108266

Câu 3. Tên bài: **TRIANGLE.CPP**

Cho một tam giác gồm các số nguyên không âm. Hãy tính tổng lớn nhất các số trên đường đi từ đỉnh tam giác xuống một điểm nào đó ở đáy tam giác nào đó. Tại mỗi ô ta chỉ có đi thẳng xuống ô bên trái hoặc thẳng xuống ô bên phải.



Dữ liệu: Vào từ file **TRIANGLE.INP** gồm

+ Dòng đầu là 1 số nguyên dương N (N <= 100) là số hàng của tam giác

+ N dòng tiếp theo, dòng thứ i gồm i số nguyên dương thể hiện 1 tầng của tam giác (các số có giá trị không quá 100)

Kết quả: ghi ra file **TRIANGLE.OUT** một số nguyên là kết quả của bài toán

Ví dụ:

TRIANGLE.INP	TRIANGLE.OUT
5 7 3 8 8 1 0 2 7 4 4 4 5 2 6 4	30

Câu 4. Tên bài: **PLAYGROUND.CPP**

Bản đồ khu vui chơi là một hình chữ nhật có kích thước MxN ô vuông. Khu vui chơi có một cổng vào đặt tại ô (1, 1) và một cổng ra đặt tại ô (M, N). Mỗi ô (i, j) được bố trí một trò chơi, giá vé vào ô (i, j) là C[i, j]. Tại mỗi ô khách có thể di chuyển sang các ô chung cạnh bên phải hoặc phía dưới. Vào ngày nghỉ, Bờm quyết định tham quan khu vui chơi. Trong khu vui chơi có một trò chơi Bờm rất thích đặt tại ô (U, V) và nhất định Bờm phải tham gia trò chơi này. Hơn nữa, sau khi chơi ở ô (U,V), Bờm phải quay về ở ô (M, N). Nhưng số tiền có hạn

Bờm không biết phải đi theo lộ trình nào để chơi được trò chơi mình thích và ít tốn tiền nhất. Bạn hãy giúp Bờm nhé.

Dữ liệu: Vào từ file **PLAYGROUND.INP** gồm

+ Dòng đầu là 4 số M, N, U, V ($3 \le M, N \le 1000$; $1 \le U \le M$; $1 \le V \le N$)

+ M dòng tiếp theo, mỗi dòng ghi N số thể hiện ma trận chi phí C, với C[i, j] là giá vé khi vào ô (i, j). ($0 \le C[i][j] \le 10000$)

Kết quả: ghi ra file **PLAYGROUND** một số nguyên duy nhất là chi phí ít nhất tìm được.

Ví dụ:

PLAYGROUND.INP	PLAYGROUND.OUT
4 5 2 3 5 6 4 4 1 1 2 9 1 5 1 1 1 2 3 4 6 5 1 4	25

Câu 5. Tên bài: **JUMP.CPP**

Trên trục tọa độ Ox có N vị trí được đánh dấu sẵn, vị trí i có tọa độ X_i. Nhà vua muốn thách đố Trạng Tí nên đã đưa ra trò chơi như sau: Trạng Tí sẽ thực hiện các bước nhảy giữa các điểm đã đánh dấu. Trước khi bắt đầu trò chơi, phải chọn điểm bắt đầu (là một trong N vị trí đã đánh dấu) và chiều nhảy thuận theo chiều tia Ox hoặc ngược lại. Trạng được thực hiện số bước nhảy tùy ý với điều kiện chiều nhảy phải giống nhau và độ dài bước nhảy phải là một số dương và lớn hơn hoặc bằng bước nhảy liền trước. Tất nhiên với những điều kiện này Trạng Tí không thể thực hiện quá N bước nhảy. Mỗi vị trí i có một số điểm tương ứng là P_i, điểm của Trạng sẽ được tính là tổng số điểm các vị trí đến được (bao gồm cả điểm xuất phát).

***Yêu Cầu*:** Tìm cách nhảy để Trạng Tí đạt được số điểm lớn nhất.

Dữ Liệu: vào từ File **JUMP.INP** gồm:

+ Dòng đầu ghi số nguyên N ($1 < N \le 10^3$)

+ N dòng tiếp theo, dòng thứ i ghi 2 số nguyên X_i và P_i. ($1 < X_i, P_i \le 10^6$)

Kết Quả: Ghi ra File **JUMP.OUT** là số điểm lớn nhất đạt được.

JUMP.INP	JUMP.OUT
6 5 6 1 1 10 5 7 6 4 8 8 10	25

Giải thích: Các vị trí nhảy qua: 4 5 7 10 (tương ứng điểm nhận được: 8+6+6+5=25)

3.6. Quy hoạch động vị trí cấu hình.

Tư tưởng:

Cho một dãy các cấu hình thỏa mãn điều kiện nào đó được sắp theo thứ tự từ điển. Bài toán thường có 2 yêu cầu:

+ Cho cấu hình (c), hỏi (c) nằm ở vị trí nào trong từ điển.

+ Cho vị trí k, yêu cầu tìm cấu hình (c) ứng với vị trí k trong từ điển.

Tư tưởng chính để giải quyết yêu cầu 1 là tìm số lượng cấu hình có vị trí nằm trước (c) trong từ điển. Từ đó suy ra vị trí của (c).

Tư tưởng chính để giải quyết yêu cầu 2 là thu hẹp dần phạm vi từ điển (lần lượt bỏ đi những cấu hình (c) có vị trí bé hơn k, tạo thành một từ điển mới đồng thời giảm chỉ số k cho tương ứng với từ điển mới).

Ví dụ: Cho từ điển bao gồm tất cả các dãy N bit nhị phân. VD với N=3 ta có từ điển dãy bit nhị phân là: 000, 001, 010, 011, 100, 101, 110, 111.

Cho một dãy (c) gồm N bit nhị phân, hỏi (c) nằm ở vị trí nào trong từ điển

Cho một số k, hỏi dãy bit nhị phân nào ở vị trí k trong từ điển.

Hướng dẫn:

Thực ra vị trí của (c) trong từ điển cũng chính là dạng thập phân của (c) cộng thêm 1. Nhưng để làm quen với tư tưởng Quy hoạch động vị trí cấu hình, ta

tạm thời quên đi cách giải đơn giản này và sử dụng một tư tưởng khác như sau:

Gọi F[i] là số lượng các dãy i bit nhị phân.

Dễ thấy F[i] = 2*F[i-1] (hay F[i] = 2^i). F[0] = 1

Giả sử (c) có dạng là : $a_1\ a_2 \ldots a_n$. Ta chỉ quan tâm đến các bit 1 của (c), giả sử bit 1 xuất hiện ở những vị trí k: $a_1\ a_2 \ldots a_{k-1}\ 1\ a_{k+1} \ldots a_n$. Khi đó (c) sẽ có vị trí lớn hơn vị trí của những cấu hình dạng $x_1\ x_2 \ldots x_{k-1}\ 0\ a_{k+1} \ldots a_n$ (trong đó x_i nhận giá trị 0 hoặc 1). Hay nói cách khác sẽ có F[k-1] cấu hình dạng $x_1\ x_2 \ldots x_{k-1}\ 0\ a_{k+1} \ldots a_n$ có vị trí bé hơn (c).

Gọi S là số lượng các cấu hình có vị trí bé hơn (c):

$$S = \sum_{i=1}^{st} F[t_i - 1]$$

Trong đó, t_i là các vị trí xuất hiện bit 1, *st* là số lượng vị trí có bit là 1 trong (c). Suy ra vị trí của (c) trong từ điển là s+1.

```
s = 0;
for (int t = 1; t <= n; t ++)
        if (c[t] == 1) s += f[t – 1];
cout << s + 1;
```

Để tìm cấu hình (c) ở vị trí k, ta duyệt lần lượt các bit của (c) từ bit thứ 1 đến N. Nếu bít thứ t nhận giá trị 1 hay (c): $a_1 a_2 \ldots a_{t-1}\ 1 \ldots$ thì rõ ràng (c) có vị trí lớn hơn vị trí của các cấu hình dạng $a_1\ a_2 \ldots a_{t-1}\ 0\ x_{t+1} \ldots x_n$ (trong đó x_i nhận giá trị 0 hoặc 1). Hay nói cách khác k sẽ lớn hơn F[n-t]. Vậy:

+ Nếu k ≤ F[n-t] thì bit thứ t của (c) nhận giá trị 0

+ Nếu k > F[n-t] thì bit thứ t của (c) nhận giá trị 1, đồng thời gán lại k = k - F[n-t].

```
for (int t = 1; t <= n ; t ++)
        if ( k <= f[n – t]) c[t] = 0;
        else {
             c[t] = 1;
             k  -= f[n – t];
        }
```

```
for (int i = 1; i <= n; i ++)
    cout << c[i];
```

Câu 1. Tên bài: **BINARY.CPP**

Một tập hợp S gồm các dãy N bit 0, 1 trong đó không có hai bit 1 nào kề nhau. Ví dụ với N = 5 thì S gồm các dãy 00000, 00001, 00010,... Tập S được săp xếp theo chiều tăng dần của số nguyên tương ứng mà dãy bit biểu diễn. Cho số N và một số nguyên M hãy cho biết dãy bit thứ M trong S.
Dữ liệu: vào từ file văn bản **BINARY.INP** gồm hai số nguyên dương N, M ($N \leq 40$, M đảm bảo có nghiệm)

Kết quả: ghi ra file **BINARY.OUT** là dãy N số 0, 1 ghi liền nhau mô tả dãy nhị phân tìm được

Ví dụ:

BINARY.INP	BINARY.OUT
5 3	00010

Câu 2. Tên bài: **SHHV.CPP**

Xét tất cả các hoán vị của dãy số tự nhiên (1, 2,..., n) (1 <= n <= 12). Giả sử rằng các hoán vị được sắp xếp theo thứ tự từ điển.

Yêu cầu: Cho trước 1 hoán vị. Tìm số hiệu hoán vị đó trong dãy đã sắp xếp

Dữ liệu: vào từ file **SHHV.INP** gồm:

+ Dòng 1: Chứa số nguyên n

+ Dòng 2: Chứa n số a_1, a_2, … a_n (dãy hoán vị n phần tử).

Kết quả: ghi ra file **SHHV.OUT** số q là số hiệu của dãy hoán vị trên.

Ví dụ:

SHHV.INP	SHHV.OUT
3 2 1 3	3

Câu 3. Tên bài: **HV2.CPP**

Xét tất cả các hoán vị của dãy số tự nhiên (1, 2,..., N) Giả sử rằng các hoán vị được sắp xếp theo thứ tự từ điển.

Yêu cầu: Cho trước số hiệu của 1 hoán vị trong dãy hoán vị đã sắp xếp. Tìm hoán vị đó.

Dữ liệu: vào từ file **HV2.INP** gồm

+ Dòng đầu chứa số N (1 <= N <= 12)

+ Dòng thứ hai chứa số hiệu hoán vị

Kết quả: ghi ra file **HV2.OUT** là hoán vị cần tìm.

HV2.INP	HV2.OUT
3 4	2 3 1

Câu 4. Tên bài: **CATALAN.CPP**

Cho số nguyên dương n, dãy Catalan cấp n là dãy C(1), C(2) … C(2n+1) gồm các số nguyên không âm thoả mãn: C(1) = C(2n+1) = 0; với i bất kì 1 ≤ i ≤ 2n thì C(i), C(i+1) hơn kém nhau 1 đơn vị. Ví dụ: 0 1 2 1 2 3 2 1 0 là 1 dãy Catalan, nhưng dãy 0 1 3 2 1 0 thì không phải.

Với mỗi n ta sắp xếp các dãy Catalan theo thứ tự từ điển, đánh số từ 1 trở đi .
Yêu cầu: Cho trước một dãy Catalan bất kỳ, và số nguyên dương k. Hãy tìm dãy Catalan có thứ tự k.

Dữ liệu: vào từ file **CATALAN.INP** gồm:

+ Dòng đầu ghi số nguyên dương n. (n <= 15)

+ Dòng hai ghi một dãy Catalan cấp n

+ Dòng 3 ghi một số nguyên dương k (k có thể rất lớn nhưng đảm bảo luôn có nghiệm)

Kết quả: ghi ra file **CATALAN.OUT:**

+ Dòng 1 ghi số thứ tự dãy ở dòng 2 của file dữ liệu vào

+ Dòng 2 ghi dãy Catalan ứng với số thứ tự là k

CATALAN.INP	CATALAN.OUT
4 0 1 2 3 2 1 2 1 0 12	12 0 1 2 3 2 1 2 1 0

Câu 5. Tên bài: **MEANK.CPP**

Trong tất cả các số tự nhiên có một chữ số, Cậu Tí thích nhất là số 0. Tuy nhiên, trong tiết Tin học tuần này, thầy giáo đã dạy cho Tí về cách biển diễn nhị phân, trong đó có các số 0 ở đầu sẽ không có ý nghĩa. Cho số nguyên không âm N ($N < 2^{31}$). Hãy xác định xem trong phạm vi từ 0 tới N có bao nhiêu số mà trong dạng biểu diễn nhị phân của nó có đúng K chữ số 0 có nghĩa. ***Ví dụ:*** N = 18, K = 3 có 3 số: 8 = 1000; 17 = 10001; 18 = 10010

Dữ liệu: vào từ file **MEANK.INP** gồm một dòng chứa hai số nguyên N và K cách nhau một dấu cách. (0 <= K <= 32)

Kết quả: ghi ra file **MEANK.OUT** là số lượng tìm được.

Ví dụ:

MEANK.INP	MEANK.OUT
18 3	3

3.7. Quy hoạch động trạng thái

Với một bài toán cụ thể, để tìm ra được lời giải cần đi từ bước 1 đến bước thứ k. Trong đó tại bước thứ i nảy sinh nhiều trạng thái hướng đi khác nhau, để tìm ra hướng đi đúng cho bước thứ i lại dựa vào trạng thái của bước đi thứ i-1 và có thể xác định bằng một công thức cụ thể nếu tập các trạng thái là hữu hạn.

Câu 1. Tên bài: **SELECT.CPP**

Cho một bảng hình chữ nhật kích thước 4×n ô vuông. Các dòng được đánh số từ 1 đến 4, từ trên xuống dưới, các cột được đánh số từ 1 đến n từ trái qua phải. Ô nằm trên giao của dòng i và cột j được gọi là ô (i,j). Trên mỗi ô (i,j) có ghi một số nguyên aij , i =1, 2, 3, 4; j =1, 2, ..., n. Một cách chọn ô là việc xác định một tập con khác rỗng S của tập tất cả các ô của bảng sao cho không có hai ô nào trong S có chung cạnh. Các ô trong tập S được gọi là ô được chọn, tổng các số trong các ô được chọn được gọi là trọng lượng của cách chọn. Tìm cách chọn sao cho trọng lượng là lớn nhất.

Ví dụ: Xét bảng với n=3 trong hình vẽ dưới đây:

	1	2	3
1	-1	9	3
2	-4	5	-6
3	7	8	9
4	9	7	2

Cách chọn là tập các ô S = {(3,1), (1,2), (4,2), (3,3)} với trọng lượng 32.

Dữ liệu: Vào từ file văn bản **SELECT.INP** gồm

+ Dòng đầu tiên chứa số nguyên dương n là số cột của bảng ($n \leq 10^4$)

+ Cột thứ j trong số n cột tiếp theo chứa 4 số nguyên a_{1j}, a_{2j}, a_{3j}, a_{4j} là 4 số trên cột j của bảng.($|a_{ij}| \leq 3.10^4$)

Kết quả: ghi ra file **SELECT.OUT** là kết quả bài toán.

SELECT.INP	SELECT.OUT
3 -1 9 3 -4 5 -6 7 8 9 9 7 2	32

Câu 2. Tên bài: **ASSET.CPP**

Nhà Vua có n người con gái, và muốn gã cho n vị tiến sỹ trẻ năm ấy. Thế nhưng không biết nên sắp xếp hôn nhân thế nào cho lợi. Một nhà hiền triết được mời đến để tham khảo ý kiến. Nhà hiền triết sau khi xem xét đã báo cáo kết quả thành 1 bảng n*n và nói rằng "nếu cô con gái thứ i kết hôn cùng tiến sỹ j thì sẽ làm ra được lượng tài sản bằng với số ở hàng i cột j từ nay đến hết đời." Nhưng khổ nỗi ông hiền triết cũng không biết sắp xếp hôn nhân sao cho nhà Vua có được ngôi mộ hoành tráng nhất sau khi bang hà. Các bạn hãy giúp nhà vua nhé.

Dữ liệu: vào từ file **ASSET.INP** gồm

+ Dòng đầu gồm số nguyên dương n (n <= 20).

+ n dòng sau mỗi dòng chứa n số nguyên A[i,j] với A[i,j] là số tiền có được khi con gái thứ i kết hôn với tiến sỹ j ($0 < a[i,j] <= 10^7$)

Kết quả: ghi ra file **ASSET.OUT** là tổng tài sản lớn nhất nhận được.

Ví dụ:

ASSET.INP	ASSET.OUT
4 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	34

Câu 3: Tên bài: **TRIP.CPP**

Trong kì nghỉ hè năm nay Trạng Tí được nhà vua thưởng cho 1 tour du lịch quanh N đất nước tươi đẹp với nhiều thắng cảnh nổi tiếng. Tí dự định sẽ đi bằng máy bay. Giá vé máy bay từ đất nước i đến đất nước j là C_{ij} (dĩ nhiên C_{ij} có thể khác C_{ji}). Tuy được nhà Vua thưởng cho nhiều tiền để đi du lịch nhưng Trạng Tí cũng muốn tìm cho mình 1 hành trình với chi phí rẻ nhất vì muốn tiết kiệm tiền cho đất nước.

Yêu cầu: Bạn hãy giúp Trạng Tí tìm 1 hành trình đi qua tất cả các nước, mỗi nước đúng 1 lần sao cho chi phí là bé nhất nhé.

Dữ liệu: vào từ file **TRIP.INP** gồm

+ Dòng 1 là số nguyên N ($5 < N < 16$)

+ Dòng thứ i trong N dòng tiếp theo: Gồm N số nguyên, số thứ j là C_{ij} ($0 < C_{ij} < 10001$)

***Kết quả*:** ghi ra file **TRIP.OUT** là chi phí bé nhất tìm được

Ví dụ:

TRIP.INP	TRIP.OUT
6 0 1 2 1 3 4 5 0 3 2 3 4 4 1 0 2 1 2 4 2 5 0 4 3 2 5 3 5 0 2 5 4 3 3 1 0	8

Câu 4. Tên bài: **MIX.CPP**

Mỗi trong N cô bò của bác John có một số seri phân biệt a[i]. Các cô bò tự hào đến nỗi mỗi cô đều đeo một chiếc vòng vàng có khắc số seri của mình trên cổ theo kiểu các băng đảng giang hồ. Các cô bò giang hồ này thích nổi loạn nên đứng xếp hàng chờ vắt sữa theo một thứ tự gọi được gọi là 'hỗn loạn'. Một thứ tự bò là 'hỗn loạn' nếu trong dãy số seri tạo bởi hàng bò, hai số liên tiếp khác biệt nhau nhiều hơn K. Ví dụ: nếu N = 6 và K = 1 thì 1, 3, 5, 2, 6, 4 là một thứ tự 'hỗn loạn', nhưng 1, 3, 6, 5, 2, 4 thì không (vì hai số liên tiếp 5 và 6 chỉ chênh lệch 1).

Yêu cầu: Hỏi có bao nhiêu cách xếp hàng khác nhau để N cô bò sắp thành thứ tự 'hỗn loạn'?

Dữ liệu: vào từ file văn bản **MIX.INP** gồm

+ Dòng 1: Hai số N và K. ($4 \le N \le 16$; $1 \le K \le 3400$)

+ Dòng 2..N+1: mỗi dòng là một số nguyên duy nhất là số seri của cô bò thứ i: a[i] ($1 \le a[i] \le 25.10^3$)

Kết quả: ghi ra file **MIX.OUT** một số nguyên duy nhất là số cách để N cô bò sắp thành thứ tự 'hỗn loạn'. Kết quả đảm bảo nằm trong phạm vi kiểu số nguyên *long long*.

Ví dụ:

MIX.INP	MIX.OUT
4 1 3 4 2 1	2

Câu 5. Tên bài: **SHEEP.CPP**

Thảo nguyên Beufati là 1 cánh đồng chia làm M*N ô vuông. Bác Hotte muốn đưa đàn cừu lên đây để ăn cỏ. Tuy vậy, mỗi con cừu chỉ đứng trong 1 ô và mỗi ô chỉ chứa 1 con cừu .Chỉ có 1 quy tắc duy nhất là không bao giờ được để 4 con cừu tạo thành 1 hình vuông 2*2 hoặc để trống 1 khu đất 2*2, nếu không, những con sói sẽ dễ dàng ăn thịt cừu. Bác Hotte muốn biết tất cả có bao nhiêu cách xếp cừu thỏa mãn quy tắc trên để đề phòng mọi trường hợp. Vì con số này rất lớn nên hãy giúp bác Hotte nhé.

Dữ liệu: vào từ file **SHEEP.INP** gồm

+ Dòng đầu gồm 1 số T duy nhất là số test ($1 \le T \le 111$)

+ T dòng tiếp theo gồm 2 số M, N cho biết kích thước của thảo nguyên ($1 \le M*N \le 30$

Kết quả: ghi ra file **SHEEP.OUT** T dòng, mỗi dòng ứng với 1 test là số cách xếp bò của test đó.

Ví dụ:

SHEEP.INP	SHEEP.OUT
2 1 1 2 2	2 14

Gợi Ý Cách Giải

Câu 1. Tên bài **LIQ.CPP**

Ta xây dựng hàm mục tiêu : F(i) là độ dài dãy con tăng dài nhất thỏa mãn từ A_1 đến A_i và thỏa mãn dãy đó có A_i là phần tử cuối cùng.

Bài toán cơ sở : F(1) = 1. (hiển nhiên vì chỉ có 1 phần tử).

Công thức truy hồi :

F(i) = max(F(j)) + 1 với mọi phần tử j thỏa mãn 1 <= j <= i-1 và $A_j \leq A_i$. Nếu không tìm thấy j nào thì F(i) = 1;

Tính F(i) : phần tử đang được xét là A_i . Với mọi j từ 1 đến i – 1, ta tìm đế phần tử $A_j \leq A_i$ có F(j) lớn nhất. Khi đó nếu bổ sung A_i vào sau dãy con A_j ta sẽ được dãy con tăng dần dài nhất xét từ $A_1 \ldots A_i$.

```
#include<bits/stdc++.h>
int n, a[1002], f[1002];
using namespace std;
int main(){
   ios_base::sync_with_stdio(0);
   cin.tie(0); cout.tie(0);
   freopen("LIQ.INP", "r", stdin);
   freopen("LIQ.OUT", "w", stdout);
   cin >> n;
   for (int i = 1; i <= n; i ++)
      cin >> a[i];
   int res = 1; // res là kết quả.
   for (int i = 1; i <= n; i ++){
      f[i] = 1; //
      for (int j = 1; j < i; j ++)
         if (a[j] <= a[i])
            f[i] = max(f[i], f[j] + 1);
      res = max(res, f[i]);// sau mỗi bước i ta tính luôn kết quả.
   }
   cout << res;
   return 0;
}
```

Độ phức tạp là $O(N^2)$.

Câu 2. Tên bài: **LIS.CPP**

Ta nhận thấy bài này tương tự như câu 1, nhưng chỉ là thay đổi giới hạn của *n* từ 100 lên 10^5 và là dãy tăng ngặt. Với giới hạn này, chúng ta không thể làm thuật toán với độ phức tạp là $O(n^2)$, như thế sẽ bị TLE (time limit exceeded nghĩa là bị chạy quá thời gian). Vì n <= 10^5 nên $O(n^2) = 10^{10}$.

Như vậy, ta cần một thuật toán khác câu 1.

Gọi F[i] là vị trí nhỏ nhất của phần tử tận cùng của dãy con tăng độ dài i. Khi đó mảng F[] là một dãy con tăng khi duyệt từ đầu dãy đến cuối dãy ban đầu (do lưu vị trí và cập nhật). Khi xét đến một phần tử i thì ta chặt nhị phân tìm dãy có độ dài j lớn nhất sao cho phần tử tận cùng của dãy này có giá trị nhỏ hơn phần tử đang xét, tức là tìm j max sao cho a[i] > a[f[j]]. Sau đó chỉ việc cập nhật lại mảng f[] với phần tử mới tại vị trí i.

Như vậy, nếu duyệt hết tất cả giá trị j mất $O(n)$, thì khi chặt nhị phân chúng ta chỉ mất $O(log(n))$.

Vậy độ phức tạp thuật toán là $O(nlog(n))$, đủ để vượt qua mà không bị *TLE*.

```
#include <bits/stdc++.h>
using namespace std;
int n, len, f[100005],a[100005];
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("LIS.inp", "r", stdin);
    freopen("LIS.out", "w", stdout);
    cin >> n;
    for(int i = 1;i <= n;i ++)
        cin >> a[i];
    f[1] = 1; //dãy độ dài 1 có phần tử cuối cùng là vị trí 1 là vị trí nhỏ nhất
    len = 1; // độ dài của dãy con.
    int lo, hi, mid, rem;
    for (int i=2;i <= n;i ++) {
        lo = 1, hi = len ,rem = 0;
        while (lo <= hi) {
            mid=(lo+hi) / 2;
            if (a[i] > a[f[mid]]) {
```

```
            rem = mid;
            lo = mid + 1;
          }
          else hi = mid - 1;
        }
        if (rem == len) {len ++; f[len] = i;}
        if (a[i] < a[f[rem+1]]) f[rem+1] = i;
    }
    cout << len;
    return 0;
}
```

Câu 3. Tên bài: **TOWER.CPP**

+ Nhận xét: Đây là bài toán tìm dãy con không tăng theo *R* và có tổng các *H* là lớn nhất.

+ Gọi *F(x)* là độ cao lớn nhất của tòa tháp xếp được với khối có *x* là đường kính hình trụ xếp cao nhất . Gọi *Rmax* là giá trị đường kính lớn nhất của các hình trụ. Với mỗi khối trụ thứ *i*, ta sẽ duyệt lại 1 vòng các đường kính từ *Ri* đến *Rmax*.

Ta thiết lập được công thức truy hồi

F(Ri) = max(F(Ri), F(j) + Hi). Với mọi *j* thỏa mãn $Ri \leq j \leq Rmax$.

Do đó, ta cần khởi tạo với mọi *i* từ 1 đến *N* thì *F(Ri) = Hi*, còn các *F* còn lại thì khởi tạo bằng 0.

Kết quả là *max (F(Ri))* với mọi *i* chạy từ 1 đến *N*.

Độ phức tạp của thuật toán là $O(N*Rmax) \approx 500 * 10^5 = 5.10^7$

```
#include <bits/stdc++.h>
using namespace std;
int n, res, f[503],r[100005], h[100005];
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("TOWER.inp", "r", stdin);
    freopen("TOWER.out", "w", stdout);
    cin >> n;
```

```
    for(int i = 1;i <= n;i ++)
        cin >> r[i] >> h[i];
    int rmax = 0;
    for (int i = 1; i <= n;i ++)
        rmax = max(rmax, r[i]);
    // khởi tạo mảng f.
    for (int i = 1; i <= rmax; i ++)
        f[i] = 0;
    f[r[1]] = h[1];
    res = f[r[1]];
    for (int i = 2; i <= n; i ++){
        for (int j = r[i]; j <= rmax; j++)
            f[r[i]] = max(f[r[i]], f[j] + h[i]);
        res = max(res, f[r[i]]); // cập nhập kết quả
    }
    cout << res;
      return 0;
}
```

Câu 4. Tên bài: **STADIUM.CPP**

+ Lưu ý ở câu này, chúng ta muốn sử dụng quy hoạch động thì sẽ phải sắp xếp các đội theo 1 chỉ số nào đó để dễ dàng quản lí và xây dựng trạng thái.

+ Nếu sắp xếp các thời gian sử dụng sân theo thời điểm kết thúc, ta sẽ đưa được về bài toán **tìm dãy con có tổng lớn nhất**. Bài toán này là biến thể của bài toán tìm dãy con tăng dài nhất.

Ta gọi *f(i)* là số tiền lớn nhất ông chủ sân có được khi đến đội thứ *i* được thuê. Ta có khi xét đến đội *i*, với $1 \le j < i$, thì nếu $bj \le ai$ thì $f(i) = max\ (f(i), f(j) + ci)$.

Kết quả bài toán là max của các *f(i)*. với mọi i chạy từ 1 đến *n*

Độ phức tạp của thuật toán là $O(n^2)$.

```
#include <bits/stdc++.h>
using namespace std;
int n, res;
struct club{
    int a, b, c;
```

```
};
club x[5005];
int f[5005];
bool cmp(club g, club h){
    if (g.b != h.b) return g.b < h.b;
    return g.a < h.a;
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("STADIUM.inp", "r", stdin);
    freopen("STADIUM.out", "w", stdout);
    cin >> n;
    for(int i = 1;i <= n;i ++)
        cin >> x[i].a >> x[i].b >> x[i].c;
    sort(x + 1, x + n + 1, cmp); // sắp xếp tăng theo chỉ số b.
    res = 0;
    for (int i =1 ; i <= n; i ++){
        f[i] = x[i].c;
        for (int j = 1; j < i; j++)
            if (x[j].b <= x[i].a)
                f[i] = max(f[i], f[j] + x[i].c);
    }
    // tìm kết quả
    for (int i = 1; i <= n; i ++) res = max(res, f[i]);
    cout << res;
    return 0;
}
```

Câu 5. Tên bài: **EXP.CPP**

Đặt *F[i, x] = 1* nếu có thể điền dấu vào *i* số đầu tiên và cho kết quả bằng *x*. Ta có công thức sau để tính *F*:

F[1, a[1]] = 1 (điều hiển nhiên)

F[i, x] = 1 nếu F[i - 1, x + a[i]] = 1 hoặc F[i - 1, x - a[i]] = 1.

Vì nếu *F[i - 1, x + a[i]] = 1* thì ta thêm dấu – vào trước *a[i]* thì sẽ được tổng bằng *x;* nếu *F[i - 1, x - a[i]] = 1* thì ta thêm dấu + vào trước *a[i]* thì cũng được tổng bằng *x.*

Nếu *F[n, S] = 1* thì câu trả lời của bài toán là YES, ngược lại in ra NO.

Ta có một lưu ý khi tiến hành cài đặt bài toán này, đó là các kết quả biểu thức có thể âm nên mảng của chúng ta phải có khả năng lưu phần âm (tức là lưu được từ *-D* đến *D* với *D* là tổng của *N* số). Do đó, vì mảng không có chỉ số âm nên ta chỉ cần cộng kết quả của biểu thức với 1 số lớn hơn D là sẽ thỏa mãn.

Độ phức tạp của thuật toán là *O(N*S)* , xấp xỉ $2.5*10^7$

```
#include <bits/stdc++.h>
using namespace std;
int n, a[1003], S, sum;
bool f[503][50003];
int main() {
   ios_base::sync_with_stdio(0);
   cin.tie(0); cout.tie(0);
   freopen("EXP.inp", "r", stdin);
   freopen("EXP.out", "w", stdout);
   cin >> n;
   for (int i = 1; i <= n; i ++) cin >> a[i];
   cin >> S;
   int dmax = 25000; // dmax sử dụng để tránh chỉ số mảng bị âm
   sum = 0;
   for (int i =1 ; i <= n; i ++) sum += a[i]; // sum là tổng lớn nhất của N số
   memset(f, false, sizeof(f)); // khởi tạo mảng f.
   f[1][a[1] + dmax] = true;
   for (int i = 2; i <= n; i ++){
      for (int j = -sum; j <= sum; j++){
         if (j + a[i] <= sum) {
            if (f[i - 1][j + a[i] + dmax]) // nếu tồn tại khả năng
               f[i][j + dmax] = 1;
         }
         if (j - a[i] >= -sum){
            if (f[i - 1][j - a[i] + dmax]) // nếu tồn tại khả năng
```

```
                f[i][j + dmax] = 1;
            }
        }
    }
    if (f[n][S + dmax]) cout << "YES";
    else cout << "NO";
    return 0;
}
```

Câu 6. Tên bài: **SPSEQ.CPP**

+ Gọi F1[i] là dãy con tăng dài nhất từ 1 đến i với mỗi $1 \le i \le n$

+ Gọi F2[i] là dãy con giảm dài nhất tử i đến n với mỗi $1 \le i \le n$. (thực chất chỉ là duyệt dãy từ n về 1, tìm dãy tăng dài nhất). Kết quả bài toán là:

max(2 * min(F1[i], F2[n – i + 1]) - 1) với mỗi $1 \le i \le n$.

Trong kết quả ta có trừ đi 1 vì a[i] được tính 2 lần (ta tưởng tượng nó như là đỉnh của cái nón), còn F2[n – i + 1] là độ dài dãy giảm dài nhất từ i đến n.

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int N=1e5+7;
const int base=1e9+7;
int f1[N],f2[N],a[N],g[N];
int n;
void lis(int f[], int g[]){
   int ln=0;
   for (int i=1;i<=n;i++){
      int dau=1;
      int cuoi=ln;
      int nho=0;
      while (dau<=cuoi){
         int giua=(dau+cuoi)/2;
         if (a[i]>a[g[giua]]) {
            nho=giua;
            dau=giua+1;
         }
         else cuoi=giua-1;
```

```
        }
        if (nho==ln) {ln++; g[ln]=i;}
        if (a[i] < a[g[nho+1]]) g[nho+1] = i;
        f[i]=ln;
    }
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(); cout.tie();
    freopen ("spseq.inp", "r", stdin);
    freopen ("spseq.out", "w", stdout);
    cin>>n;
    for (int i=1;i<=n;i++) cin>>a[i];
    f1[1]=1;
    lis(f1,g);
    reverse(a+1,a+n+1); // Hàm đảo mảng trong thư viện
    f2[1]=1;
    lis(f2,g);
    int res=0;
    for (int i=1;i<=n;i++)
        res=max(res,2*min(f1[i], f2[n-i+1]) -1);
    cout<<res;
    return 0;
}
```

Câu 7. Tên bài: **BEADS.CPP**

Cần chú ý đến vỏ ốc đầu tiên được cho vào chuỗi. Giả sử chuỗi ốc đầu tiên được cho vào là A[i], ta thấy dãy các vỏ được thêm vào sau cuối chuỗi sẽ tạo nên một dãy con tăng bắt đầu từ A[i], còn các vỏ được thêm vào đầu chuỗi sẽ tạo nên dãy con giảm bắt đầu từ A[i]. Do đó, với mỗi vị trí ta tìm độ dài dãy con tăng dài nhất (LIS) bắt đầu từ vị trí `i` và dãy con giảm dài nhất (LDS) bắt đầu từ vị trí i. Kết quả là `max(LIS + LDS - 1)` với mỗi vị trí i.

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5+5;
int n, a[N], s[N], F0[N], F1[N], k, res;
int main() {
```

```
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("beads.inp", "r", stdin);
    freopen("beads.out", "w", stdout);
      cin >> n;
      for (int i = 1; i <= n; i++)
       cin >> a[i];
      k = 1;
      s[1] = a[n];
      F0[n] = 1;
      for (int i = n-1; i >= 1; i--) {
              int j = k;
              while (j > 0 && a[i] >= s[j]) j--;
              if (j == k) s[++k] = 0;
              s[j+1] = max(s[j+1], a[i]);
              F0[i] = j+1;
      }
      k = 1;
      memset(s, 0, sizeof(s));
      s[1] = a[n];
      F1[n] = 1;
      for (int i = n-1;i >= 1; i--) {
              int j = k;
              while (j > 0 && a[i] <= s[j]) j--;
              if (j == k) s[++k] = 1e9+7;
              s[j+1] = min(s[j+1], a[i]);
              F1[i] = j+1;
      }
      for(int i = 1; i <= n; i ++)
       res = max(res, F0[i]+F1[i]-1);
    cout << res;
      return 0;
}
```

Câu 8. Tên bài: **STICK.CPP**

Đầu tiên, ta sẽ sắp xếp N đoạn gỗ này theo chiều tăng dần của độ dài L, nếu chúng có độ dài bằng nhau thì ta sẽ sắp xếp tăng dần theo trọng lượng W.

Ta thấy, để Bình không tốn thời gian thì 2 đoạn gỗ liên tiếp (lúc này đã được sắp xếp tăng dần theo L) phải có W tăng dần. Tức là khi đó, một số lượng đoạn gỗ thỏa mãn sẽ là 1 dãy con tăng theo chỉ số W. Lúc này, ta chỉ cần áp dụng tư tưởng bài LIS (dãy con tăng dài nhất) để áp dụng vào bài toán này theo chiều W. Bởi lẽ ta cần chia N đoạn gỗ thành ít nhất số lượng dãy con tăng này, nên mỗi dãy con tăng phải là dài nhất có thể. Lúc này, số lượng dãy con là kết quả của bài toán.

```
#include <bits/stdc++.h>
using namespace std;
const int N = 5010;
struct   doango{
      int l;
      int w;
};
int T, n, s[N];
doango  a[N];
bool cmp(doango  u, doango  v) {
      return (u.l < v.l) || (u.l == v.l && u.w < v.w);
}
int main() {
   ios_base::sync_with_stdio(0);
   cin.tie(0); cout.tie(0);
   freopen("stick.inp", "r", stdin);
   freopen("stick.out", "w", stdout);
   cin >> T;
     while(T--){
              cin >> n;
             for (int i = 1; i <= n; i++)
                      cin >> a[i].l >> a[i].w;
             sort(a+1, a+n+1, cmp);
             int k = 1;
             s[1] = a[1].w;
             for(int i = 2; i <= n; i ++) {
                      int j = k;
                      while (j > 0 && s[j] <= a[i].w) j--;
                      if (j == k) s[++k] = a[i].w;
                      s[j+1] = max(s[j+1], a[i].w);
```

```
            }
            cout << k << '\n';
        }
        return 0;
    }
```

Câu 9. Tên bài: **NEMCHUA.CPP**

Đầu tiên ta nhận thấy chi phí 1 nhóm là chênh lệch phần tử lớn nhất và bé nhất trong nhóm, vì thế nên nếu trong nhóm đó phần tử lớn nhất giá trị là b, phần tử bé nhất giá trị là a, thì nhóm đó sẽ chứa tất cả các số từ a → b. Vì vậy ta sắp xếp dãy số $a_1, a_2 \ldots a_N$ theo thứ tự tăng dần → mỗi nhóm sẽ là 1 đoạn các số liên tiếp.

Quy hoạch động: gọi f[i][j] là chi phí nhỏ nhất chia dãy thành i nhóm khi xét từ vị trí 1 đến vị trí j → xem xét nhóm thứ i chứa bao nhiêu phần tử: f[i][j] = min(f[i-1][k] + a[j]-a[k+1]) nhóm thứ i chứa các số a[k+1], a[k+2],..., a[j]; max là a[j]; min là a[k+1] với k = 0…j-1.

Kết quả bài toán là f[m][n]. Độ phức tạp là $O(m*n^2)$

```
#include <bits/stdc++.h>
const int maxn = 222;
using namespace std;
int f[maxn][maxn];
int n, m;
int a[maxn];
int solve() {
    int largeNumber = 1e9+1;
    sort(a+1, a+n+1);
    for(int i=0; i<=m; i++)
        for(int j=0; j<=n; j++) {
            f[i][j] = largeNumber;
        }
  for(int i=0; i<=m; i++) {
    f[i][0] = 0;
  }
    for(int i=1; i<=n; i++) {
        f[1][i] = a[i] - a[1];
    }
```

```
    for(int i=2; i<=m; i++)
            for(int j=1; j<=n; j++) {
                    for(int k=j-1; k>=0; k--) {
                            f[i][j] = min(f[i][j], f[i-1][k] + a[j]-a[k+1]);
                    }
            }
     return f[m][n];
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("NEMCHUA.INP", "r", stdin);
   freopen("NEMCHUA.OUT", "w", stdout);
     cin>>n>>m;
   for(int i=1; i<=n; i++) {
      cin>>a[i];
   }
   cout<<solve();
   return 0;
}
```

Câu 10. Tên bài: **BUNDAU.CPP**

Bài toán được phát biểu lại là cho dãy số a_1, a_2, …a_N. Hãy tìm 1 dãy con liên tiếp có nhiều phần tử nhất có tổng các phần tử là số nguyên tố:

+ Gọi F(i) là tổng các phần tử khi xét dãy từ vị trí 1 đến vị trí i. Dễ dàng ta thấy F(0)= 0; F(i) = F(i-1) + a[i] với mọi i= 1..N. Ta duyệt tất cả các dãy con từ vị trí i đến vị trí j: ta tính tổng các phần tử chính bằng F(j) – F(i-1). Nhận thấy tổng các phần tử luôn nằm trong đoạn [0 …5.10^6], nên ta sàng nguyên tố trong đoạn này để kiểm tra tính nguyên tố của F(j) – F(i-1).

Kết quả bài toán là max của tất cả các dãy con tìm được thoả mãn đề bài.

Độ phức tạp của thuật toán là $O(n^2)$

```
#include <bits/stdc++.h>
const   int N = 5E3+5;
using namespace std;
int i, n;
int a[N], f[N];
```

```
bool check[N*1000+1];
bool  sangnt(int n){
   memset(check, true, sizeof(check));
   check[0] = check[1] = false;
   for (int i= 2; i*i <= n; i++)
      if (check[i])
         for (int j = i*2; j <= n; j+= i)
            check[j] = false;
}
void  solve(){
   f[0] = 0;
   for(int i= 1; i<= n; i++) f[i]= f[i-1]+ a[i];
   int res = -1;
   for(int i= 1; i<= n; i++)
      for (int j= i; j<= n; j++)
         if (check[f[j]-f[i-1]])
            if (res< j-i+1) res= j- i+ 1;
   cout << res;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("BUNDAU.INP", "r", stdin);
   freopen ("BUNDAU.OUT", "w", stdout);
   cin >> n;
   for(int i = 1; i <= n; i++) cin >> a[i];
   sangnt(N*1000);
   solve();
   return 0;
}
```

3.2 Biến đổi xâu

Câu 1. Tên bài: **STRING.CPP**

Dễ thấy số phép biến đổi phụ thuộc vào vị trí *i* đang xét của xâu S và vị trí *j* đang xét của xâu T. Do vậy để cài đặt cho bảng phương án ta sẽ dùng mảng 2 chiều. Gọi *F[i,j]* là số phép biến đổi ít nhất để biến xâu S_i gồm *i* kí tự

phần đầu của S ($S_i = S[1..i]$) thành xâu T_j gồm j kí tự phần đầu của T ($T_j = T[1..j]$).

Dễ thấy: $F[0 , j] = j$ và $F[i , 0] = i$.

Có 2 trường hợp xảy ra:

- Nếu $S[i] = T[j]$: thì ta chỉ phải biến đổi xâu S_{i-1} thành xâu T_{j-1}. Do đó :

$$F[i,j] = F[i-1, j-1].$$

- Ngược lại, ta có 3 cách biến đổi:

+ Xoá kí tự $S[i]$: Xâu S_{i-1} thành T_j. Khi đó :

$$F[i , j] = F[i-1 , j] + 1$$ (Cộng 1 là do ta đã dùng 1 phép xóa)

+ Thay thế $S[i]$ bởi $T[j]$: Xâu S_{i-1} thành T_{j-1} Khi đó :

$$F[i , j] = F[i-1 , j-1] + 1.$$

+ Chèn $T[j]$ vào sau $S[i]$: Xâu S_i thành T_{j-1}. Khi đó:

$$F[i , j] = F[i , j-1] + 1.$$

Tổng kết lại, ta có công thức QHĐ:

$$F[0,j]=j$$

$$F[i,0]=i$$

$$F[i,j] =F[i-1,j-1] \text{ nếu } S[i] = T[j]$$

$$F[i,j] = min(F[i-1,j], F[i,j-1], F[i-1,j-1]) + 1 \text{ nếu } S[i] \neq T[j]$$

Kết quả bài toán là $F[n][m]$. Trong đó n là độ dài xâu S và m là độ dài xâu T. Chú ý ta chèn thêm 1 dấu cách vào đầu 2 xâu để ta duyệt chỉ số từ 1.

Độ phức tạp của thuật toán là $O(m*n)$. Với m, n là độ dài của 2 xâu S và T

```
#include <bits/stdc++.h>
using namespace std;
string a, b;
int f[1003][1003], n, m;
int main(){
   ios_base::sync_with_stdio(0);
   cin.tie(0); cout.tie(0);
```

```
    freopen("STRING.inp", "r", stdin);
    freopen("STRING.out", "w", stdout);
    cin >> a >> b;
    memset(f,0, sizeof(f));
    n = a.size(); m = b.size();
    a = ' ' + a;
    b = ' ' + b; // chỉ số a,b sẽ bắt đầu từ 1.
    for (int i =1 ; i <= n; i ++) f[i][0] = i;
    for (int i = 1; i <= m; i ++) f[0][i] = i;
    for (int i =1 ; i <= n; i++)
       for (int j = 1; j <= m; j++){
          if (a[i] == b[j]) f[i][j] = f[i - 1][j - 1];
          else f[i][j] = min(f[i- 1][j], min(f[i][j - 1],f[i - 1][j - 1])) + 1;
       }
    cout << f[n][m]<<'\n';
    return 0;
}
```

Câu 2. Tên bài: **QBSTR.CPP**

Ta xây dựng hàm mục tiêu là *F(i, j)* có ý nghĩa là độ dài xâu con chung dài nhất của xâu X_i (xâu gồm *i* kí tự đầu tiên của xâu *X*) và xâu Y_j (xâu gồm *j* kí tự đầu tiên của xâu *Y*). Tương tự như bài toán 1, ta có thể suy ra công thức quy hoạch động sau:

- *F(0,j) = F(i, 0) = 0.*
- *F(i, j) = F(i – 1, j – 1) + 1 nếu X[i] = Y[j]*
- *F(i, j) = max(F(i – 1, j), F(i, j – 1)) nếu X[i] ≠ Y[j].*

Kết quả bài toán là *F(m, n).* Trong đó *m, n* là độ dài 2 xâu *X* và *Y*. Cộng thêm dấu cách vào đầu 2 xâu để cho chạy chỉ số từ 1.

Độ phức tạp của thuật toán là *O(m*n).* Trong đó *m, n* là độ dài 2 xâu.

```
#include <bits/stdc++.h>
using namespace std;
string x, y;
int f[1003][1003], n, m;
int main() {
   ios_base::sync_with_stdio(0);
```

```
    cin.tie(0); cout.tie(0);
    freopen("QBSTR.inp", "r", stdin);
    freopen("QBSTR.out", "w", stdout);
    cin >> x >> y;
    m = x.length();
    n = y.length();
    x = " " + x;
    y = " " + y;
    for (int i = 0; i <= m; i++)
       f[i][0] = 0;
    for (int j = 0;j <= n;j ++)
       f[0][j]=0;
    for (int i = 1; i <= m; i ++)
       for (int j = 1; j <= n; j ++)
          if (x[i] == y[j]) f[i][j] = f[i-1][j-1]+1;
          else f[i][j] = max(f[i][j-1] , f[i-1][j]);
    cout << f[m][n];
    return 0;
}
```

Câu 3. Tên bài: **NKPALIN.CPP**

Bài này ta có thể đảo ngược xâu đầu vào và tìm xâu con chung dài nhất của 2 xâu đó. Xâu con chung này chính là xâu đối xứng cần tìm. Công thức tìm xâu con chung dài nhất của 2 xâu A, B:

Gọi F[i][j] là độ dài xâu con chung lớn nhất đến vị trí A[i] và B[j].

+ Nếu A[i] = B[j] thì F[i][j] = F[i−1][j−1]+1.

+ Ngược lại, F[i][j] = Max(F[i−1][j] , F[i][j−1]).

Kết quả của bài toán là F[n][n] với n là độ dài của xâu (xâu được bắt đầu tính từ 1). Sau đó, ta chỉ cần truy vết để tìm ra xâu thỏa mãn.

```
#include <bits/stdc++.h>
using namespace std;
unsigned int f[2004][2004];
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
```

```
    freopen("nkpalin.inp", "r", stdin);
    freopen("nkpalin.out", "w", stdout);
    string a, b;
    cin>>a;
    int t = a.length();
    for (int i=0;i<=t;i++) b=a[i]+b;
    string z="";
    int m,n;
    m=a.length();
    n=b.length();
    for (int i=0;i<=m;i++) f[i][0]=0;
    for (int j=0;j<=n;j++) f[0][j]=0;
    for (int i=1;i<=m;i++)
       for (int j=1;j<=n;j++)
          if (a[i-1]==b[j-1]) f[i][j]=f[i-1][j-1]+1;
          else f[i][j]=max(f[i][j-1],f[i-1][j]);
   int x, y;
   x= m; y= n;
   while(true){
      if (a[x-1]==b[y-1]){
         z=a[x-1]+z;
         x--;
         y--;
      }
      else{
         if (f[x][y]==f[x-1][y]) x--;
         else y--;
      }
     if (z.length()==f[m][n]) break;
    }
   cout<<z;
   return 0;
}
```

Câu 4. Tên bài: **COUNTPL.CPP**

Đầu tiên, ta sẽ nhận thấy, nếu gọi *F(i)* là số lượng xâu *palindrome* nhỏ nhất xây được từ vị trí 0 đến vị trí *i*.

Ta dễ dàng nhận thấy *F(0) = 1;* (vì khi xét 1 kí tự đầu tiên của xâu)

Với một vị trí *j* bất kì *(1 ≤ j ≤ i)*, nếu xâu $S_j, S_{j+1}, ..., S_i$ là một xâu đối xứng, thì ta dễ dàng tìm được công thức là:

$$F(i) = min(F(i), F(j-1) + 1)).$$

Tuy vậy, để bài toán chạy nhanh hơn, chúng ta cần phải xét xem xâu $S_j, S_{j+1}, ..., S_i$ là một xâu đối xứng hay không ở ngoài vòng lặp for cuối cùng.

Như thế, ta sẽ tạo 1 mảng *bool C(i, j) = True* nếu xâu từ *i* đến *j* là 1 *palindrome* và ngược lại.

Với cách tính mảng *C(i, j)*, ta có :

+ Nếu $i = j$ thì $C(i, j) = 1$.

+ $C(i , j) = false/0$ nếu $Si \neq Sj$.

+ $C(i, j) = C(i + 1, j – 1)$ nếu $Si = Sj$.

Kết quả bài toán là *F(n-1)*. Độ phức tạp của thuật toán là $O(n^2)$

```
#include<bits/stdc++.h>
using namespace std;
bool C[1005][1005];
int f[1005],n;
string s;
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("COUNTPL.inp", "r", stdin);
   freopen("COUNTPL.out", "w", stdout);
   cin>>s;
   n = s.size();
   for (int i=0;i<n; i++)
      for (int j=0;j<n-i;j++){
         if (i==0) C[j][j]=true;
         else if (i==1){
            if (s[j] == s[j+1]) C[j][j+1]=true;
            else C[j][j+1]=false;}
         else{
            if (s[j]!=s[j+i]) C[j][j+i]=false;
```

```
            else  C[j][j+i] = C[j+1][j+i-1];
        }
      }
    f[0]=1;
    for (int i=1;i<n;i++)
    {
      if (C[0][i]==true) f[i]=1;
      else
      {
        f[i]=1001; // khởi tạo mảng F bằng 1 số lớn bất kì
        for (int j=1;j<=i;j++)
          if (C[j][i]==true) f[i]=min(f[i],f[j-1]+1);
      }
    }
    cout<<f[n-1];
    return 0;
}
```

Câu 5. Tên bài: **PAL.CPP**

Ta tạo hàm mục tiêu là *F(i , j)*, mang ý nghĩa là số xâu con đối xứng trong đoạn từ *i* đến *j* của xâu ban đầu.

Khởi tạo *:*

+ F(i, i) = 1 // Xâu con có 1 ký tự

+ F[i][i-1] = 0 // Trường hợp không có xâu con

Công thức: khi xét xâu con từ vị trí *i* đến vị trí *j* (1 <= *i* < *j* <= *n*)

Nếu $S_i = S_j$ *thì F(i, j) = F(i + 1, j) + F(i, j – 1) + 1.*

Ngược lại ta thấy rằng : *F(i, j)* hình thành từ *F(i, j – 1)* và *F(i – 1, j)*. Nhưng nếu ta sử dụng phép cộng thì đồng nghĩa với việc ta đã tính 2 lần các xâu con đối xứng từ *i + 1* đến *j – 1*. Do đó ta có công thức :

$$F(i , j) = F(i + 1, j) + F(i, j – 1) \ - F(i + 1, j – 1).$$

Kết quả bài toán là *F(1, n)*, với *n* là độ dài xâu S và ký tự đầu tính từ vị trí 1.

Độ phức tạp của thuật toán $O(n^2)$.

```
#include<bits/stdc++.h>
```

```
using namespace std;
const int mod = 1e9 + 7;
int f[205][205], n;
string s;
int main()
{
   ios_base::sync_with_stdio(false);
   cin.tie(0);
   cout.tie(0);
   freopen("PAL.inp", "r", stdin);
   freopen("PAL.out", "w", stdout);
   cin >> s;
   n = s.length();
   s = " " + s; // chỉ số xâu bắt đầu từ 1
   for (int i = 1;i <= n;i++)      f[i][i] = 1;
   for (int i = 2;i <= n;i++)      f[i][i-1] = 0;
   for (int k = 1;k <= n-1 ; k++)
      for (int i = 1;i <= n-k; i++){
         int j = i + k;
         if (s[i] == s[j])
            f[i][j] = f[i+1][j] + f[i][j-1] + 1;
         else
            f[i][j] = f[i+1][j] + f[i][j-1] - f[i+1][j-1];
         f[i][j] = ((f[i][j] % mod) + mod) % mod;// lấy modun
      }
   cout << f[1][n];
   return 0;
}
```

Sẽ có nhiều trường hợp code như sai như sau:

```
for (int i = 1;k <= n-1 ; k++)
      for (int j = i+1; i <= n; j++){
         if (s[i] == s[j])
            f[i][j] = f[i+1][j] + f[i][j-1] + 1;
         else
            f[i][j] = f[i+1][j] + f[i][j-1] - f[i+1][j-1];
         f[i][j] = ((f[i][j] % mod) + mod) % mod;// lấy modun
      }
```

Code như trên sẽ sai vì chúng ta tính *f[i][j]* mà các *f[i+1][j], f[i][j-1], f[i+1][j-1]* có thể chưa được tính trước. Ví dụ đầu tiên tính *f[1][2]* dựa vào *f[2][2], f[1][1], f[2][1]*. Như vậy tính được *f[1][2]* thành công vì các *f[2][2], f[1][1], f[2][1]* đã được tính trước. Tiếp theo ta tính *f[1][3]* dựa vào các *f[2][3], f[1][2], f[2][2]*. Như vậy *f[1][2], f[2][2]* đã được tính từ trước, còn *f[2][3]* chưa được tính, mà ta lại dùng nó để tính *f[1][3]* nên dẫn đến sẽ sai kết quả. Nguyên tắc của quy hoạch động: khi ta tính các bài toán lớn dựa vào các bài toán con, nhưng các bài toán con này phải được tính trước.

Câu 6. Tên bài: **GONME.CPP**

Ta lưu *m* dãy vào ma trận *A*, *A[i, j]* lưu số thứ *j* trong dãy hoán vị thứ *i*. Gọi *D[i, j]* là vị trí của số *j* trong dãy hoán vị thứ *i*, *F[i]* là độ dài dãy con chung dài nhất kết thúc bằng số *i*. Với mỗi số thứ *i* trong dãy hoán vị đầu tiên, lần lượt ta xét những số thứ *j* đứng sau nó, kiểm tra xem cặp số này có thỏa mãn tính thứ tự trong *m – 1* dãy còn lại hay không, tức là số *A[1,i]* có đứng trước số *A[1, j]* trong *m – 1* dãy còn lại hay không. Nếu đúng hết thì *F[A[1, i]] = Max(F[A[1, i]], F[A[1, j]] + 1)*. Sau mỗi lần tính chọn lại giá trị lớn nhất cho biến *res* và cuối cùng ghi ra *res*.

Độ phức tạp của thuật toán là $O(n^2 * m)$.

```
#include<bits/stdc++.h>
using namespace std;
int n, m;
int a[12][1002], d[12][1002], f[1002];
bool check(int i, int j){
   for (int k = 2; k <= m; k++){
      if (d[k][a[1][j]] > d[k][a[1][i]]) // không thỏa mãn
         return false;
   }
  return true;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0);
   cout.tie(0);
   freopen("GONME.inp", "r", stdin);
```

```
    freopen("GONME.out", "w", stdout);
    cin >> n >> m;
    for (int i =1 ; i <= m; i++)
       for (int j = 1; j <= n; j ++){
          cin >> a[i][j];
          d[i][a[i][j]] = j;// lưu vị trí mỗi số trong từng dãy hoán vị
       }
    int res = 0;
    for (int i =1 ; i <= n; i++){
       f[a[1][i]] = 1;
       for (int j =1 ; j < i; j ++){
          if (check(i, j)) //kiểm tra tính thỏa mãn của 2 cặp số thứ i và j
             f[a[1][i]] = max(f[a[1][i]], f[a[1][j]] + 1);
       }
       res = max(res, f[a[1][i]]);
    }
    cout << res;
    return 0;
}
```

Câu 7: Tên bài: **SUBSTRK.CPP**

+ Thêm dấu cách vào đầu xâu: s = " " + s; để ta xét xâu từ vị trí 1.

+ Gọi F[i] là số lượng kí tự 1 khi xét xâu s từ vị trí 1 đến vị trí i, ta có công thức tính

+ F[i] = F[i-1] + 1; nếu s[i] = '1'

+ Ngược lại F[i] = F[i-1];

Như vậy, ta có F[j] – F[i] chính là số lượng kí tự 1 của xâu s xét từ vị trí i+1 đến vị j. Giả sử n là vị trí cuối của xâu. Ta duyệt i từ 0 đến n, với mỗi vị trí của i: ta tìm left là vị trí phía sau i mà là vị trí đầu tiên thoả mãn f[left] – f[i] = k; Nếu không có left thoả mãn thì ta quay lên tiếp tục lần lặp tiếp theo. Khi tìm được left thì ta tìm tiếp right là vị trí cuối cùng thoả mãn f[left] – f[i] = k; Từ đây là cập nhật thêm vào kết quả: right – left + 1; (vì trong các đoạn này sẽ có đúng k kí tự 1)

Độ phức tạp của thuật toán là *O(nlogn)*

```
#include <bits/stdc++.h>=
```

```
using namespace std;
const int N = 1E6+2;
string s;
int i, n, k;
int f[N], trai[N], phai[N];
long long dem =0;
void   solve(){
   f[0] = 0;
   for(int i = 1; i<= n; i++)
      if (s[i] == '1') f[i]= f[i-1] + 1;
      else f[i] = f[i-1];
    int left, right;
   for(int i= 0; i< n; i++){
      left = lower_bound (f+i+1, f+n+1, f[i] + k) - f;
      if (left == n+1) continue;
      right = upper_bound (f+i+1, f+n+1, f[i] + k) - f - 1;
      dem = dem + right - left +1;
   }
   cout << dem;
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("substrk.inp", "r", stdin);
    freopen("substrk.out", "w", stdout);
    cin >> k;
    cin >> s;
    n = s.length();
    s = " " + s;
    solve();
   return 0;
}
```

Câu 8. Tên bài: **BLAST.CPP**

+ Gọi n là độ dài xâu A, m là độ dài xâu B

+ Gọi F[i,j] là khoảng cách nhỏ nhất giữa hai xâu mở rộng mà khi xét xâu A đến vị trí thứ i và xâu B đến vị trí thứ j.

+ Nhận thấy F[0,0] = 0; F[i,0] = i*k; F[0,j] = j*k;

+ Công thức quy hoạch động

F[i,j]:=min(F[i,j-1] + k, F[i-1,j] + k, F[i-1,j-1] + 2*k, F[i-1,j-1] + |a[i] - b[j]|)

+ Kết quả của bài toán là F[n,m]

```
#include<bits/stdc++.h>
using namespace std;
const int N=2e3+7;
const int base = 1e9+7;
int n,k,m;
string a,b;
int f[N][N];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);  cout.tie(0);
    freopen("blast.inp","r",stdin);
    freopen("blast.out","w", stdout);
    cin >> a >> b; cin >> k;
    n=a.size(); m=b.size();
    a=" "+a;   b=" "+b;
    for (int i=1;i<=n;i++) f[i][0]=k*i;
    for (int i=1;i<=m;i++) f[0][i]=k*i;
    for (int i=1;i<=n;i++)
       for (int j=1;j<=m;j++){
          f[i][j]=f[i-1][j-1]+abs(a[i]-b[j]);
          f[i][j]=min(f[i][j],f[i-1][j-1]+2*k);
          f[i][j]=min(f[i][j],f[i-1][j]+k);
          f[i][j]=min(f[i][j],f[i][j-1]+k);
       }
    cout<<f[n][m];
    return 0;
}
```

Câu 9. Tên bài: **STMERGE.CPP**

+ Gọi F[i, j, 0] là tổng nhỏ nhất tạo thành từ xâu X[1..i] và Y[1..j], trong đó kí tự cuối là X[j]. F[i,j,1] là tổng nhỏ nhất tạo thành từ xâu X[1..i] và Y[1..j], trong đó kí tự cuối là Y[j].

+ Nhận thấy f[i, 0, 0] = 0; f[0, j, 1] = 0;

+ Công thức: f[i, j, 0]:=min(f[i-1, j, 0], f[i-1, j, 1]+c[i, j]);

f[i, j, 1]:=min(f[i, j-1, 1], f[i, j-1, 0]+c[i, j]);

+ Kết quả bài toán min(f[m,n,0], f[m,n,1]). Độ phức tạp thuật toán là *O(m*n)*

```
#include<bits/stdc++.h>
#define ll long long
const   int N = 1E3+5;
const   ll MAXV = 1E18;
using namespace std;
int t,n,m;
ll c[N][N];
ll f[N][N][2];
void    solve (){
   cin >> m >> n;
   for(int i= 1; i <= m; i++)
      for(int j= 1; j <= n; j++)
         cin >> c[i][j];
   for (int i= 0; i<= m; i++)
   for(int j= 0; j<= n; j++){
      f[i][j][0]= MAXV;
      f[i][j][1]= MAXV;
   }
   for(int i= 0; i<= m; i++) f[i][0][0]= 0;
   for(int i= 0; i<= n; i++) f[0][i][1]= 0;
   for(int i= 1; i<= m; i++)
      for(int j= 1; j<= n; j++){
         f[i][j][0]= min(f[i-1][j][0], f[i-1][j][1]+c[i][j]);
         f[i][j][1]= min(f[i][j-1][1], f[i][j-1][0]+c[i][j]);
      }
   cout << min(f[m][n][0],f[m][n][1]) << endl;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0);  cout.tie(0);
   freopen("stmerge.inp","r",stdin);
   freopen("stmerge.out","w", stdout);
```

```
    cin>>t;
    for (int i = 1; i <= t; i++)  solve();
    return 0;
}
```

Câu 10. Tên bài: **SUBSTR**

- Thay vì ta đi tìm xóa số kí tự ít nhất để xâu trở thành xâu đặc biệt, thì ta đi tìm xâu con có độ dài lớn nhất thõa mãn yêu cầu đề bài:

- $n \leq 20$ ta có thể làm bằng cách đệ quy sinh nhị phân để xét tất cả các xâu con, với mỗi xâu ta đi kiểm tra nó có thoả mãn đề bài hay không, và cập nhật độ dài xâu vào kết quả. Lấy n trừ đi độ dài đó là ra kết quả bài toán.
- Với n lớn hơn ta làm tương tự như bài xâu con chung:

+ Gọi f(i, j) là độ dài xâu con lớn nhất thỏa mãn đề bài mà 2 đầu là nguyên âm khi xét xâu từ vị trí i đến vị trí j.

+ Gọi g(i, j) là độ dài xâu con lớn nhất thỏa mãn đề bài mà 2 đầu là phụ âm khi xét xâu từ vị trí i đến vị trí j.

- f(i,j) = max(f(i, j-1), f(i+1, j), g(i+1, j-1) +2) nếu 2 đầu i, j là nguyên âm
- g(i,j) = max (g(i, j-1), g(i+1, j), f(i+1, j-1) +2) nếu 2 đầu i, j là phụ âm

Kết quả bài toán là: n- max(f(1, n), g(1, n)). Độ phức tạp bài toán là $O(n^2)$

```
#include <bits/stdc++.h>
const int N = 5002;
using namespace std;
int i, j, n;
char  a[N];
int f[N][N], g[N][N];
bool check (char ch){
  switch (ch){
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u': return true;
    default: return false;
  }
```

```
}
void  xuly (){
  for (int i = 1; i <= n-1; i++)
     if (check(a[i]) == true && check(a[i+1]) == true)  f[i][i+1] = 2;
     else if (check(a[i]) == false && check(a[i+1]) == false)  g[i][i+1] = 2;
  for (int ii =2; ii <= n-1; ii++)
     for (int i = 1; i <= n-ii; i++){
          j = i + ii;
          f[i][j] = max(f[i+1][j], f[i][j-1]);
          g[i][j] = max(g[i+1][j], g[i][j-1]);
          if (check(a[i]) == true && check(a[j]) == true)
            f[i][j] = max(f[i][j], g[i+1][j-1] + 2);
          else  if (check(a[i]) == false && check(a[j]) == false)
                    g[i][j]=max(g[i][j], f[i+1][j-1]+2);
  }
  return;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("substr.inp","r", stdin);
   freopen("substr.out","w", stdout);
   cin >> n;
   for (int i = 1; i <= n; i++) cin >> a[i];
   memset (f, 0, sizeof(f));
   memset (g, 0, sizeof(g));
   xuly();
  cout << n - max (f[1][n], g[1][n]);
  return 0;
}
```

3.3. Dạng bài toán cái túi và ứng dụng.

Câu 1. Tên bài: **VALIA.CPP**

Nhận xét: giá trị của vali phụ thuộc vào 2 yếu tố: có bao nhiêu vật đang được xét và trọng lượng của các vật. Do đó bảng phương án sẽ là bảng 2 chiều:

- *F(i,j):* là tổng giá trị lớn nhất của vali khi xét từ vật 1, vật 2, …vật *i* và trọng lượng của vali chưa vượt quá *j*. Chú ý rằng khi xét đến *F[i,j]* thì các giá trị trên bảng phương án đều đã được tối ưu.

Tính *F(i,j)*: vật đang xét là A_i với trọng lượng của vali không được quá *j*. Có 2 khả năng xảy ra:

+ Nếu chọn A_i đưa vào vali, trọng lượng vali trước đó phải không quá $j-A_i$ .Vì mỗi vật chỉ được chọn 1 lần nên giá trị lớn nhất của vali lúc đó là $F(i-1,j-A_i) + B_i$

+ Nếu không chọn A_i, giá trị của vali là như cũ (như lúc trước khi chọn A_i): $F(i-1,j)$

Tóm lại ta có: $F(i, j) = max(F(i-1, j-A_i)+ B_i ,F(i-1 , j))$.

Kết quả bài toán là *F(n, w)*

Độ phức tạp của thuật toán là O(n*W).

```
#include<bits/stdc++.h>
using namespace std;
int n, w, a[102], b[102], f[102][1002];
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0);
   cout.tie(0);
   freopen("VALIA.inp", "r", stdin);
   freopen("VALIA.out", "w", stdout);
   cin >> n >> w;
   for (int i =1 ;i <= n; i ++ )
      cin >> a[i] >> b[i];
   memset(f, 0, sizeof(f));
   for (int i = 1; i <= n; i ++)
      for (int j =1 ;j <= w; j++){
         if (a[i] <= j)
            f[i][j] = max(f[i - 1][j - a[i]] + b[i], f[i - 1][j]);
         else
            f[i][j] = f[i - 1][j];
      }
   cout << f[n][w];
```

```
    return 0;
}
```

Câu 2: Tên bài: **VALIB.CPP**

Lưu ý: Trong bài toán này, mỗi đồ vật có thể lựa chọn nhiều lần.

Ta cũng xây dựng các hàm và công thức giống như bài toán 1, chỉ có điều đặc biệt ở chỗ: Khi xét đến đồ vật i và có trọng lượng vali tối đa là j, ta thấy trường hợp nếu $A_i <= j$ thì $F(i, j) = max(F(i , j – A_i) + B_i, F(i – 1, j))$. Bởi vì lúc này 1 đồ vật hoàn toàn có thể lấy nhiều lần, nên khi lấy 1 đồ vật thứ i thì hoàn toàn có khả năng đồ vật thứ i đã được lấy trước đó.

Kết quả bài toán là $f[n][w]$

Độ phức tạp $O(n * w)$.

```
#include<bits/stdc++.h>
using namespace std;
int n, w, a[102], b[102], f[102][1002];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("VALIB.inp", "r", stdin);
    freopen("VALIB.out", "w", stdout);
    cin >> n >> w;
    for (int i =1 ;i <= n; i ++ )
        cin >> a[i] >> b[i];
    memset(f, 0, sizeof(f));
    for (int i = 1; i <= n; i ++)
        for (int j =1 ;j <= w; j++){
            if (a[i] <= j)
                f[i][j] = max(f[i][j - a[i]] + b[i], f[i - 1][j]);
            else
                f[i][j] = f[i - 1][j];
        }
    cout << f[n][w];
    return 0;
}
```

Câu 3: Tên bài: **SEQ.CPP**

Ta chọn hàm mục tiêu là *F(i, j)* có ý nghĩa là nếu có thể tạo ra tổng bằng *j* từ *i* phần tử đầu tiên của dãy thì *F(i, j) = true*, ngược lại *F(i, j) = false*.

Do đó, nếu *F(n, S) = true* thì bài toán có nghiệm.

Ta có thể tính *F(i,j)* theo công thức:

F(i, j) = true Nếu *F(i−1 , j) = true* hoặc *F(i−1,j−a[i]) = true*.

Ta có thể nhận xét rằng để tính dòng thứ *i*, ta chỉ cần dòng *i−1*. Bảng phương án khi đó chỉ cần 1 mảng 1 chiều *F [0..S]*.

Độ phức tạp của thuật toán là *O(n*m)* (với *m* là tổng của *n* số).

```
#include<bits/stdc++.h>
using namespace std;
int n, s, a[202];
bool f[40003];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("SEQ.inp", "r", stdin);
    freopen("SEQ.out", "w", stdout);
    cin >> n >> s;
    for (int i =1 ;i <= n; i ++ )
        cin >> a[i];
    memset(f, false, sizeof(f));
    f[0] = true;
    for (int i = 1; i <= n; i++)
        for (int j = s; j >= a[i]; j --){
            if (f[j] == false && f[j - a[i]] == true)
                f[j] = true;
        }
    if (f[s] == true) cout << "YES";
    else cout << "NO";
    return 0;
}
```

Câu 4. Tên bài: **FARMER.CPP**

Dễ thấy mảnh đất thứ i có A_i cây ôliu và dải đất thứ j có $B_j - 1$ cây ôliu. Ta xem các mảnh đất và dải đất là các "đồ vật", đồ vật thứ k có khối lượng W_k và giá trị V_k (nếu k là mảnh đất i thì $W_k = V_k = A_i$, nếu k là dải đất j thì $W_k = B_j$, $V_k = B_j - 1$). Ta cần chọn các "đồ vật", sao cho tổng "khối lượng" của chúng không vượt Q và tổng "giá trị" là lớn nhất. Đây chính là bài toán *VALIA* đã trình bày ở trên.

Độ phức tạp của thuật toán là $O(Q*(m + n))$.

```
#include<bits/stdc++.h>
using namespace std;
int Q, n, m;
int w[2002], v[2002];
int f[2002][30003];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("FARMER.inp", "r", stdin);
    freopen("FARMER.out", "w", stdout);
    cin >> Q >> n >> m;
    int tmp;
    for (int i = 1; i <= n; i++){
        cin >> tmp;
        v[i] = w[i] = tmp;
    }
    for (int i = 1; i <= m; i++){
        cin >> tmp;
        v[i + n] = tmp - 1;
        w[i + n] = tmp;
    }
    n += m;
    memset(f, 0, sizeof(f));
    for (int i = 1; i <= n; i ++)
        for (int j =1 ;j <= Q; j++){
            if (w[i] <= j)
```

```
            f[i][j] = max(f[i - 1][j - w[i]] + v[i], f[i - 1][j]);
        else
            f[i][j] = f[i - 1][j];
    }
  cout << f[n][Q];
  return 0;
}
```

Câu 5. Tên bài: **MONSTER.CPP**

Gọi *F[i, j]* là tổng độ hung dữ lớn nhất của các tên cướp mà Bờm mua chuộc được khi đi qua *i* tên cướp đầu tiên và dùng *j* đồng tiền vàng. Nếu Bờm không thể đi qua *i* tên cướp đầu tiên với *j* đồng vàng thì *F[i, j] = -1*.
- Các bài toán cơ sở:

F[1, j] = H[1] nếu j ≥ T[1]

Ngược lại: *F[1, j] = -1*.

Công thức truy hồi: *(i > 1)*

+ Bờm mua chuộc tên cướp thứ *i* thì :

F[i, j] = F[i - 1, j - T[i]] + H[i] (điều kiện: *F[i - 1, j - T[i]] ≠ -1*)

+ Bờm không mua chuộc tên cướp thứ *i* thì :

F[i, j] = F[i-1, j] (điều kiện: *F[i-1, j] ≥ H[i])*

Nghiệm của bài toán là *j* nhỏ nhất sao cho *F[n, j] <> -1* hoặc *F[n,v]* nếu *F[n,v]= -1*

Độ phức tạp của thuật toán là *O(n * V)*.

```
#include<bits/stdc++.h>
using namespace std;
int n, gold, h[1003], t[1003], f[1002][10003];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("MONSTER.inp","r",stdin);
    freopen("MONSTER.out","w", stdout);
    cin >> n >> gold;
```

```
    for (int i = 1; i <= n; i++) cin >> h[i];
    for (int i = 1; i <= n; i++) cin >> t[i];
    memset(f, -1, sizeof(f));
    for (int j = 1; j <= gold; j ++){
        if (j >= t[1]) f[1][j] = h[1];
        else f[1][j] = -1;
    }
    for (int i = 2; i <= n; i++){
        for (int j = 1; j <= gold; j++){
            f[i][j] = -1;
            if (f[i - 1][j] >= h[i])
                f[i][j] = max(f[i][j], f[i - 1][j]);
            if (j >= t[i]) {
                if (f[i - 1][j - t[i]] != -1)
                    f[i][j] = max(f[i][j], f[i - 1][j - t[i]] + h[i]);
            }
        }
    }
    if (f[n][gold] == -1) cout << -1;
    else {
        for (int j = 1; j <= gold; j ++)
            if (f[n][j] != -1) {
                cout << j; break;
            }
    }
    return 0;
}
```

3.4. Ghép cặp

Câu 1. Tên bài: **MPILOT.CPP**

Gọi F[i][j] là số tiền tối ưu khi xét hết i người , trong đó còn j trợ lí là chưa có lái chính bay kèm. Do đó, có hai trường hợp xảy ra :

+ Nếu i chọn làm lái chính thì F[i][j] = F[i-1][j+1] + giá_lái_chính(i). (Sẽ mất đi 1 trợ lí vì đi kèm với lái chính i)

+ Nếu i chọn làm trợ lí thì F[i][j] = F[i-1][j-1] + giá_trợ_lí(i). (Vì đám trợ lí lúc này sẽ tăng lên 1 người)

Khởi tạo: Nhận xét người thứ nhất luôn phải làm trợ lí, (vì tuổi nhỏ nhất) nên F[1][1] = giá_trợ_lí(1).

Nhận xét là số lượng trợ lí mỗi giai đoạn không lớn hơn i (số phi công hiện tại) và n/2 (tối đa n/2 trợ lí), do đó thuật có độ phức tạp là $O(N^2/2)$

Nhận xét kế tiếp là mảng F[i][] chỉ tính thông qua F[i-1][] nên ta có thể tạo mảng F[2][] (như hai mảng 1 chiều) để tiết kiệm bộ nhớ vì N <= 10000.

```
#include <bits/stdc++.h>
const int N = 1E4+2;
using namespace std;
long long  x[N], y[N], f[2][N];
int n;
int main(){
   ios_base::sync_with_stdio(0);
   cin.tie(0); cout.tie(0);
   freopen("mpilot.inp","r",stdin);
   freopen("mpilot.out","w",stdout);
   cin >> n;
   for(int i = 1; i <= n; i++)
      cin >> x[i] >> y[i];
   f[1][1] = y[1];
   int m, id;
   for (int i = 2; i <= n; i++){
      m = min(i, n/2); // tối đa có n/2 người trợ lí
      id = i % 2; // tối ưu bộ nhớ
      f[id][0] = f[1 - id][1] + x[i];
      f[id][m] = f[1 - id][m - 1] + y[i];
      for (int j = 1; j < m; j ++){
         f[id][j] = min(f[1 - id][j- 1] + y[i], f[1 - id][j + 1] + x[i]);
      }
   }
   cout << f[0][0];
}
```

Câu 2. Tên bài: **FLOWER.CPP**

Nhận xét rằng bài toán nêu trên là một bài toán ghép cặp có yêu cầu về thứ tự nên ta có thể giải quyết bằng phương pháp QHĐ.

Hàm mục tiêu: F tổng giá trị thẩm mỹ của cách cắm.

Giá trị thẩm mỹ phụ thuộc vào các hoa và các lọ đang được xét nên ta sẽ dùng mảng 2 chiều để lưu bảng phương án.

F(i,j) là tổng giá trị thẩm mỹ lớn nhất khi xét đến hoa *i* và lọ *j*.

Khi tính *F(i,j)*: hoa đang xét sẽ là hoa *i* và lọ *j*.

+ Nếu *i* = *j*. Chỉ có một cách cắm:

$$F[i,j] = V[1,1] + V[2,2] + ... + V[i,i]$$

+ Nếu *i* > *j* . Không có cách cắm hợp lý

+ Nếu *i* < j. Có 2 trường hợp xảy ra:

- Cắm hoa *i* vào lọ *j*. Tổng giá trị thẩm mỹ là $F[i-1,j-1]+V(i,j)$. (Bằng tổng giá trị trước khi cắm cộng với giá trị thẩm mỹ khi cắm hoa *i* vào lọ *j*)
- Không cắm hoa *i* vào lọ *j* (có thể cắm vào lọ trước *j*), giá trị thẫm mỹ của cách cắm là như cũ: $F[i,j-1]$

Kết quả bài toán là *F[K][N]*

Độ phức tạp của thuật toán là *O(N*K)*.

```
#include<bits/stdc++.h>
using namespace std;
int n, k;
int a[102][102], f[102][102], sum[102];
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("FLOWER.inp","r",stdin);
   freopen("FLOWER.out","w", stdout);
   cin >> k >> n;
   for (int i =1 ; i <= k; i ++)
      for (int j =1 ; j <= n; j ++)
         cin >> a[i][j];
   // khởi tạo hàm mục tiêu F
   for (int j = 0; j <= n; j ++) f[0][j] = 0;
   for (int i = 1; i <= k; i ++)
      for (int j = 0; j <= n; j ++)
```

```
            f[i][j] = -1000000007; // khởi tạo f bằng một số âm cực lớn.
        // khởi tạo hàm tính trường hợp i = j
        sum[0] = 0;
        for (int i = 1; i <= k; i++ ) sum[i] = sum[i - 1] + a[i][i];
        for (int i =1 ; i <= k; i++)
            for (int j = i; j <= n; j ++){
                if (i == j) f[i][j] = sum[i];
                else if (i < j)
                    f[i][j] = max(f[i - 1][j - 1] + a[i][j], f[i][j - 1]);
            }
        cout << f[k][n];
    }
```

Câu 3. Tên bài: **CHAIR.CPP**

Ta nhận thấy, nếu tạo 1 mảng 2 chiều *V[i][j]* tức là số ghế chuyển ra hoặc vào nếu nhóm học sinh *i* vào phòng *j*. Ta sẽ có $V[i][j] = |A_j - B_i|$

Đến đây, ta có thể hoàn toàn làm như câu 2.

Độ phức tạp $O(n * k)$

```
#include<bits/stdc++.h>
using namespace std;
int n, k;
int a[1002], b[1002], v[1002][1002], f[1002][1002], sum[1002];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("CHAIR.inp","r",stdin);
    freopen("CHAIR.out","w", stdout);
    cin >> n >> k;
    for (int i = 1; i <= n; i ++)
        cin >> a[i];
    for (int j = 1; j <= k; j++)
        cin >> b[j];
    for (int i = 1; i <= k; i++)
        for (int j = 1; j <= n; j ++)
            v[i][j] = abs(a[j] - b[i]);
```

```
    for (int j = 0; j <= n; j ++) f[0][j] = 0;
    for (int i = 1; i <= k; i ++)
        for (int j = 0; j <= n; j ++)
            f[i][j] = 1000000007;
    sum[0] = 0;
    for (int i = 1; i <= k; i++ ) sum[i] = sum[i - 1] + v[i][i];
    for (int i =1 ; i <= k; i++)
        for (int j = i; j <= n; j ++){
            if (i == j) f[i][j] = sum[i];
            else if (i < j)
                f[i][j] = min(f[i - 1][j - 1] + v[i][j], f[i][j - 1]);

        }
    cout << f[k][n];
}
```

Câu 4. Tên bài: **SHOES.CPP**

Theo bổ đề **Heuristic**: cho 2 dãy tăng dần các số *dương* $A_1, A_2, \ldots, A_n$ và $B_1, B_2, \ldots, B_n$. Gọi $C_1, C_2, \ldots, C_n$ là một hoán vị bất kì của dãy B. Khi đó:

$$|A_1 - B_1| + |A_2 - B_2| + \ldots + |A_n - B_n| \le |A_1 - C_1| + |A_2 - C_2| + \ldots + |A_n - C_n|.$$

Như vậy, ta sẽ sắp xếp N đôi giày và K cầu thủ theo thứ tự tăng dần. Sau đó, ta lại đưa vào mảng 2 chiều *A[i][j]* tức là độ chênh lệch của cầu thủ i khi đi đôi giày j: *A[i][j] = | S[i] – H[j] |*

Sau đó làm tương tự câu 2 và 3 ở trên.

Độ phức tạp của thuật toán là $O(N * K)$.

```
#include<bits/stdc++.h>
using namespace std;
int n, k;
int a[1002], b[1002], v[1002][1002], f[1002][1002], sum[1002];
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("SHOES.inp","r",stdin);
```

```
    freopen("SHOES.out","w", stdout);
    cin >> n >> k;
    for (int i = 1; i <= n; i ++)
        cin >> a[i];
    for (int j = 1; j <= k; j++)
        cin >> b[j];
    sort(a + 1, a + n + 1);
    sort(b + 1, b + k
          + 1);
    for (int i = 1; i <= k; i++)
        for (int j = 1; j <= n; j ++)
            v[i][j] = abs(a[j] - b[i]);
    for (int j = 0; j <= n; j ++) f[0][j] = 0;
    for (int i = 1; i <= k; i ++)
        for (int j = 0; j <= n; j ++)
            f[i][j] = 1000000007;
    sum[0] = 0;
    for (int i = 1; i <= k; i++ ) sum[i] = sum[i - 1] + v[i][i];
    for (int i =1 ; i <= k; i++)
        for (int j = i; j <= n; j ++){
            if (i == j) f[i][j] = sum[i];
            else if (i < j)
                f[i][j] = min(f[i - 1][j - 1] + v[i][j], f[i][j - 1]);
        }
    cout << f[k][n];
   return 0;
}
```

Câu 5. Tên bài: **BUILT.CPP**

Ta thấy rằng, nếu kho A cung cấp x đơn vị vật liệu cho công trường thứ i thì đồng nghĩa với việc kho B phải cung cấp D[i] – x đơn vị vật liệu cho công trường đó.

Ta xây dựng hàm mục tiêu F(i, j) là cước phí vận chuyển nhỏ nhất trong trường hợp kho A cung cấp j đơn vị vật liệu đến các công trường từ 1, 2, .. i (j ≤ R). Công thức truy hồi :

+ F(1, j) = Min(A[1] * x + B[1] *(D[1] – x)) với x là đơn vị vật liệu kho A cung cấp (x ≤ D[1] và x ≤ j)

+ Do đó F(i, j) = Min {A[i]*x + B[i]*(D[i]-x) + F(i-1, j-x)}.

Kết quả bài toán là F[N[R]

Độ phức tạp của thuật toán là O(N * R * max(D[i])).

```
#include<bits/stdc++.h>
using namespace std;
int n, r;
int d[505], a[505], b[505], f[505][505];
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0);
   cout.tie(0);
   freopen("BUILT.inp","r",stdin);
   freopen("BUILT.out","w", stdout);
   cin >> n >> r;
   for (int i = 1; i <= n ; i++)
      cin >> d[i];
   for (int i = 1; i <= n ; i++)
      cin >> a[i];
   for (int i = 1; i <= n ; i++)
      cin >> b[i];
   for (int i = 1; i <= n; i++)
      for (int j = 0; j <= r; j ++)
         f[i][j] = 1000000007;
   for (int j = 0; j <= r; j ++)
      for (int x = 0; x <= min(j, d[1]); x++)
         f[1][j] = min(f[1][j], a[1] * x + b[1] * (d[1] - x));
   for (int i = 2; i <= n; i ++)
```

```
    for (int j = 0; j <= r; j ++){
        for (int x = 0; x <= min(j, d[i]); x ++){
            if (f[i - 1][j - x] == 1000000007) continue;
            // không tồn tại trường hợp trên nên bỏ qua
            f[i][j] = min(f[i][j], a[i] * x + b[i] * (d[i] - x) + f[i - 1][j - x]);
        }
    }
  cout << f[n][r];
 return 0;
}
```

Câu 6. Tên bài: **CABLE.CPP**

+ Gọi F[i][0] là độ dài ngắn nhất xét đến máy i, với máy i không được nối với máy i-1. Gọi F[i][1] là độ dài ngắn nhất xét đến máy i, với máy i được nối với máy i-1

Khi đó: F[2][0] = +oo (không tồn tại cách này)

F[2][1] = a[1].

Lập công thức:

+ F[i][0] = F[i-1][1] (nếu i không nối i-1 thì bắt buộc i-1 phải nối i-2)

+ F[i][1] = min(F[i-1][0] , F[i-1][1]) + a[i-1] (i nối i-1 thì i-1 có thể nối hoặc không nối với i-2)

Kết quả bài toán là F[N][1]

Độ phức tạp của thuật toán là *O(N)*.

```
#include<bits/stdc++.h>
using namespace std;
int n, a[25002], f[25002][2];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("CABLE.inp","r",stdin);
    freopen("CABLE.out","w", stdout);
    cin >> n;
    for (int i =1 ; i < n; i ++)
```

```
        cin >> a[i];
    f[2][1] = a[1];
    f[2][0] = 1000000007;
    for (int i = 3; i <= n; i ++){
        f[i][0] = f[i - 1][1];
        f[i][1] = min(f[i - 1][0], f[i - 1][1]) + a[i - 1];
    }
    cout << f[n][1];
    return 0;
}
```

3.5. Di chuyển.

Phương pháp chung: Đối với các bài toán có dạng di chuyển thì việc xác định hàm mục tiêu cần căn cứ vào quy luật di chuyển cho phép và cần phải chú ý vào giá trị biên. Đây là dạng toán thường gặp tuy nhiên việc xây dựng hàm mục tiêu và công thức truy hồi là khá dễ dàng

Câu 1. Tên bài: **MAX.CPP**

Gọi F(i,j) là giá trị lớn nhất có được khi di chuyển đến ô (i, j). Có 3 ô có thể đi đến ô (i, j) là ((i−1, j), (i−1, j−1) và (i−1, j+1). Do đó ta có công thức QHĐ như sau:

F[1][j] = A[1][j] với mọi j =1, 2, … n

F[i][j]=max(F[i−1][j], F([i−1][j−1], F[i−1][j+1]) + A[i][j] với i > 1

Để xử lý các F nằm ở viền (cột 1 và cột n) ta khởi tạo các F ở cột 0 và cột n+1 là các số nhỏ nhất có thể.

F[i][0] = F[i][n+1] = $-\infty$ với mọi i = 1…m-1

Kết quả bài toán là max(F[m][j]) với mọi j = 1..n

Độ phức tạp của thuật toán là *O(m*n)*.

```
#include<bits/stdc++.h>
using namespace std;
int m,n,a[102][102], f[102][102];
const int inf = 1000000007;
int main(){
   ios_base::sync_with_stdio(0);
```

```
    cin.tie(0); cout.tie(0);
    freopen("MAX.inp", "r", stdin);
    freopen("MAX.out", "w", stdout);
    cin>> m >> n;
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j ++)
            cin >> a[i][j];
    for (int j = 1; j <= n; j++)
        f[1][j] = a[1][j];
    for (int i = 1; i <= m-1; i++)
       f[i][0] = f[i][n+1] = -inf;
    for (int i = 2; i <= m; i ++)
        for (int j = 1; j <= n; j++)
            f[i][j] = max(f[i - 1][j], max(f[i- 1][j - 1], f[i - 1][j + 1])) + a[i][j];
    int res = 0;
    for (int j = 1; j <= n; j++) res = max(res, f[m][j]);
    cout << res;
    return 0;
}
```

Câu 2. Tên bài: **VSTEPS.CPP**

Gọi F[i] là số cách leo đến bậc thang thứ i. Dễ dàng nhận thấy F[0] = 0; F[1] = 1; Nếu bậc thứ i thủng thì F[i] = 0, trái lại F[i] chỉ có thể leo từ bậc i-1 hoặc i-2, nên F[i] = F[i-1] + F[i-2] (không quan tâm i-1 với i-2 có thủng hay không vì nếu có thủng thì nó cũng bằng 0, không ảnh hưởng đến F[i])

Kết quả bài toán là F[n]. Độ phức tạp: *O(N)*.

```
    #include <bits/stdc++.h>
    const   int N = 1E5+5;
    const  int BASE = 14062008;
    using namespace std;
    int f[N], a[N];
    bool d[N];
    int main(){
        ios_base::sync_with_stdio(0);
        cin.tie(0); cout.tie(0);
        freopen("vsteps.inp", "r", stdin);
        freopen("vsteps.out", "w", stdout);
```

```
    int n,k;
    cin>>n>>k;
    for (int i=1;i<=n;i++)
        d[i]=true;
    for (int i=1;i<=k;i++){
        cin>>a[i];
        d[a[i]]=false;
    }
    f[1]=1; f[0]=0;
    for (int i=2;i<=n;i++){
        if (d[i]==true) f[i]=(f[i-1]% BASE +f[i-2]% BASE)% BASE;
        if (d[i]==false) {f[i]=0; continue;}
    }
    cout<<f[n];
    return 0;
}
```

Câu 3. Tên bài: **TRIANGLE.CPP**

Mô tả các phần tử của tam giác số như một ma trận, A[i,j] là phần tử thứ j trên dòng i (với $1 \le i \le N$ và $1 \le j \le i$). Có 2 ô có thể di chuyển đến ô (i,j) là ô (i−1,j−1) và ô (i−1,j). Gọi F(i,j) là tổng lớn nhất có thể có khi đi đến ô (i,j) ta có:

+ F[1][1] = A[1][1].

+ F[i][1]=F[i−1][1]+A[i][1].

+ F[i][j]=max(F[i−1][j−1], F[i−1][j])+A[i][j]

Kết quả bài toán là max(F[N][j]) với mọi j = 1, 2,…N.

Độ phức tạp của thuật toán là $O(N^2)$.

```
#include<bits/stdc++.h>
using namespace std;
int n, a[102][102], f[102][102];
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("TRIANGLE.inp", "r", stdin);
    freopen("TRIANGLE.out", "w", stdout);
```

```
    cin >> n;
    for (int i = 1; i <= n; i ++)
       for (int j = 1; j <= i; j ++)
          cin >> a[i][j];
    memset(f, 0, sizeof(f));
    f[1][1] = a[1][1];
    for (int i = 2; i <= n; i ++)
       f[i][1] = f[i - 1][1] + a[i][1];
    for (int i = 2; i <= n;  i++)
       for (int j = 2; j <= i; j ++)
          f[i][j] = max(f[i - 1][j - 1], f[i - 1][j]) + a[i][j];
    int res = 0;
    for (int j = 1; j <= n; j ++)
      res = max (res, f[n][j]);
    cout << res;
    return 0;
}
```

Câu 4. Tên bài: **PLAYGROUND.CPP**

Gọi F[i][j] là chi phí ít nhất đi từ ô (1, 1) đến ô (i, j)

Công thức truy hồi:

F[i][j] = Min{F[i][j-1], F[i-1][j]}+C[i][j] với mọi $1 \le i \le U$ và $1 \le j \le V$;

Ta tính được F[U][V].

Gọi G[i][j] là chi phí ít nhất đi từ ô (U, V) đến ô (i, j). Ta có công thức:

G[i][j] = Min{G[i][j-1], G[i-1][j]}+C[i][j] với mọi $U \le i \le M$ và $V \le j \le N$;

Kết quả bài toán: F[U][V] + G[M][N] – C[U][V] vì chi phí C[U][V] được tính 2 lần.

Chú ý: trong khi code ta tính các F, G ở phía ngoài biên bên trái và phía trên trước và tận dụng chỉ dùng một mảng F để tính cho mảng G.

Độ phức tạp của thuật toán là *O(M*N)*.

```
#include<bits/stdc++.h>
using namespace std;
int m, n, u, v;
```

```
int c[1003][1003], f[1003][1003];
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("PLAYGROUND.inp", "r", stdin);
    freopen("PLAYGROUND.out", "w", stdout);
    cin >> m >> n >> u >> v;
    for (int i = 1; i <= m; i ++)
        for (int j = 1; j <= n; j ++)
            cin >> c[i][j];
    memset(f, 0, sizeof(f));
    f[1][1] = c[1][1];
    for (int i = 2; i <= m; i++)
        f[i][1] = f[i - 1][1] + c[i][1];
    for (int j = 2; j <= n; j ++)
        f[1][j] = f[1][j - 1] + c[1][j];
    for (int i = 2; i <= m; i ++)
        for (int j = 2; j <= n; j ++)
            f[i][j] = min(f[i][j - 1], f[i - 1][j] ) + c[i][j];
    int res = f[u][v];
    memset(f, 0, sizeof(f));
    f[u][v] = c[u][v];
    for (int i = u + 1; i <= m; i ++)
        f[i][v] = f[i - 1][v] + c[i][v];
    for (int i = v + 1; i <= n; i ++)
        f[u][i] = f[u][i - 1] + c[u][i];
    for (int i = u + 1; i <= m; i++)
        for (int j = v + 1; j <= n; j++)
            f[i][j] = min(f[i][j - 1], f[i - 1][j] ) + c[i][j];
    res = res + f[m][n] - c[u][v];
    cout << res;
    return 0;
}
```

Câu 5. Tên bài: **JUMP.CPP**

Để đơn giản ta có thể xem như các vị trí đã được sắp xếp tăng dần theo X_i. Ngoài ra ta chỉ xét việc nhảy theo chiều X_i tăng, việc nhảy ngược lại có thể được làm tương tự bằng cách đảo dấu các tọa độ X_i và thứ tự các vị trí.

Gọi F[i][j] là số điểm đạt được lớn nhất có thể nếu vị trí cuối cùng là vị trí i và vị trí liền trước là j.

Gọi k là vị trí liền trước j, ta có X[j] - X[k] <= X[i] - X[j].

Do đó F[i][j]= Max{F[j][k]}+ P[i]. ∀k liền trước j

Độ phức tạp của thuật toán là $O(N^2 + N)$.

```
#include<bits/stdc++.h>
using namespace std;
struct  vitri{
    int x;
    int y;
};
int n;
vitri   pnt[1003];
int  f[1003][1003], g[1003];
bool  cmp (vitri  a, vitri  b){
    return a.x < b.x;
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("JUMP.inp", "r", stdin);
    freopen("JUMP.out", "w", stdout);
    cin >> n;
    for (int i = 1; i <= n ; i ++)
        cin >> pnt[i].x >> pnt[i].y;
    sort(pnt +1, pnt + n + 1, cmp); // sắp xếp theo tọa độ x.
    for (int i = 1; i <= n; i ++)
       f[i][i] = pnt[i].y;
    int dis1, dis2, res = 0;
    for (int i = 2; i <= n; i ++)
       for (int j = 1; j < i; j ++){
          f[i][j] = 0;
          dis1 = pnt[i].x - pnt[j].x;
          for (int k = j; k >= 1; k --){
              dis2 = pnt[j].x - pnt[k].x;
              if (dis2 > dis1) break;
```

```
            f[i][j] = max(f[i][j], f[j][k] + pnt[i].y);
        }
        res = max(res, f[i][j]);
    }
  cout << res;
  return 0;
}
```

3.6. Quy hoạch động vị trí cấu hình.

Câu 1. Tên bài: **BINARY.CPP**

Gọi F[i] là số phần tử của tập S ứng với N = i.

+ F[i] = F[i-1] + F[i-2]

+ F[0] = 1; F[1] = 2.

Cách xây dựng dãy bit thứ M tương tự như bài trên.

```
#include<bits/stdc++.h>
using namespace std;
int f[42], c[42];
int n, m;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);  cout.tie(0);
    freopen("BINARY.inp","r", stdin);
    freopen("BINARY.out", "w", stdout);
    cin >> n >> m;
    f[0] = 1;  f[1] = 2;
    for (int i = 2; i <= n; i++)
        f[i] = f[i - 1] + f[i - 2];
    for (int t = 1; t <= n ; t ++)
        if ( m <= f[n - t])
            c[t] = 0;
        else {
            c[t] = 1;
            m  -= f[n - t];
        }
    for (int i = 1; i <= n; i ++)
        cout << c[i];
```

```
    return 0;
}
```

Câu 2. Tên bài: **SHHV.CPP**

Gọi F[i] là số lượng các hoán vị có i chữ số. Ta có:

+ F[i] = F[i-1]*i (hay F[i] = i!).

+ F[1] = 1

Fin(i) là hàm cho biết số lượng các số trong đoạn $a_{i+1}…a_n$ và có giá trị bé hơn a[i].

```
#include<bits/stdc++.h>
using namespace std;
long long a[15], f[15];
int n;
long long fin(int i){
    long long cur = 0;
    for (int j = i + 1; j <= n; j++)
        if (a[j] < a[i]) cur++;
    return cur;
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("SHHV.inp","r", stdin);
    freopen("SHHV.out", "w", stdout);
    cin >> n;
    for (int i = 1; i <= n ; i++)
        cin >> a[i];
    f[1] = f[0] = 1;
    for (int i = 2; i <= n; i ++)
        f[i] = f[i - 1] * i;
    long long res = 0;
    for (int i = 1; i <= n; i++){
        res += fin(i) * f[n - i];
    }
    cout << res + 1;
```

```
    return 0;
}
```

Câu 3. Tên bài: **HV2.CPP**

Gọi F[i] là số lượng các hoán vị có i chữ số. Ta có

+ F[i] = F[i-1]*i (hay F[i] = i!).
+ F[1] = 1

Để tìm hoán vị, ta lần lượt thử từng số vào từng vị trí i xem có thỏa mãn yêu cầu hay không.

```
#include<bits/stdc++.h>
using namespace std;
long long n, m,f[15], a[15];
bool kt[15];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("HV2.inp","r", stdin);
    freopen("HV2.out", "w", stdout);
    cin >> n >> m;
    f[1] = f[0] = 1;
    for (int i = 2; i <= n; i ++)
        f[i] = f[i - 1] * i;
    memset(kt, false, sizeof(kt));
    for (int i = 1; i <= n; i ++){
        int j;
        for (j = 1; j <= n; j++)
            if (kt[j] == false){ // chưa có trong hoán vị
                if (m > f[n - i]) m -= f[n - i];
                else break;
            }
        a[i] = j;
        kt[j] = true;
    }
    for (int i = 1; i <= n; i++) cout << a[i] << " ";
    return 0;
}
```

Câu 4. Tên bài: **CATALAN.CPP**

Xét lưới vuông (n + 1) * (n + 1). Một quy tắc đi thoả mãn:

+ Xuất phát từ ô (1, 1) đến ô (n + 1, n + 1)

+ Mỗi bước đi chỉ được di chuyển đến ô kề cạnh bên phải hoặc bên dưới.
+ Không được di chuyển qua ranh giới là đường chéo nối đỉnh trái trên và phải dưới của lưới

+ Nếu quy định a[1][1] = 0 và a[i + 1][j] = a[i][j] + 1, a[i][j + 1] = a[i][j] – 1.
+ Mỗi cách đi từ ô (1,1) đến ô (n + 1,n + 1) là 1 dãy catalan tương ứng
- Mảng qhđ:

+ F[i][j]: là số cách để đi từ ô (i, j) đến ô (n + 1, n + 1)

+ Công thức : F[i][j] = F[i – 1][j] + F[i][j – 1] với F[n + 1][n + 1] = 1;

+ Giả sử tại ô (u,v) ta di chuyển xuống ô phía dưới thì cách đi này sẽ có số thứ tự lớn hơn cách đi từ ô (u,v) di chuyển sang ô bên phải (với u > v). Do đó ta chỉ quan tâm đến những ô (u, v) mà tại đó thực hiện di chuyển xuống ô phía dưới, ta cộng vào res thêm F[u][v + 1].

Kết quả số thứ tự dãy Catalan tương ứng là res + 1.

Nếu trong trường hợp cho số thứ tự, yêu cầu tìm dãy thì ta làm ngược lại.

Độ phức tạp của thuật toán là $O(n^2)$.

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
int n, a[40];
ll f[40][40];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);  cout.tie(0);
    freopen("CATALAN.inp","r", stdin);
    freopen("CATALAN.out", "w", stdout);
    cin >> n;
    n = 2 * n +1;
    for (int i = 1; i <= n; i++) cin >> a[i];
    f[0][0] = f[1][1] = 1ll;
```

```
    for (int j = 2; j < n; j ++){
        f[0][j] = f[1][j - 1];
        for (int i = 1; i<= n + 1; i++)
            f[i][j] = f[i - 1][j - 1] + f[i +1][j - 1];
    }
    ll res = 0;
    for (int i = 2; i <= n; i++){
        if (a[i] < a[i - 1] || a[i - 1] == 0) continue;
        res += f[a[i - 1] - 1][n - i];
    }
    cout << res + 1 << '\n';
    ll x;
    cin >> x; x -= 1;
    a[1] = 0;
    for (int i = 2; i <= n; i++){
        if (a[i - 1] == 0) a[i] = 1;
        else {
            if (x >= f[a[i - 1] - 1][n - i]){
                x -= f[a[i - 1] - 1][n - i];
                a[i] = a[i - 1] + 1;
            }
            else a[i] = a[i - 1] - 1;
        }
    }
    for (int i = 1; i<= n ; i++) cout << a[i] << " ";
    return 0;
}
```

Câu 5. Tên bài: **MEANK.CPP**

Một bài toán tổ hợp. Biến N thành 1 dãy nhị phân. Nhìn chung, có 2 cách để tạo 1 dãy nhị phân nhỏ hơn dãy ban đầu là :

+ Dãy đó có độ dài nhỏ hơn dãy ban đầu

+ Dãy có cùng độ dài và tồn tại 1 vị trí đầu tiên khác nhau (tính từ trái qua phải).

Ví dụ:

10100 >1010 (độ dài nhỏ hơn)

100101>100011 (2 bit đầu tiên khác nhau)

Vậy, từ 2 ví dụ trên, ta dễ dàng tìm được cách để xử lý bài toán này:

+ Lấy tất cả các trường hợp có độ dài nhỏ hơn và phù hợp bằng tổ hợp làm kết quả

+ Tìm tất cả các trường hợp cùng độ dài với dãy ban đầu mà nhỏ hơn N theo trường hợp thứ 2. Bằng cách với mỗi vị trí 1 trong dãy ban đầu, ta thử cho nó là 0 và dùng tổ hợp tính toán số trường hợp.

Trong code dưới ta có 1 hàm xử lý bit:

```
int getbit(int x,int i){
    i--;
    return (x>>i)&1;
}
```

Hàm trên có ý nghĩa: cho giá trị bit thứ i của số x, trong cách biểu diễn nhị phân của số x. Trong hàm trên ta xử dụng 2 phép xử lý bit:

+ >> nghĩa là phép dịch phải (giống như shr trong pascal). ***Ví dụ:***

cout << (12 >> 2); //kết quả in ra 3, ta giải thích như sau, số 12 và 2 đều là kiểu *int* (32 bit): 12 >> 2 nghĩa là dịch chuyển số 12 ở hệ nhị phân sang bên phải 2 bit. Số 12 = 0...01100 (dãy nhị phân gồm 32 bit, dấu … là 26 số 0), khi ta dịch chuyển sang bên phải 2 bit (hiểu nôm na là bỏ 2 bit bên phải đi) thì ta thu được số 0…011 (dãy gồm 32 bit, dấu … là 28 số 0). Đây chính là số 3 trong hệ thập phân. Phép & là phép *and* trên bit. Ví dụ ta muốn biết bit thứ 3 của số 12 là 0 hay 1 thì ta làm như sau: cout << ((12>>(3-1))&1); kết quả in ra số 1. Giải thích: 12 >> 3 ta được kết quả 0...011 (vì dịch sang phải 3 bit, dấu … là 28 số 0). Lúc này ta có phép and như sau

```
    12 >> 3 = 0000 0000 0000 0000 0000 0000 0000 0011
          1 = 0000 0000 0000 0000 0000 0000 0000 0001
(12 >> 3) & 1 = 0000 0000 0000 0000 0000 0000 0000 0001
```

Như vậy kết quả là số 0000 0000 0000 0000 0000 0000 0000 0001 chính là số 1 ở hệ thập phân.

Như vậy muốn lấy bit thứ k của số x thì ta cho x dịch phải (k-1) bit rồi đem *and* với 1. Để hiểu rõ hơn nữa ta có thể search thêm google: "xử lý bit c++"

```
#include <bits/stdc++.h>
```

```
using namespace std;
string m;
int f[50][50];
int d[50];
int l;
int n,k;
void init(){
   memset(f,0,sizeof(f));
   for (int i=0; i<=34;i++)
      f[0][i]=1;
   for (int i=1; i<=34;i++)
      for (int j=i; j<=34;j++)
         f[i][j]=f[i-1][j-1]+f[i][j-1];
}
int getbit(int x,int i){
   i--;
   return (x>>i)&1;
}
void xuly(){
   int kq=0;
   if (k==1) kq++;
   int kt=0;
   int cn0=0;
   memset(d,0,sizeof(d));
   for (int i=32;i>=1;i--){
      if (kt==0) {if (getbit(n,i)==1) {d[i]=1; kt=i;}}
      else{
         if (getbit(n,i)==0) cn0++;
         else d[i]=1;
      }
   }
   for (int i=kt-1;i>=k+1;i--) kq=kq+f[k][i-1];
   int x=0;
   int y;
   for (int i=kt-1;i>=1;i--) {
      if (d[i]==0) x++;
      else {
         y=k-x-1;
```

```
            if (y<0||y>i-1) continue;
            kq=kq+f[y][i-1];
        }
    }
    if (k==cn0) kq++;
    cout<<kq;
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    freopen("MEANK.inp","r", stdin);
    freopen("MEANK.out", "w", stdout);
    init();
    cin >> n >> k;
    xuly();
    return 0;
}
```

3.7. Quy hoạch động trạng thái

Câu 1. Tên bài: **SELECT.CPP**

Mỗi cột có 4 dòng nên có thể dùng biến x có 4 bit nhị phân để mô tả trạng thái chọn của cột i: bit thứ k bằng 1 (0) tương ứng với ô dòng thứ k của cột được (không được) chọn.

Gọi F[i,x] là trọng lượng lớn nhất nếu xét từ cột 1 đến cột i và trạng thái chọn của cột i được biểu diễn bằng biến x. Công thức Quy hoach động là:

$$F[i,x] = \max (F[i-1,x'] + sum(i,x))$$

Trong đó x và x' là hai trạng thái chọn của 2 cột liên tiếp nhau (i và i-1) do đó 2 trạng thái phải thỏa mãn điều kiện không có 2 ô nào được chọn kề nhau.

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const ll inf= 1e18 + 7;
const int N = 10007;
ll n, a[5][N], f[N][20];
```

```
int getbit(int x, int i){
    return (x >> i) & 1;
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);  cout.tie(0);
    freopen("SELECT.inp","r", stdin);
    freopen("SELECT.out", "w", stdout);
    cin >> n;
    ll r = -inf; //r là giá trị lớn nhất của các ô
    for (int i = 1; i <= 4; i++)
        for (int j = 1; j <= n; j++){
            cin >> a[i][j];
            r = max(r, a[i][j]);
        }
    if (r <= 0) {cout << r; return 0;} // trường hợp đặc biệt
    for (int i = 0; i < 20; i++) f[0][i] = 0;
    ll res = -inf;
    for (int j = 1; j <= n ;j++){
        for (int msk = 0; msk < (1 << 4) ; msk ++){
            f[j][msk] = -inf;
            ll tmp = 0;
            bool ch = true;
            for (int i = 0; i < 3; i++)
                if (getbit(msk, i) && getbit(msk, i + 1)) {
                    ch = false;
                    break;
                }
            if (! ch) continue;
            for (int i = 0 ;i < 4; i++)
                if (getbit(msk, i)) tmp += a[i + 1][j];
            for (int msk2 = 0; msk2 < (1 << 4) ; msk2 ++)
                if ((msk2 & msk) == 0 ) // msk2 và msk là 2 trạng thái liên tiếp
thỏa mãn
                    f[j][msk] = max(f[j][msk], f[j - 1][msk2] + tmp);
            res = max(res, f[j][msk]);
        }
    }
```

```
    cout << res;
    return 0;
}
```

Câu 2. Tên bài: **ASSET.CPP**

+ Mảng qhđ: gọi F[i][msk] ($0 \le msk \le 2^n - 1$, $1 \le i \le n$) là tổng tài sản sau khi ghép i con gái với i tiến sĩ có trạng thái là msk (msk có i bit 1, bit 1 thứ j của msk ở vị trí x thì tương ứng với con gái i sẽ cưới tiến sĩ x).
+ Sau đó, với mỗi msk, mình duyệt các trạng thái trước của nó (tức là duyệt các trường hợp j xem con gái thứ i sẽ cưới anh tiến sĩ thứ j hay không), đó là trạng thái msk2.

Công thức : f[i][msk] = max(f[i][msk], f[i - 1][msk2] + a[i][j]) // j ở đây đánh số từ 1 đến n.

Kết quả bài toán là $F[n][2^n-1]$.

Trong *code* dưới ta có sử dụng hàm *__builtin_popcount(x)* có sẵn trong thư viện, với ý nghĩa là số lượng bit 1 của số x khi biểu diễn nhị phân (__ là 2 dấu gạch nối)

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const int N = 21;
int n, a[N][N];
ll f[N][1050000];
void input(){
   cin >> n;
   for (int i = 1; i<= n; i++)
      for (int j = 1; j <= n; j++)
         cin >> a[i][j];
}
void init(){
   memset(f, 0, sizeof(f));
}
void sol(){
   int q = (1 << n) - 1; // số này là 2^n-1
   for(int i = 1; i <= n; i++){
```

```
    for (int msk = 0; msk <= q; msk ++){
        if (__builtin_popcount(msk) != i) continue;// nếu msk không có đúng i
bit 1 thì bỏ qua
        for(int j = 0; j <= n - 1; j ++){
            if (((msk >> j) & 1) == 0) continue;
            int msk2 = (msk ^ (1 << j));
            if (f[i][msk] < f[i - 1][msk2] + a[i][j + 1]){
               f[i][msk] = f[i - 1][msk2] + a[i][j + 1];
            }
        }
    }
  }
  cout << f[n][q] ;
}
int main(){
  ios_base::sync_with_stdio(false);
  cin.tie(0);  cout.tie(0);
  freopen("ASSET.inp","r", stdin);
  freopen("ASSET.out", "w", stdout);
  input();
  init();
  sol();
  return 0;
}
```

Câu 3: Tên bài: **TRIP.CPP**

Gọi F[i,x] là tổng chi phí nếu Trạng Tí đang ở đất nước i và trạng thái các đất nước đã đi qua được lưu vào biến x: gồm N bit nhị phân 0,1 (bit thứ k có giá trị bằng 1/0 có ý nghĩa đã đi qua đất nước k hay chưa). Công thức Quy hoạch động: F[i,x] = min(F[i',x'] + C[i',i]). Trong đó:

+ i' là đất nước đã được đánh dấu đi qua trong x.

+ x' là trạng thái giống như trạng thái x nhưng đất nước i chưa được đánh dấu

Ở *code* dưới ta có dùng thủ tục tắt bit thứ k + 1 của x. ~ là phép đảo bit; << là phép dịch trái bit (như shl trong pascal):

```
  int tatbit(int x,int k){
     return x&(~(1<<k));
```

```
}
```

Ta có 1 << k nghĩa là dịch sang trái k bit. Ví dụ trong trường hợp k = 5;

1 = 0000 0000 0000 0000 0000 0000 0000 0001

1 << 5 = 0000 0000 0000 0000 0000 0000 0010 0000

~ (1 << 5) = 1111 1111 1111 1111 1111 1111 1101 1111

Như vậy tại vị trí k+1 thì là bit 0, còn lại tất cả các vị trí khác đều là bit 1. Ta dùng phép *and* với x thì ta được giá trị mà bit ở vị trí k+1 của giá trị x là bit 0, tất cả các bit còn lại của x vẫn giữ nguyên vì nó and với bit 1 thì vẫn là bit đó.

```
#include <bits/stdc++.h>
using namespace std;
#define mm 100000
int k, n;
int t[20][(1<<16)+5];
int c[20][20];
int getbit(int x,int k) {
   return (x>>k)&1;
}
int tatbit(int x,int k){
   return x&(~(1<<k));
}
int xuly(){
   k=(1<<n)-1;
   for (int i=1;i<=n;i++)
      for (int j=0;j<=k;j++)
         t[i][j]=mm;
   for (int i=1;i<=n;i++)
      t[i][1<<(i-1)]=0;
   for (int sta=1;sta<=k;sta++){
      int b[k+1];
      int y=0;
      for (int j=1;j<=n;j++) {
         if (getbit(sta,j-1)==1){
            y++;
            b[y]=j;
         }
      }
```

```
        for (int i=1;i<=y;i++){
            int u=b[i];
            int st=tatbit(sta,b[i]-1);
            for (int j=1; j<=y; j++)
                if (i!=j)
                    t[u][sta]=min(t[u][sta],t[b[j]][st]+c[b[j]][u]);
        }
    }
    int kq=mm;
    for (int i=1;i<=n;i++)
        kq=min(kq,t[i][k]);
    return kq;
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0);  cout.tie(0);
    freopen("TRIP.inp", "r", stdin);
    freopen("TRIP.out", "w", stdout);
    cin>>n;
    for (int i=1;i<=n;i++)
        for (int j=1;j<=n;j++)
            cin>>c[i][j];
    cout<<xuly();
    return 0;
}
```

Câu 4. Tên bài: **MIX.CPP**

+ Gọi F[x, i] là số cách sắp các cô bò được đánh dấu trong trạng thái x (là biến gồm n bit nhị phân) mà cô bò thứ i được xếp sau cùng.

+ Quy ước trạng thái như sau: ở vị trí i là bit 1 là trong dãy có cô bò i, ngược lại nếu là bit 0 là không có cô bò i

+ Vậy F[x, i] sẽ bằng các tổng các F[y, j] với y = x ^ (1 << i) tức là dãy không có cô bò i, và chênh lệch bò i và bò j phải vượt quá k (|s[i] - s[j]| > k)

Kết quả bài toán sẽ là tổng các F[(1 << n) - 1][i] (với i = 0 … n-1)

Ta có ^ là phép *xor* trong xử lý bit (ta nhớ đơn giản chính là phép cộng tương ứng các bit, nhưng cộng không nhớ); 1 << i là dịch trái i bit của số 1 = 2^i

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
int  n;
ll k;
ll a[20], f[66000][20];
ll getbit(ll x,int j){
   return (x>>j)&1;
}
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0);  cout.tie(0);
   freopen("MIX.inp", "r", stdin);
   freopen("MIX.out", "w", stdout);
   cin>> n>> k;
   for (int i=0; i<n; i++) cin >> a[i];
   int m=(1 <<n ) - 1; // = 2^n – 1 (có giá trị nhị phân là n bit 1)
   for (int i=0; i<n; i++)  f[1<<i][i]=1;
   for (int x=0; x<=m; x++){
      for (int i=0; i<n; i++){
         if (getbit(x,i)==0) continue;
         for (int j=0; j<n; j++){
            if (getbit(x,j)&&abs(a[i]-a[j])>k){
               f[x][i]+=f[x^(1<<i)][j];
            }
         }
      }
   }
   ll res=0;
   for (int i=0;i<n;i++)
      res+=f[m][i];
   cout<<res;
   return 0;
}
```

Câu 5. Tên bài: **SHEEP.CPP**

Do m*n ≤ 30 nên trong 2 số m, n luôn tồn tại 1 số ≤ 5, lấy số nhỏ hơn làm cột. Giả sử số đó là n.

Hàm mục tiêu : F[i][x] là số cách để sắp xếp bò từ hàng 1 đến hàng i, với hàng i có trạng thái có bò/không có bò là x (xét theo hệ nhị phân).

Vì mỗi hàng có tối đa N con bò nên trạng thái x chạy từ 0 đến $2^N - 1$.

Công thức quy hoạch động F[i][x] như sau: duyệt tất cả các trạng thái y mà có thể nằm trước x và không vi phạm yêu cầu đề bài (bò không tạo thành hình vuông 2x2 và không có hình vuông 2x2 trống) :

F[i][x]= F[i][x]+ F[i−1][y].

Để kiểm tra xem 2 trạng thái x và y có thỏa mãn hay không, ta có :

+ Gọi u = (x&y) thì u không có 2 bit 1 nào kề nhau.

+ Gọi v = (x | y) thì v không có 2 bit 0 nào kề nhau (trong N bit đầu tiên).

Để kiểm tra u không có 2 bit 1 nào kề nhau thì ta kiểm tra *u & (u>>1)=0* (vì nếu u có 2 bit 1 kề nhau thì *u & (u>>1)* chắc chắn sẽ > 0 (*u>>1* nghĩa là dịch u sang phải 1 bit). Thay vì kiểm tra v không có 2 bit 0 kề nhau trong N bit đầu tiên (từ phải qua trái) thì ta kiểm tra *(~v)&(2^n -1)* không có 2 bit 1 nào kề nhau. Cách kiểm tra u và v chi tiết như hàm *check* ở code

Kết quả bài toán là tổng F[m – 1][j] với j chạu từ 0 đến $2^N - 1$ (trong code chạy chỉ số hàng và cột bắt đầu từ 0 để dễ so khớp với phép xử lí bit).

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
bool check(int x, int y, int s) {
   int u = x & y;
   int v = x|y;
   int v1 = (~v)&(s -1);
   if ((u& (u>>1) )==0&&(v1&(v1>>1))==0) return true;
   else return false;
}
ll solve(int m, int n) {
   int s = 1<<n;
   ll f[m][s];
   for(int i = 0; i < s; i++) f[0][i] = 1;
   for(int i = 1; i < m; i++ )
      for(int j = 0; j < s; j++) {
```

```
            f[i][j] = 0;
            for(int k = 0; k < s; k ++)
                if (check(j,k,s))
                    f[i][j] += f[i-1][k];
            }
        ll res = 0;
        for(int i = 0; i < s; i ++) res += f[m-1][i];
        return res;
    }
    int main() {
        ios_base::sync_with_stdio(false);
        cin.tie(0);  cout.tie(0);
        freopen("SHEEP.inp", "r", stdin);
        freopen("SHEEP.out", "w", stdout);
        int T; cin >> T;
        while (T--) {
            int m, n;
            cin >> m >> n;
            cout << solve(max(m, n), min(m, n)) << '\n';
        }
        return 0;
    }
```

Ngoài những dạng quy hoạch động nêu ở trên, chúng ta có thể tham khảo thêm trên mạng về các dạng khác như: quy hoạch động nhân ma trận, quy hoạch động trên cây…

Những lỗi Thí sinh hay mắc phải trong Kỳ thi

- Đặt tên bài làm không đúng, mặc dù bài làm có đúng thì vẫn được 0 điểm bài đó. Ví dụ như: bài làm là nguyento.cpp lại đặt là nguyento.cpp.cpp, lỗi này là máy học sinh để chế độ không hiển thị đuôi của file nên học sinh thấy không có lại F2 để thêm .cpp vào.

- Bị sai tên file đọc hoặc ghi dữ liệu, nhiều lúc tên file vào ra viết bằng tiếng anh, học sinh hay đọc lướt qua nên dễ nhầm.

- Có thói quen check bằng nhập dữ liệu bằng bàn phím và xuất ra màn hình hoặc thói quen làm bài trên mạng hay bỏ phần đọc và ghi tệp bằng thêm dấu // vào phía đầu dòng đọc và ghi. Đến lúc nộp bài rồi không để ý vẫn để vậy. Vì hiện nay kỳ thi cấp Tỉnh chấm bằng đọc và ghi bằng tệp.

- Khi khai báo không để ý kiểu dữ liệu, dẫn đến hiện tượng tràn số => các trường hợp số lớn sẽ bị sai đáp án.

- Dùng phần tử ở vị trí ngoài mảng khai báo. Ví dụ như *int a[10];* đến lúc trong chương trình ta lại sử dụng a[10] (vì mảng trên chỉ có 10 phần tử từ a[0], a[1], … a[9]. Nên khi dùng mảng ta nên khai báo thừa ra từ 2 đến 5 phần tử.

- Dùng phần tử ở vị trí ngoài xâu. Ví dụ: *string st;* đến lúc trong chương trình lại sử dụng *st[st.size()]* hoặc *st[st.length()]* (vì xâu *st* trên có độ dài là *st.size()* hoặc *st.length()* và các kí tự của nó được đánh từ vị trí *0* đến vị trí *st.zize()-1*)

- Khi dùng so sánh bằng lại thực hiện dấu = chứ không phải ==

- Chương trình dịch đang bị lỗi mà vẫn nạp bài.

- Không biết đánh giá độ phức tạp, tưởng làm đúng được test ví dụ là làm đúng cả bài.

- In dữ liệu ra không đúng yêu cầu: ví dụ ghi ra chữ “Yes” lại ghi ra chữ “yes”; ghi ra chữ “No” lại ghi ra chữ “no”

- Đọc nhầm dữ liệu đề bài: ví dụ: cho bảng có *n* hàng và *m* cột (*n* đứng trước *m*) thì lúc thí sinh lại đọc hai số *m, n* (do test ví dụ để bài, *m* và *n* bằng nhau nên thí sinh không để ý dẫn đến không phát hiện được lỗi (thói quen lâu nay toàn đọc *m* trước *n*)

Ngoài những lỗi nêu trên, tôi tin chắc sẽ còn nhiều lỗi khác trong thực tế nữa.

LỜI NÓI CUỐI

Mặc dù chúng tôi đã cố gắng hoàn thiện nhưng chắc chắn sách còn nhiều thiếu sót, các tác giả mong nhận được nhiều ý kiến đóng góp để sách sẽ hoàn thiện hơn, phục vụ bạn đọc được hiệu quả hơn. Các góp ý xin gửi về địa chỉ email: *dangphuvpt@gmail.com;* Hoặc trang facebook hỗ trợ sách: *Lập Trình C cộng cộng Thật Giản Đơn*

Bộ Sách Chinh Phục C++ gồm các cuốn:

+ Lập Trình C++ Thật Giản Đơn Tập 1+2

+ Lập Trình C++ Cấu Trúc Dữ Liệu Nâng Cao

+ Lập Trình C++ Chuyên Đề Đồ Thị

Chúng tôi xin cảm ơn quý độc giả đã quan tâm!

Link download Test:

https://drive.google.com/drive/folders/1zjBG-h5EsW5nxEw_RKpynwi3m-1HhPoJ

Hoặc lấy link tại trang Facebook: Lập Trình C cộng cộng Thật Giản Đơn